

JON BONSO AND KENNETH SAMONTE

**AWS CERTIFIED
DEVOPS
ENGINEER
PROFESSIONAL**



Tutorials Dojo Study Guide



TABLE OF CONTENTS

INTRODUCTION	8
AWS CERTIFIED DEVOPS ENGINEER PROFESSIONAL EXAM OVERVIEW	9
Exam Details	10
Exam Domains	11
Exam Domain I: SDLC Automation	12
Exam Domain II: Configuration Management and IaC	14
Exam Domain III: Resilient Cloud Solutions	15
Exam Domain IV: Monitoring and Logging	17
Exam Domain V: Incident and Event Response	19
Exam Domain VI: Security and Compliance	20
Old DOP-C01 vs the New DOP-C02 Exam Version	22
Exam Scoring System	23
Related Exam Topics	24
Excluded Exam Topics	27
Exam Benefits	27
AWS CERTIFIED DEVOPS ENGINEER PROFESSIONAL EXAM - STUDY GUIDE AND TIPS	28
Study Materials	28
AWS Services to Focus On	30
Common Exam Scenarios	31
Validate Your Knowledge	36
Sample Question 1	38
Sample Question 2	40
Domain 1: Software Development Life Cycle (SDLC) Automation	44
Overview	45
What is DevOps?	46
A Brief History of the DevOps Exam in AWS	48
Why Automate?	49
Types of Blue Green Deployment via ELB, Route 53, Elastic Beanstalk	50
AWS Lambda Function Alias Traffic Shifting	55
Basic Blue/Green Deployment using Route 53	58
Lifecycle Event Hook Availability (CodeDeploy Concept)	60
Automatically Run CodeBuild Tests After a Developer Creates a Pull Request	63
Automating Pull Request Validation with AWS CodeBuild	63
DevOps Exam Notes:	63



Automated Pull Request Validation Workflow	63
Managing Artifacts in AWS CodeBuild and CodePipeline	65
DynamoDB – Fetch vs Projected Attributes	70
CodeBuild with CloudWatch Logs, Metrics, and Alarms	72
CodeDeploy with CloudWatch Logs, Metrics, and Alarms	80
CodePipeline and Amazon EventBridge Integration	89
CodeDeploy - Linear, Canary and All-at-Once (Pre-defined Deployments)	92
Elastic Beanstalk - Deployment Policies and Settings	94
Domain 2: Configuration Management and Infrastructure-as-Code	96
Overview	97
What is Configuration Management?	98
What is Infrastructure-as-Code (IaC)?	99
CloudFormation Cross-Stack Reference	100
Lambda Function Artifact from S3 or CloudFormation Inline	102
AutoScalingReplacingUpdate vs AutoScalingRollingUpdate Policy	105
Discovery Agent vs Discovery Connector	107
CloudFormation Template for ECS, Auto Scaling and ALB	108
Domain 3: Resilient Cloud Solutions	110
Overview	111
High Availability vs. Fault Tolerance	112
Multi-AZ vs Multi-Region Architecture	114
Disaster Recovery Objectives	115
Amazon Route 53 Routing Policies	117
Amazon RDS Disaster Recovery Strategies	119
Auto Scaling Group with MinSize = 1 and MaxSize = 1	122
Auto Scaling Lifecycle Hooks	125
Amazon EKS Auto Scaling	127
Amazon EKS Networking	132
Automated Patching for Hybrid Environments	137
Amazon EC2 Image Builder	142
Automated RDS Read Replica Promotion	150
Amazon S3 Object tagging and Access control	154
Amazon Storage Gateway - S3 File Gateway	158
Amazon CodeGuru Reviewer	162
Amazon CodeGuru Profiler	167
Domain 4: Monitoring and Logging	171
Overview	172



AWS Config Multi-Account Multi-Region Data Aggregation	173
Consolidating VPC Flow Logs From Multiple Sources	174
Consolidating CloudTrail Log Files from Multiple Sources	176
Ensuring the Integrity of the CloudTrail Log Files	178
Fetching Application Logs from Amazon EC2, ECS and On-premises Servers	179
CloudWatch Logs Agent to CloudWatch Logs Subscription	181
Monitoring Service Limits with Trusted Advisor	183
Domain 5: Incident and Event Response	187
Overview	188
Incident and Event Response Management on AWS	189
Amazon S3 Event Notifications	191
Amazon RDS Event Notifications	193
AWS_RISK_CREDENTIALS_EXPOSED Event	194
AWS-Scheduled Maintenance Notification to Slack Channel	198
Using AWS Health API and Amazon EventBridge for Monitoring AWS-Scheduled Deployments/Changes	201
Monitoring Amazon EC2 Auto Scaling Events	203
Monitoring Amazon S3 Data Events in AWS CloudTrail	205
AWS CodePipeline Event Patterns	206
Monitoring Deployments in AWS CodeDeploy	209
Orchestrating Events in AWS CodePipeline	210
Domain 6: Security and Compliance	212
Overview	213
Management and Governance on AWS	214
AWS CodeBuild Configuration Best Practices	218
S3 Bucket Policy to Only Allow HTTPS Requests	220
Secrets Manager vs. Systems Manager Parameter Store	222
AWS Managed Policy	224
Using Systems Manager Automation to create AMIs	227
AWS SSM Session Manager to Record Sessions on your Instances	229
AWS Systems Manager Inventory	233
Systems Manager Patch Manager and Maintenance Windows	237
AWS IAM Identity Center	242
RELATED AWS SERVICES TO FOCUS	243
Amazon Elastic MapReduce (EMR)	244
Amazon QuickSight	244
Amazon Kinesis Data Streams	244
Amazon AppFlow	245



Amazon Eventbridge	245
AWS App Runner	245
AWS App2Container	246
Amazon EKS Deployment Options	246
Amazon EKS on AWS Outposts	246
Amazon EKS Distro	247
Red Hat OpenShift Service on AWS (ROSA)	248
AWS Database Migration Service (AWS DMS)	248
Amazon DocumentDB (with MongoDB compatibility)	248
Amazon MemoryDB for Redis	248
Amazon ElastiCache	249
AWS Command Line Interface (AWS CLI)	249
AWS Cloud Development Kit (AWS CDK)	250
AWS CloudShell	250
AWS CodeArtifact	250
Amazon CodeGuru	252
AWS Fault Injection Simulator (AWS FIS)	252
AWS Control Tower	252
Customizations for AWS Control Tower (CfCT)	253
Amazon Compute Optimizer	254
Amazon Managed Grafana	254
Amazon Managed Service for Prometheus	254
Amazon FSx	255
AWS Backup	256
AWS Elastic Disaster Recovery	256
AWS Proton	256
AWS CloudHSM	257
AWS Network Firewall	257
Amazon Detective	258
Amazon Cognito	258
AWS Virtual Private Network (AWS VPN)	259
AWS PrivateLink	259
AWS License Manager	259
AWS Service Health Dashboard	260
AWS Health API	260
AWS Resilient Hub	260
AWS CHEAT SHEETS	261



AWS Compute Services	262
Amazon Elastic Compute Cloud (EC2)	262
Amazon Elastic Container Registry (ECR)	266
Amazon Elastic Container Service (ECS)	267
AWS Elastic Beanstalk	270
AWS Lambda	271
AWS Serverless Application Model (SAM)	272
AWS Serverless Application Repository	272
AWS Storage Services	274
Amazon EBS	274
Amazon EFS	274
Amazon S3	274
Amazon S3 Bucket Policies for VPC Endpoints	277
Amazon S3 Glacier Flexible Retrieval (formerly S3 Glacier)	277
AWS Storage Gateway	279
AWS Database Services	281
Amazon Aurora	281
Aurora Serverless v2	281
Amazon DynamoDB	282
Lambda Integration With Amazon DynamoDB Streams	284
Amazon RDS	286
Amazon Redshift	288
AWS Networking & Content Delivery	289
Amazon API Gateway	289
Amazon Route 53	291
Amazon CloudFront	292
AWS Elastic Load Balancing (ELB)	294
AWS Transit Gateway	295
AWS Security & Identity Services	296
Amazon GuardDuty	296
Amazon Inspector	297
Amazon Macie	299
AWS Identity & Access Management (IAM)	300
AWS Key Management Service	302
AWS Secrets Manager	304
AWS Certificate Manager	305
AWS Directory Service	308



AWS Resource Access Manager	313
AWS Security Hub	315
AWS Security Token Service (AWS STS)	316
AWS Shield	318
AWS WAF	320
AWS Management Tools	322
AWS Organizations	322
Amazon CloudWatch	324
AWS Auto Scaling	326
AWS CloudFormation	329
AWS CloudTrail	332
AWS Config	333
AWS Health	335
AWS Systems Manager	336
AWS Trusted Advisor	340
AWS Service Catalog	341
AWS Analytics Services	344
Amazon OpenSearch Service	344
Amazon Kinesis	346
Amazon Athena	350
AWS Developer Tools	352
AWS CodeBuild	352
AWS CodeDeploy	354
AWS CodePipeline	359
AWS X-Ray	361
AWS SDKs and Tools	361
AWS Application Services	363
Amazon SNS	363
AWS Step Functions	365
Comparison of AWS Services	367
AWS CloudTrail vs Amazon CloudWatch	367
CloudWatch Agent vs SSM Agent vs Custom Daemon Scripts	368
EC2 Container Services ECS vs Lambda	369
EC2 Instance Health Check vs ELB Health Check vs Auto Scaling and Custom Health Check	371
Elastic Beanstalk vs CloudFormation vs CodeDeploy	374
Service Control Policies vs IAM Policies	376
FINAL REMARKS AND TIPS	378





INTRODUCTION

As more companies build their DevOps practices, there will always be a growing demand for certified IT Professionals that can do agile software development, configuration management, task automation, and continuous integration/continuous delivery (CI/CD). This Study Guide and Cheat Sheets eBook for AWS Certified DevOps Engineer - Professional aims to equip you with the necessary knowledge and practical skill sets needed to pass the latest version of the AWS Certified DevOps Engineer – Professional exam.

This eBook contains the essential concepts, exam domains, exam tips, sample questions, cheat sheets, and other relevant information about the AWS Certified DevOps Engineer – Professional exam. This study guide begins with the presentation of the exam structure, giving you an insight into the question types, exam domains, scoring scheme, and the list of benefits you'll receive once you pass the exam. We used the official AWS [exam guide](#) to structure the contents of this guide, where each section discusses a particular exam domain. Various DevOps concepts, related AWS services, and technical implementations are covered to provide you with an idea of what to expect on the actual exam.

DevOps Exam Notes:

Don't forget to read the boxed "**exam tips**" (like this one) scattered throughout the eBook, as these are the key concepts that you will likely encounter on your test. After covering the six domains, we have added a bonus section containing a curated list of AWS Cheat Sheets to fast-track your review. The last part of this guide includes a collection of articles that compares two or more similar AWS services to supplement your knowledge.

The AWS Certified DevOps Engineer - Professional certification exam is a difficult test to pass; therefore, anyone who wants to take it must allocate ample time for review. The exam registration cost is not cheap, which is why we spent considerable time and effort to ensure that this study guide provides you with the essential and relevant knowledge to increase your chances of passing the DevOps exam.

**** Note:** *This eBook is meant to be just a supplementary resource when preparing for the exam. We highly recommend working on [hands-on sessions](#) and [practice exams](#) to further expand your knowledge and improve your test-taking skills.*



AWS CERTIFIED DEVOPS ENGINEER PROFESSIONAL EXAM OVERVIEW

The AWS Certified DevOps Engineer - Professional certification exam validates various skills which are necessary to become a full-fledged DevOps engineer. The exam will check your capability in implementing and managing continuous delivery systems and methodologies on AWS. Automating security controls, validating compliance, and optimizing the governance processes are also included in the test.

This professional-level AWS certification exam validates your technical expertise in provisioning, operating, and managing distributed systems and services on the AWS Cloud. It also verifies your ability to complete the following DevOps tasks:

- Implement and manage continuous delivery systems and methodologies on AWS.
- Implement and automate security controls, governance processes, and compliance validation.
- Define and deploy monitoring, metrics, and logging systems on AWS.
- Implement systems that are highly available, scalable, and self-healing on AWS.
- Design, manage, and maintain tools to automate operational processes

Before we discuss the details of the new exam, it's important to know the history of this certification test to better understand the changes it entails. We will go back in time and re-discover the history of the AWS Certified DevOps Engineer Professional exam and other exam-related information.

Amazon Web Services (AWS) started its Global Certification Program in 2013, which was about a decade ago. The primary purpose of this program is to validate the necessary technical skills and knowledge required for building secure and reliable cloud-based applications using the AWS Cloud. By passing the AWS Certification exam, IT professionals can prove their expertise and knowledge in the AWS Cloud to their current employers or even to the prospective companies they wish to apply for. AWS unveiled the Professional and Specialty-level certifications in an effort to expand its certification program and continuously release new updates.

These Professional-level exams have covered various domains, namely monitoring, security, SDLC, Infrastructure as Code(IaC), data analytics, advanced networking, machine learning, and many others. There are a bunch of new and updated versions of AWS certification exams that are released on a regular basis to include the new services offered by AWS and as well as to incorporate the new knowledge areas.

There are basically two Professional-level exams offered by AWS, which are the AWS Certified Solutions Architect – Professional and the AWS Certified DevOps Engineer – Professional. The first version of the AWS Certified Solutions Architect Professional exam (SAP-C00) was released in May 2014. This was followed by the first version of the AWS Certified DevOps Engineer Professional exam (DOP-C00) on February 2015. After 4 years, an updated version of the AWS Certified DevOps Engineer – Professional certification was launched on February 2019 with an exam code of DOP-C01.



After 4 years, the AWS Certification and Training team released yet another version of this certification test with an exam code of DOP-C02. The latest version of the AWS Certified DevOps Engineer – Professional certification exam was released on the 7th day of March 2023. Based on this trend, it can be assumed that the new version of the DevOps Professional exam will be coming in around 2025 or 2026 with an exam code of DOP-C03.

Exam Details

The AWS Certified DevOps Engineer Professional certification is intended for IT Professionals who perform a Solutions Architect or DevOps role and have substantial years of hands-on experience designing available, cost-efficient, fault-tolerant, and scalable distributed systems on the AWS platform. It is composed of scenario-based questions that can either be in multiple-choice or multiple-response formats. The first question type has one correct answer and three incorrect responses, while the latter has two or more correct responses out of five or more options. You can take the exam from a local testing center or online from the comfort of your home.

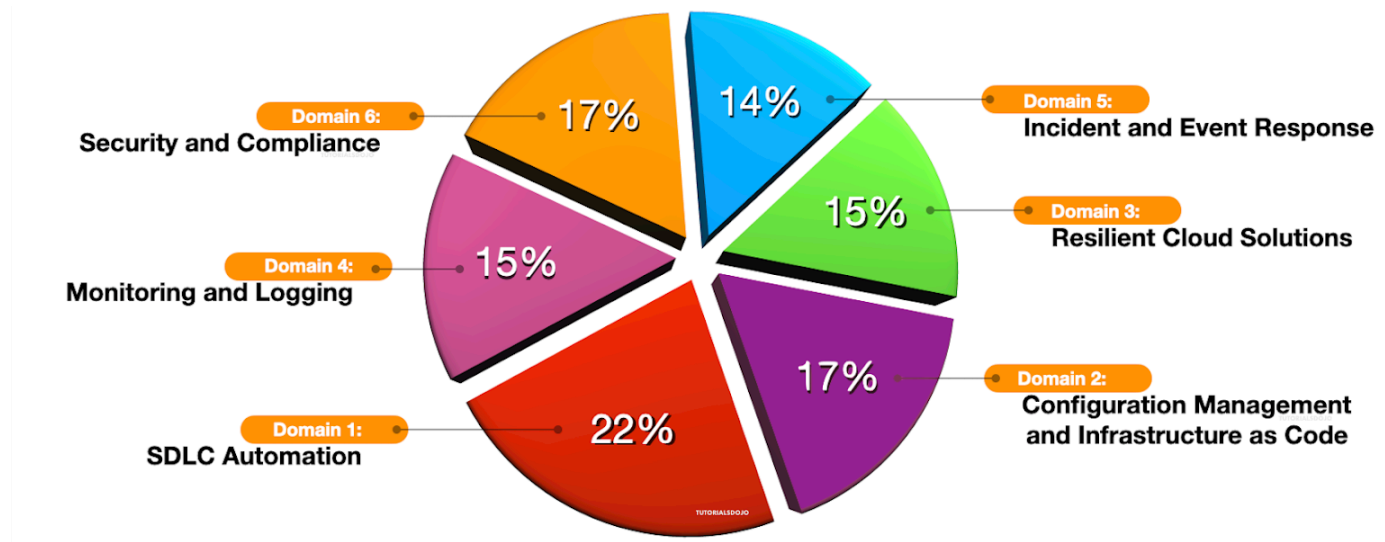
Exam Code:	DOP-C02
Release Date:	March 2023
Prerequisites:	None
No. of Questions:	75
Score Range:	100 - 1000
Passing Score:	750/1000
Time Limit:	3 hours (180 minutes)
Format:	Scenario-based. Multiple choice/multiple answers.
Delivery Method:	Testing center or online proctored exam.

Don't be confused if you see in your Pearson Vue booking that the duration is 190 minutes since they included an additional 10 minutes for reading the Non-Disclosure Agreement (NDA) at the start of the exam and the survey at the end of it.

Exam Domains

The AWS Certified DevOps Engineer Professional (DOP-C02) exam has 6 different domains, each with corresponding weight and topic coverage. The exam domains are as follows:

- Domain 1: SDLC Automation (22%)
- Domain 2: Configuration Management and IaC (17%)
- Domain 3: Resilient Cloud Solutions (15%)
- Domain 4: Monitoring and Logging (15%)
- Domain 5: Incident and Event Response (14%)
- Domain 6: Security and Compliance (17%)



Let's The list of exam domains can be found on the official Exam Guide for the AWS Certified DevOps Engineer - Professional exam. Each exam domain is comprised of several task statements. A task statement is a sub-category of the exam domain that contains the required cloud concepts, knowledge, and skills for you to accomplish a particular task or activity in AWS.

Let's look at each of these domains one by one.



Exam Domain I: SDLC Automation

Task Statement 1: Implement CI/CD pipelines.

Knowledge of:

- Software development lifecycle (SDLC) concepts, phases, and models
- Pipeline deployment patterns for single- and multi-account environments

Skills in:

- Configuring code, image, and artifact repositories
- Using version control to integrate pipelines with application environments
- Setting up build processes (for example, AWS CodeBuild)
- Managing build and deployment secrets (for example, AWS Secrets Manager, AWS Systems Manager Parameter Store)
- Determining appropriate deployment strategies (for example, AWS CodeDeploy)

Task Statement 2: Integrate automated testing into CI/CD pipelines.

Knowledge of:

- Different types of tests (for example, unit tests, integration tests, acceptance tests, user interface tests, security scans)
- Reasonable use of different types of tests at different stages of the CI/CD pipeline

Skills in:

- Running builds or tests when generating pull requests or code merges (for example, Git, CodeBuild)
- Running load/stress tests, performance benchmarking, and application testing at scale
- Measuring application health based on application exit codes
- Automating unit tests and code coverage
- Invoking AWS services in a pipeline for testing



Task Statement 3: Build and manage artifacts.

Knowledge of:

- Artifact use cases and secure management
- Methods to create and generate artifacts
- Artifact lifecycle considerations

Skills in:

- Creating and configuring artifact repositories (for example, AWS CodeArtifact, Amazon S3, Amazon Elastic Container Registry [Amazon ECR])
- Configuring build tools for generating artifacts (for example, CodeBuild, AWS Lambda)
- Automating Amazon EC2 instance and container image build processes (for example, EC2 Image Builder)

Task Statement 4: Implement deployment strategies for instance, container, and serverless environments.

Knowledge of:

- Deployment methodologies for various platforms (for example, Amazon EC2, Amazon Elastic Container Service [Amazon ECS], Amazon Elastic Kubernetes Service [Amazon EKS], Lambda)
- Application storage patterns (for example, Amazon Elastic File System [Amazon EFS], Amazon S3, Amazon Elastic Block Store [Amazon EBS])
- Mutable deployment patterns in contrast to immutable deployment patterns
- Tools and services available for distributing code (for example, CodeDeploy, EC2 Image Builder)

Skills in:

- Configuring security permissions to allow access to artifact repositories (for example, AWS Identity and Access Management [IAM], CodeArtifact)
- Configuring deployment agents (for example, CodeDeploy agent)
- Troubleshooting deployment issues
- Using different deployment methods (for example, blue/green, canary)



Exam Domain II: Configuration Management and IaC

Task Statement 1: Define cloud infrastructure and reusable components to provision and manage systems throughout their lifecycle.

Knowledge of:

- Infrastructure as code (IaC) options and tools for AWS
- Change management processes for IaC-based platforms
- Configurations management services and strategies

Skills in:

- Composing and deploying IaC templates (for example, AWS Serverless Application Model [AWS SAM], AWS CloudFormation, AWS Cloud Development Kit [AWS CDK])
- Applying AWS CloudFormation StackSets across multiple accounts and AWS Regions
- Determining optimal configuration management services (for example, AWS Systems Manager, AWS Config, AWS AppConfig)
- Implementing infrastructure patterns, governance controls, and security standards into reusable IaC templates (for example, AWS Service Catalog, CloudFormation modules, AWS CDK)

Task Statement 2: Deploy automation to create, onboard, and secure AWS accounts in a multi-account/multi-Region environment.

Knowledge of:

- AWS account structures, best practices, and related AWS services

Skills in:

- Standardizing and automating account provisioning and configuration
- Creating, consolidating, and centrally managing accounts (for example, AWS Organizations, AWS Control Tower)
- Applying IAM solutions for multi-account and complex organization structures (for example, SCPs, assuming roles)
- Implementing and developing governance and security controls at scale (AWS Config, AWS Control Tower, AWS Security Hub, Amazon Detective, Amazon GuardDuty, AWS Service Catalog, SCPs)



Task Statement 3: Design and build automated solutions for complex tasks and large-scale environments.

Knowledge of:

- AWS services and solutions to automate tasks and processes
- Methods and strategies to interact with the AWS software-defined infrastructure

Skills in:

- Automating system inventory, configuration, and patch management (for example, Systems Manager, AWS Config)
- Developing Lambda function automation for complex scenarios (for example, AWS SDKs, Lambda, AWS Step Functions)
- Automating the configuration of software applications to the desired state (for example, Systems Manager State Manager)
- Maintaining software compliance (for example, Systems Manager)

Exam Domain III: Resilient Cloud Solutions

Task Statement 1: Implement highly available solutions to meet resilience and business requirements.

Knowledge of:

- Multi-AZ and multi-Region deployments (for example, compute layer, data layer)
- Service Level Agreements (SLAs)
- Replication and failover methods for stateful services
- Techniques to achieve high availability (for example, Multi-AZ, multi-Region)

Skills in:

- Translating business requirements into technical resiliency needs
- Identifying and remediating single points of failure in existing workloads
- Enabling cross-Region solutions where available (for example, Amazon DynamoDB, Amazon RDS, Amazon Route 53, Amazon S3, Amazon CloudFront)
- Configuring load balancing to support cross-AZ services
- Configuring applications and related services to support multiple Availability Zones and Regions while minimizing downtime



Task Statement 2: Implement solutions that are scalable to meet business requirements.

Knowledge of:

- Appropriate metrics for scaling services
- Loosely coupled and distributed architectures
- Serverless architectures
- Container platforms

Skills in:

- Identifying and remediating scaling issues
- Identifying and implementing appropriate auto-scaling, load balancing, and caching solutions
- Deploying container-based applications (for example, Amazon ECS, Amazon EKS)
- Deploying workloads in multiple AWS Regions for global scalability
- Configuring serverless applications (for example, Amazon API Gateway, Lambda, AWS Fargate)

Task Statement 3: Implement automated recovery processes to meet RTO/RPO requirements.

Knowledge of:

- Disaster recovery concepts (for example, RTO, RPO)
- Backup and recovery strategies (for example, pilot light, warm standby)
- Recovery procedures

Skills in:

- Testing failover of Multi-AZ/multi-Region workloads (for example, Amazon RDS, Amazon Aurora, Route 53, CloudFront)
- Identifying and implementing appropriate cross-Region backup and recovery strategies (for example, AWS Backup, Amazon S3, Systems Manager)
- Configuring a load balancer to recover from backend failure



Exam Domain IV: Monitoring and Logging

Task Statement 1: Configure the collection, aggregation, and storage of logs and metrics.

Knowledge of:

- How to monitor applications and infrastructure
- Amazon CloudWatch metrics (for example, namespaces, metrics, dimensions, and resolution)
- Real-time log ingestion
- Encryption options for at-rest and in-transit logs and metrics (for example, client-side and server-side, AWS Key Management Service [AWS KMS])
- Security configurations (for example, IAM roles and permissions to allow for log collection)

Skills in:

- Securely storing and managing logs
- Creating CloudWatch metrics from log events by using metric filters
- Creating CloudWatch metric streams (for example, Amazon S3 or Amazon Data Firehose options)
- Collecting custom metrics (for example, using the CloudWatch agent)
- Managing log storage lifecycles (for example, S3 lifecycles, CloudWatch log group retention)
- Processing log data by using CloudWatch log subscriptions (for example, Kinesis, Lambda, Amazon OpenSearch Service)
- Searching log data by using filter and pattern syntax or CloudWatch Logs Insights
- Configuring encryption of log data (for example, AWS KMS)

Task Statement 2: Audit, monitor, and analyze logs and metrics to detect issues.

Knowledge of:

- Anomaly detection alarms (for example, CloudWatch anomaly detection)
- Common CloudWatch metrics and logs (for example, CPU utilization with Amazon EC2, queue length with Amazon RDS, 5xx errors with an Application Load Balancer)
- Amazon Inspector and common assessment templates
- AWS Config rules
- AWS CloudTrail log events



Skills in:

- Building CloudWatch dashboards and Amazon QuickSight visualizations
- Associating CloudWatch alarms with CloudWatch metrics (standard and custom)
- Configuring AWS X-Ray for different services (for example, containers, API Gateway, Lambda)
- Analyzing real-time log streams (for example, using Kinesis Data Streams)
- Analyzing logs with AWS services (for example, Amazon Athena, CloudWatch Logs Insights)

Task Statement 3: Automate monitoring and event management of complex environments.

Knowledge of:

- Event-driven, asynchronous design patterns (for example, S3 Event Notifications or Amazon EventBridge events to Amazon Simple Notification Service [Amazon SNS] or Lambda)
- Capabilities of auto scaling a variety of AWS services (for example, EC2 Auto Scaling groups, RDS storage auto scaling, DynamoDB, ECS capacity provider, EKS autoscalers)
- Alert notification and action capabilities (for example, CloudWatch alarms to Amazon SNS, Lambda, EC2 automatic recovery)
- Health check capabilities in AWS services (for example, Application Load Balancer target groups, Route 53)

Skills in:

- Configuring solutions for auto scaling (for example, DynamoDB, EC2 Auto Scaling groups, RDS storage auto scaling, ECS capacity provider)
- Creating CloudWatch custom metrics and metric filters, alarms, and notifications (for example, Amazon SNS, Lambda)
- Configuring S3 events to process log files (for example, by using Lambda), and deliver log files to another destination (for example, OpenSearch Service, CloudWatch Logs)
- Configuring EventBridge to send notifications based on a particular event pattern
- Installing and configuring agents on EC2 instances (for example, AWS Systems Manager Agent [SSM Agent], CloudWatch agent)
- Configuring AWS Config rules to remediate issues
- Configuring health checks (for example, Route 53, Application Load Balancer)



Exam Domain V: Incident and Event Response

Task Statement 1: Manage event sources to process, notify, and take action in response to events.

Knowledge of:

- AWS services that generate, capture, and process events (for example, AWS Health, EventBridge, CloudTrail)
- Event-driven architectures (for example, fan out, event streaming, queuing)

Skills in:

- Integrating AWS event sources (for example, AWS Health, EventBridge, CloudTrail)
- Building event processing workflows (for example, Amazon Simple Queue Service [Amazon SQS], Kinesis, Amazon SNS, Lambda, Step Functions)

Task Statement 2: Implement configuration changes in response to events.

Knowledge of:

- Fleet management services (for example, Systems Manager, AWS Auto Scaling)
- Configuration management services (for example, AWS Config)

Skills in:

- Applying configuration changes to systems
- Modifying infrastructure configurations in response to events
- Remediating a non-desired system state

Task Statement 3: Troubleshoot system and application failures.

Knowledge of:

- AWS metrics and logging services (for example, CloudWatch, X-Ray)
- AWS service health services (for example, AWS Health, CloudWatch, Systems Manager OpsCenter)
- Root cause analysis



Skills in:

- Analyzing failed deployments (for example, AWS CodePipeline, CodeBuild, CodeDeploy, CloudFormation, CloudWatch synthetic monitoring)
- Analyzing incidents regarding failed processes (for example, auto-scaling, Amazon ECS, Amazon EKS)

Exam Domain VI: Security and Compliance

Task Statement 1: Implement techniques for identity and access management at scale.

Knowledge of:

- Appropriate usage of different IAM entities for human and machine access (for example, users, groups, roles, identity providers, identity-based policies, resource-based policies, session policies)
- Identity federation techniques (for example, using IAM identity providers and AWS IAM Identity Center)
- Permission management delegation by using IAM permissions boundaries
- Organizational SCPs

Skills in:

- Designing policies to enforce least privilege access
- Implementing role-based and attribute-based access control patterns
- Automating credential rotation for machine identities (for example, Secrets Manager)
- Managing permissions to control access to human and machine identities (for example, enabling multi-factor authentication [MFA], AWS Security Token Service [AWS STS], IAM profiles)

Task Statement 2: Apply automation for security controls and data protection.

Knowledge of:

- Network security components (for example, security groups, network ACLs, routing, AWS Network Firewall, AWS WAF, AWS Shield)
- Certificates and public key infrastructure (PKI)
- Data management (for example, data classification, encryption, key management, access controls)



Skills in:

- Automating the application of security controls in multi-account and multi-Region environments (for example, Security Hub, Organizations, AWS Control Tower, Systems Manager)
- Combining security controls to apply defense in depth (for example, AWS Certificate Manager [ACM], AWS WAF, AWS Config, AWS Config rules, Security Hub, GuardDuty, security groups, network ACLs, Amazon Detective, Network Firewall)
- Automating the discovery of sensitive data at scale (for example, Amazon Macie)
- Encrypting data in transit and data at rest (for example, AWS KMS, AWS CloudHSM, ACM)

Task Statement 3: Implement security monitoring and auditing solutions.

Knowledge of:

- Security auditing services and features (for example, CloudTrail, AWS Config, VPC Flow Logs, CloudFormation drift detection)
- AWS services for identifying security vulnerabilities and events (for example, GuardDuty, Amazon Inspector, IAM Access Analyzer, AWS Config)
- Common cloud security threats (for example, insecure web traffic, exposed AWS access keys, S3 buckets with public access enabled or encryption disabled)

Skills in:

- Implementing robust security auditing
- Configuring alerting based on unexpected or anomalous security events
- Configuring service and application logging (for example, CloudTrail, CloudWatch Logs)
- Analyzing logs, metrics, and security findings

Old DOP-C01 vs the New DOP-C02 Exam Version

In terms of the number of exam domains, the old version of the AWS Certified DevOps Engineer Professional (DOP-C01) has the exact same number of domains as compared with the new DOP-C02 version. However, there are differences in terms of exam coverage and names for some domains.

The biggest exam domain is still the SDLC (Software Development Lifecycle) Automation domain which retains its 22% percent exam coverage. The same goes for the Monitoring and Logging domain, which still has 15 percent. This is followed by the Configuration Management and Infrastructure as Code (IaC) domain which is down to only 17% exam coverage from the previous 19% percent. The Incident and Event Response domain has a huge 4% decline as it only has 14% coverage coming from an 18% high on the previous version.

You can also notice that two exam domains have changed their name:

- The **“High Availability, Fault Tolerance, and Disaster Recovery”** domain has been renamed and is now called the **“Resilient Cloud Solutions”** domain.
- The **“Policies and Standards Automation”** domain is now **“Security and Compliance”**

DOP-C01 Valid until March 6, 2023		aws certified DevOps Engineer PROFESSIONAL	DOP-C02 ^{NEW} Available on March 7, 2023	
DOMAIN	% of Examination		DOMAIN	% of Examination
SDLC Automation	22%		SDLC Automation	22%
Configuration Management and Infrastructure as Code	19%		Configuration Management and Infrastructure as Code	17% ↓
Incident and Event Response	18%		^{RENAMED} Security and Compliance	17% ↑
High Availability, Fault Tolerance, and Disaster Recover	16%		^{RENAMED} Resilient Cloud Solutions	15% ↓
Monitoring and Logging	15%		Monitoring and Logging	15%
Policies and Standards Automation	10%		Incident and Event Response	14% ↓

- There is no BETA exam for the DOP-C02 exam version



The concept of resiliency is related to High Availability, Fault Tolerance, and Disaster Recovery. This is the primary reason why AWS renamed this lengthy domain as “Resilient Cloud Solutions” for brevity. From 16%, this exam domain has a slight decrease of coverage at 15% percent.

Security in AWS can be implemented through IAM Policies, Service Control Policies (SCPs), Bucket Policies, VPC Endpoint Policies, and other types of policies. The term “standards” is synonymous with the word “compliance” in the IT industry. The name of the Policies and Standards Automation exam domain was simplified and is now officially the Security and Compliance domain. It’s interesting to note that on the previous DOP-C01 version, this domain has the lowest exam coverage at 10%, but now, it has become the second largest exam domain for DOP-C02 with 17% coverage.

As you can notice, the DevOps Pro exam has significantly included many security-related topics based on its new exam domain content distribution. This means that you have to focus on various security topics and security services offered by AWS.

Exam Scoring System

You can get a score from 100 to 1,000 with a minimum passing score of **750** when you take the DevOps Engineer Professional exam. AWS uses a scaled scoring model to equate scores across multiple exam types that may have different difficulty levels. The complete score report will be sent to you by email after a few days. Right after you complete the actual exam, you’ll immediately see a pass or fail notification on the testing screen. A *“Congratulations! You have successfully passed...”* message will be shown if you pass the exam.

Individuals who unfortunately do not pass the AWS exam must wait 14 days before they are allowed to retake the exam. Fortunately, there is no hard limit on exam attempts until you pass the exam. Take note that on each attempt, the full registration price of the AWS exam must be paid.

Within 5 business days of completing your exam, your AWS Certification Account will have a record of your complete exam results. The score report contains a table of your performance at each section/domain, which indicates whether you met the competency level required for these domains or not. AWS is using a compensatory scoring model, which means that you do not necessarily need to pass each and every individual section, only the overall examination. Each section has a specific score weighting that translates to the number of questions; hence, some sections have more questions than others. The Score Performance table highlights your strengths and weaknesses that you need to improve on.



Related Exam Topics

The new AWS Certified DevOps Engineer – Professional exam (DOP-C02) is focused on the various tools, services, and knowledge areas that revolve around DevOps in AWS. The official exam guide provides a list of AWS services, general tools, and technologies that are grouped according to their primary functions. Keep in mind that even though some of these topics will likely be covered more than others on the exam, the placement or order of these exam topics/ AWS services in this list is not an indication of any relative weight or importance.

The relevant exam topics that you should be familiar with on your upcoming DOP-C02 exam are:

- Application Deployment
- Application Integration
- Application pipelines
- Automation
- Code repository best practices
- Cost optimization
- Deployment requirements
- Hybrid deployments
- IAM policies
- Metrics, monitoring, alarms, and logging
- Network ACL and security group design and implementation
- Operational best practices
- Rollback procedures



Here is the list of relevant AWS services that are covered in the AWS Certified DevOps Engineer – Professional (DOP-C02) exam based on the official exam guide. You must focus on these AWS services and their respective features for your upcoming test:

Analytics: • Amazon Athena • Amazon Elastic MapReduce (Amazon EMR) • Amazon Data Firehose • Amazon Kinesis Data Streams • Amazon OpenSearch Service • Amazon QuickSight Application Integration: • Amazon AppFlow • Amazon EventBridge Compute: • AWS App Runner • Amazon EC2 • Amazon EC2 Auto Scaling • EC2 Image Builder • AWS Elastic Beanstalk • AWS Serverless Application Repository Containers: • AWS App2Container • Amazon Elastic Container Registry (Amazon ECR) • Amazon Elastic Container Service (Amazon ECS) • Amazon Elastic Kubernetes Service (Amazon EKS) • Amazon EKS Distro • AWS Fargate • Red Hat OpenShift Service on AWS (ROSA) Database: • Amazon Aurora • Amazon Aurora Serverless v2 • AWS Database Migration Service (AWS DMS) • Amazon DocumentDB (with MongoDB compatibility) • Amazon DynamoDB • Amazon ElastiCache • Amazon MemoryDB for Redis • Amazon RDS • Amazon Redshift Developer Tools: • AWS Cloud Development Kit (AWS CDK)	Management and Governance: • AWS Auto Scaling • AWS CloudFormation • AWS CloudTrail • Amazon CloudWatch • Amazon CloudWatch Logs • AWS Compute Optimizer • AWS Config • AWS Control Tower • AWS Health • AWS License Manager • Amazon Managed Grafana • Amazon Managed Service for Prometheus • AWS Organizations • AWS Proton • AWS Resilience Hub • AWS Service Catalog • AWS Systems Manager • AWS Trusted Advisor Networking and Content Delivery: • Amazon API Gateway • AWS Client VPN • Amazon CloudFront • Elastic Load Balancing (ELB) • AWS PrivateLink • AWS Site-to-Site VPN • Amazon Route 53 • AWS Transit Gateway • Amazon VPC Serverless: • AWS Lambda • AWS Serverless Application Model (AWS SAM) • Amazon Simple Notification Service (Amazon SNS) • Amazon Simple Queue Service (Amazon SQS) • AWS Step Functions Security, Identity, and Compliance: • AWS Certificate Manager (ACM) • AWS CloudHSM • Amazon Cognito • Amazon Detective • AWS Directory Service
--	--



- AWS CloudShell
- AWS CodeArtifact
- AWS CodeBuild
- AWS CodeDeploy
- Amazon CodeGuru
- AWS CodePipeline
- AWS Command Line Interface (AWS CLI)
- AWS Fault Injection Simulator
- AWS SDKs and Tools
- AWS X-Ray

- Amazon GuardDuty
- AWS Identity and Access Management (IAM)
- Amazon Inspector
- AWS Key Management Service (AWS KMS)
- Amazon Macie
- AWS Network Firewall
- AWS Resource Access Manager (AWS RAM)
- AWS Secrets Manager
- AWS Security Hub
- AWS Security Token Service (AWS STS)
- AWS Shield
- AWS IAM Identity Center
- AWS WAF

Storage:

- AWS Backup
- Amazon Elastic Block Store (Amazon EBS)
- AWS Elastic Disaster Recovery (AWS DRS)
- Amazon Elastic File System (Amazon EFS)
- Amazon FSx for Lustre
- Amazon FSx for NetApp ONTAP
- Amazon FSx for OpenZFS
- Amazon FSx for Windows File Server
- Amazon S3
- Amazon S3 Glacier Flexible Retrieval (formerly S3 Glacier)
- AWS Storage Gateway



Excluded Exam Topics

Usually, the official exam guides provide a list of both the relevant and irrelevant AWS services for the exam. This is not the case for the latest AWS Certified DevOps Engineer Professional DOP-C02 exam. The official DOP-C02 exam guide doesn't come with a list of exam topics that are not in scope for this certification test. However, we can deduce the out-of-scope AWS topics by comparing the [exam guide for the AWS Certified Solutions Architect – Professional SAP-C02 exam](#).

Just a friendly reminder that the following AWS services and features do not represent each and every AWS offering that is excluded from the DOP-C02 exam content. This list is only a hint of what topics are not covered on the AWS Certified DevOps Engineer – Professional exam, which you should not focus on:

- Machine Learning
- Internet-of-Things (IoT)
- Frontend development for mobile apps
- 12-factor app methodology
- AWS Direct Connect

Exam Benefits

If you successfully passed any AWS exam, you will be eligible for the following benefits:

- **Exam Discount** - You'll get a 50% discount voucher that you can apply for your recertification or any other exam you plan to pursue. To access your discount voucher code, go to the "Benefits" section of your AWS Certification Account, and apply the voucher when you register for your next exam.
- **Certification Digital Badges** - You can showcase your achievements to your colleagues and employers with digital badges on your email signatures, LinkedIn profile, or on your social media accounts. You can also show your Digital Badge to gain exclusive access to Certification Lounges at AWS re:Invent, regional Appreciation Receptions, and select AWS Summit events. To view your badges, simply go to the "Digital Badges" section of your AWS Certification Account.
- **Event Recognition** - You can receive invitations to regional Appreciation Receptions and use your digital badge for exclusive access to AWS Certification Lounges at AWS re:Invent and select AWS Summit events.



AWS CERTIFIED DEVOPS ENGINEER PROFESSIONAL EXAM - STUDY GUIDE AND TIPS

This certification is the pinnacle of your DevOps career in AWS. The AWS Certified DevOps Engineer Professional (or AWS DevOps Pro) is the advanced certification of both [AWS SysOps Administrator Associate](#) and [AWS Developer Associate](#). This is similar to how the AWS Solutions Architect Professional role is a more advanced version of the AWS Solutions Architect Associate.

Generally, AWS recommends that you first take (and pass) both AWS SysOps Administrator Associate and AWS Developer Associate certification exams before taking on this certification. Previously, it was a prerequisite that you obtain the associate level certifications before you are allowed to go for the professional level. Last October 2018, AWS removed this ruling to provide customers with a more flexible approach to the certifications.

Study Materials

The [FREE AWS Exam Readiness course](#), [official AWS sample questions](#), Whitepapers, FAQs, AWS Documentation, Re:Invent videos, forums, labs, [AWS cheat sheets](#), [practice tests](#), and personal experiences are what you will need to pass the exam. Since the DevOps Pro is one of the most difficult AWS certification exams out there, you have to prepare yourself with every study material you can get your hands on. If you need a review on the fundamentals of AWS DevOps, then do check out our review guides for the [AWS SysOps Administrator Associate](#) and [AWS Developer Associate](#) certification exams. Also, visit this [AWS exam blueprint](#) to learn more details about your certification exam.

For virtual classes, you can attend the [DevOps Engineering on AWS](#) and [Systems Operations on AWS](#) classes since they will teach you concepts and practices that are expected to be in your exam.

For whitepapers, focus on the following:

1. [Running Containerized Microservices on AWS](#)
2. [Microservices on AWS](#)
3. [Infrastructure as Code](#)
4. [Introduction to DevOps](#)
5. [Practicing Continuous Integration and Continuous Delivery on AWS](#)
6. [Jenkins on AWS](#)
7. [Blue/Green Deployments on AWS whitepaper](#)
8. [Development and Test on AWS](#)

Almost all online training you need can be found on the AWS web page. One digital course that you should check out is the [Exam Readiness: AWS Certified DevOps Engineer – Professional](#) course. This digital course

contains lectures on the different domains of your exam, and they also provide a short quiz right after each lecture to validate what you have just learned.

Exam Readiness:
AWS Certified
DevOps Engineer -
Professional

13% COMPLETE

- Course Overview ✓
- Domain 1: SDLC Automation ○
- Domain 2: Configuration Management and Infrastructure as ... ○
- Domain 3: Monitoring and Logging ○
- Domain 4: Policies and Standards Automation ○
- Domain 5: Incident and Event ○

AWS CloudFormation: update options

- **MaxBatchSize:** maximum number of instances that are updated
- **MinInstancesInService:** minimum number of instances that must be in service within the Auto Scaling group while old instances are updated
- **PauseTime:** amount of time given to instances to start applications

```

"Resources": {
  "WebServerGroup": {
    "Type": "AWS::AutoScaling::AutoScalingGroup",
    "CreationPolicy": {
      "ResourceSignal": {
        "Timeout": "PT15M",
        "Count": "2"
      }
    }
  },
  "UpdatePolicy": {
    "AutoScalingRollingUpdate": {
      "MaxBatchSize": "1",
      "MinInstancesInService": "1",
      "PauseTime": "PT15M",
      "WaitOnResourceSignals": "true"
    }
  }
}

```

SAMPLE QUESTIONS

Lastly, do not forget to study the AWS CLI, SDKs, and APIs. Since DevOps Pro is also an advanced certification for Developer Associate, you need to have knowledge of programming and scripting in AWS. Go through the AWS documentation to review the syntax of the CloudFormation template, Serverless Application Model template, CodeBuild buildspec, CodeDeploy appspec, and IAM Policy.

Also, check out this article: [Top 5 FREE AWS Review Materials](#).



AWS Services to Focus On

Since this exam is a professional level one, you should already have a deep understanding of the AWS services listed under our SysOps Administrator Associate and Developer Associate review guides. In addition, you should familiarize yourself with the following services since they commonly come up in the DevOps Pro exam:

1. [AWS CloudFormation](#)
2. [AWS Lambda](#)
3. [Amazon CloudWatch](#)
4. [Amazon EventBridge](#)
5. [Amazon CloudWatch Alarms](#)
6. [AWS CodePipeline](#)
7. [AWS CodeDeploy](#)
8. [AWS CodeBuild](#)
9. [AWS Config](#)
10. [AWS Systems Manager](#)
11. [Amazon ECS](#)
12. [Amazon Elastic Beanstalk](#)
13. [AWS CloudTrail](#)
14. [AWS Trusted Advisor](#)

The FAQs provide a good summary for each service, however, the AWS documentation contains more detailed information that you'll need to study. These details will be the deciding factor in determining the correct choice from the incorrect choices in your exam. To supplement your review of the services, we recommend that you take a look at [Tutorials Dojo's AWS Cheat Sheets](#). Their contents are well-written and straight to the point, which will help reduce the time spent going through FAQs and documentation.



Common Exam Scenarios

Scenario	Solution
Software Development and Lifecycle (SDLC) Automation	
An Elastic Beanstalk application must not have any downtime during deployment and requires an easy rollback to the previous version if an issue occurs.	Set up Blue/Green deployment, deploy a new version on a separate environment then swap environment URLs on Elastic Beanstalk.
A new version of an AWS Lambda application is ready to be deployed, and the deployment should not cause any downtime. A quick rollback to the previous Lambda version must be available.	Publish a new version of the Lambda function. After testing, use the production Lambda Alias to point to this new version.
In an AWS Lambda application deployment, only 10% of the incoming traffic should be routed to the new version to verify the changes before eventually allowing all production traffic.	Set up Canary deployment for AWS Lambda. Create a Lambda Alias pointed to the new Version. Set Weighted Alias value for this Alias as 10%.
An application hosted in Amazon EC2 instances behind an Application Load Balancer. You must provide a safe way to upgrade the version on Production and allow easy rollback to the previous version.	Launch the application in Amazon EC2 that runs the new version with an Application Load Balancer (ALB) in front. Use Route 53 to change the ALB A-record Alias to the new ALB URL. Rollback by changing the A-record Alias to the old ALB.
During the deployment, you need to run custom actions before deploying the new version of the application using AWS CodeDeploy.	Add lifecycle hook action BeforeAllowTraffic
You need to run custom verification actions after the new version is deployed using AWS CodeDeploy.	Add lifecycle hook action AfterAllowTraffic
You need to set up AWS CodeBuild to automatically run after a pull request has been successfully merged.	Create Amazon EventBridge rule to detect pull requests and action set to trigger CodeBuild Project. Use AWS Lambda to update the pull request with the result of the project Build



You need to use AWS CodeBuild to create artifact and automatically deploy the new application version	Set CodeBuild to save artifact to S3 bucket. Use CodePipeline to deploy using CodeDeploy and set the build artifact from the CodeBuild output.
You need to upload the AWS CodeBuild artifact to Amazon S3	S3 bucket needs to have versioning and encryption enabled.
You need to review AWS CodeBuild Logs and have an alarm notification for build results on Slack	Send AWS CodeBuild logs to CloudWatch Log group. Create Amazon EventBridge rule to detect the result of your build and target a Lambda function to send results to the Slack channel (or SNS notification)
Need to get a Slack notification for the status of the application deployments on AWS CodeDeploy	Create Amazon EventBridge rule to detect the result of CodeDeploy job and target a notification to Amazon SNS or a Lambda function to send results to Slack channel
Need to run an AWS CodePipeline every day for updating the development progress status	Create Amazon EventBridge rule to run on schedule every day and set a target to the AWS CodePipeline ARN
Automate deployment of a Lambda function and test for only 10% of traffic for 10 minutes before allowing 100% traffic flow.	Use CodeDeploy and select deployment configuration CodeDeployDefault.LambdaCanary10Percent10M inutes
Deployment of Elastic Beanstalk application with absolutely no downtime. The solution must maintain full compute capacity during deployment to avoid service degradation.	Choose the "Rolling with additional Batch" deployment policy in Elastic Beanstalk
Deployment of Elastic Beanstalk application where the new version must not be mixed with the current version.	Choose the "Immutable deployments" deployment policy in Elastic Beanstalk
Configuration Management and Infrastructure-as-Code (IaC)	
The resources on the parent CloudFormation stack needs to be referenced by other nested CloudFormation stacks	Use Export on the Output field of the main CloudFormation stack and use Fn::ImportValue function to import the value on the other stacks



On which part of the CloudFormation template should you define the artifact zip file on the S3 bucket?	The artifact file is defined on the AWS::Lambda::Function code resource block
Need to define the AWS Lambda function inline in the CloudFormation template	On the AWS::Lambda::Function code resource block, the inline function must be enclosed inside the ZipFile section.
Use CloudFormation to update Auto Scaling Group and only terminate the old instances when the newly launched instances become fully operational	Set AutoScalingReplacingUpdate : WillReplace property to TRUE to have CloudFormation retain the old ASG until the instances on the new ASG are healthy.
You can scale down the EC2 instances at night when there is low traffic using AWS Auto Scaling and Scheduled Scaling.	Create <i>Time-based</i> instances for automatic scaling of predictable workload.
Can't install an agent on on-premises servers but need to collect information for migration	Deploy the Agentless Discovery Connector VM on your on-premises data center to collect information.
Syntax for CloudFormation with an Amazon ECS cluster with ALB	Use the AWS::ECS::Service element for the ECS Cluster, AWS::ECS::TaskDefinition element for the ECS Task Definitions, and the AWS::ElasticLoadBalancingV2::LoadBalancer element for the ALB.
Monitoring and Logging	
Need to centralize audit and collect configuration settings on all regions of multiple accounts	Setup an Aggregator on AWS Config.
Consolidate CloudTrail log files from multiple AWS accounts	Create a central S3 bucket with bucket policy to grant cross-account permission. Set this as destination bucket on the CloudTrail of the other AWS accounts.
Ensure that CloudTrail logs on the S3 bucket are protected and cannot be tampered with.	Enable Log File Validation on CloudTrail settings
Need to collect/investigate application logs from EC2 or on-premises server	Install CloudWatch Logs Agent to send the logs to CloudWatch Logs for storage and viewing.



Need to review logs from running ECS Fargate tasks	Enable awslogs log driver on the Task Definition and add the required logConfiguration parameter.
Need to run real-time analysis for collected application logs	Send logs to CloudWatch Logs, create a Lambda subscription filter, OpenSearch subscription filter, or Kinesis stream filter.
Need to be automatically notified if you are reaching the limit of running EC2 instances or limit of Auto Scaling Groups	Track service limits with Trusted Advisor on CloudWatch Alarms using the ServiceLimitUsage metric.
Security and Compliance	
Need to secure the buildspec.yml file, which contains the AWS keys and database password stored in plaintext.	Store these values as encrypted parameter on SSM Parameter Store
You need to secure an S3 bucket by ensuring that only HTTPS requests are allowed for compliance purposes.	Create an S3 bucket policy that Deny if checks for condition aws:SecureTransport is false
Need to store a secret, database password, or variable, in the most cost-effective solution	Store the variable on SSM Parameter Store and enable encryption
Need to generate a secret password and have it rotated automatically at regular intervals	Store the secret on AWS Secrets Manager and enable key rotation.
Several team members with designated roles need to be granted permission to use AWS resources	Assign AWS-managed policies on the IAM accounts such as, ReadOnlyAccess, AdministratorAccess, PowerUserAccess
Apply latest patches on EC2 and automatically create an AMI	Use Systems Manager automation to execute an Automation Document that installs OS patches and creates a new AMI.
Need to have a secure SSH connection to EC2 instances and have a record of all commands executed during the session	Install SSM Agent on EC2 and use SSM Session Manager for the SSH access. Send the session logs to S3 bucket or CloudWatch Logs for auditing and review.
Ensure that the managed EC2 instances have the correct application version and patches installed.	Use SSM Inventory to have visibility of your managed instances and identify their current configurations.



Apply custom patch baseline from a custom repository, and schedule patches to managed instances	Use SSM Patch Manager to define a custom patch baseline and schedule the application patches using SSM Maintenance Windows
Incident and Event Response	
Need to get a notification if somebody deletes files in your S3 bucket	Setup Amazon S3 Event Notifications to get notifications based on specified S3 events on a particular bucket.
Need to be notified when an RDS Multi-AZ failover happens	Setup Amazon RDS Event Notifications to detect specific events on RDS.
Get a notification if somebody uploaded IAM access keys on any public GitHub repositories	Create an Amazon EventBridge rule for the AWS_RISK_CREDENTIALS_EXPOSED event from AWS Health Service. Use AWS Step Functions to automatically delete the IAM key.
Get notified on Slack when your EC2 instance is having an AWS-initiated maintenance event	Create an Amazon EventBridge rule for the AWS Health Service to detect EC2 Events. Target a Lambda function that will send a notification to the Slack channel
Get notified of any AWS maintenance or events that may impact your EC2 or RDS instances	Create an Amazon EventBridge rule for detecting any events on AWS Health Service and send a message to an SNS topic or invoke a Lambda function.
Monitor scaling events of your Amazon EC2 Auto Scaling Group, such as launching or terminating an EC2 instance.	Use Amazon EventBridge for monitoring the Auto Scaling Service and monitor the EC2 Instance-Launch Successful and EC2 Instance-Terminate Successful events.
View object-level actions of S3 buckets, such as upload or deletion of object in CloudTrail	Set up Data events on your CloudTrail trail to record object-level API activity on your S3 buckets.
Execute a custom action if a specific CodePipeline stage has a FAILED status	Create CloudWatch Event rule to detect failed state on the CodePipeline service, and set a target to SNS topic for notification or invoke a Lambda function to perform custom action.



Automatically rollback a deployment in AWS CodeDeploy when the number of healthy instances is lower than the minimum requirement.	On CodeDeploy, create a deployment alarm that is integrated with Amazon CloudWatch. Track the MinimumHealthyHosts metric for the threshold of EC2 instances and trigger the rollback if the alarm is breached.
Need to complete QA testing before deploying a new version to the production environment	Add a Manual approval step on AWS CodePipeline, and instruct the QA team to approve the step before the pipeline can resume the deployment.
Resilient Cloud Solutions	
Need to ensure that both the application and the database are running in the event that one Availability Zone becomes unavailable.	Deploy your application on multiple Availability Zones and set up your Amazon RDS database to use Multi-AZ Deployments.
In the event of an AWS Region outage, you have to make sure that both your application and database will still be running to avoid any service outages.	Create a copy of your deployment on the backup AWS region. Set up an RDS Read-Replica on the backup region.
Automatically switch traffic to the backup region when your primary AWS region fails	Set up Route 53 Failover routing policy with health check enabled on your primary region endpoint.
Need to ensure the availability of a legacy application running on a single EC2 instance	Set up an Auto Scaling Group with MinSize=1 and MaxSize=1 configuration to set a fixed count and ensure that it will be replaced when the instance becomes unhealthy
Ensure that every EC2 instance on an Auto Scaling group downloads the latest code first before being attached to a load balancer	Create an Auto Scaling Lifecycle hook and configure the Pending:Wait hook with the action to download all necessary packages.
Ensure that all EC2 instances on an Auto Scaling group upload all log files in the S3 bucket before being terminated.	Use the Auto Scaling Lifecycle and configure the Terminating:Wait hook with the action to upload all logs to the S3 bucket.

Validate Your Knowledge

After your review, you should take some practice tests to measure your preparedness for the real exam. AWS offers a sample practice test for free which you can find [here](#). You can also opt to buy the longer AWS sample



practice test at aws.training and use the discount coupon you received from any previously taken certification exams. Be aware though, that the sample practice tests do not mimic the difficulty of the real DevOps Pro exam.

Therefore, we highly encourage using other mock exams such as our very own [**AWS Certified DevOps Engineer Professional Practice Exam**](#) course, which contains high-quality questions with complete explanations on correct and incorrect answers, visual images and diagrams, YouTube videos as needed, and also contains reference links to official AWS documentation as well as our cheat sheets and study guides. You can also pair our practice exams with our [**AWS Certified DevOps Engineer Professional Exam Study Guide eBook**](#) to further help in your exam preparations.



Sample Question 1

An application is hosted in an Auto Scaling group of Amazon EC2 instances with public IP addresses in a public subnet. The instances are configured with a user data script that fetches and installs the required system dependencies of the application from the Internet upon launch. A change was recently introduced to prohibit any Internet access from these instances to improve the security, but after its implementation, the instances could not get the external dependencies anymore. Upon investigation, all instances are properly running, but the hosted application is not starting up completely due to the incomplete installation.

Which of the following is the MOST secure solution to solve this issue and also ensure that the instances do not have public Internet access?

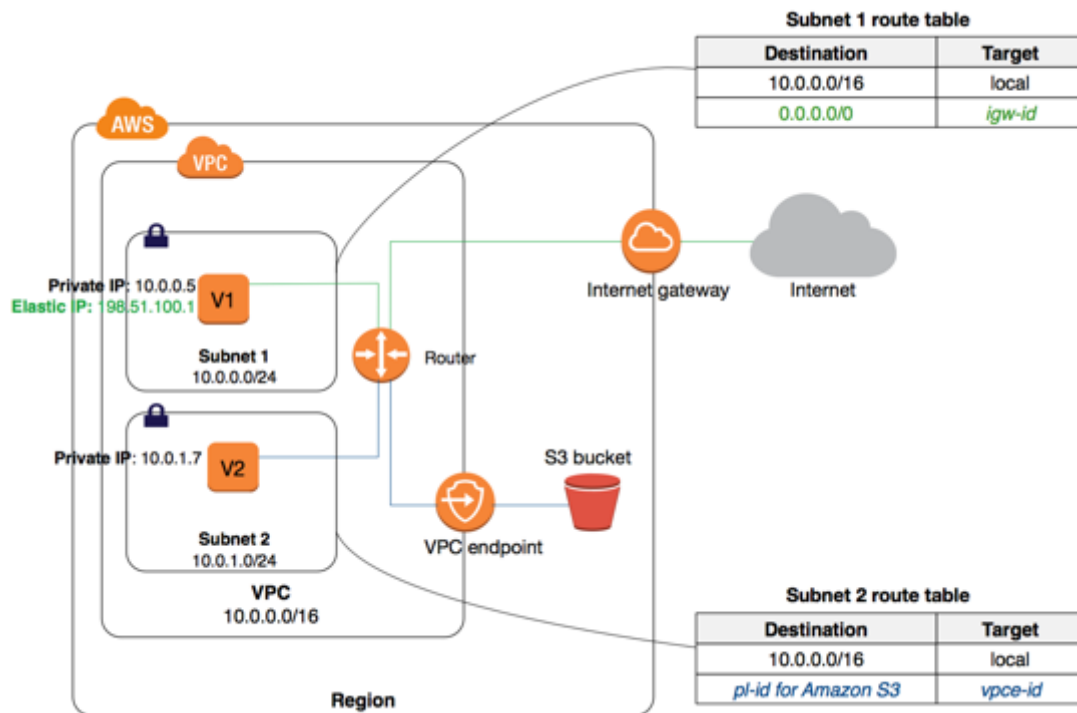
1. Download all of the external application dependencies from the public Internet and then store them in an S3 bucket. Set up a VPC endpoint for the S3 bucket and then assign an IAM instance profile to the instances in order to allow them to fetch the required dependencies from the bucket.
2. Deploy the Amazon EC2 instances in a private subnet and associate Elastic IP addresses on each of them. Run a custom shell script to disassociate the Elastic IP addresses after the application has been successfully installed and is running properly.
3. Use a NAT gateway to disallow any traffic to the VPC which originated from the public Internet. Deploy the Amazon EC2 instances to a private subnet then set the subnet's route table to use the NAT gateway as its default route.
4. Set up a brand new security group for the Amazon EC2 instances. Use a whitelist configuration to only allow outbound traffic to the site where all of the application dependencies are hosted. Delete the security group rule once the installation is complete. Use AWS Config to monitor the compliance.

Correct Answer: 1

A VPC endpoint enables you to privately connect your VPC to supported AWS services and VPC endpoint services powered by PrivateLink without requiring an Internet gateway, NAT device, VPN connection, or AWS Direct Connect connection. Instances in your VPC do not require public IP addresses to communicate with resources in the service. Traffic between your VPC and the other service does not leave the Amazon network.

Endpoints are virtual devices. They are horizontally scaled, redundant, and highly available VPC components that allow communication between instances in your VPC and services without imposing availability risks or bandwidth constraints on your network traffic.

There are two types of VPC endpoints: *interface endpoints* and *gateway endpoints*. You can create the type of VPC endpoint required by the supported service. S3 and DynamoDB are using Gateway endpoints, while most of the services are using Interface endpoints.



You can use an S3 bucket to store the required dependencies and then set up a VPC Endpoint to allow your EC2 instances to access the data without having to traverse the public Internet.

Hence, the correct answer is the option that says: **Download all of the external application dependencies from the public Internet and then store them to an S3 bucket. Set up a VPC endpoint for the S3 bucket and then assign an IAM instance profile to the instances in order to allow them to fetch the required dependencies from the bucket.**

The option that says: **Deploy the Amazon EC2 instances in a private subnet and associate Elastic IP addresses on each of them. Run a custom shell script to disassociate the Elastic IP addresses after the application has been successfully installed and is running properly** is incorrect because it is possible that the custom shell script may fail and the disassociation of the Elastic IP addresses might not be fully implemented, which will allow the EC2 instances to access the Internet.

The option that says: **Use a NAT gateway to disallow any traffic to the VPC which originated from the public Internet. Deploy the Amazon EC2 instances to a private subnet then set the subnet's route table to use the NAT gateway as its default route** is incorrect because although a NAT Gateway can safeguard the instances from any incoming traffic that were initiated from the Internet, it still permits them to send outgoing requests externally.

The option that says: **Set up a brand new security group for the Amazon EC2 instances. Use a whitelist configuration to only allow outbound traffic to the site where all of the application dependencies are hosted.**



Delete the security group rule once the installation is complete. Use AWS Config to monitor the compliance is incorrect because this solution has a high operational overhead since the actions are done manually. This is susceptible to human error, such as in the event that the DevOps team forgets to delete the security group. The use of AWS Config will just monitor and inform you about the security violation, but it won't do anything to remediate the issue.

References:

<https://docs.aws.amazon.com/vpc/latest/userguide/vpc-endpoints.html>

<https://docs.aws.amazon.com/vpc/latest/userguide/vpce-gateway.html>

Check out this Amazon VPC Cheat Sheet:

<https://tutorialsdojo.com/amazon-vpc/>

Sample Question 2

Due to the growth of its regional e-commerce website, the company has decided to expand its operations globally in the coming months ahead. The REST API web services of the app are currently running in an Auto Scaling group of EC2 instances across multiple Availability Zones behind an Application Load Balancer. For its database tier, the website is using a single Amazon Aurora MySQL database instance in the AWS Region where the company is based. The company wants to consolidate and store the data of its offerings into a single data source for its product catalog across all regions. For data privacy compliance, they need to ensure that the personal information of their users, as well as their purchases and financial data, are kept in their respective regions.

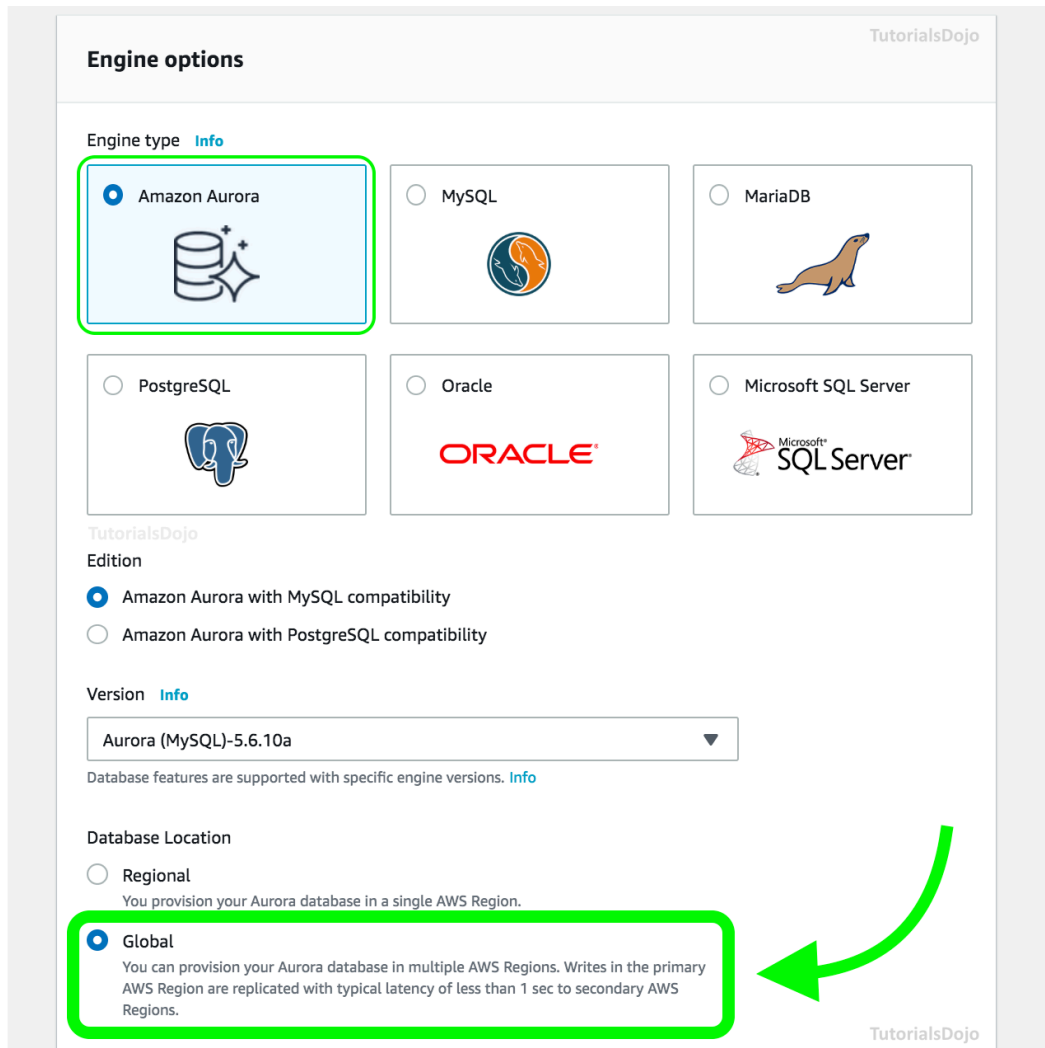
Which of the following options can meet the above requirements and entails the LEAST amount of change to the application?

1. Set up a new Amazon Redshift database to store the product catalog. Launch a new set of Amazon DynamoDB tables to store the personal information and financial data of their customers.
2. Set up a DynamoDB global table to store the product catalog data of the e-commerce website. Use regional DynamoDB tables for storing the personal information and financial data of their customers.
3. Set up multiple read replicas in your Amazon Aurora cluster to store the product catalog data. Launch an additional local Amazon Aurora instances in each AWS Region for storing the personal information and financial data of their customers.
4. Set up multiple read replicas in your Amazon Aurora cluster to store the product catalog data. Launch a new DynamoDB global table for storing the personal information and financial data of their customers.

Correct Answer: 3

An Aurora global database consists of one primary AWS Region where your data is mastered, and one read-only, secondary AWS Region. Aurora replicates data to the secondary AWS Region with typical latency of

under a second. You issue write operations directly to the primary DB instance in the primary AWS Region. An Aurora global database uses dedicated infrastructure to replicate your data, leaving database resources available entirely to serve application workloads. Applications with a worldwide footprint can use reader instances in the secondary AWS Region for low-latency reads. In the unlikely event, your database becomes degraded or isolated in an AWS region, you can promote the secondary AWS Region to take full read-write workloads in under a minute.



Engine options

Engine type [Info](#)

- ☒ Amazon Aurora
- ☐ MySQL
- ☐ MariaDB
- ☐ PostgreSQL
- ☐ Oracle
- ☐ Microsoft SQL Server

TutorialsDojo Edition

- ☒ Amazon Aurora with MySQL compatibility
- ☐ Amazon Aurora with PostgreSQL compatibility

Version [Info](#)

Aurora (MySQL)-5.6.10a

Database features are supported with specific engine versions. [Info](#)

Database Location

- ☐ Regional
You provision your Aurora database in a single AWS Region.
- ☒ Global
You can provision your Aurora database in multiple AWS Regions. Writes in the primary AWS Region are replicated with typical latency of less than 1 sec to secondary AWS Regions.

The Aurora cluster in the primary AWS Region where your data is mastered performs both read and write operations. The cluster in the secondary region enables low-latency reads. You can scale up the secondary cluster independently by adding one or more DB instances (Aurora Replicas) to serve read-only workloads. For disaster recovery, you can remove and promote the secondary cluster to allow full read and write operations.



Only the primary cluster performs write operations. Clients that perform write operations connect to the DB cluster endpoint of the primary cluster.

Hence, the correct answer is: **Set up multiple read replicas in your Amazon Aurora cluster to store the product catalog data. Launch an additional local Amazon Aurora instances in each AWS Region for storing the personal information and financial data of their customers.**

The option that says: **Set up a new Amazon Redshift database to store the product catalog. Launch a new set of Amazon DynamoDB tables to store the personal information and financial data of their customers** is incorrect because this solution entails a significant overhead of refactoring your application to use Redshift instead of Aurora. Moreover, Redshift is primarily used as a data warehouse solution and is not suitable for OLTP or e-commerce websites.

The option that says: **Set up a DynamoDB global table to store the product catalog data of the e-commerce website. Use regional DynamoDB tables for storing the personal information and financial data of their customers** is incorrect because although the use of Global and Regional DynamoDB is acceptable, this solution still entails a lot of changes to the application. There is no assurance that the application can work with a NoSQL database, and even so, you have to implement a series of code changes in order for this solution to work.

The option that says: **Set up multiple read replicas in your Amazon Aurora cluster to store the product catalog data. Launch a new DynamoDB global table for storing the personal information and financial data of their customers** is incorrect because although the use of Read Replicas is appropriate, this solution still requires you to do a lot of code changes since you will use a different database to store your regional data.

References:

<https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/aurora-global-database.html#aurora-global-database.advantages>

<https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/AuroraMySQL.Replication.CrossRegion.html>

Check out this Amazon Aurora Cheat Sheet:

<https://tutorialsdodo.com/amazon-aurora/>

Click [here](#) for more **AWS Certified DevOps Engineer Professional practice exam questions**.

More AWS reviewers can be found [here](#)



At this point, you should already be very knowledgeable on the following topics:

1. Continuous Integration/Continuous Delivery (CI/CD)
2. Application Development
3. Automation
4. Configuration Management and Infrastructure as Code
5. Monitoring and Logging
6. Incident Mitigation and Event Response
7. Implementing Resilient Cloud Solutions
8. Security and Compliance

As an AWS DevOps practitioner, you shoulder a lot of roles and responsibilities. Many professionals in the industry have attained proficiency through continuous practice and producing results of value. Therefore, you should properly review all the concepts and details that you need to learn so that you can also achieve what others have achieved.

The day before your exam, be sure to double-check the schedule, location, and items to bring for your exam. During the exam itself, you have 180 minutes to answer all questions and recheck your answers. Be sure to manage your time wisely. It will also be very beneficial for you to review your notes before you go in to refresh your memory. The AWS DevOps Pro certification is very tough to pass, and the choices for each question can be very misleading if you do not read them carefully. Be sure to understand what is being asked in the questions and what options are offered to you. With that, we wish you all the best in your exam!



Domain 1: Software Development Life Cycle (SDLC) Automation



Overview

The first domain of the AWS Certified DevOps Engineer Professional exam checks your preparedness on how well you understand the integration between the AWS services necessary for code development and deployment, such as AWS CodeBuild, AWS CodeDeploy, and AWS CodePipeline. You will also need a working knowledge of how they complement each other for software development as well as integrations with Amazon EventBridge, Amazon S3, and AWS Lambda. A big part of a normal workday for a DevOps Engineer deals with the software development life cycle.

Roughly 22% of questions in the actual DevOps exam revolve around these topics. The Software Development Life Cycle (SDLC) Automation domain is the biggest exam domain of the DOP-C02 exam, so ensure that you allocate ample time reviewing the topics under this section.

This domain will challenge your know-how in doing the following:

- Configuring code, image, and artifact repositories
- Using version control to integrate pipelines with application environments
- Setting up build processes via AWS CodeBuild and other related services
- Managing build and deployment secrets using AWS Secrets Manager and AWS Systems Manager Parameter Store as well as securing other sensitive credentials
- Determining appropriate deployment strategies in AWS CodeDeploy, AWS Elastic Beanstalk and other deployment services
- Running builds or tests when generating pull requests or code merges via AWS CodeBuild.
- Running load/stress tests, performance benchmarking, and application testing at scale
- Measuring application health based on application exit codes
- Automating unit tests and code coverage
- Invoking AWS services in a pipeline for testing
- Creating and configuring artifact repositories using AWS CodeArtifact, Amazon S3, Amazon Elastic Container Registry (Amazon ECR) and other related services.
- Configuring build tools for generating artifacts using AWS CodeBuild and AWS Lambda
- Automating Amazon EC2 instance and container image build processes via EC2 Image Builder
- Configuring security permissions to allow access to artifact repositories using the AWS Identity and Access Management, CodeArtifact, et cetera.
- Configuring deployment agents via CodeDeploy agent, SSM agent, and others.
- Troubleshooting deployment issues
- Using different deployment methods such as blue/green and canary deployments

In this chapter, we will cover all of the related topics for SDLC automation in AWS that will likely show up in your DevOps Professional exam.



What is DevOps?

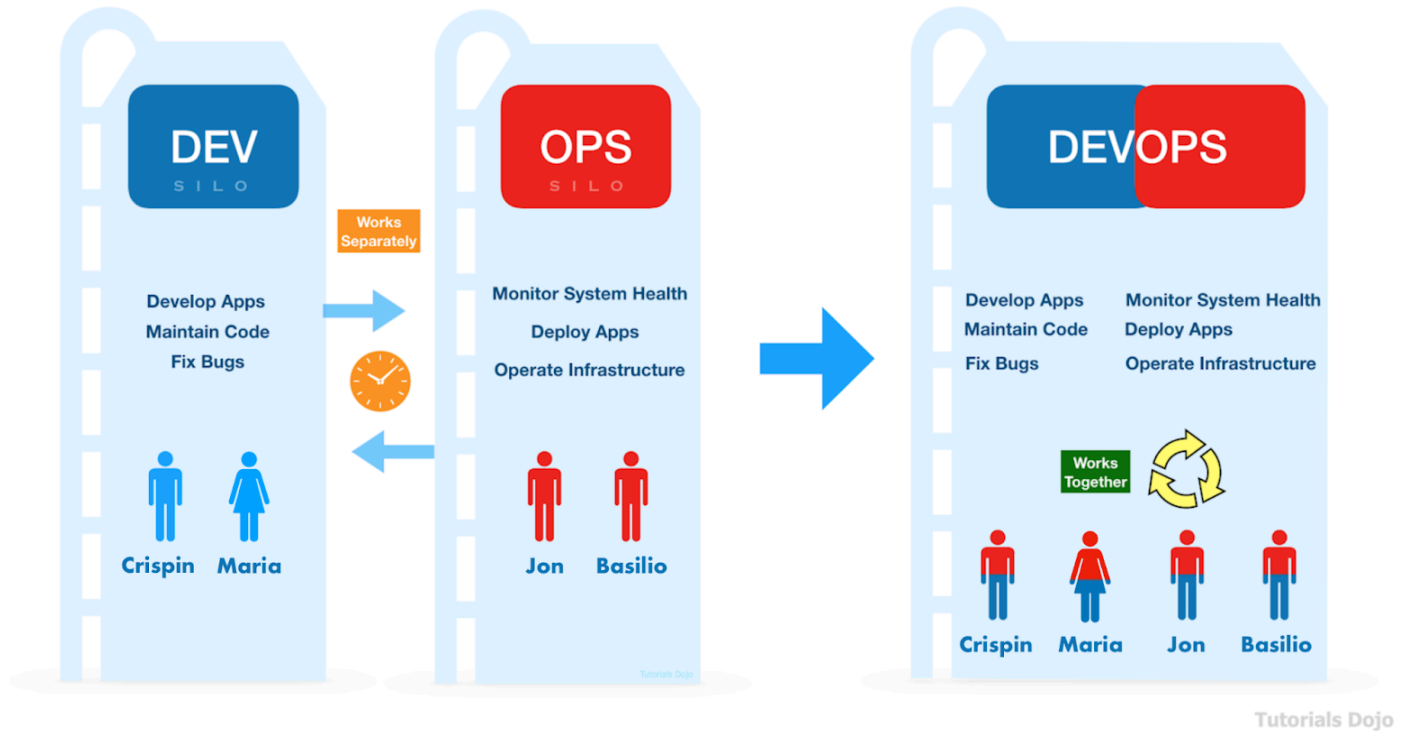
Do you ever wonder what DevOps is and why it is so popular in the IT Industry today? Is it a tool, a process, a corporate culture, or a combination of all of these? Why are companies giving competitive salaries for this kind of role?

If you typed the word “DevOps” on any job market website today, you would see many available positions that require knowledge of both programming and infrastructure management. You will usually see an advertisement looking for a candidate who knows how to program in Python or any other language. The requirements include being capable of managing servers with configuration management tools such as Ansible, Chef, or Puppet, as well as the provisioning of entire cloud environments using Infrastructure-as-code tools like Terraform or CloudFormation. The salary range offered for these positions is remarkably high too!

Traditional IT companies have a dedicated Development (Dev) team that builds enterprise applications and an Operations (Ops) team that handles the servers and network infrastructure. These two teams are often siloed or isolated from each other. While the Dev team writes the software code, the Ops team prepares the server, database, and other infrastructure needed to run the soon-to-be-released application. In this setup, the developers are entirely oblivious to what the system operators are doing and vice versa. A lot of time is wasted waiting for the Dev Team to fix minor bugs while developing new features and for the Ops team to provision, deploy and scale the needed server resources. When bugs and incompatibility issues are detected in the development cycle, the Ops team waits for the Dev team to address the issue since it is strictly the job of the Developers to fix it. The same is true when there are issues during deployments when the Ops are not familiar with the application and make wrong assumptions which can cause further delays in the deployment targets. Due to this lack of coordination, both the business and its customers are impacted. This is where DevOps comes in!

DevOps is not just the combination of Development (Dev) and Operations (Ops). DevOps is the fusion of practices, processes, tools, and corporate culture that expedite the organization’s ability to deliver applications and services at a higher velocity, faster than traditional software development processes. It’s not merely a tool or a process that your team adopts, but a synergy of values, corporate structure, and internal processes to attain the digital transformation of the business enterprise. It tears down the traditional and isolated silos of the Development, Operations, IT Security, and other teams, enabling collaboration and improving overall business performance. With DevOps, Developers are empowered to directly influence the deployment life cycle, and the IT Operations folks have the ability to report and fix possible bugs or incompatibilities in the application.

DevOps is not just a framework, rather, it’s a cultural approach and a mindset combining operations and development skills and delivering a product (or service) from inception to retirement. Company executives also play a crucial role in allocating budgets and adopting this new status quo within their respective organizations.



With the advent of Cloud Computing, companies can easily unify their software development and system operation processes. AWS enables organizations to rapidly build, deliver, and manage their products, following DevOps practices with just a click of a button. The efficiency of provisioning new resources, managing infrastructure, deploying application code, automating software release processes, and many other tasks in AWS contributes to overall productivity and business profitability. Because of this massive benefit, companies are willing to pay competitive remuneration for their DevOps Engineers, especially those who are AWS Certified.



A Brief History of the DevOps Exam in AWS

In 2013, Amazon Web Services (AWS) began the Global Certification Program to validate the technical skills and knowledge for building secure and reliable cloud-based applications using the AWS platform. The first-ever certification launched by Amazon is the AWS Certified Solutions Architect – Associate, followed by SysOps Administrator and Developer Associate. A year later, AWS released the first Professional-level certification: AWS Certified Solutions Architect - Professional, and in February 2015, they released the AWS Certified DevOps Engineer Professional.

The AWS Certified DevOps Engineer Professional certification enables technology professionals to showcase their DevOps skills, and it allows companies to identify top candidates to lead their internal DevOps initiatives. It validates your technical expertise in provisioning, managing, and operating distributed application systems on the AWS Cloud platform. It tests your ability to implement and manage Continuous Integration/Continuous Delivery (CI/CD) systems and methodologies on AWS following the industry's best practices, as well as to automate security controls and handle governance processes and meet compliance. The exam also covers core topics such as Software Development Lifecycle (SDLC) automation, security, compliance, monitoring, logging, configuration management, and incident/event response.

As Amazon Web Services continue to evolve, new and updated versions of the AWS certification exams are released regularly to reflect the service changes and to include new knowledge areas. Four years after its initial release, an updated version of the AWS Certified DevOps Engineer - Professional certification was launched in February 2019 with an exam code of DOP-C01.

The latest version of the AWS Certified DevOps Engineer - Professional certification exam was unveiled on March 2023 with an exam code of DOP-C02. AWS is continuously adding more services and features to help organizations and companies to improve their DevOps processes.



Why Automate?

Automation is at the heart of every DevOps engineer. Automation is a key highlight of DevOps practice. There is a saying in the DevOps community to “Automate Everything” and it starts from code inception, releasing to production, application retirement, and everything in between. Eliminating repetitive tasks, reducing toil, and minimizing manual work are the key aspects that you want to solve through automation. Automation in DevOps fosters speed, greater accuracy, consistency, reliability, and rapid delivery.

Here are some major benefits of automation:

- **Speed** – to innovate your product faster and adapt to changing market trends. Team members are empowered to make changes quickly as needed, either on the development side or the operational side.
- **Rapid delivery** – increase the pace of your releases by automating your entire deployment pipeline. This is the concept of “fail-fast, iterate faster” in which your companies are incentivized to release minor changes as often as possible, which keeps them on top of competitors.
- **Reliability** – continuous integration and continuous delivery processes allow you to reliably and consistently deliver your product to end-users. This also reduces human error as automation rarely makes mistakes as humans do.
- **Scale** - infrastructure as code helps you manage your environments in a repeatable and more efficient manner and scale easily as needed. It gives you a robust system to manage your infrastructure no matter how big or small it is.
- **Improved collaboration** – reduce inefficiencies when collaborating with teams. Automation allows the easier integration of development, testing, and deployment processes. It facilitates faster collaboration between Dev and Ops, which results in an improved turnaround time for bug fixing, deployment, etc.
- **Security** - reduces risk through integrated security testing tools, automating adoption of compliance requirements. It allows you to declare and script your security compliance requirement and make sure they are applied to needed resources in your environments.



Types of Blue Green Deployment via ELB, Route 53, Elastic Beanstalk

AWS Elastic Beanstalk Blue/Green

Elastic Beanstalk, by default, performs an in-place update when you deploy a newer version of your application. This can cause a short downtime since your application will be stopped while Elastic Beanstalk performs the application update.

Blue/Green deployments allow you to deploy without application downtime.

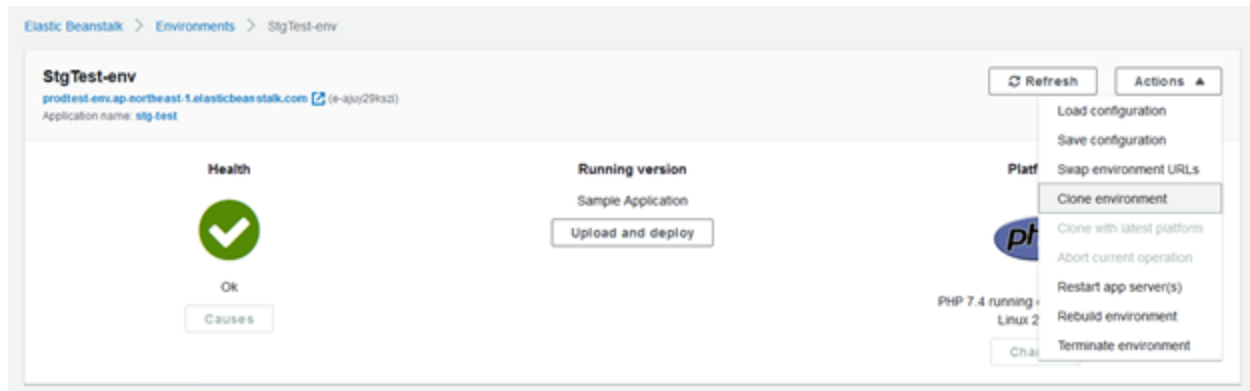
DevOps Exam Notes:

Remember these key points on when to use blue/green deployments:

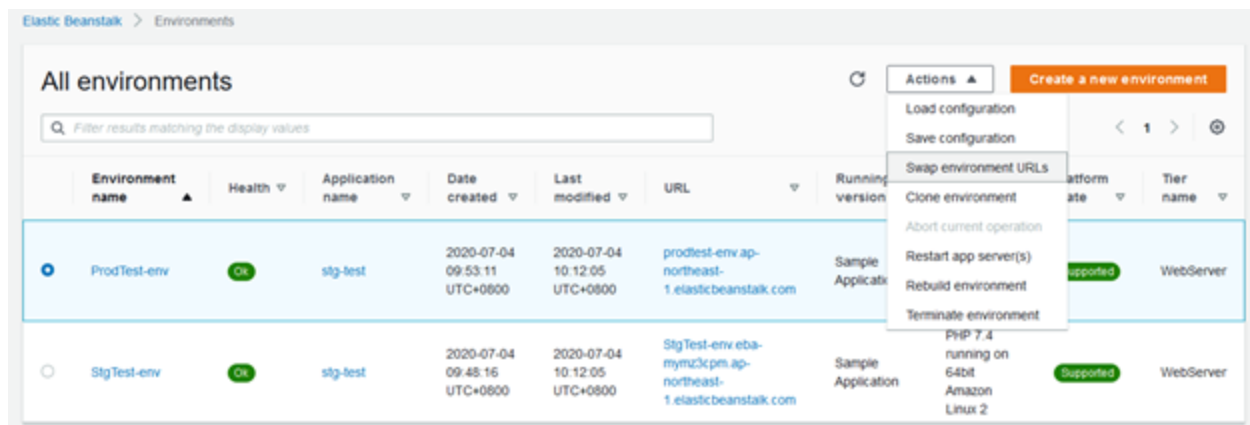
- No downtime during deployment because you are deploying the newer version on a separate environment
- CNAMEs of the environment URLs are swapped to redirect traffic to the newer version.
- Route 53 will swap the CNAMEs of the application endpoints.
- Fast deployment time and quick rollback since both old and new versions are running at the same time, you just have to swap back the URLs if you need to rollback.
- Useful if your newer version is incompatible with the current platform version of your application. (ex. Jumping from major versions of NodeJS, Python, Ruby, PHP, etc.)
- Your RDS Database instance should be on a separate stack because the data will not transfer to your second environment. You should decouple your database from the web server stack.

To implement a Blue/Green deployment for your Elastic Beanstalk application, you can perform the following steps:

1. Create another environment on which you will deploy the newer version of your application. You can clone your current environment for easier creation.



2. Once the new environment is ready, deploy a new version of your application. Perform your tests on the URL endpoint of your new environment.
3. After testing, select your Production environment, click Actions > Swap environment URLs.



4. On the Swap Environment URLs page, select the newer environment and click Swap to apply the changes.



Elastic Beanstalk > Environments > ProdTest-env

Swap environment URLs

When you swap an environment's URL with another environment's URL, you can deploy versions with no downtime. [Learn more](#)

 Swapping the environment URL will modify the Route 53 DNS configuration, which may take a few minutes. Your application will continue to run while the changes are propagated.

Environment details

Environment name:
ProdTest-env (e-fim82brguw)

Environment URL:
prodtest-env.ap-northeast-1.elasticbeanstalk.com

Select an environment to swap

Environment name:

Environment URL:
StgTest-env.eba-mymz3cpm.ap-northeast-1.elasticbeanstalk.com



AWS Lambda Blue/Green

You can also implement Blue/Green deployments on your Lambda functions. The concept is the same as in Elastic beanstalk blue/green deployment i.e. you will need to create two versions of your Lambda function and use function Aliases to swap the traffic flow.

Lambda versions – lets you publish a new version of a function that you can test without affecting the current application accessed by users. You can create multiple versions as needed for your testing environments. The ARN of Lambda version is the same as the ARN of the Lambda function with added version suffix.

```
arn:aws:lambda:aws-region:acct-id:function:helloworld:$LATEST
```

Lambda aliases – Aliases are merely pointers to specific Lambda versions. You can't select a Lambda alias and edit the function. You need to select the LATEST version if you want to edit the function. Aliases are helpful for blue/green deployments because it allows you to use a fixed ARN and point it to a particular Lambda version that you want to deploy.

DevOps Exam Notes:

Remember the difference between Lambda \$LATEST, Lambda Versions, and Lambda Aliases:

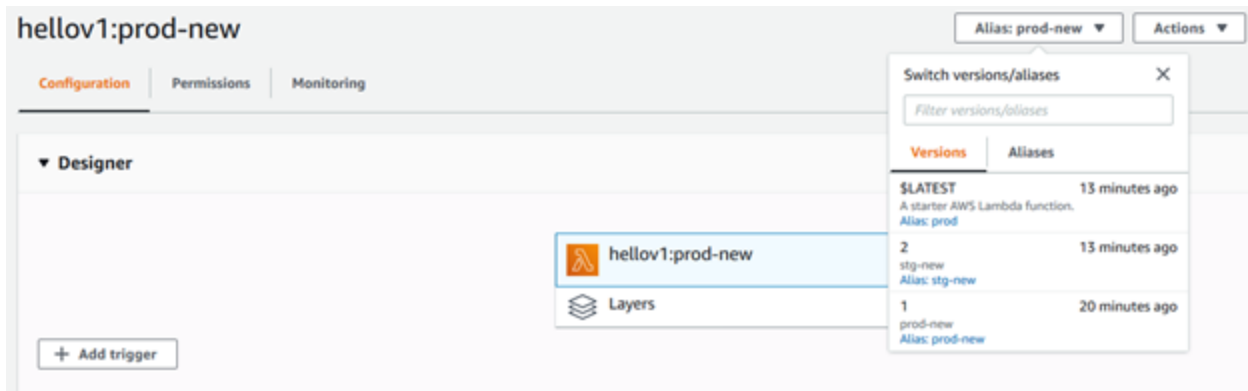
\$LATEST - this is the latest version of your Lambda function. You can freely edit this version.

Lambda Version - fixed version of your function. You can't edit this directly.

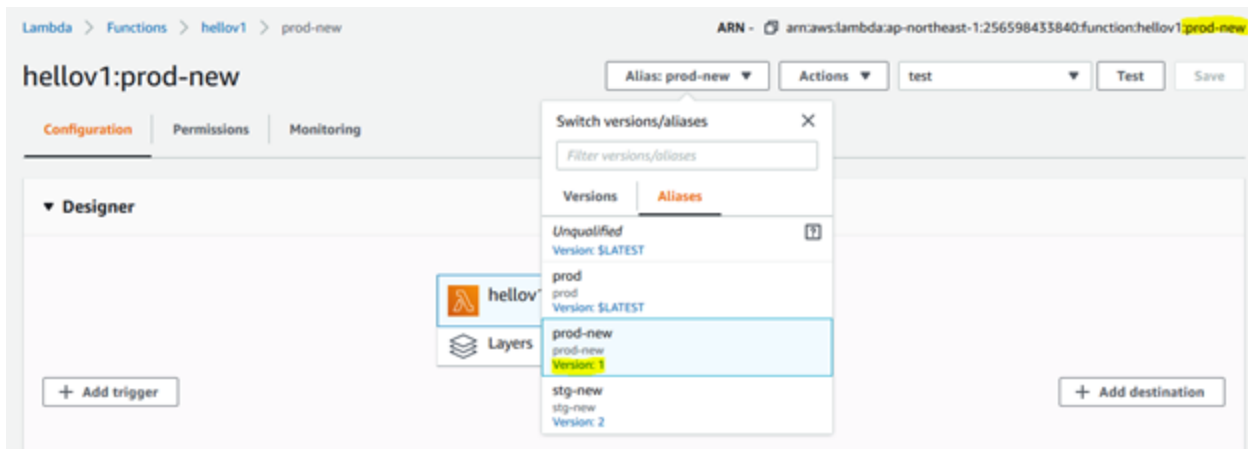
Lambda Alias - a pointer to a specific Lambda version. You can perform blue/green deployment with Aliases by pointing to a newer version.

The following steps will show how blue/green deployment can be done on Lambda functions.

1. The current version of your Lambda function is deployed on Version 1. Create another version and make your changes, this will be Version 2.



2. Create an Alias that will point to the current production version. Use this alias as your fixed production ARN.



3. Create another Alias that you will use for your newer version. Perform your testing and validation on this newer version. Once testing is complete, edit the production alias to point to the newer version. Traffic will now instantly be shifted from the previous version to the newer version.

Sources:

<https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/using-features.CNAMEswap.html>

<https://docs.aws.amazon.com/lambda/latest/dg/configuration-aliases.html>

<https://docs.aws.amazon.com/lambda/latest/dg/configuration-versions.html>

AWS Lambda Function Alias Traffic Shifting

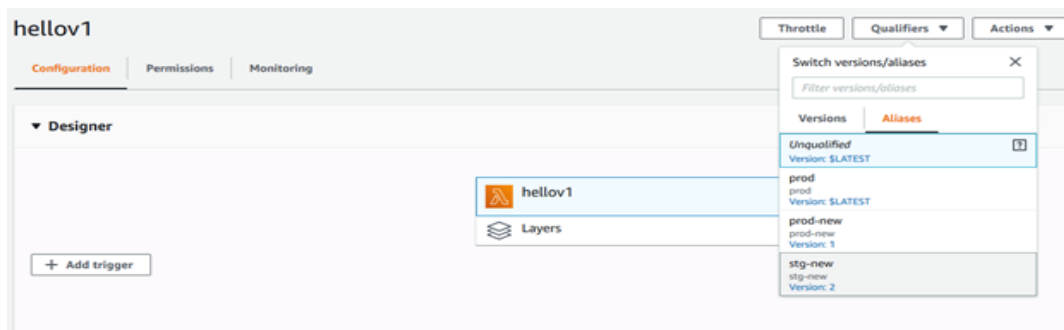
On EC2 instances, you can perform a canary deployment by deploying a newer application version on a single EC2 instance and analyzing the production traffic flowing through it. If you are satisfied with it, you'll proceed to deploy the newer version on all EC2 instances. However, for deployments on your Lambda functions, you can't use a canary deployment since you don't deploy your application directly on EC2 instances.

DevOps Exam Notes:

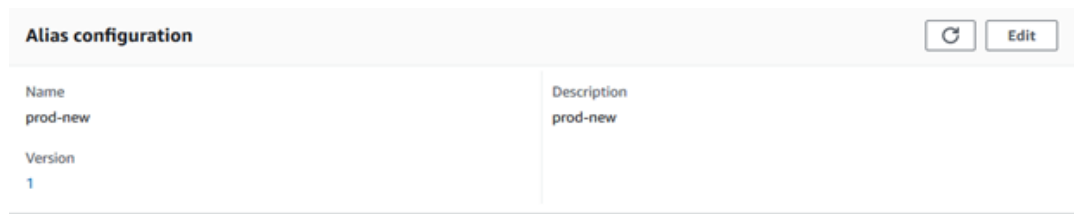
To provide similar functionality as a canary deployment, AWS Lambda gives you the ability to use Function Aliases to shift the percentage of traffic from one version to another. Essentially, you will create an Alias that points to the current version of the Lambda function, then use a weighted alias to define a newer version of the Lambda function. You can then define the weight (percent of traffic) that you want to forward to this version. After validation, you can completely shift traffic to the newer version.

You can consult the previous section (Types of Blue/Green deployment - AWS Lambda) on how to create AWS Lambda Versions and Aliases. Here's an example of how to control that percentage of traffic flowing to different Lambda functions using function alias. This is similar to the way a canary deployment works.

1. Select the function alias pointing to the current production version.



2. Edit the alias configuration.



- On the weighted alias section, select the newer version of your Lambda function and assign the percentage of traffic to shift to the newer version. You can repeat this step multiple times if you want to slowly shift traffic from the older version to the newer version.

Edit alias

Alias configuration

Description - *optional*

Version

Weight (%)

90

▼ Weighted alias

You can shift traffic between two versions, based on weights (%) that you assign. Click [here](#) to learn more.

Additional version - *optional*

Weight (%)

Cancel

Save

If you are using AWS CodeDeploy to deploy your Lambda functions, CodeDeploy uses Aliases to shift traffic to the newer version. As you can see on the options on the deployment configurations, CodeDeploy can automate this gradual traffic shifting for your Lambda Functions.



The screenshot displays the AWS CodeDeploy console interface. On the left, a sidebar menu lists navigation options: Source (CodeCommit), Artifacts (CodeArtifact), Build (CodeBuild), Deploy (CodeDeploy), Pipeline (CodePipeline), and Settings. The 'Deploy' section is expanded, showing 'Getting started', 'Deployments', 'Applications', 'Deployment configurations' (highlighted in orange), and 'On-premises instances'. The main content area shows a grid of six deployment configurations for AWS Lambda functions. Each configuration card includes the name, compute platform (AWS Lambda), configuration type, step percentage (10%), and interval.

Configuration Name	Compute platform	AWS Lambda	Configuration type	Step	Interval
CodeDeployDefault.LambdaAllAtOnce	Compute platform	AWS Lambda	All at once	10%	1 minutes
CodeDeployDefault.LambdaLinear10PercentEvery1Minute	Compute platform	AWS Lambda	Linear	10%	1 minutes
CodeDeployDefault.LambdaLinear10PercentEvery2Minutes	Compute platform	AWS Lambda	Linear	10%	2 minutes
CodeDeployDefault.LambdaLinear10PercentEvery3Minutes	Compute platform	AWS Lambda	Linear	10%	3 minutes
CodeDeployDefault.LambdaLinear10PercentEvery10Minutes	Compute platform	AWS Lambda	Linear	10%	10 minutes
CodeDeployDefault.LambdaCanary10Percent5Minutes	Compute platform	AWS Lambda	Canary	10%	5 minutes

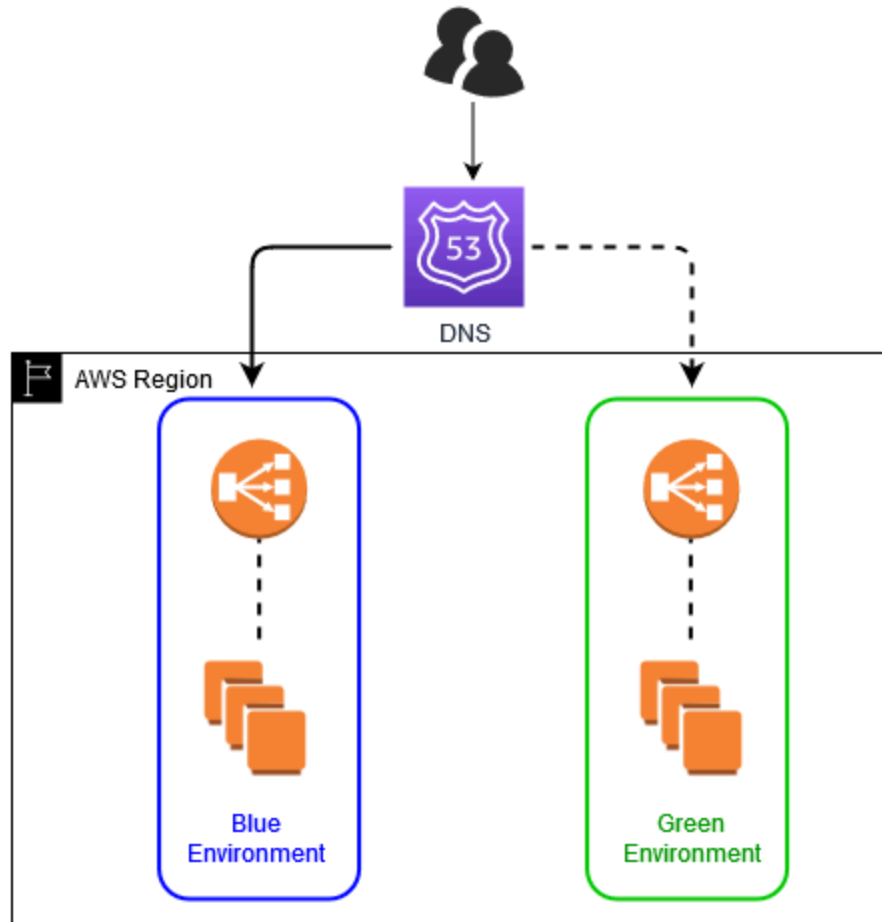
See the [CodeDeploy - Linear, Canary and All-at-Once \(Pre-defined Deployments\)](#) topic for more discussion on these deployment strategies.

Source:

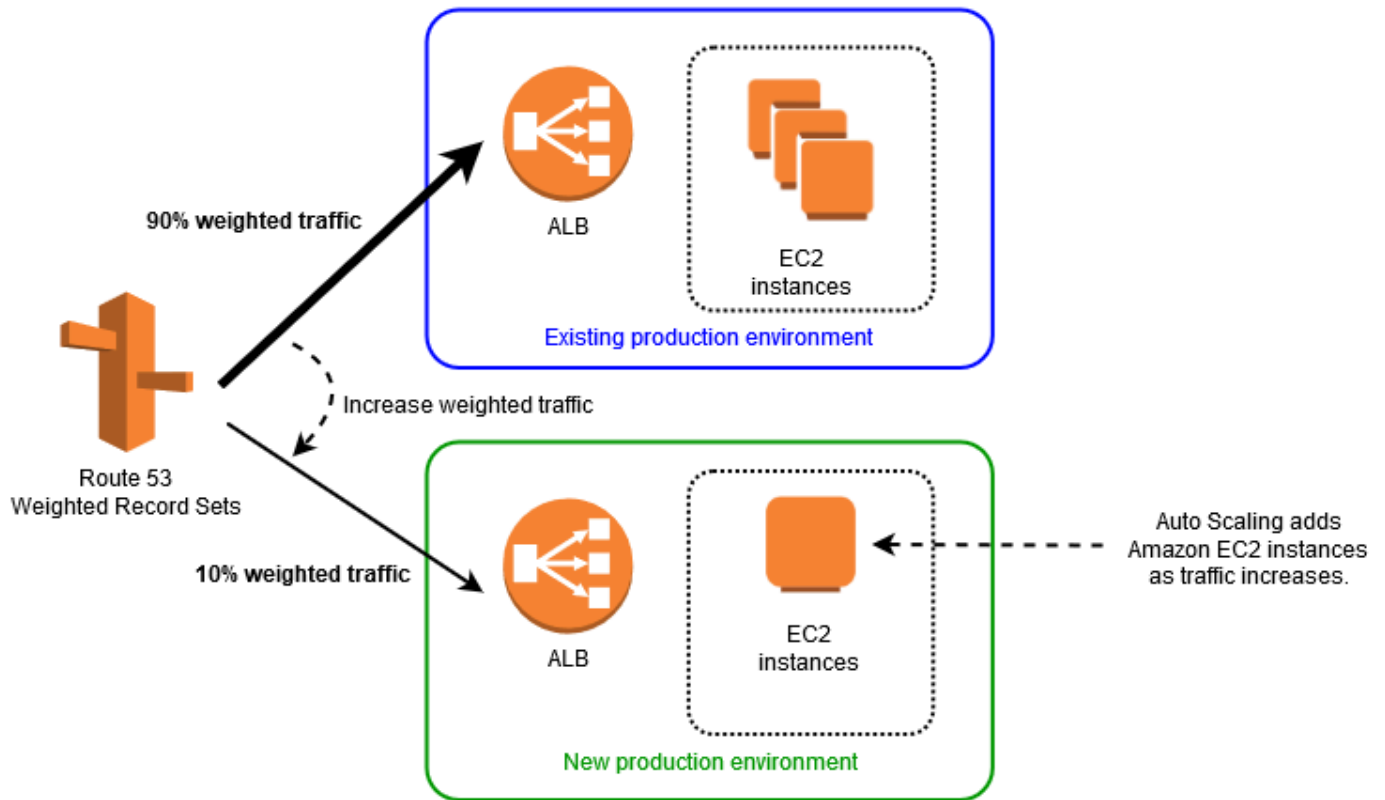
<https://aws.amazon.com/blogs/compute/implementing-canary-deployments-of-aws-lambda-functions-with-alias-traffic-shifting/>

Basic Blue/Green Deployment using Route 53

Blue/green deployment on the AWS platform provides a safer way to upgrade production software. This deployment usually involves two environments, the production environment (blue) and the new updated environment (green).



Once the new version is deployed on the green environment, you can validate the new software before going live. Then, you start shifting traffic away from the blue environment and sending it to the green one. Normally, you'd use Route 53 weighted routing policy because it gives you an easy way to push incremental traffic to the green environment or revert traffic back to the blue environment in case of issues. If you want to, you can switch the traffic immediately by updating the production Route 53 record to point to the green endpoint. Users will not see that you changed the endpoint since from their perspective, the production URL is the same.



You can also shift a small portion (like 10%) of traffic on the Green environment by using a weighted routing policy on Route 53. This way, you can test live traffic on the new environment, analyze the new logs, and then you can easily revert to the original environment if you find any problems. This process is also called a canary deployment.

Source:

<https://aws.amazon.com/blogs/startups/upgrades-without-tears-part-2-bluegreen-deployment-step-by-step-on-aws/>



Lifecycle Event Hook Availability (CodeDeploy Concept)

AWS CodeDeploy gives you several lifecycle event hooks to perform actions at different stages of the deployment. You can run scripts on your desired stage to run specific actions needed for that stage.

For example, you can use the **BeforeInstall** lifecycle event hook to create a backup of your current version before CodeDeploy installs the new version on your instances. You can use the **BeforeAllowTraffic** lifecycle event hook to perform some tasks or run scripts on the instances before they are registered on the load balancer.

Another example is when using blue/green deployment, and you want to run validation scripts after the new instances have been registered on the load balancer. You want to validate the new version before you remove the old version instances. For this scenario, you will use the **AfterAllowTraffic** lifecycle event hook.

The stages are available depending on which deployment method you have chosen, such as in-place deployment or blue-green deployment.

The following table lists the lifecycle event hooks available for each deployment and rollback scenario.

Lifecycle event name	In-place deployment ¹	Blue/green deployment: Original instances	Blue/green deployment: Replacement instances	Blue/green deployment rollback: Original instances	Blue/green deployment rollback: Replacement instances
ApplicationStop	✓		✓		
DownloadBundle ²	✓		✓		
BeforeInstall	✓		✓		
Install ²	✓		✓		
AfterInstall	✓		✓		
ApplicationStart	✓		✓		
ValidateService	✓		✓		
BeforeBlockTraffic	✓	✓			✓
BlockTraffic ²	✓	✓			✓
AfterBlockTraffic	✓	✓			✓



BeforeAllowTraffic	✓		✓	✓	
AllowTraffic ²	✓		✓	✓	
AfterAllowTraffic	✓		✓	✓	

¹Also applies to the rollback of an in-place deployment.

² Reserved for CodeDeploy operations. Cannot be used to run scripts.

Here's a summary of the Lifecycle event hooks and what actions are performed on that stage.

ApplicationStop – This deployment lifecycle event occurs even before the application revision is downloaded.

DownloadBundle – During this deployment lifecycle event, the CodeDeploy agent copies the application revision files to a temporary location.

BeforeInstall – You can use this deployment lifecycle event for preinstall tasks, such as decrypting files and creating a backup of the current version.

Install – During this deployment lifecycle event, the CodeDeploy agent copies the revision files from the temporary location to the final destination folder.

AfterInstall – You can use this deployment lifecycle event for tasks such as configuring your application or changing file permissions.

ApplicationStart – You typically use this deployment lifecycle event to restart services that were stopped during ApplicationStop.

ValidateService – This is the last deployment lifecycle event. It is used to verify that the deployment was completed successfully.

BeforeBlockTraffic – You can use this deployment lifecycle event to run tasks on instances before they are deregistered from a load balancer.

BlockTraffic – During this deployment lifecycle event, internet traffic is blocked from accessing instances that are currently serving traffic.

AfterBlockTraffic – You can use this deployment lifecycle event to run tasks on instances after they are deregistered from a load balancer.

BeforeAllowTraffic – You can use this deployment lifecycle event to run tasks on instances before they are registered with a load balancer.



AllowTraffic – During this deployment lifecycle event, internet traffic is allowed to access instances after a deployment.

AfterAllowTraffic – You can use this deployment lifecycle event to run tasks on instances after they are registered with a load balancer.

Going to the exam, you don't have to remember all stages, but you need to know the important lifecycle hooks such as **BeforeInstall**, **BeforeAllowTraffic**, and **AfterAllowTraffic**.

Here's an example structure of a "hooks" section on your YAML file:

```
hooks:
  deployment-lifecycle-event-name:
    - location: script-location
      timeout: timeout-in-seconds
      runas: user-name
```

Source:

<https://docs.aws.amazon.com/codedeploy/latest/userguide/reference-appspec-file-structure-hooks.html#reference-appspec-file-structure-hooks-availability>



Automatically Run CodeBuild Tests After a Developer Creates a Pull Request

AWS now provides alternative solutions for managing source code repositories. Developers can utilize **Amazon CodeCatalyst**, **GitHub**, or **Bitbucket** for repository hosting and pull request management. Changes from feature branches can be merged into the main branch, typically used for production releases. To integrate updates into the main branch, developers create pull requests. However, these changes must be validated to ensure they integrate properly with the existing codebase.

Automating Pull Request Validation with AWS CodeBuild

To validate changes before merging, you can configure an AWS CodeBuild project to automatically build and test pull requests. The results of the build can then be used to determine whether to approve the merge or reject the pull request.

DevOps Exam Notes:

You can automate pull request validation using **AWS CodeBuild**, **AWS Lambda**, and **Amazon EventBridge**. The process works as follows:

1. **Amazon EventBridge detects the pull request** in your repository and triggers an AWS Lambda function.
2. **The Lambda function posts an automated comment** on the pull request indicating that the build process is about to start.
3. **Amazon EventBridge triggers an AWS CodeBuild project**, which builds and validates the changes.
4. **A second EventBridge rule detects the build results** and triggers another Lambda function.
5. **The Lambda function posts a comment** on the pull request with the build results and a link to the build logs.

Automated Pull Request Validation Workflow

The following workflow illustrates how a source code repository, AWS CodeBuild, Amazon EventBridge, and AWS Lambda work together to automate pull request validation:

1. The repository contains two branches:
 - **Main branch** that holds the approved code.
 - **Development branch** where changes are implemented.
2. A developer pushes new code to the **development branch**.
3. The developer creates a **pull request** to merge changes into the main branch.
4. An **Amazon EventBridge rule** detects the pull request and triggers an AWS Lambda function to post an automated comment, notifying that the validation build is about to start.
5. The same **Amazon EventBridge rule** triggers an **AWS CodeBuild project** to build and validate the changes.



6. A **second Amazon EventBridge rule** detects the build results and triggers another AWS Lambda function.
7. The Lambda function posts an automated comment on the pull request, displaying the **build results and a link to the build logs**.
8. Based on the test results, the developer can address any failures and update the pull request.
9. The validation workflow runs again when updates are pushed.
10. Once the pull request successfully passes validation, it can be **merged into the main branch**.

This automated validation workflow ensures that only tested and validated code is merged, reducing the risk of integration issues and improving the overall development process.

Sources:

<https://docs.aws.amazon.com/codebuild/latest/userguide/how-to-create-pipeline.html>



Managing Artifacts in AWS CodeBuild and CodePipeline

In AWS CodeBuild, you can run build projects that will build and test your code. After the build process, you have the option to store the artifact on an Amazon S3 bucket which can then be used by AWS CodeDeploy to deploy on your instances. You can create an AWS Pipeline for the whole process to be automated – from building, testing, and validation, up to the deployment of your artifact.

Here's the reference snippet of code on your CodeBuild buildspec.yml file showing how to declare the output artifact file.

```
phases:
  install:
    runtime-versions:
      java: corretto11
  pre_build:
    commands:
      - echo Nothing to do in the pre_build phase...
  build:
    commands:
      - echo Build started on `date`
      - mvn install
  post_build:
    commands:
      - echo Build completed on `date`
artifacts:
  files:
    - target/CodeDeploySample.zip
```

The **buildspec.yml** file should be in the same folder as your source code. The source code will be built based on the contents of the **buildspec.yml** file, and the output will be sent to the S3 bucket that you have specified on the build project.

This is an example of a JSON file that you can use when creating a CodeBuild project. Notice that the input and output buckets are specified.

```
{
  "name": "sample-codedeploy-project",
  "source": {
    "type": "S3",
    "location": "my-codepipeline-website-bucket/CodeDeploySample.zip"
  },
}
```



```
"artifacts": {  
  "type": "S3",  
  "location": "my-codepipeline-website-bucket",  
  "packaging": "ZIP",  
  "name": "CodeDeployOutputArtifact.zip"  
},  
"environment": {  
  "type": "LINUX_CONTAINER",  
  "image": "aws/codebuild/standard:4.0",  
  "computeType": "BUILD_GENERAL1_SMALL"  
},  
"serviceRole": "arn:aws:iam::account-ID:role/role-name",  
"encryptionKey": "arn:aws:kms:region-ID:account-ID:key/key-ID"  
}
```

If you are using the AWS CodeBuild, this is where you specify the S3 bucket for the artifact.

ArtifactsAdd artifact

Artifact 1 - Primary

Type

Amazon S3

You might choose no artifacts if you are running tests or pushing a Docker image to Amazon ECR.

Bucket name

Q mytest-bucket-cicd X

Name

The name of the folder or compressed file in the bucket that will contain your output artifacts. Use Artifacts packaging under Additional configuration to choose whether to use a folder or compressed file. If the name is not provided, defaults to project name.

TutorialsDojo-build

☐ **Enable semantic versioning**
Use the artifact name specified in the buildspec file

Path - optional

The path to the build output ZIP file or folder.

Example: MyPath/MyArtifact.zip.

Package type - optional



DevOps Exam Notes:

After the build runs, you can use the output artifact to be deployed by AWS CodeDeploy manually. Or using CodePipeline, you can create another stage on which CodeDeploy will automatically pick up this artifact and run the deployment on your desired instances.

Here's how to specify the artifact filename on CodeDeploy.

Create deployment

Deployment settings

Application
TutorialsDojoApp

Deployment group
TutorialsDojoDev

Compute platform
EC2/On-premises

Deployment type
In-place

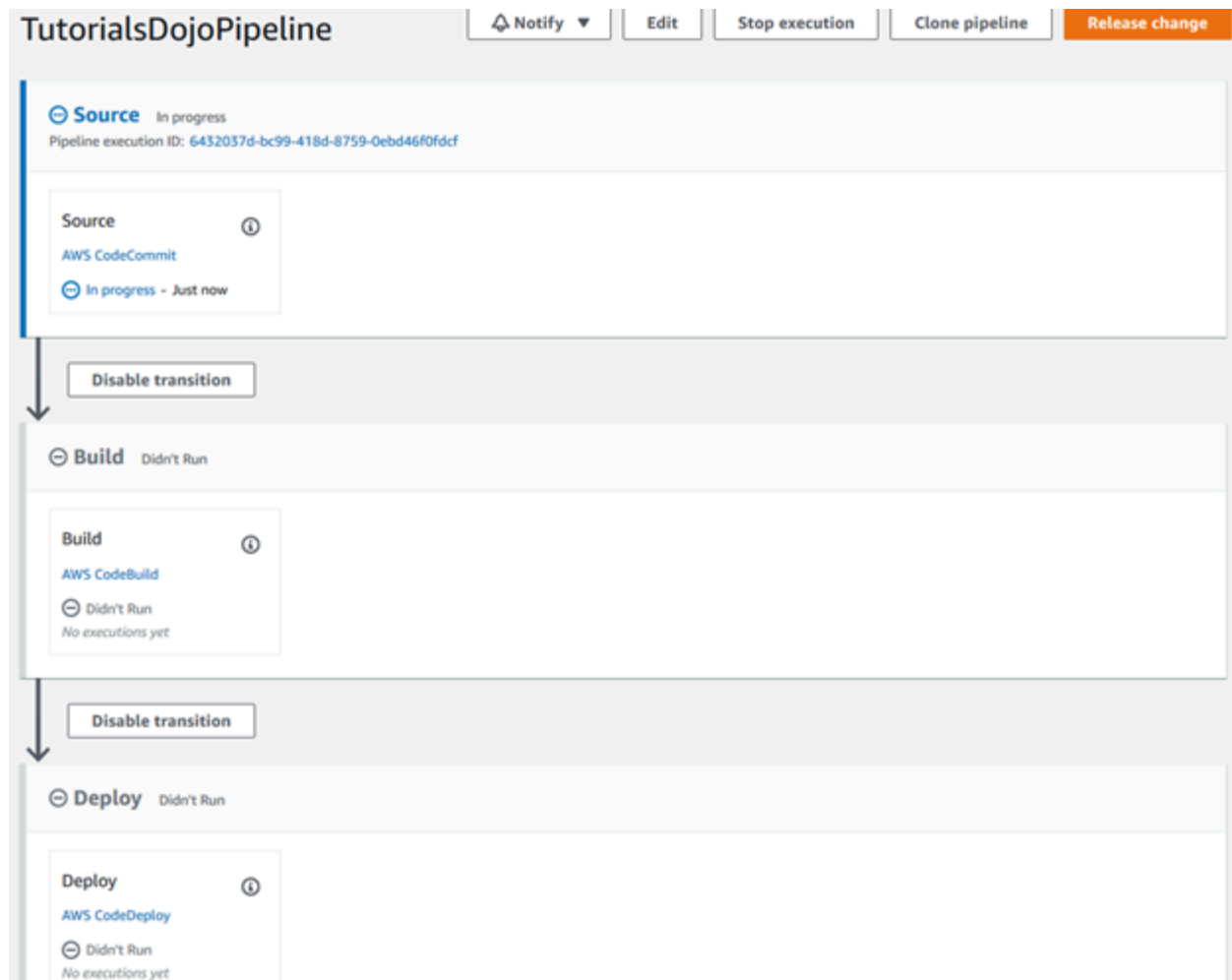
Revision type
☒ My application is stored in Amazon S3 ☐ My application is stored in GitHub

Revision location
Copy and paste the Amazon S3 bucket where your revision is stored
s3://mytest-bucket-cicd/myapp.zip?versionId=kwfk7xCNv_ZqgGLTshk5NY3q9
s3://bucket-name/folder/object.[zip|tar|tgz]

Revision file type
.zip



Here's how it should look on your CodePipeline run with multiple stages.



S3 Versioning and Encryption

By default, all the artifacts that you upload to an S3 bucket from AWS CodeBuild are encrypted. The default encryption is Amazon S3 Server-side encryption using AES-256.



Amazon S3 > Buckets > mytest-bucket-cicd > myapp.zip

myapp.zip Info

Properties | Permissions | **Versions**

Versions (2)

☐	Version ID	Type	Last modified
☒	_tzCiK5_AkXVra4TKb6X0o87sa16Z_Pf (Current version)	zip	February 13, 2024, 15:18:51 (UTC+08:00)
☐	YHJw5nNdoJDOq_5zuiFg1yYh2GWZbN2u	zip	February 13, 2024, 15:17:45 (UTC+08:00)

DevOps Exam Notes:

If you are using CodePipeline and you reference an Amazon S3 object artifact (like the output from CodeDeploy), you need to have versioning enabled. When you create your source bucket, make sure that you enable versioning on the bucket first. When you specify the S3 object name on your artifact parameter, you can specify the specific version ID that you want to deploy.

Also, remember that when you use the console to create or edit your pipeline, CodePipeline creates an Amazon EventBridge rule that starts your pipeline when a change occurs in the S3 source bucket or when the CodeBuild stage completes and successfully uploads the artifact to S3.

Sources:

<https://docs.aws.amazon.com/codepipeline/latest/userguide/tutorials-s3deploy.html>
https://docs.aws.amazon.com/codebuild/latest/APIReference/API_ProjectArtifacts.html
<https://docs.aws.amazon.com/codebuild/latest/userguide/getting-started-output-console.html>
<https://docs.aws.amazon.com/codepipeline/latest/userguide/action-reference-S3.html>



DynamoDB – Fetch vs Projected Attributes

A projection is the set of attributes that are copied (projected) from a table into a secondary index. These are in addition to the primary key attributes and index key attributes, which are automatically projected.

For example, we can see this base table (GameScores) with a partition key (UserID) and sort key (GameTitle).

GameScores

UserId	GameTitle	TopScore	TopScoreDateTime	Wins	Losses
"101"	"Galaxy Invaders"	5842	"2015-09-15:17:24:31"	21	72
"101"	"Meteor Blasters"	1000	"2015-10-22:23:18:01"	12	3
"101"	"Starship X"	24	"2015-08-31:13:14:21"	4	9
"102"	"Alien Adventure"	192	"2015-07-12:11:07:56"	32	192
"102"	"Galaxy Invaders"	0	"2015-09-18:07:33:42"	0	5
"103"	"Attack Ships"	3	"2015-10-19:01:13:24"	1	8
"103"	"Galaxy Invaders"	2317	"2015-09-11:06:53:00"	40	3
"103"	"Meteor Blasters"	723	"2015-10-19:01:13:24"	22	12
"103"	"Starship X"	42	"2015-07-11:06:53:00"	4	19

You can create a global secondary index (GameTitleIndex) with a new partition key (GameTitle) and sort key (TopScore). The base table's primary key attributes are always projected into an index, so the UserID attribute is also present. This improves searching when not using the primary keys of the base table.

GameTitleIndex

GameTitle	TopScore	UserId	Wins	Losses
"Alien Adventure"	192	"102"	32	192
"Attack Ships"	3	"103"	1	8
"Galaxy Invaders"	0	"102"	0	5
"Galaxy Invaders"	2317	"103"	40	3
"Galaxy Invaders"	5842	"101"	21	72
"Meteor Blasters"	723	"103"	22	12
"Meteor Blasters"	1000	"101"	12	3
"Starship X"	24	"101"	4	9
"Starship X"	42	"103"	4	19



When you query an index, Amazon DynamoDB can access any attribute in the projection as if those attributes were in a table of their own. When you create a secondary index, you need to specify the attributes that will be projected into the index. DynamoDB provides three different options for this:

- **KEYS_ONLY** – Each item in the index consists only of the table partition key and sort key values, plus the index key values. The KEYS_ONLY option results in the smallest possible secondary index.
- **INCLUDE** – In addition to the attributes described in KEYS_ONLY, the secondary index will include other non-key attributes that you specify.
- **ALL** – The secondary index includes all of the attributes from the source table. Because all of the table data are duplicated in the index, an ALL projection results in the largest possible secondary index.

You can project other base table attributes into the index if you want. When you query the index, DynamoDB can retrieve these projected attributes efficiently. However, global secondary index queries cannot fetch attributes from the base table. For example, if you query GameTitleIndex as shown in the diagram above, the query could not access any non-key attributes other than TopScore (although the key attributes GameTitle and UserID would automatically be projected).

Sources:

https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_Projection.html

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/GSI.html#GSI.Projections>



CodeBuild with CloudWatch Logs, Metrics, and Alarms

AWS Codebuild allows you to compile, build, run tests, and produce an artifact of your code. This can be integrated into your CI/CD process. After a developer pushes new code to the Development branch, you can have an automatic trigger for CodeBuild to run your project build, test your application, and then upload the output artifact to S3. The artifact file on S3 can then be used by deployment tools such as AWS CodeDeploy to have it deployed on your EC2 instances, ECS, or Lambda functions.

Here are the steps on how to create a CodeBuild build project and save the artifact on an S3 bucket. We will also demonstrate how CodeBuild integrates with Amazon CloudWatch.

1. Go to AWS Code Build > Build Projects and click Create Build Project. Input your details for this project.

Create build project

Project configuration

Project name
TutorialsDojoBuild
A project name must be 2 to 255 characters. It can include the letters A-Z and a-z, the numbers 0-9, and the special characters - and _.

Description - optional
Test if webpage has the word "TutorialsDojo" in it.

Build badge - optional
☐ Enable build badge

► Additional configuration
tags

2. CodeBuild supports several sources for your application code, including Amazon S3, GitHub, Bitbucket, etc. Using CodeBuild, select your source repository and branch.



Source

Add source

Source 1 - Primary

Source provider

AWS CodeCommit

Repository

Q TutorialsDojoRepo X

Reference type

Choose the source version reference type that contains your source code.

☒ Branch☐ Git tag☐ Commit ID

Branch

Choose a branch that contains the code to build.

master

Commit ID - optional

Choose a commit ID. This can shorten the duration of your build.

Q

Source version [Info](#)

refs/heads/master

9178776c second commit

3. Use the Amazon Linux runtime since we'll build this for the Amazon Linux AMI.



Runtime(s)
Standard ▼

Image
aws/codebuild/amazonlinux2-x86_64-standard:3.0 ▼

Image version
Always use the latest image for this runtime version ▼

Environment type
Linux ▼

Privileged
☐ Enable this flag if you want to build Docker images or want your builds to get elevated privileges

Service role

☒ New service role
Create a service role in your account

☐ Existing service role
Choose an existing service role from your account

Role name
codebuild-TutorialDojoBuild-service-role
Type your service role name

4. By default, the build specification filename is "buildspec.yml". This should be in the root folder of your application code.

Buildspec

Build specifications

☒ Use a buildspec file
Store build commands in a YAML-formatted buildspec file

☐ Insert build commands
Store build commands as build project configuration

Buildspec name - optional
By default, CodeBuild looks for a file named buildspec.yml in the source code root directory. If your buildspec file uses a different name or location, enter its path from the source root here (for example, buildspec-two.yml or configuration/buildspec.yml).

5. Specify which S3 bucket you are going to send the output artifact of your build.



Artifacts

Add artifact

Artifact 1 - Primary

Type

Amazon S3

You might choose no artifacts if you are running tests or pushing a Docker image to Amazon ECR.

Bucket name

Q mytest-bucket-cicd X

Name

The name of the folder or compressed file in the bucket that will contain your output artifacts. Use Artifacts packaging under Additional configuration to choose whether to use a folder or compressed file. If the name is not provided, defaults to project name.

TutorialsDojo-build

☐ Enable semantic versioning

Use the artifact name specified in the buildspec file

Path - optional

The path to the build output ZIP file or folder.

Example: MyPath/MyArtifact.zip.

Namespaces - optional

- On the logs part, you can have the option to send the build logs to CloudWatch Log group or send them to an S3 bucket. The Amazon CloudWatch log group or S3 bucket must exist first before you specify it here.



Logs

CloudWatch

- ☒ **CloudWatch logs - optional**
Checking this option will upload build output logs to CloudWatch.

Group name

/aws/codebuild/buildlog

Stream name

S3

- ☒ **S3 logs - optional**
Checking this option will upload build output logs to S3.

Bucket



Path prefix

- ☐ **Disable S3 log encryption**

7. Click “Create build project”. Select your project and click the “Start build” button.

TutorialsDojoBuild Notify Share Edit Delete build project Start build

Configuration

Source provider AWS CodeCommit	Primary repository TutorialsDojoRepo	Artifacts upload location mytest-bucket-cicd	Build badge Disabled
-----------------------------------	---	---	-------------------------

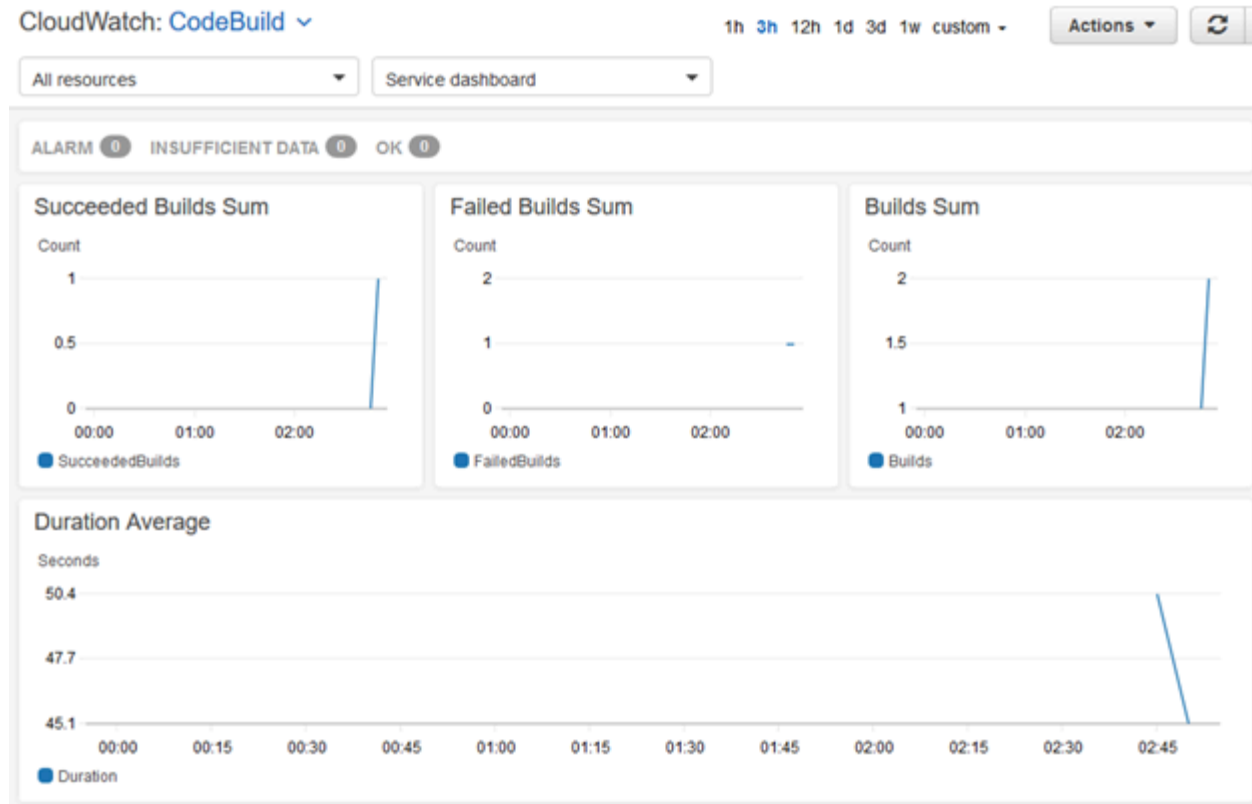
Build history | Build details | Build triggers | Metrics

Build history Stop build View artifacts View logs Delete builds Retry build

☐	Build run	Status	Build number	Source version	Submitter	Duration	Completed
☐	TutorialsDojoBuild:7062e be2-60cc- 4b9b-9699-79943f9c36b c	✔ Succeeded	3	refs/heads/master	k.samonte	44 seconds	Just now



8. After the build, you should see the artifact on the S3 bucket. You will also see the build logs of your project if you click on the Build run ID of your project.
9. AWS CodeBuild is also integrated on CloudWatch Logs. When you go to CloudWatch Dashboard, you can select the pre-defined CodeBuild Dashboard where you will see the metrics such as Successful builds, Failed builds, etc.



10. You should also be able to see the CloudWatch Log Group you created and that CodeBuild logs are delivered to it.



CloudWatch > CloudWatch Logs > Log groups > /aws/codebuild/buildlog Switch to the original interface.

/aws/codebuild/buildlog

Delete Actions ▼ View in Logs Insights Search log group

▼ **Log group details**

Retention	Creation time	Stored bytes	ARN
Never expire	9 minutes ago	-	arn:aws:logs:ap-northeast-1:██████:log-group:/aws/codebuild/buildlog:*
KMS key ID	Metric filters	Subscriptions	Contributor Insights rules
-	0	-	-

Log streams | Metric filters | Contributor Insights

Log streams (2) Refresh Delete Create log stream Search all

< 1 > ⌕

☐	Log stream	Last event time
☐	7062ebe2-60cc-4b9b-9699-79943f9c36bc	2020-07-26T02:53:31.987Z

11. Using CloudWatch Event rules, you can also create a rule to detect Successful or Failed builds and then use the Trigger to invoke a Lambda function to send you a slack notification or use an SNS Topic target to send you an email notification about the build status.



Event Source

Build or customize an Event Pattern or set a Schedule to invoke Targets.

☒ Event Pattern ⓘ ☐ Schedule ⓘ

Build event pattern to match events by service

Service Name **CodeBuild**

Event Type **CodeBuild Build State Change**

☐ Any state ☒ Specific state(s)

FAILED

▼ Event Pattern Preview

[Copy to clipboard](#) [Edit](#)

```
{
  "source": [
    "aws.codebuild"
  ],
  "detail-type": [
    "CodeBuild Build State Change"
  ],
  "detail": {
    "build-status": [
      "FAILED"
    ]
  }
}
```

Targets

Select Target to invoke when an event matches your Event Pattern or when schedule is triggered.

SNS topic

Topic* **Select topic**

► Configure input

Lambda function

Function* **Select function**

► Configure version/alias

► Configure input

Add target*

DevOps Exam Notes:

AWS CodeBuild can be integrated with Amazon EventBridge. A predefined dashboard is created on CloudWatch to view metrics regarding project builds on your account. You can send CodeBuild build logs to a CloudWatch log group, so you have a central location to review them. You can set filters and alarms on these logs and set up a notification. You can also use an Amazon EventBridge rule to detect changes on your build project, such as FAILED builds, and have it invoke a Lambda function to send you a message on your Slack channel or an SNS topic to send you an email notification.

Sources:

<https://docs.aws.amazon.com/codebuild/latest/userguide/create-project.html>

<https://docs.aws.amazon.com/codebuild/latest/userguide/create-project-console.html>



CodeDeploy with CloudWatch Logs, Metrics, and Alarms

AWS CodeDeploy allows you to automate software deployments to a variety of services such as EC2, AWS Fargate, AWS Lambda, and your on-premises servers.

For example, after you have produced from AWS CodeBuild, you can have CodeDeploy fetch the artifact from the Amazon S3 bucket and then deploy it to your AWS instances. Please note that the target instances need to have the AWS CodeDeploy agent installed on them for this to be successful and have proper Tags.

Here are the steps to create a deployment on AWS CodeDeploy, including a discussion on how it integrates with Amazon CloudWatch.

1. Go to AWS CodeDeploy > Applications and click "Create Application". Input details of your application and which platform it is running.

Create application

Application configuration

Application name
Enter an application name
TutorialsDojoApp
100 character limit

Compute platform
Choose a compute platform
EC2/On-premises

Cancel Create application

2. Select your application and create a Deployment Group. CodeDeploy needs to have an IAM permission to access your targets as well as read the Amazon S3 bucket containing the artifact to be deployed.



Create deployment group

Application

Application
TutorialsDojoApp
Compute type
EC2/On-premises

Deployment group name

Enter a deployment group name

100 character limit

Service role

Enter a service role
Enter a service role with CodeDeploy permissions that grants AWS CodeDeploy access to your target instances.

3. Select your deployment type. You can use In-place deployment if you already have the instances running. Specify the Tags of those EC2 instances, for example, **Environment:Dev**. CodeDeploy will use this as an identifier for your target instances.

Deployment type

Choose how to deploy your application

☒ **In-place**
Updates the instances in the deployment group with the latest application revisions. During a deployment, each instance will be briefly taken offline for its update

☐ **Blue/green**
Replaces the instances in the deployment group with new instances and deploys the latest application revision to them. After instances in the replacement environment are registered with a load balancer, instances from the original environment are deregistered and can be terminated.

Environment configuration

Select any combination of Amazon EC2 Auto Scaling groups, Amazon EC2 instances, and on-premises instances to add to this deployment

☐ Amazon EC2 Auto Scaling groups

☒ **Amazon EC2 instances**
1 unique matched instance. [Click here for details](#)

You can add up to three groups of tags for EC2 instances to this deployment group.
One tag group: Any instance identified by the tag group will be deployed to.
Multiple tag groups: Only instances identified by all the tag groups will be deployed to.

Tag group 1

Key	Value - optional	
Environment	Dev	Remove tag
Add tag		
+ Add tag group		

- Select how you want the new code to be deployed, such as All-at-once, or one-at-a-time, etc. These deployment settings will be discussed in the succeeding sections.



Deployment settings

Deployment configuration

Choose from a list of default and custom deployment configurations. A deployment configuration is a set of rules that determines how fast an application is deployed and the success or failure conditions for a deployment.

CodeDeployDefault.AllAtOnce ▲

 or

Create deployment configuration

CodeDeployDefault.OneAtATime

CodeDeployDefault.HalfAtATime

CodeDeployDefault.AllAtOnce

Select a load balancer to manage incoming traffic during the deployment process. The load balancer blocks traffic from each instance while it's being deployed to and allows traffic to it again after the deployment succeeds.

☐ Enable load balancing

- Now on your application Deployment group, create a Deployment. Input details for the artifact source. On the S3 bucket, you can specify the versionID of the artifact file.



Create deployment

Deployment settings

Application
TutorialsDojoApp

Deployment group

Compute platform
EC2/On-premises

Deployment type
In-place

Revision type
☒ My application is stored in Amazon S3 ☐ My application is stored in GitHub

Revision location
Copy and paste the Amazon S3 bucket where your revision is stored

s3://bucket-name/folder/object.[zip|tar|tgz]

Revision file type

6. After you click “Create deployment”, the deployment of the artifact on your EC2 instances will begin. You should see that the deployment will succeed.

Deployment lifecycle events						
Instance ID	Duration	Status	Most recent event	Events	Start time	End time
i-013f44fc2cb1443cc	8 seconds	✓ Succeeded	ValidateService	View events	Jul 26, 2020 11:28 AM (UTC+8:00)	Jul 26, 2020 11:28 AM (UTC+8:00)



7. You can click on “View events” to see the stages of the deployment process. Here you can view the deployment events as well as the status of the deployment lifecycle hooks you have defined. See the Lifecycle Event hooks section of this book for the details regarding each event.

Deployment details

Application	Deployment ID	Status
TutorialsDojoApp	d-SNK4KZN65	✔ Succeeded
Deployment configuration	Deployment group	Initiated by
CodeDeployDefault.AllAtOnce	TutorialsDojoDev	User action
Deployment description		
-		

Revision details

Revision location	Revision created	Revision description
s3://mytest-bucket-cicd/myapp.zip?versionId=kwfk7xCNv_ZqgGLTshk5NY3q9aam1C.M	Just now	Application revision registered by Deployment ID: d-SNK4KZN65

Event	Duration	Status	Error code	Start time	End time
ApplicationStop	less than one second	✔ Succeeded	-	Jul 26, 2020 11:28 AM (UTC+8:00)	Jul 26, 2020 11:28 AM (UTC+8:00)
DownloadBundle	less than one second	✔ Succeeded	-	Jul 26, 2020 11:28 AM (UTC+8:00)	Jul 26, 2020 11:28 AM (UTC+8:00)
BeforeInstall	2 seconds	✔ Succeeded	-	Jul 26, 2020 11:28 AM (UTC+8:00)	Jul 26, 2020 11:28 AM (UTC+8:00)
Install	less than one second	✔ Succeeded	-	Jul 26, 2020 11:28 AM (UTC+8:00)	Jul 26, 2020 11:28 AM (UTC+8:00)
AfterInstall	less than one second	✔ Succeeded	-	Jul 26, 2020 11:28 AM (UTC+8:00)	Jul 26, 2020 11:28 AM (UTC+8:00)
ApplicationStart	less than one second	✔ Succeeded	-	Jul 26, 2020 11:28 AM (UTC+8:00)	Jul 26, 2020 11:28 AM (UTC+8:00)
ValidateService	less than one second	✔ Succeeded	-	Jul 26, 2020 11:28 AM (UTC+8:00)	Jul 26, 2020 11:28 AM (UTC+8:00)

8. CodePipeline is integrated with the Amazon CloudWatch rule. You can create an Amazon EventBridge rule that will detect CodeDeploy status changes, such as a Successful or Failed deployment. Then have it invoke a Lambda function to perform a custom action, such as sending a notification on a Slack channel or setting an SNS topic to send an email to you about the status of your deployment.



Deployment details

Application	Deployment ID	Status
TutorialsDojoApp	d-SNK4KZN65	✔ Succeeded
Deployment configuration	Deployment group	Initiated by
CodeDeployDefault.AllAtOnce	TutorialsDojoDev	User action
Deployment description		
-		

Revision details

Revision location	Revision created	Revision description
s3://mytest-bucket-cicd/myapp.zip?versionId=kwfk7xCNv_ZqgGLTshk5NY3q9aam1C.M	Just now	Application revision registered by Deployment ID: d-SNK4KZN65

Event	Duration	Status	Error code	Start time	End time
ApplicationStop	less than one second	✔ Succeeded	-	Jul 26, 2020 11:28 AM (UTC+8:00)	Jul 26, 2020 11:28 AM (UTC+8:00)
DownloadBundle	less than one second	✔ Succeeded	-	Jul 26, 2020 11:28 AM (UTC+8:00)	Jul 26, 2020 11:28 AM (UTC+8:00)
BeforeInstall	2 seconds	✔ Succeeded	-	Jul 26, 2020 11:28 AM (UTC+8:00)	Jul 26, 2020 11:28 AM (UTC+8:00)
Install	less than one second	✔ Succeeded	-	Jul 26, 2020 11:28 AM (UTC+8:00)	Jul 26, 2020 11:28 AM (UTC+8:00)
AfterInstall	less than one second	✔ Succeeded	-	Jul 26, 2020 11:28 AM (UTC+8:00)	Jul 26, 2020 11:28 AM (UTC+8:00)
ApplicationStart	less than one second	✔ Succeeded	-	Jul 26, 2020 11:28 AM (UTC+8:00)	Jul 26, 2020 11:28 AM (UTC+8:00)
ValidateService	less than one second	✔ Succeeded	-	Jul 26, 2020 11:28 AM (UTC+8:00)	Jul 26, 2020 11:28 AM (UTC+8:00)

9. CodeDeploy also has built-in notification triggers. Click “Notify” on your application.

Developer Tools > CodeDeploy > Applications > TutorialsDojoApp

TutorialsDojoApp

⚠ Notify ⚠

Delete application

Create notification rule

Manage notification rules

Application details

Name	Compute platform
TutorialsDojoApp	EC2/On-premises

10. For events happening on your deployment, you can have targets much like Amazon EventBridge; however, this is limited to only an SNS Topic or AWS chatbot to Slack.

Targets

Create a target to use specifically for this notification rule. SNS topics created as targets have no subscribers but have all policies applied to act as a target for notifications. If you choose AWS Chatbot, you will be redirected to create a client in the AWS Chatbot console. [Learn more](#)

Create target

Configured targets

Choose target type	Choose target	
SNS topic		Remove row
AWS Chatbot (Slack)		Remove row

Add row

DevOps Exam Notes:

AWS CodePipeline is integrated with Amazon EventBridge. You can create Amazon EventBridge rule that will detect CodeDeploy status changes, such as a Successful or Failed deployment. With the rule targets, you invoke a Lambda function to perform a custom action i.e. set an SNS topic to send an email to you about the status of your deployment.

CodeDeploy also has built-in notification triggers to notify you of your deployment status, however, this is limited to only an SNS Topic or AWS Chatbot to Slack.

CodeDeploy Supports ECS and Lambda Deployments

Aside from EC2 instances, AWS CodeDeploy also supports deployment for ECS instances and Lambda deployments. The general steps for deployment are still the same - create an **Application**, create a **deployment group** for your instances, and create a **deployment** for your application.



However, for ECS, the artifact is a Docker image that can be stored from AWS ECR or from DockerHub. For Lambda deployments, the artifact will come from a zip file from an S3 bucket. Be sure to have the proper filename of your Lambda handler file for successful code deployments.

Deployments for ECS also support deployment configurations such as all-at-once, one-at-a-time, and half-of-the-time. These deployment configurations will be discussed in a separate section. Lambda, on the other hand, supports a percentage of traffic shifting, such as linear or canary deployment. This is also discussed in a separate section.

Sources:

<https://docs.aws.amazon.com/codedeploy/latest/userguide/monitoring-create-alarms.html>

<https://docs.aws.amazon.com/codedeploy/latest/userguide/monitoring.html>

<https://docs.aws.amazon.com/codedeploy/latest/userguide/deployments-create-console-ecs.html>

<https://docs.aws.amazon.com/codedeploy/latest/userguide/deployments-create-console-lambda.html>



CodePipeline and Amazon EventBridge Integration

AWS CodePipeline is a continuous delivery service that helps you automate your release pipelines for fast and reliable application and infrastructure updates. For example, you can use CodePipeline to have specific stages that build, test, and deploy your code to your target instances.

You can have CodePipeline trigger a full deployment pipeline when your developer pushes code to your Git or Github repository, which will then start the series of pipeline stages:

- Pull source code from the Git or Github repository.
- Run the CodeBuild project to build and test the artifact file. Upload the artifact to the Amazon S3 bucket.
- Trigger CodeDeploy deployment to fetch the artifact and deploy it to your instances.

This whole cascading process is triggered only by a single Git or Github repository push event.

Here are the steps to create a pipeline on AWS CodePipeline as well as its integration with Amazon CloudWatch.

1. Go to AWS CodePipeline > Pipeline and click Create pipeline. Input the name of the service for CodePipeline and the S3 bucket that holds the artifacts for this pipeline.
2. Add a source stage such as Git or Github repository and branch on which code version you want to deploy.
3. Create a build stage, such as CodeBuild, that will build and test the artifact for you. You must have an existing CodeBuild build project for this.
4. Add a deploy stage using AWS CodeDeploy. The details such as the Application and Deployment group must exist first before you proceed here.
5. After creating the pipeline, you should see your stages, and the pipeline starts the whole process, from Git or Github, CodeBuild to CodeDeploy. You should be able to see the status of each stage.

Devops Exam Notes:

AWS CodePipeline can be integrated with Amazon EventBridge. For example, you have your entire pipeline running on schedule every day so you will have an updated environment based on the daily progress of development. You can have an Amazon EventBridge Events rule that runs on schedule and have it target an AWS CodePipeline ARN.

Here's a sample rule on specifying a daily run of a pipeline on AWS CodePipeline:

Step 1: Create rule

Create rules to invoke Targets based on Events happening in your AWS environment.

Event Source

Build or customize an Event Pattern or set a Schedule to invoke Targets.

☐ Event Pattern ⓘ ☒ Schedule ⓘ

☒ Fixed rate of

☐ Cron expression

[Learn more about CloudWatch Events schedules.](#)

▶ Show sample event(s)

Targets

Select Target to invoke when an event matches your Event Pattern or when schedule is triggered.

Pipeline ARN* ⓘ

[Learn more about AWS CodePipeline ARN formats](#)

▶ Configure input

CloudWatch Events needs permission to start pipeline executions in AWS CodePipeline. You can create a role or use an existing role to provide these permissions.

☒ Create a new role for this specific resource

☐ Use existing role ⓘ

[Learn more about CloudWatch Events identity-based policies.](#)

Devops Exam Notes:

With Amazon EventBridge, you can also detect the Status of each stage of the pipeline. You can have a rule for the Pipeline execution state change, such as a FAILED stage, and have it invoke a Lambda function or an SNS topic to send you an email notification.

Here's a screenshot of an EventBridge Events rule that targets a Lambda function and an SNS topic.



The screenshot shows the 'Create rule' wizard in the Amazon EventBridge console. The 'Define rule detail' step is active, showing the rule name 'CodePipeline' and the event bus 'default'. The 'Event pattern' step is also visible, showing the event source 'CodePipeline' and the event type 'CodePipeline Pipeline Execution State Change'. The event pattern is defined as:

```
1 {
2   "source": ["aws.codepipeline"],
3   "detail-type": ["CodePipeline Pipeline Execution State Change"],
4   "detail": {
5     "state": ["SUCCEEDED", "FAILED"]
6   }
7 }
```

The 'Event pattern' step shows the event source 'CodePipeline' and the event type 'CodePipeline Pipeline Execution State Change'. The event pattern is defined as:

```
1 {
2   "source": ["aws.codepipeline"],
3   "detail-type": ["CodePipeline Pipeline Execution State Change"],
4   "detail": {
5     "state": ["SUCCEEDED", "FAILED"]
6   }
7 }
```

The 'Event pattern' step shows the event source 'CodePipeline' and the event type 'CodePipeline Pipeline Execution State Change'. The event pattern is defined as:

```
1 {
2   "source": ["aws.codepipeline"],
3   "detail-type": ["CodePipeline Pipeline Execution State Change"],
4   "detail": {
5     "state": ["SUCCEEDED", "FAILED"]
6   }
7 }
```

Source:

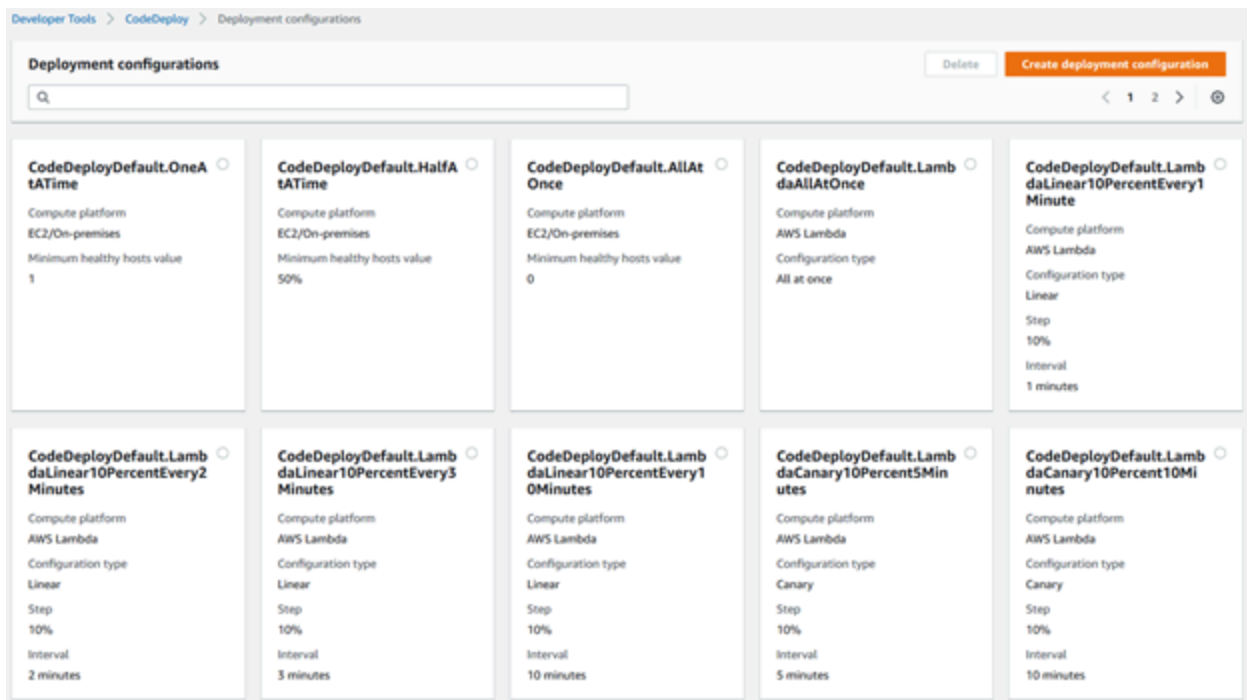
<https://docs.aws.amazon.com/codepipeline/latest/userguide/pipelines-create.html>

CodeDeploy - Linear, Canary and All-at-Once (Pre-defined Deployments)

When you deploy a new application version on your target compute platform (EC2 or ECS or Lambda), you can have several options for shifting network traffic to the newer version.

Using CodeDeploy, you can control how many instances are getting updated at any given time during the deployment. This is important because, during deployments, the application is stopped while the new version is deployed. You want to make sure that enough instances are online to serve the traffic as well as the ability to roll-back the changes when an error during the deployment occurs.

On the AWS Console, when you click on CodeDeploy > Deployment configurations, you will see the list of AWS-defined deployment strategies that you can use for your deployments. You can create your own deployment configuration if you want to, but we'll discuss the most common ones here, which you will likely encounter in the exam.



CodeDeployDefault.AllAtOnce – this is the fastest deployment. The application will stop on all EC2 instances and CodeDeploy will install the newer version on all instances. The application will stop serving traffic during the deployment as all instances are offline.



CodeDeployDefault.OneAtATime – this is the slowest deployment. CodeDeploy will stop only one instance at a time. This will take time to deploy on all instances but the application will remain online since only one instance is offline at any given time.

CodeDeployDefault.HalfAtATime – half, or 50% of the instances will be offline during the deployment, while the other half is online to serve traffic. This is a good balance between a fast and safe deployment.

CodeDeployDefault.LambdaLinear10PercentEvery10Minutes – Deployment for Lambda functions. This deployment will use Aliases on the backend to shift traffic from the old version to the newer version. Ten percent of traffic will be shifted to the newer version every 10 minutes. Deployment will run at 10-minute intervals until 100% of the traffic is shifted to the newer version.

CodeDeployDefault.LambdaCanary10Percent10Minutes – Deployment for Lambda functions. This deployment will use Aliases on the backend to shift traffic from the old version to the newer version. Initially, 10% of the traffic will be shifted to the newer version. This will only last 10 minutes so you can have time to check the application logs. After 10 minutes, the remaining 90% of the traffic will be shifted to the newer version.

Source:

<https://docs.aws.amazon.com/codedeploy/latest/userguide/deployment-configurations.html>



Elastic Beanstalk - Deployment Policies and Settings

AWS Elastic Beanstalk provides several options for how deployments are processed, including deployment policies (All at once, Rolling, Rolling with additional batch, Immutable, and Traffic splitting) and options that let you configure the batch size and health check behavior during deployments. By default, your environment uses all-at-once deployments.

All at once – The quickest deployment method. Suitable if you can accept a short loss of service, and if quick deployments are important to you. With this method, Elastic Beanstalk deploys the new application version to each instance.

Rolling deployments - Elastic Beanstalk splits the environment's Amazon EC2 instances into batches and deploys the new version of the application to one batch at a time. During a rolling deployment, some instances serve requests with the old version of the application, while instances in completed batches serve other requests with the new version.

Rolling deployment with additional batch - launches new batches during the deployment. To maintain full capacity during deployments, you can configure your environment to launch a new batch of instances before taking any instances out of service. When the deployment completes, Elastic Beanstalk terminates the additional batch of instances.

Immutable deployments - perform an immutable update to launch a full set of new instances running the new version of the application in a separate Auto Scaling group, alongside the instances running the old version. Immutable deployments can prevent issues caused by partially completed rolling deployments. If the new instances don't pass health checks, Elastic Beanstalk terminates them, leaving the original instances untouched.

Traffic-splitting deployments - let you perform canary testing as part of your application deployment. In a traffic-splitting deployment, Elastic Beanstalk launches a full set of new instances just like during an immutable deployment. It then forwards a specified percentage of incoming client traffic to the new application version for a specified evaluation period. If the new instances stay healthy, Elastic Beanstalk forwards all traffic to them and terminates the old ones. If the new instances don't pass health checks, or if you choose to abort the deployment, Elastic Beanstalk moves traffic back to the old instances and terminates the new ones.

Here's a summary of the deployment methods, how long each deployment takes, and how rollback is handled.



Method	Impact of failed deployment	Deploy time	Zero downtime	No DNS change	Rollback process	Code deployed to
All at once	Downtime	⌚	No	Yes	Manual redeploy	Existing instances
Rolling	Single batch out of service; any successful batches before failure running new application version	⌚ ⌚ †	Yes	Yes	Manual redeploy	Existing instances
Rolling with an additional batch	Minimal if first batch fails; otherwise, similar to Rolling	⌚ ⌚ ⌚ †	Yes	Yes	Manual redeploy	New and existing instances
Immutable	Minimal	⌚ ⌚ ⌚ ⌚	Yes	Yes	Terminate new instances	New instances
Traffic splitting	Percentage of client traffic routed to new version temporarily impacted	⌚ ⌚ ⌚ ⌚ ††	Yes	Yes	Reroute traffic and terminate new instances	New instances
Blue/green	Minimal	⌚ ⌚ ⌚ ⌚	Yes	No	Swap URL	New instances

† Varies depending on batch size.

†† Varies depending on **evaluation time** option setting.

Sources:

<https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/using-features.rolling-version-deploy.html>

<https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/using-features.deploy-existing-version.html>



Domain 2: Configuration Management and Infrastructure-as-Code



Overview

The second domain of the AWS Certified DevOps Engineer Professional exam focuses on the topics related to configuration management and infrastructure-as-code. You should have working knowledge in crafting CloudFormation templates and how to use it for deployments, operating with ElasticBeanstalk environments, AWS Lambda deployments, and deployments on the Elastic Container Service. To be an effective DevOps Engineer, it is important that you understand these key concepts. Roughly 17% of questions in the actual DevOps exam revolve around these topics.

This domain will challenge your know-how in doing the following:

- Composing and deploying IaC templates using AWS Serverless Application Model (AWS SAM), AWS CloudFormation and AWS Cloud Development Kit (AWS CDK)
- Applying AWS CloudFormation StackSets across multiple accounts and AWS Regions
- Determining optimal configuration management services for a particular use case. Related AWS configuration services are AWS Systems Manager, AWS Config and AWS AppConfig.
- Implementing infrastructure patterns, governance controls, and security standards into reusable IaC templates using AWS Service Catalog, CloudFormation modules and AWS CDK.
- Standardizing and automating account provisioning and configuration
- Creating, consolidating, and centrally managing through AWS Organizations and AWS Control Tower.
- Applying IAM solutions for multi-account and complex organization structures with SCPs and IAM Roles.
- Implementing and developing governance and security controls at scale using AWS Config, AWS Control Tower, AWS Security Hub, Amazon Detective, Amazon GuardDuty, AWS Service Catalog and Service Control Policies (SCPs)
- Automating system inventory, configuration, and patch management with AWS Systems Manager and AWS Config
- Developing Lambda function automations for complex scenarios with the help of AWS SDKs, AWS CDKs, AWS Lambda, and AWS Step Functions.
- Automating the configuration of software applications to the desired state using Systems Manager State Manager.
- Maintaining software compliance using Systems Manager, Security Hub and other related services.

In this chapter, we will cover all of the related topics for configuration management and infrastructure-as-code in AWS that will likely show up in your DevOps Professional exam.



What is Configuration Management?

Configuration management is the process of standardizing resource configurations and maintaining the consistency of your application and server components. Configuration management deals with several areas such as source code repositories, artifact and image repositories, and configuration repositories.

For example, when deploying your application on EC2 instances, you want to make sure that the correct artifact version is deployed and that the required dependencies are installed on the servers. If your new application version requires another dependency package, you should configure all related servers to accommodate this change as well.

If you want to make changes on the OS configuration, for example - an updated logging configuration, you will want to apply it on all running servers as well as to new servers that will be created in the future, plus have the ability to roll back the changes in case you find any errors.

Benefits of configuration management:

Scalability – you don't have to manually configure your servers, such as installation of OS updates, application dependencies, and security compliance configurations. The same process will apply no matter how many servers you have.

Reliability - configuration management offers a reliable way for you to deploy your code. There is central management for your changes and updates so it reduces human errors compared to applying changes manually across your systems.

Disaster Recovery – If you happen to deploy an artifact with bad code or a new config file causing an error, you will have a quick and easy way to rollback since you can go back to the last working version of your configuration.

With proper configuration management tools, you will only have to make changes on your configuration code and it will be applied to all related instances. This process is consistent and scalable in such a way that it's applicable from a range of few instances to several hundreds of instances. Automation is a key component for configuration management so there are several AWS tools available for you.



What is Infrastructure-as-Code (IaC)?

Infrastructure-as-code (IaC) takes the concept of configuration management to the next level. Imagine your entire AWS infrastructure and resources described inside a YAML or JSON file. Just like how your application source code outputs an artifact, IaC generates a consistent environment when you apply it.

For example, Infrastructure as Code enables DevOps teams to easily and quickly create test environments that are similar to the production environment. IaC allows you to deliver stability rapidly, consistently, and at scale.

Another example is when you need to create a Disaster Recovery site in another region. With IaC, you can quickly create resources on the new region and be assured that the environment is consistent with the current live environment because everything is defined and described on your JSON or YAML code. You can also save your code to repositories and you can version control it to track changes on your infrastructure. AWS CloudFormation is the main service that you can use if you have codified your infrastructure.

AWS CloudFormation gives developers and systems administrators an easy way to create and manage a collection of related AWS resources, provisioning and updating them in an orderly and predictable fashion. You can use templates to describe the AWS resources and any associated dependencies or runtime parameters required to run your application.

Benefits of using CloudFormation

- **Extensibility** - Using the AWS CloudFormation Registry, you can model, provision, and manage third-party application resources alongside AWS resources with AWS CloudFormation.
- **Authoring with JSON/YAML** - allows you to model your entire infrastructure in a text file. You can use JSON or YAML to describe what AWS resources you want to create and configure.
- **Safety controls** - automates the provisioning and updating of your infrastructure in a safe and controlled manner. You can use Rollback Triggers to rollback in case errors are encountered during the update.
- **Preview changes to your environment** - Change Sets allow you to preview how proposed changes to a stack might impact your running resources. For example, whether your changes will delete or replace any critical resources.
- **Dependency management** - automatically manages dependencies between your resources during stack management actions. The sequence of creating, updating, or deleting the dependencies and resources is automatically taken care of.
- **Cross account & cross-region management** - AWS StackSets that lets you provision a common set of AWS resources across multiple accounts and regions with a single CloudFormation template.

Sources:

<https://aws.amazon.com/cloudformation/>



CloudFormation Cross-Stack Reference

CloudFormation allows you to reference resources from one CloudFormation stack and use those resources on another stack. This is called **cross-stack reference**. It allows for a layering of stacks, which is useful for separating your resources based on your services. Instead of putting all resources on one stack, you can create resources from one stack and reference those resources on other CloudFormation stacks.

This also allows you to reuse the same CloudFormation stacks so that you can build faster if you need a new environment with minimal changes.

Example:

- Network stack – contains VPC, public and private subnets, and security groups.
- Web server stack – contains webserver and referencing the public subnets and security groups from the network stack
- Database stack – contains your database server and referencing the private subnets and security groups from the network stack

DevOps Exam Notes:

The requirement for cross stack reference is that you need to export the resources that you want to be referenced by other stacks. Use **Export** on the output Field of your main CloudFormation stack to define the resources that you want to expose to other stacks. On the other stacks, use the **Fn::ImportValue** intrinsic function to import the value that was previously exported.

Here's an example of CloudFormation exporting a subnet and a security group, and referencing it on another CloudFormation stack.



Tutorials Dojo Manila Stack A - EXPORT

```
"Outputs" : {
  "PublicSubnet" : {
    "Description" : "The subnet ID to use for public web servers",
    "Value" : { "Ref" : "PublicSubnet" },
    "Export" : { "Name" : "${AWS::StackName}-SubnetID" }}
},
"WebServerSecurityGroup" : {
  "Description" : "The security group ID to use for public web servers",
  "Value" : { "Fn::GetAtt" : [ "WebServerSecurityGroup", "GroupId" ] },
  "Export" : { "Name" : "${AWS::StackName}-SecurityGroupID" }}
}
```

Tutorials Dojo Manila Stack B - IMPORT

```
"Resources" : {
  "WebServerInstance" : {
    "Type" : "AWS::EC2::Instance",
    "Properties" : {
      "InstanceType" : "t2.micro",
      "ImageId" : "ami-a1b23456",
      "NetworkInterfaces" : [{
        "GroupSet" : [{ "Fn::ImportValue" : { "Fn::Sub" : "${NetworkStackNameParameter}-SecurityGroupID" } }],
        "AssociatePublicIpAddress" : "true",
        "DeviceIndex" : "0",
        "DeleteOnTermination" : "true",
        "SubnetId" : { "Fn::ImportValue" : { "Fn::Sub" : "${NetworkStackNameParameter}-SubnetID" } }
      ]
    }
  }
}
```

Source:

<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/walkthrough-crossstackref.html>



Lambda Function Artifact from S3 or CloudFormation Inline

S3 bucket on CloudFormation

Using AWS CloudFormation, you can deploy AWS Lambda functions, which is an easy way to reliably reproduce and version your application deployments.

DevOps Exam Notes:

On your CloudFormation template, the **AWS::Lambda::Function** resource creates a Lambda function. To create a function, you need a deployment package and an execution role. The deployment package contains your function code. The artifact file is defined on the **AWS::Lambda::Function Code** resource.

Here is an example of Node.js lambda function that uses an artifact saved on an S3 bucket on CloudFormation (uses JSON Format).

```
"AMIIDLookup": {
  "Type": "AWS::Lambda::Function",
  "Properties": {
    "Handler": "index.handler",
    "Role": {
      "Fn::GetAtt": [
        "LambdaExecutionRole",
        "Arn"
      ]
    },
    "Code": {
      "S3Bucket": "lambda-functions",
      "S3Key": "amillookup.zip"
    },
    "Runtime": "nodejs12.x",
    "Timeout": 25,
    "TracingConfig": {
      "Mode": "Active"
    }
  }
}
```

Note that Changes to a deployment package in Amazon S3 are not detected automatically during stack updates. To update the function code, change the object key or version in the template. Or you can use a



parameter on your CloudFormation template to input the name of the artifact on S3 that contains the latest version of your code.

If you have a zip file on your local machine, you can use the package command to create a template that generates a template that you can use.

```
aws cloudformation package --template /path_to_template/template.json --s3-bucket mybucket  
--output json > packaged-template.json
```

This will save the output on a json template that you can upload on CloudFormation.

Inline Lambda functions in CloudFormation

For Node.js and Python functions, you can specify the function code inline in the template.

DevOps Exam Notes:

Note that you need to use the **AWS::Lambda::Function Code** to define that function and it should be enclosed inside the **ZipFile: |** section to ensure that CloudFormation correctly parses your code.

Here is an example of Node.js lambda function inline on CloudFormation template using the YAML format.

```
AWSTemplateFormatVersion: '2010-09-09'  
Description: Lambda function with cfn-response.  
Resources:  
  primer:  
    Type: AWS::Lambda::Function  
    Properties:  
      Runtime: nodejs12.x  
      Role: arn:aws:iam::123456789012:role/lambda-role  
      Handler: index.handler  
      Code:  
      ZipFile: |  
var aws = require('aws-sdk')  
var response = require('cfn-response')  
exports.handler = function(event, context) {  
  console.log("REQUEST RECEIVED:\n" + JSON.stringify(event))  
  // For Delete requests, immediately send a SUCCESS response.  
  if (event.RequestType == "Delete") {  
    response.send(event, context, "SUCCESS")  
    return  
  }  
}
```




```
var responseStatus = "FAILED"
var responseData = {}
var functionName = event.ResourceProperties.FunctionName
var lambda = new aws.Lambda()
lambda.invoke({ FunctionName: functionName }, function(err,
invokeResult) {
    if (err) {
        responseData = {Error: "Invoke call failed"}
        console.log(responseData.Error + ":\n", err)
    }
    else responseStatus = "SUCCESS"
    response.send(event, context, responseStatus, responseData)
})
}
Description: Invoke a function during stack creation.
TracingConfig:
Mode: Active
```

Sources:

<https://docs.aws.amazon.com/lambda/latest/dg/deploying-lambda-apps.html>

<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/using-cfn-cli-package.html>



AutoScalingReplacingUpdate vs AutoScalingRollingUpdate Policy

When using AWS CloudFormation to provision your Auto Scaling groups, you can control how CloudFormation handles updates for your Auto Scaling Group. You need to define the proper **UpdatePolicy** attribute for your ASG depending on your desired behavior during an update.

DevOps Exam Notes:

You can use **AWS::AutoScaling::AutoScalingGroup** resource type on CloudFormation if you want to create an AutoScaling group for your fleet of EC2 instances. Going into the exam, you will need to distinguish the difference between the **AutoScalingReplacingUpdate** and **AutoScalingRollingUpdate** UpdatePolicy attribute, which define how the instances on your group will be updated when you deploy a new revision of your application.

AutoScalingReplacingUpdate - will create a new auto scaling group with new launch template. This is more like an immutable type of deployment.

AutoScalingRollingUpdate - will replace the instances on the current auto-scaling group. You can control if instances will be replaced "all-at-once" or use a rolling update by batches. The default behavior is to delete instances first, before creating the new instances.

The **AutoScalingReplacingUpdate** policy specifies how AWS CloudFormation handles replacement updates for an Auto Scaling group. This policy enables you to specify whether AWS CloudFormation replaces an Auto Scaling group with a new one or replaces only the instances in the Auto Scaling group.

```
"UpdatePolicy" : {  
  "AutoScalingReplacingUpdate" : {  
    "WillReplace" : Boolean  
  }  
}
```

For example, you can set **AutoScalingReplacingUpdate WillReplace** property to TRUE to have CloudFormation retain the old ASG and the instances it contains. CloudFormation will wait for the successful creation of the new ASG and its instances before it deletes the old ASG. This is helpful when the update fails; CloudFormation can quickly rollback as it will only delete the new ASG. The current ASG and its instances are not affected during the deployment and rollback process.



The **AutoScalingRollingUpdate** policy specifies how AWS CloudFormation handles rolling updates for an Auto Scaling group. Rolling updates enable you to specify whether AWS CloudFormation updates instances that are in an Auto Scaling group in batches or all at once.

```
"UpdatePolicy" : {  
  "AutoScalingRollingUpdate" : {  
    "MaxBatchSize" : Integer,  
    "MinInstancesInService" : Integer,  
    "MinSuccessfulInstancesPercent" : Integer,  
    "PauseTime" : String,  
    "SuspendProcesses" : [ List of processes ],  
    "WaitOnResourceSignals" : Boolean  
  }  
}
```

For example, **AutoScalingRollingUpdate** allows you to specify the **MaxBatchSize** property to set the maximum number of instances that AWS CloudFormation updates at any given time. Or use the **MinInstancesInService** property to ensure that there is a minimum number of instances that are in service while CloudFormation updates the old instances.

Sources:

<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/aws-attribute-updatepolicy.html>
<https://aws.amazon.com/premiumsupport/knowledge-center/auto-scaling-group-rolling-updates/>



Discovery Agent vs Discovery Connector

Application Discovery Agent and Agentless Discovery Connector are helpful tools in the AWS Application Discovery Service to help you plan your migration from on-premises servers and VMs to AWS cloud. Here's a quick description and summary of the differences between the two.

The **Application Discovery Agent** is a software that you install on on-premises servers and VMs targeted for discovery and migration. The agent is needed by AWS Discovery Service to help you plan your migration to the AWS cloud by collecting usage and configuration data about your on-premises servers. The Agent captures system configuration, system performance, running processes, and details of the network connections between systems.

You can then view the discovered servers, group them into applications, and then track the migration status of each application from the AWS Migration Hub console.

If you can't install the agent on your on-premises servers, AWS Application Discovery Service offers another way of performing discovery through the AWS **Agentless Discovery Connector**. This agentless discovery is performed by deploying an OVA file in VMware vCenter.

The Discovery Connector identifies virtual machines (VMs) and hosts associated with vCenter, collects static configuration data such as Server hostnames, IP addresses, MAC addresses, and disk resource allocations. Additionally, it collects the utilization data for each VM and computes average and peak utilization for metrics such as CPU, RAM, and Disk I/O.

DevOps Exam Notes:

Know the differences between **Application Discovery Agent** and **Agentless Discovery Connector**.

- **Application Discovery Agent** - agent package is installed on on-premises VMs and servers for migration.
- **Agentless Discovery Connector** - standalone VM to be deployed on on-premises data center to collect information for migration.

Sources:

<https://docs.aws.amazon.com/application-discovery/latest/userguide/what-is-appdiscovery.html>

<https://docs.aws.amazon.com/application-discovery/latest/userguide/discovery-agent.html>

<https://docs.aws.amazon.com/application-discovery/latest/userguide/discovery-connector.html>



CloudFormation Template for ECS, Auto Scaling and ALB

Amazon Elastic Container Service (ECS) allows you to manage and run Docker containers on clusters of EC2 instances. You can also configure your ECS to use Fargate launch type which eliminates the need to manage EC2 instances.

With CloudFormation, you can define your ECS clusters and task definitions to easily deploy your containers. For the high availability of your Docker containers, ECS clusters are usually configured with an auto scaling group behind an application load balancer. These resources can also be declared on your CloudFormation template.

DevOps Exam Notes:

Going on to the exam, be sure to remember the syntax needed to declare your ECS cluster, Auto Scaling group, and application load balancer. The **AWS::ECS::Cluster** resource creates an ECS cluster and the **AWS::ECS::TaskDefinition** resource creates a task definition for your container. The **AWS::ElasticLoadBalancingV2::LoadBalancer** resource creates an application load balancer and the **AWS::AutoScaling::AutoScalingGroup** resource creates an EC2 auto scaling group.

AWS provides an example template that you can use to deploy a web application in an Amazon ECS container with auto-scaling and an application load balancer. Here's a snippet of the template with the core resources:

```
{
  "AWSTemplateFormatVersion": "2010-09-09",
  "Resources": {
    "ECSCluster": {
      "Type": "AWS::ECS::Cluster"
    },
    .....,
    "taskdefinition": {
      "Type": "AWS::ECS::TaskDefinition",
      "Properties": {
        .....,
        "ECSALB": {
          "Type": "AWS::ElasticLoadBalancingV2::LoadBalancer",
```



```
    "Properties":{  
.....  
    "ECSAutoScalingGroup":{  
      "Type": "AWS::AutoScaling::AutoScalingGroup",  
      "Properties":{  
        "VPCZoneIdentifier":{  
          "Ref": "SubnetId"  
        }  
      },
```

Sources:

<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/quickref-ecs.html>

<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/aws-resource-ecs-service.html>

<https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/aws-properties-ecs-service-loadbalancers.html>



Domain 3: Resilient Cloud Solutions



Overview

The third exam domain of the AWS Certified DevOps Engineer Professional test covers the topic of Resilient Cloud Solutions. This particular knowledge area is related to the concepts of High Availability, Fault Tolerance, and Disaster Recovery. 15% of the DevOps exam questions would be related to the subject of resiliency so make sure that you allocate ample time studying for this domain.

This domain will verify your capability in implementing the following:

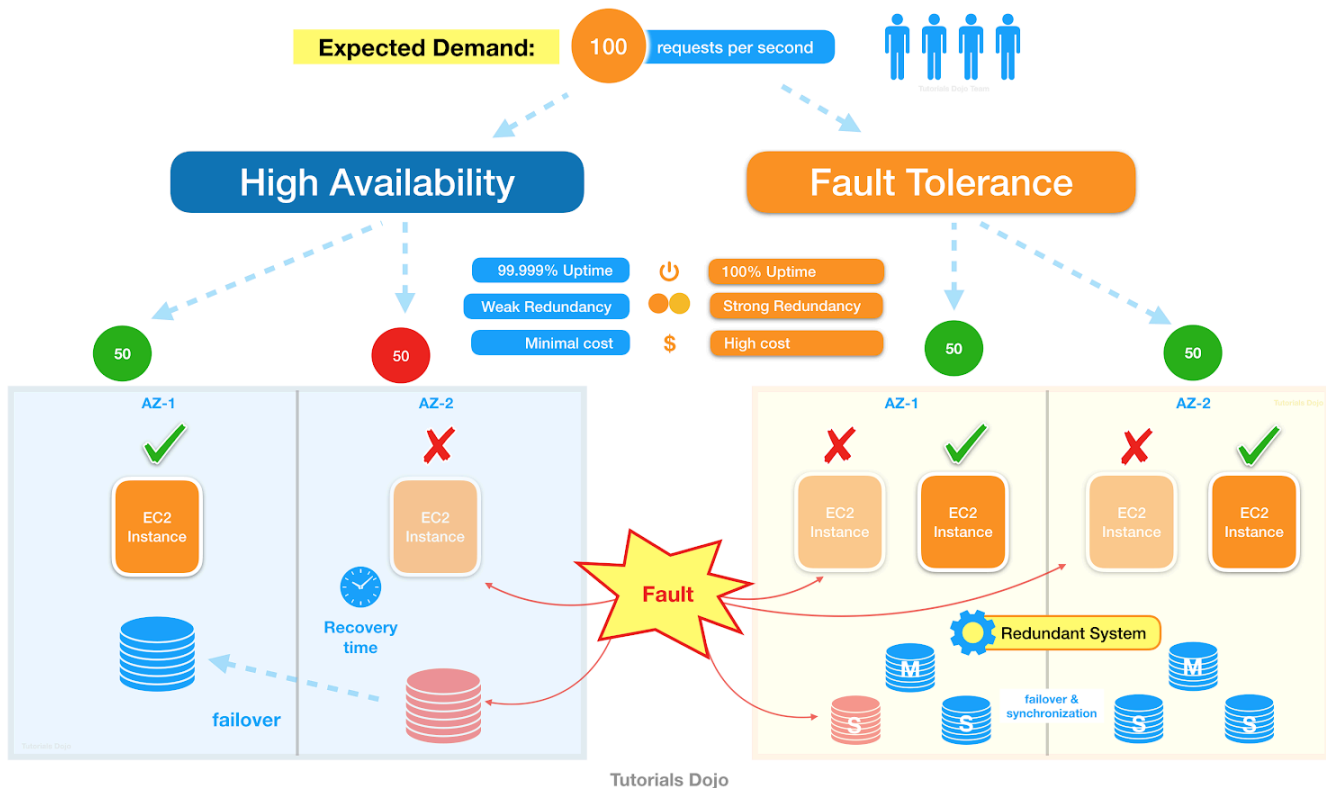
- Translating business requirements into technical resiliency needs
- Identifying and remediating single points of failure in existing workloads
- Enabling cross-Region solutions for certain scenarios using Amazon DynamoDB Global Tables, Amazon RDS Read Replica, Amazon Aurora Global Database, Amazon Route 53, Amazon S3, Amazon CloudFront, Amazon S3 Cross-Region Replication and other replication features.
- Configuring load balancing to support cross-AZ services
- Configuring applications and related services to support multiple Availability Zones and Regions while minimizing downtime
- Identifying and remediating scaling issues
- Identifying and implementing appropriate auto-scaling, load balancing, and caching solutions
- Deploying container-based applications using Amazon ECS, Amazon EKS and AWS Fargate
- Deploying workloads in multiple AWS Regions for global scalability
- Configuring serverless applications using Amazon API Gateway, AWS Lambda and AWS Fargate
- Testing failover of Multi-AZ/multi-Region workloads using Amazon RDS, Amazon Aurora, Route 53, CloudFront and other related features
- Identifying and implementing appropriate cross-Region backup and recovery strategies via AWS Backup, Amazon S3, AWS Systems Manager and the likes.
- Configuring a load balancer to recover from backend failure

Let's discover how to implement resiliency in your AWS cloud architecture in this section.

High Availability vs. Fault Tolerance

Many IT Professionals nowadays do not have the right understanding of High Availability and Fault Tolerance, even those who are already in the IT industry for a long time. These two concepts are quite similar to each other, and sometimes, people use these two terms interchangeably. But are they theoretically the same?

Both High Availability and Fault Tolerance have the same objective of ensuring that your application runs all the time without any system degradation. However, these concepts have unique attributes that differentiate them from each other. These two differ in cost, design, redundancy level, and behavior on component faults or failures.



High Availability aims for your application to run 99.999% of the time. Its design ensures that the entire system can quickly recover if one of its components crashes. It has an ample number of redundant resources to allow failover to another resource if the other one fails. This concept accepts that a failure will occur but provides a way for the system to recover fast.



Fault Tolerance, on the other hand, has the goal of keeping your application running with zero downtime. It has a more complex design, and higher redundancy to sustain any fault in one of its components. Think of it as an upgraded version of High Availability. As its name implies, it can *tolerate* any component fault to avoid any performance impact, data loss, or system crashes by having redundant resources beyond what is typically needed. The caveat for implementing a fault-tolerant system is its cost as companies have to shoulder the capital and operating expenses for running its required numerous resources.

A system can be highly available but not fault-tolerant, and it can be both. If an application is said to be fault-tolerant then it is also considered highly available. However, there are situations in which a highly available application is not considered fault-tolerant.

There are various services, features, and techniques in AWS that you can use to implement a highly available and fault-tolerant architecture. You can ensure high availability by deploying your application to multiple Availability Zones or several AWS Regions. Auto Scaling can dynamically scale your systems depending on the incoming demand, and an active-active or active-passive failover policy can be implemented in Route 53 to reduce downtime. Amazon RDS offers Automated Snapshots, Read Replica, and Multi-AZ Deployments to strengthen your database tier to remove a single point of failure in your system. Alternatively, you can opt to use the Amazon Aurora Global database or DynamoDB Global tables for your globally-accessible applications. You can also leverage on the self-healing capabilities of AWS services to achieve fault tolerance.



Multi-AZ vs Multi-Region Architecture

The AWS Global Architecture is composed of an extensive worldwide network of Local Zones, Availability Zones (AZs), Regions, Regional Edge Caches, and Edge locations to enable organizations to scale and go global in minutes. Cloud computing has eliminated the boundaries of deploying and running applications in limited and costly on-premises data centers. Both small and large companies can now serve their customers more efficiently and cost-effectively using AWS. Businesses can opt to deploy non-prod environments in a single Availability Zone to save on cost or design a Multi-AZ architecture for its mission-critical applications. Deploying clusters of AWS resources across multiple Availability Zones can withstand an outage in one or more zones in your primary AWS Region. You can also design a Multi-Region architecture to serve your global users and comply with your disaster recovery requirements.

You can adopt these types of architectures both on your application and database tier. AWS Auto Scaling helps your web servers handle sudden or expected demand in your application. For your database, you can deploy one or more Read Replicas to offload the surge of incoming load from the primary database instance. For Multi-AZ, the OS patching or DB Instance scaling operations are applied first on the standby instance before triggering the automatic failover to reduce interruptions and downtime.

	MULTI-AZ DEPLOYMENTS	MULTI-REGION DEPLOYMENTS	READ REPLICAS
OBJECTIVE	High Availability	Disaster recovery and local performance	Scalability
REPLICATION TYPE	Non-Aurora: Synchronous replication; Aurora: Asynchronous replication	Asynchronous replication	Asynchronous replication
OPERABILITY	Non-Aurora: Only the primary instance is active; Aurora: All instances are active	All regions are accessible and can be used for reads	All read replicas are accessible and can be used for readscaling
BACKUPS	Non-Aurora: Automated backups are taken from standby; Aurora: Automated backups are taken from shared storage layer	Automated backups can be taken in each region	No backups configured by default
SCOPE	Always span at least two	Each region can have a	Can be within an



	Availability Zones within a single region	Multi-AZ deployment	Availability Zone, Cross-AZ, or Cross-Region
DB UPGRADE	Non-Aurora: Database engine version upgrades happen on primary; Aurora: All instances are updated together	Non-Aurora: Database engine version upgrade is independent in each region; Aurora: All instances are updated together	Non-Aurora: Database engine version upgrade is independent from source instance; Aurora: All instances are updated together
FAILOVER PROCESS	Automatic failover to standby (Non-Aurora) or read replica (Aurora) when a problem is detected	Aurora allows promotion of a secondary region to be the master	Can be manually promoted to a standalone database instance (Non-Aurora) or to be the primary instance (Aurora)

Disaster Recovery Objectives

There are two types of Objectives in Disaster Recovery:

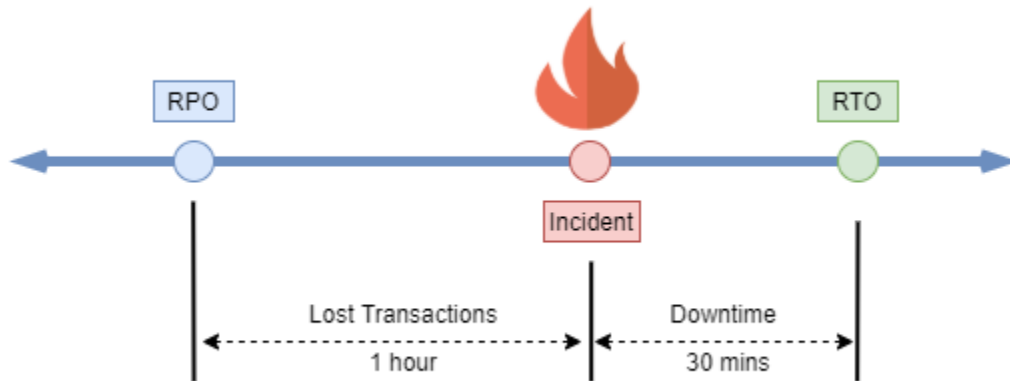
1. RTO or Recovery Time Objective
2. RPO or Recovery Point Objective

Basically, RTO refers to the time and RPO is all about the data point. RTO is the time it takes after a disruption to restore a business process to its service level, as defined by the operational level agreement (OLA).

For example, if a disaster occurs at 12 noon and the RTO is 3 hours, the Disaster Recovery process should restore service to the acceptable service level on or before 3 PM.

RPO, on the other hand, is the acceptable amount of data loss measured in time. For example, if a disaster occurs at 12 noon and the RPO is one hour, the system should recover all data that was in the system before 11 AM. The acceptable data loss should only be one hour, between 11:00 AM and 12:00 PM (noon). If you cannot recover a transaction or data made before 11:00 AM, say 10:30 or 9:30 then this means that you have failed your RPO.

RTO refers to the amount of time for the system to recover from an outage. RPO is the specific point, or state, of your data store that needs to be recoverable.





Amazon Route 53 Routing Policies

Most large organizations often have complex network structures to support their hybrid cloud architecture, distributed systems, and global users. They have several on-premises networks integrated with AWS Direct Connect or AWS VPN to connect to their AWS cloud resources across multiple Availability Zones and AWS Regions. To ensure business continuity, companies implement a disaster recovery plan that fails over the production traffic from the primary environment to the disaster recovery (DR) site.

Amazon Route 53 is a global Domain Name System (DNS) service that allows you to route traffic across various AWS Regions and external systems outside of AWS. It provides a variety of routing policies that you can implement to meet your required use cases and automatically monitor the state and performance of your applications, servers, and other resources using health checks. You can combine two or more routing policies to comply with your company's strict RTO and RPO requirements. It helps simplify the process of setting up an active-passive or active-active failover for your disaster recovery plan by intelligently routing traffic from your primary resources to the secondary resources based on the rules you specify.

Your globally distributed resources can either be considered active or passive. It's active if it accepts live production traffic and passive if it is just on standby, which will only be activated during a failover event. You can set up an active-active failover to improve your systems' fault tolerance and performance. By having several active environments, you can ensure the high availability and resiliency of your global applications. To set up an active-active failover, you can use a single or a combination of routing policies such as latency, geolocation, geoproximity, and others to configure Route 53 to respond to a DNS query using any healthy record.

Below are the different types of Amazon Route 53 routing policies that you can use in your architecture:

- **Simple** - This routing policy is commonly used for a single resource that performs a straightforward function for your domain records. For example, you can use this policy to route traffic from `tutorialsdojo.com` apex domain to an NGINX web server running on an Amazon EC2 instance.
- **Failover** – As the name implies, you can use this policy to set up an active-passive failover for your network architecture.
- **Geolocation** – Amazon Route 53 can detect the geographical location where the DNS queries originated. This routing policy lets you choose the specific resources that serve incoming traffic based on your users' geographic location. Say, you might want all user traffic from North America routed to an Application Load Balancer in the Singapore region. It works by mapping the IP addresses to geographical areas using the Extension Mechanisms for DNS version 0 (EDNS0).
- **Geoproximity** – This one is similar to the Geolocation routing policy except that it uses the Traffic Flow feature of Route 53 and has an added capability of shifting more or less traffic to your AWS services in one geographical location using a bias. It concentrates on the proximity of the resource in a given geographic area rather than its exact location.



- **Latency** – You can improve the application performance for the benefit of your global users by serving their requests from the AWS Region that provides the lowest latency. This routing policy is suitable for organizations that have resources in multiple AWS Regions.
- **Multivalue Answer** – Unlike the *Simple* routing policy, this type can route traffic to numerous resources in response to DNS queries with up to eight active records selected randomly. This policy is perfect if you are configuring an active-active failover for your network.
- **Weighted** – This policy allows you to route traffic to multiple resources in proportions that you specify. It acts as a load balancer that routes requests to a record based on the relative percentage of traffic or weight that you specified.

To monitor the system status or health, you can use Amazon Route 53 health checks to properly execute automated tasks to ensure the availability of your system. Health checks can also track the status of another health check or an Amazon CloudWatch Alarm.

Sources:

<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/routing-policy.html>

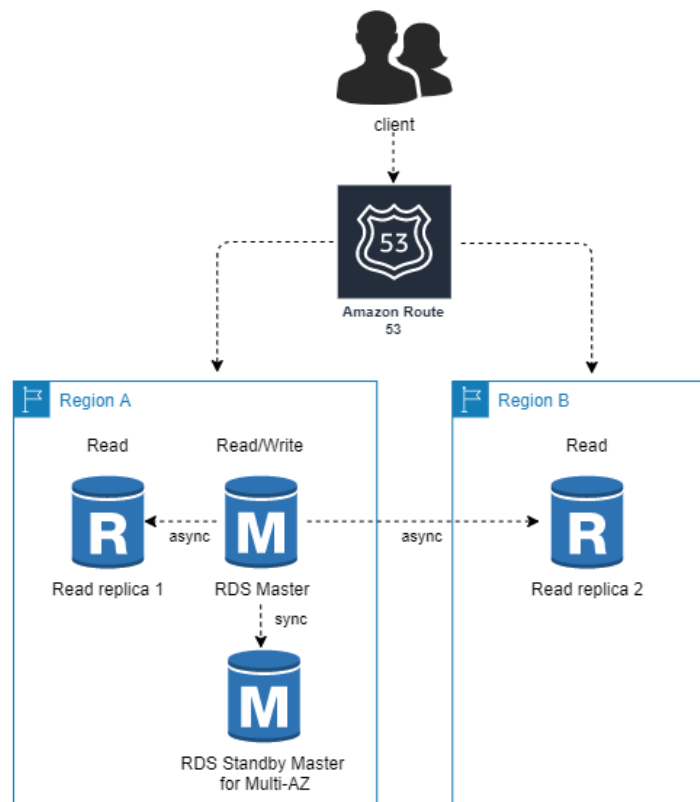
<https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/disaster-recovery-resiliency.html>

<https://tools.ietf.org/html/rfc2671>

Amazon RDS Disaster Recovery Strategies

AWS provides a plethora of database services and database engines that you can use for your applications. If you need a relational database type, you can use Amazon Aurora, Amazon RDS, or Amazon Redshift. For key-value data store, you can use an Amazon DynamoDB table that you can upgrade to DynamoDB Global to support your users worldwide. Amazon ElastiCache is perfect for in-memory data stores where you can use a Redis or Memcached engine. You can choose many other database services for document, wide column, graph, time series, and ledger-type databases.

Amazon Aurora is a fully managed MySQL and PostgreSQL-compatible relational database that provides high performance, availability, and scalability to your applications. Since it is a fully managed service, Amazon handles all of the underlying resources in your Aurora database and ensures that your cluster is highly available to meet your disaster recovery objectives and achieves fault tolerance. Aurora is excellent, but it has certain limitations, which compels companies to choose Amazon RDS as their database tier. Aurora does not use a native MySQL or PostgreSQL engine like RDS and can't directly run Oracle and Microsoft SQL Server databases unless you migrate them using the AWS Database Migration Service (AWS DMS) and AWS Schema Conversion Tool (AWS SCT). These constraints are the reasons why thousands of companies are still using Amazon RDS in their cloud architecture.





Amazon RDS is a managed database service. Unlike its "fully managed" counterparts, AWS does not entirely 'manage' or control all of the components of an Amazon RDS database compared to what it does for Amazon Aurora. If you launched an RDS database, you are responsible for making it highly scalable and highly available by deploying Read Replicas or using Multi-AZ Deployments configurations. You can also improve the data durability of your database-tier by taking automated or manual snapshots in RDS. For disaster recovery planning, you can set up a disaster recovery (DR) site to another AWS Region if the primary region becomes unavailable.

	DISASTER RECOVERY		COST	SCOPE
	RTO	RPO		
AUTOMATED BACKUPS	GOOD	BETTER	LOW	SINGLE REGION
MANUAL SNAPSHOTS	BETTER	GOOD	MEDIUM	CROSS-REGION
READ REPLICAS	BEST	BEST	HIGH	CROSS-REGION

An RDS Read Replica is mainly used to vertically scale your application by offloading the read requests from your primary DB instance. But do you know that it is tremendously useful for disaster recovery too? It uses asynchronous replication to mirror all the changes from the primary instance to the replica, located on the same or a different AWS Region. In contrast, the Multi-AZ Deployments configuration uses synchronous replication to keep its standby instance up-to-date. As its name implies, the standby instance is just on *standby*, meaning it neither accepts read nor write requests. This standby instance can only run on the same AWS Region, unlike a Read Replica with a cross-region capability. These unique attributes enable the Read Replica to provide the best RTO and RPO for your disaster recovery plan. You can deploy a Read Replica of your RDS database to another AWS Region to expedite the application failover if the primary region becomes unavailable without having to wait for hours to migrate and launch the automated/manual RDS snapshots to the other region.

You should also know the difference between automated backups, manual snapshots, and Read Replicas for your Business Continuity Plan (BCP). Amazon RDS has a built-in automated backups feature that regularly takes snapshots of your database and stores it on an Amazon S3 bucket that is owned and managed by AWS. The retention period of these backups varies between 0 and 35 days. It provides a low-cost DR solution for your database tier but is only limited to a single AWS Region. Manual snapshots are the ones that you manually take yourself, hence the name. In contrast with the automated backups, the S3 bucket where the snapshots are stored is owned by you, which means that you can control its retention period and deploy cross-region snapshots. Since you manage your own RDS snapshots, you can move these across AWS Regions using a shell script or a Lambda function run by Amazon EventBridge regularly.



The advantage of using Read Replicas over automated backups and manual snapshots is its near real-time synchronous replication. To put it into perspective, the replication time of the primary DB instance to the replica instance is less than a second! Compare that to the time required to move an RDS snapshot across another region and wait for it to start up. Hence, Read Replicas provide the fastest RTO and the best RPO for your architecture. The only setback is its high cost since you have to run your replica continuously.

Sources:

<https://aws.amazon.com/blogs/database/implementing-a-disaster-recovery-strategy-with-amazon-rds/>

https://d0.awsstatic.com/whitepapers/Backup_and_Recovery_Approaches_Using_AWS.pdf

https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_CopySnapshot.html



Auto Scaling Group with MinSize = 1 and MaxSize = 1

With Auto Scaling, you can set the number of EC2 instances that you need depending on the traffic of your application. However, there will be scenarios on the exam where you will need to set a fixed number of instances.

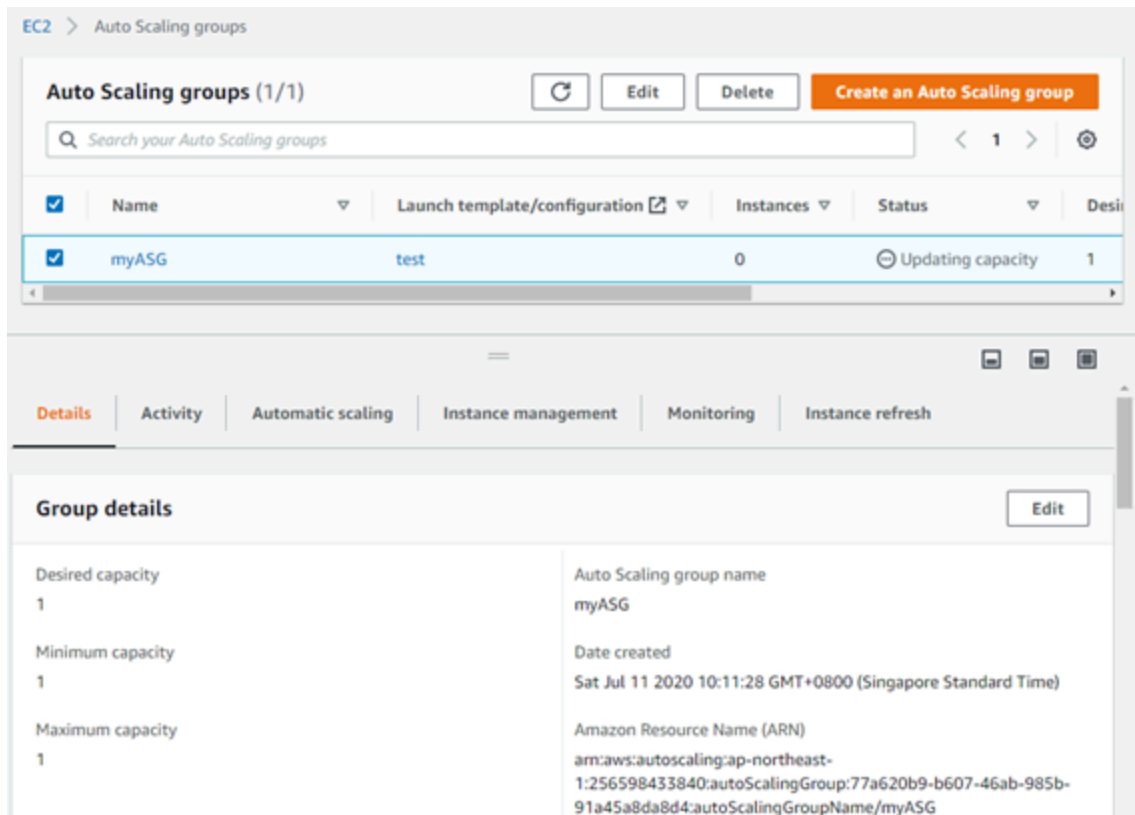
For example, you have a legacy application hosted on an EC2 instance. The application does not support working on a cluster of EC2 instances so it needs to run on a single EC2 instance and you need to make sure that the application is available and will heal itself even if that EC2 instance crashes.

In this scenario, you will create an Auto Scaling Group using the EC2 AMI in the Launch template and set the size of the group to Min 1 and Max 1. This ensures that only instances of the application are running. Auto Scaling will perform health checks for the EC2 instances periodically. If the EC2 instance fails the health check, Auto Scaling will replace the instance.

Hence, it will always be available and self-healing. This makes your application fault-tolerant.

To set the MinSize and MaxSize of the Auto Scaling group:

1. Go to the EC2 Console page, on the left pane, choose Auto Scaling Groups.
2. Select the check box next to your Auto Scaling group. The bottom pane will show information on the selected Auto Scaling group.



The screenshot shows the AWS Management Console interface for the 'Auto Scaling groups' section. At the top, there's a search bar and buttons for 'Edit', 'Delete', and 'Create an Auto Scaling group'. Below this is a table listing the Auto Scaling groups. The table has columns for 'Name', 'Launch template/configuration', 'Instances', 'Status', and 'Desired capacity'. One group, 'myASG', is listed with a launch template of 'test', 0 instances, and a status of 'Updating capacity'. Below the table, there are tabs for 'Details', 'Activity', 'Automatic scaling', 'Instance management', 'Monitoring', and 'Instance refresh'. The 'Details' tab is selected, showing the 'Group details' section. This section contains two columns of information: 'Desired capacity' (1), 'Minimum capacity' (1), and 'Maximum capacity' (1) on the left; and 'Auto Scaling group name' (myASG), 'Date created' (Sat Jul 11 2020 10:11:28 GMT+0800 (Singapore Standard Time)), and 'Amazon Resource Name (ARN)' (arn:aws:autoscaling:ap-northeast-1:256598433840:autoScalingGroup:77a620b9-b607-46ab-985b-91a45a8da8d4:autoScalingGroupName/myASG) on the right.

3. On the Details tab, view or change the current settings for minimum, maximum, and desired capacity. Set the Desired capacity to 1, Minimum capacity to 1 and Maximum capacity to 1. Make sure that you don't have any Automatic Scaling policies configured for this group.



Group size

×

Specify the size of the Auto Scaling group by changing the desired capacity. You can also specify minimum and maximum capacity limits. Your desired capacity must be within the limit range.

Desired capacity

Minimum capacity

Maximum capacity

Cancel

Update

Sources:

<https://docs.aws.amazon.com/autoscaling/ec2/userguide/as-maintain-instance-levels.html>
<https://docs.aws.amazon.com/autoscaling/ec2/userguide/asg-capacity-limits.html>

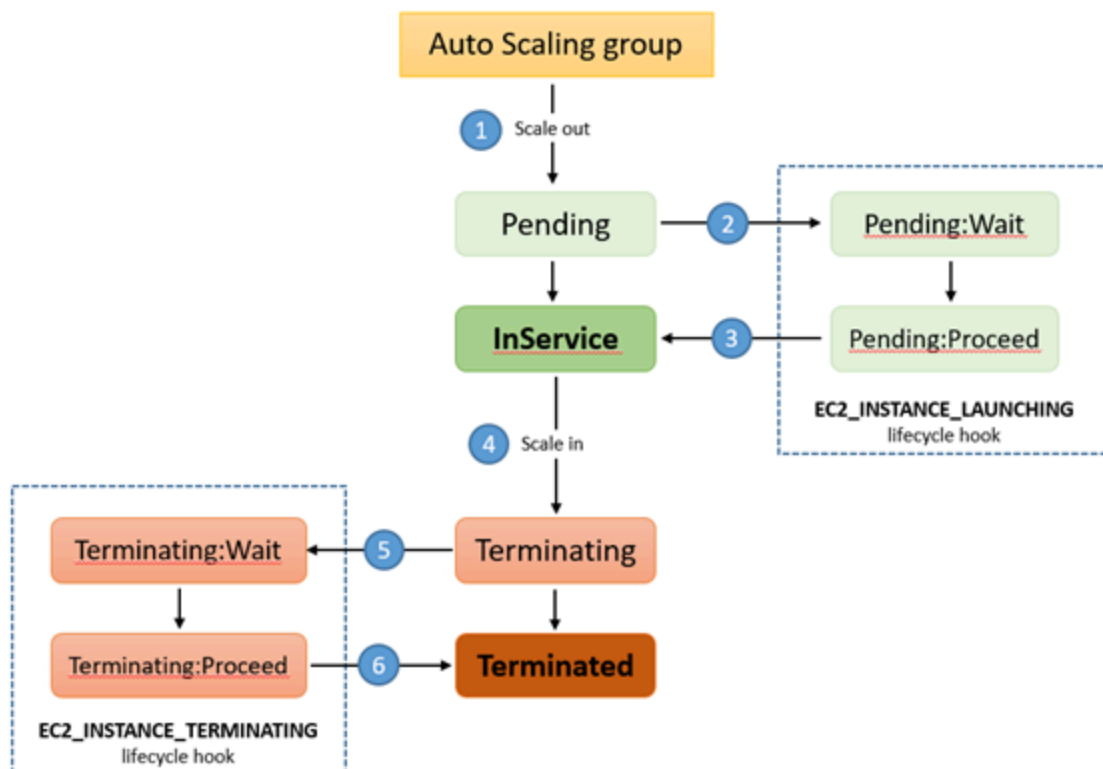
Auto Scaling Lifecycle Hooks

As your Auto Scaling group scales out or scales in your EC2 instances, you may want to perform custom actions before they start accepting traffic or before they get terminated. Auto Scaling Lifecycle Hooks allow you to perform custom actions during these stages. A lifecycle hook puts your EC2 instance into a wait state (**Pending:Wait** or **Terminating:Wait**) until your custom action has been performed or when the timeout period ends. The EC2 instance stays in a wait state for one hour by default, and then the Auto Scaling group resumes the launch or terminate process (**Pending:Proceed** or **Terminating:Proceed**).

For example, during the scale-out event of your ASG, you want to make sure that new EC2 instances download the latest code base from the repository and that your EC2 user data has been completed before it starts accepting traffic. You can use the **Pending:Wait** hook. This way, the new instances will be fully ready and will quickly pass the load balancer health check when they are added as targets.

Another example: during the scale-in event of your ASG, suppose your instances upload data logs to S3 every minute. You may want to pause the instance termination for a certain amount of time to allow the EC2 to upload all data logs before it gets completely terminated.

The following diagram shows the transitions between the EC2 instance states with lifecycle hooks.





DevOps Exam Notes:

During the paused state (either launch or terminate), you can do more than just run custom scripts or wait for timeouts. Amazon EventBridge can receive the scaling action and you can define a target to invoke a Lambda function that can perform custom actions, have it send a notification to your email via SNS, or trigger an SSM Run Command or SSM Automation to perform specific EC2 related tasks.

Sources:

<https://docs.aws.amazon.com/autoscaling/ec2/userguide/lifecycle-hooks.html>

<https://docs.aws.amazon.com/cli/latest/reference/autoscaling/put-lifecycle-hook.html>



Amazon EKS Auto Scaling

Kubernetes is the most widely used open-source container orchestration platform that helps DevOps engineers automate the deployment, scaling, and management of containerized applications. Kubernetes acts as a higher level of abstraction which allows developers to focus on writing code rather than managing infrastructure. With Kubernetes, developers can easily manage deploy and manage containerized workloads across distributed compute resources.

However, one of the challenges of managing a Kubernetes cluster on any deployment is configuring how the cluster scales up or down depending on demand. Adding or removing more compute nodes to the Kubernetes cluster is a major function to meet changing application demands.

In this section, we'll talk about the autoscaling options that are supported by Amazon Elastic Kubernetes Service (Amazon EKS):

- Kubernetes Cluster Autoscaler
- Karpenter

Kubernetes Cluster Autoscaler

The Kubernetes Cluster Autoscaler runs as a deployment on your Kubernetes cluster. It handles scheduling and scaling as it watches the Kubernetes cluster for failed pods or rescheduled pods. It then leverages Amazon EC2 Auto Scaling Groups in order to scale up or scale down the node groups.

The Kubernetes Cluster Autoscaler can be deployed on an existing Amazon EKS cluster. But it needs permission to examine and modify EC2 Auto Scaling Groups. Using IAM roles for service accounts or using an IAM OIDC provider is the recommended approach for providing proper permissions. The following are steps on deploying Cluster Autoscaler on Amazon EKS.

1. Create an IAM policy to allow the Cluster Autoscaler to describe and modify the capacity of the Auto Scaling groups. Use the below example as the cluster-autoscaler-policy.json.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "VisualEditor0",
      "Effect": "Allow",
      "Action": [
        "autoscaling:SetDesiredCapacity",
        "autoscaling:TerminateInstanceInAutoScalingGroup"
      ],
      "Resource": "*",
      "Condition": {
        "StringEquals": {
```




```
        "aws:ResourceTag/k8s.io/cluster-autoscaler/my-cluster":
            "owned"
            }
        },
        {
            "Sid": "VisualEditor1",
            "Effect": "Allow",
            "Action": [
                "autoscaling:DescribeAutoScalingInstances",
                "autoscaling:DescribeAutoScalingGroups",
                "ec2:DescribeLaunchTemplateVersions",
                "autoscaling:DescribeTags",
                "autoscaling:DescribeLaunchConfigurations",
                "ec2:DescribeInstanceTypes"
            ],
            "Resource": "*"
        }
    ]
}
```

2. Create the IAM policy using AWS CLI command.

```
aws iam create-policy \
    --policy-name AmazonEKSClusterAutoscalerPolicy \
    --policy-document file://cluster-autoscaler-policy.json
```

3. Use eksctl command to create an IAM role and attach the IAM policy. Update the name of the cluster and the name of the EKS policy created from the previous step.

```
eksctl create iamserviceaccount \
    --cluster=my-cluster \
    --namespace=kube-system \
    --name=cluster-autoscaler \
    --attach-policy-arn=arn:aws:iam::111122223333:policy/eksctl-my-cluster-n
odegroup-ng-xxxxxxx-PolicyAutoScaling \
    --override-existing-serviceaccounts \
    --approve
```

4. You can deploy the Cluster Autoscaler by downloading the YAML file from github and then using kubectl to apply the deployment. Ensure that the CLUSTER NAME is updated on the YAML file.

```
curl -O
https://raw.githubusercontent.com/kubernetes/autoscaler/master/cluster-autosca
ler/cloudprovider/aws/examples/cluster-autoscaler-autodiscover.yaml
kubectl apply -f cluster-autoscaler-autodiscover.yaml
```

5. Once deployment is applied, annotate the cluster-autoscaler service account with the ARN of the IAM role that you created previously.

```
kubectl annotate serviceaccount cluster-autoscaler \
-n kube-system \
```



```
eks.amazonaws.com/role-arn=arn:aws:iam::
```

```
111122223333:role/AmazonEKSClusterAutoscalerRole
```

6. Add an annotation to the cluster-autoscaler deployment using the kubectl patch command:

```
kubectl patch deployment cluster-autoscaler \
  -n kube-system \
  -p
'{"spec":{"template":{"metadata":{"annotations":{"cluster-autoscaler.kubernetes.io/safe-to-evict": "false"}}}}}'
```

7. Edit the Cluster Autoscaler deployment using the kubectl command and add the below container commands to ensure compute nodes are distributed across availability zones.

```
kubectl -n kube-system edit deployment.apps/cluster-autoscaler
--balance-similar-node-groups
--skip-nodes-with-system-pods=false
```

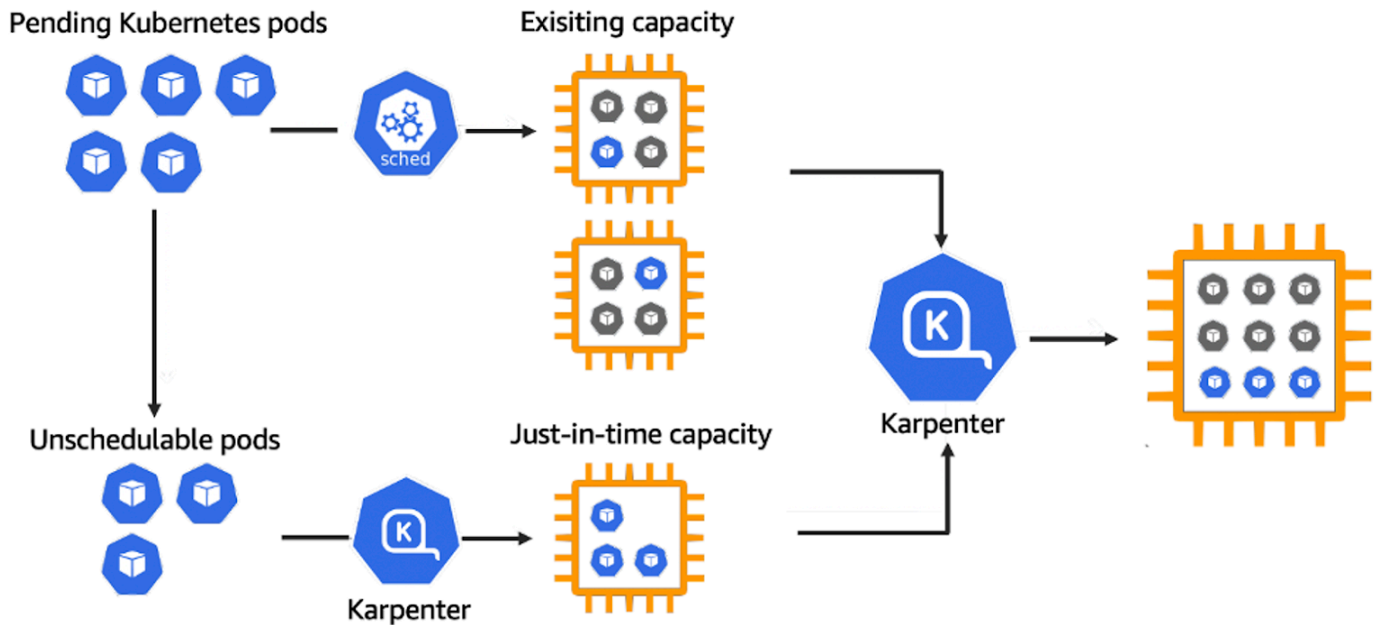
8. Ensure that the image deployed for Cluster Autoscaler is the same version of your Kubernetes cluster. If you need to update the image version, use the following eksctl command:

```
kubectl set image deployment cluster-autoscaler \
  -n kube-system \
  cluster-autoscaler=registry.k8s.io/autoscaling/cluster-autoscaler:v1.25.n
```

Karpenter

Karpenter is an open-source project from AWS built to handle node-provisioning on Kubernetes clusters. Karpenter simplifies cluster scaling by watching aggregate resource requests of unscheduled pods and making decisions to launch new nodes. It evaluates scheduling constraints requested by pods to properly provision compute nodes as they are required. Karpenter can also remove nodes when they are no longer needed or when there is excess capacity on the cluster.

See the diagram below for reference:



Karpenter can be deployed on an existing Amazon EKS cluster, and it also needs proper IAM permissions in order to describe and modify the sizes of Auto Scaling groups. The following steps will walk through the deployment of Karpenter on an existing EKS cluster.

1. Karpenter provides a CloudFormation template to create the required IAM policy and roles. Download the template, modify the parameters, and deploy the stack.

```
curl -fsSL https://karpenter.sh/v0.27.0
/getting-started/getting-started-with-eksctl/cloudformation.yaml >
karpenter-cfn.yaml \
&& aws cloudformation deploy \
  --stack-name "Karpenter-Demo" \
  --template-file karpenter-cfn.yaml \
  --capabilities CAPABILITY_NAMED_IAM \
  --parameter-overrides "Karpenter-Demo"
```

2. Use Helm Charts to deploy Karpenter to the existing EKS cluster. Add the helm repo and install karpenter to the EKS cluster.

```
$ helm repo add karpenter https://charts.karpenter.sh
$ helm repo update
$ helm upgrade --install --skip-crds karpenter karpenter/karpenter
--namespace karpenter \
  --create-namespace --set serviceAccount.create=false --version 0.5.0 \
  --set controller.clusterName=eks-karpenter-demo \
```



```
--set controller.clusterEndpoint=$(aws eks describe-cluster --name  
Karpenter-Demo --query "cluster.endpoint" --output json) \  
--wait
```

3. Create a provisioner to define how Karpenter provisions additional compute nodes. Using the default provisioner, Karpenter automatically discovers node properties such as instance types, zones, architectures, operating systems, and purchase types of instances. The spec:requirement section is optional.

```
cat <<EOF | kubectl apply -f -  
apiVersion: karpenter.sh/v1alpha5  
kind: Provisioner  
metadata:  
name: default  
spec:  
  requirements:  
    - key: "node.kubernetes.io/instance-type"  
      operator: In  
      values: ["m5.large", "m5.2xlarge"]  
    - key: "topology.kubernetes.io/zone" #If not included, all zones are  
      considered  
      operator: In  
      values: ["us-east-1a", "us-east-1b"]  
    - key: "karpenter.sh/capacity-type"  
      operator: In  
      values: ["spot", "on-demand"]  
  provider:  
    instanceProfile: KarpenterNodeInstanceProfile-eks-karpenter-demo  
    ttlSecondsAfterEmpty: 30  
EOF
```

Sources:

<https://karpenter.sh/>

<https://aws.amazon.com/blogs/aws/introducing-karpenter-an-open-source-high-performance-kubernetes-cluster-autoscaler/>

<https://github.com/kubernetes/autoscaler/blob/master/cluster-autoscaler/cloudprovider/aws/README.md>

<https://docs.aws.amazon.com/eks/latest/userguide/autoscaling.html>

Amazon EKS Networking

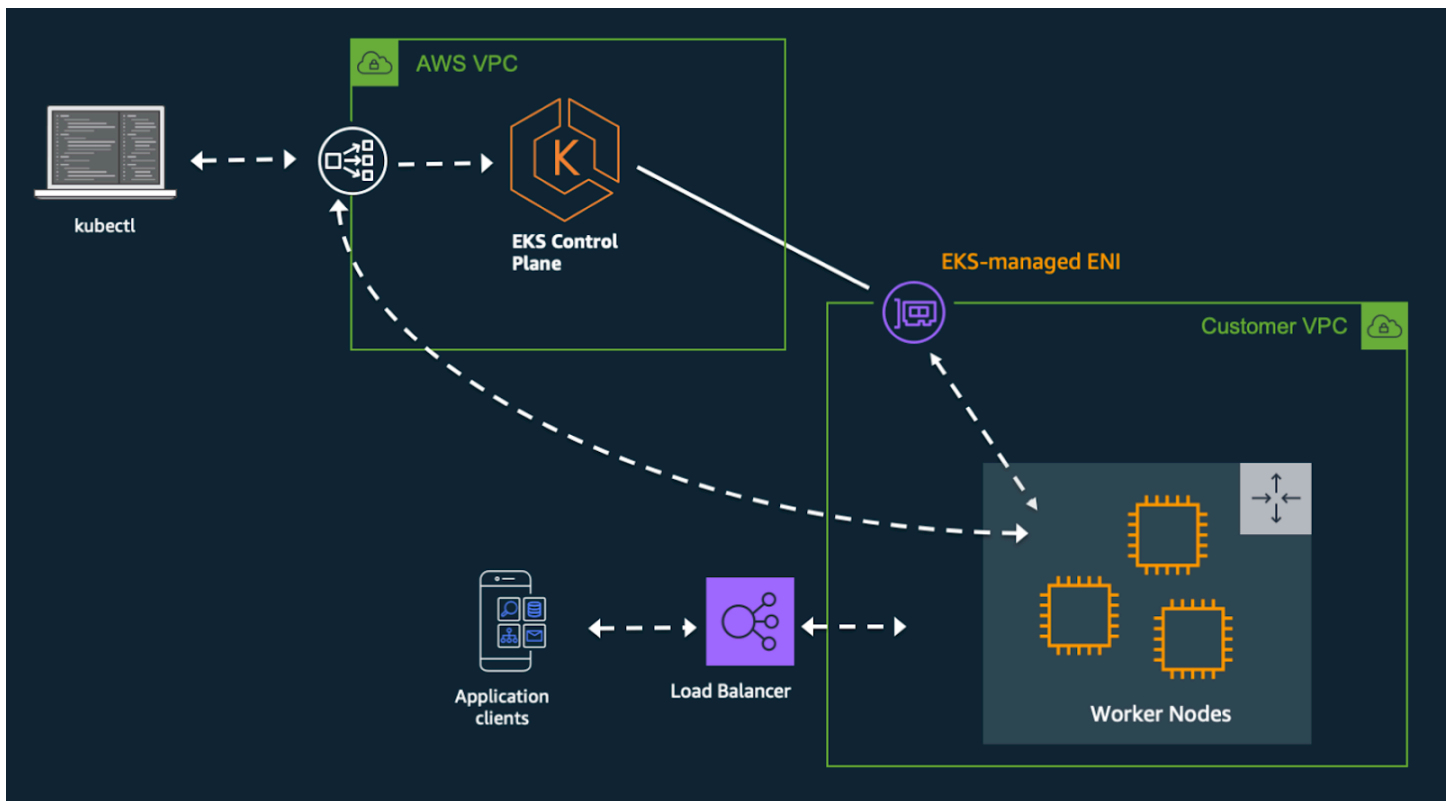
When deploying Kubernetes clusters on Amazon Elastic Kubernetes Service (Amazon EKS) a DevOps engineer must understand how to configure Amazon VPC and Kubernetes networking.

Amazon Elastic Kubernetes Service (Amazon EKS) uses a networking model similar to Kubernetes for its pod networking. This model is a flat overlay network that allows the pods to communicate with each other regardless of which physical node they are located in the cluster.

This section will provide an overview of how Amazon EKS Networking works and the Network add-ons that extend the features of your Kubernetes cluster.

There are two VPCs created when deploying a Kubernetes cluster on Amazon EKS. The first VPC is AWS managed and is not visible to the user. This VPC hosts the Kubernetes control plane, which contains the Kubernetes API Server. All user commands using kubectl or any API calls for the cluster are sent to the API server endpoint. The second VPC hosts the Kubernetes worker nodes, which could be Amazon EC2 instances. The worker nodes host the application workloads in pods. All worker nodes must be able to communicate to the API server.

See the diagram below for reference:





Amazon EKS creates a public API endpoint in which kubectl commands are received. AWS also creates EKS-managed Elastic Network Interfaces (ENIs) to allow internal communication with the nodes inside the VPC.

You can enable it whether you want to use the public endpoint which is reachable via the internet, or use the private endpoint from inside the VPC. You can enable both endpoints depending on your requirements.

By default, only the public endpoint is enabled and you can control who can connect to the endpoint using CIDR restrictions to allow only certain IPs to connect to the Kubernetes control plane. When you enable private endpoints, any traffic must come from within the cluster's VPC or connected network such as AWS VPN or AWS Direct Connect. You can use AWS PrivateLink to create a private connection from another VPC to Amazon EKS. You have to create an interface endpoint for AWS PrivateLink and enable the interface for each subnet in the VPC.

Amazon EKS networking add-ons

You can extend the functionality of Amazon EKS networking by using add-ons that provide more functionality depending on what is required by your application workloads. The following are some of the networking add-ons that you can install on your Amazon EKS cluster:

Amazon VPC CNI plugin

The Amazon VPC CNI (container network interface) plugin provides networking for pods. This plugin for Kubernetes is deployed on each node on the cluster as a daemon set. It creates an elastic network interface that is attached to each Amazon EC2 worker node. It also assigns a private IP address to each service and pod on the cluster. This plugin is helpful if you require individual IP addresses assigned to the network interface of each Amazon EC2 node.

This plugin is installed for new clusters in Amazon EKS. If your EKS cluster does not have it installed, you can create the Amazon EKS plugin using the AWS CLI command below.

Replace the fields with the appropriate values

```
aws eks create-addon --cluster-name my-cluster --addon-name vpc-cni
--addon-version v1.12.5-eksbuild.2 \
--service-account-role-arn
arn:aws:iam::111122223333:role/AmazonEKSVPCCNIRole
```

You can confirm if the add-on is applied on the EKS cluster using the below command:

```
aws eks describe-addon --cluster-name my-cluster --addon-name vpc-cni
--query addon.addonVersion --output text
```



AWS Load Balancer Controller add-on

As we design for highly available systems, pods deployed on the Kubernetes cluster are spread through multiple nodes. And to distribute traffic on the pods, we can use Amazon Application Load Balancers. The AWS Load Balancer Controller handles the provisioning of ALBs as required by the applications deployed on the Amazon EKS cluster.

Amazon EKS creates an ALB if you create a Kubernetes Ingress object while an NLB is created if you use a service type LoadBalancer.

When installing this add-on you need to create an IAM policy and role that have permission to create ALB's on your behalf.

1. Download the JSON file that contains the required IAM permissions.

```
curl -O
https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-controller/v2.4.7/docs/install/iam_policy_us-gov.json
```

Create the IAM policy using the AWS CLI command:

```
aws iam create-policy \
  --policy-name AWSLoadBalancerControllerIAMPolicy \
  --policy-document file://iam_policy.json
```

2. Create an IAM service account role based on the created IAM policy from the previous step

```
eksctl create iamserviceaccount \
  --cluster=my-cluster \
  --namespace=kube-system \
  --name=aws-load-balancer-controller \
  --role-name AmazonEKSLoadBalancerControllerRole \
  --attach-policy-arn=arn:aws:iam::111122223333:policy/AWSLoadBalancerControllerIAMPolicy \
  --approve
```

3. Use Helm charts to install the AWS Load Balancer Controller add-on to the EKS cluster. Use the following helm commands:

```
helm repo add eks https://aws.github.io/eks-charts
helm repo update
helm install aws-load-balancer-controller
eks/aws-load-balancer-controller \
  -n kube-system \
  --set clusterName=my-cluster \
  --set serviceAccount.create=false \
  --set serviceAccount.name=aws-load-balancer-controller
```



4. . You can verify that the add-on is installed using the helm command:

```
kubectl get deployment -n kube-system aws-load-balancer-controller
```

CoreDNS Amazon EKS add-on

The CoreDNS Amazon EKS add-on is an extensible DNS server used to provide more flexibility for providing name resolution for all pods in the EKS cluster. If you don't want to be limited by the functionality of Amazon Route or if you need a fast DNS service inside the Kubernetes cluster that can resolve internal and external DNS queries, CoreDNS can provide features depending on specific requirements. It also provides Service Discovery to automatically add/remove DNS entries for new/delete pods or services.

Once deployed, this add-on creates two replicas of CoreDNS on the cluster regardless of how many nodes are active. Amazon EKS automatically installs CoreDNS as a self-managed add-on for every cluster.

If your cluster does not have it installed, you can create the add-on and apply it on the EKS cluster using the following commands:

```
aws eks create-addon --cluster-name my-cluster --addon-name coredns  
--addon-version v1.9.3-eksbuild.2
```

Replace the version with the specific version needed for your cluster.

Once the add-on is installed, you can check it using the following command:

```
aws eks describe-addon --cluster-name my-cluster --addon-name coredns  
--query addon.addonVersion --output text
```

Kubernetes kube-proxy add-on

kube-proxy is the default networking add-on for Kubernetes. It is deployed on every Amazon EC2 compute node on your EKS cluster. This plugin is not deployed on EKS Fargate clusters.

kube-proxy runs as daemon set in the Kubernetes cluster, and it maintains network rules on nodes, thus, allowing communication between pods, not just inside the cluster but also for external communication.

Using this add-on is no longer recommended by AWS so we won't discuss further details for this add-on. AWS recommends using the Amazon EKS type of the add-on to your cluster instead of kube-proxy as they are robust and provide extended functionalities.

Calico network policy engine add-on

With Calico network policy enforcement, you can implement network segmentation and tenant isolation. This is useful in multi-tenant environments where you must isolate tenants from each other or when you want to create separate environments for development, staging, and production.

EKS has built-in support for Calico, providing a robust implementation of the full Kubernetes Network Policy API. EKS users wanting to go beyond Kubernetes network policy capabilities can make full use of the Calico Network Policy API.



You can also use Calico for networking on EKS in place of the default AWS VPC networking without the need to use IP addresses from the underlying VPC. Calico is not supported when using Fargate with Amazon EKS.

Create the calico-operator namespace.

```
kubectl create namespace calico- system
```

Install the Tigera Calico operator and custom resource definitions using the Helm chart:

```
helm install calico projectcalico/tigera-operator --version v3.25.0  
--namespace calico- system
```

or if you created a values.yaml above:

```
helm install calico projectcalico/tigera-operator --version v3.25.0 -f  
values.yaml --namespace calico- system
```

Confirm that all of the pods are running with the following command.

```
watch kubectl get pods -n calico-system
```

Sources:

<https://docs.aws.amazon.com/eks/latest/userguide/eks-networking.html>

<https://aws.amazon.com/blogs/containers/de-mystifying-cluster-networking-for-amazon-eks-worker-nodes/>

<https://docs.aws.amazon.com/eks/latest/userguide/vpc-interface-endpoints.html>

<https://docs.aws.amazon.com/eks/latest/userguide/eks-networking-add-ons.html>

<https://docs.aws.amazon.com/eks/latest/userguide/aws-load-balancer-controller.html>



Automated Patching for Hybrid Environments

Hybrid environments pose the challenge of managing both on-premises data center servers and cloud-hosted servers. Hybrid environments are common for companies that want to augment their on-premises data center capacity with the power of AWS cloud. One of the major security tasks done for hybrid environments is to ensure that security patches are installed in a timely manner across all environments.

AWS Systems Manager offers several tools that help automate and standardize applying OS patches across all your environments.

To configure patching for on-premises servers in a hybrid environment, the first step is to enroll the servers to AWS Systems Manager. Follow these steps to enroll servers to AWS SSM.

1. Create an IAM service role for a hybrid environment as they are required in order to communicate with the AWS Systems Manager service. AWS has a managed IAM policy already created for this. Just attach the **AmazonSSMManagedInstanceCore** to the IAM role with the Trusted entity type: **Systems Manager**.
2. On the System Manager console, create a Hybrid Activation and specify the role that is created on the first step.

AWS Systems Manager > Activations > Create activation

Create activation

Activation setting

Create a new activation. After you complete the activation, you receive an activation code and ID on hybrid and on-premises servers or virtual machines. [Learn more](#)

Activation description- *Optional*

TD-hybrid-activation

Maximum 256 characters.

Instance limit

Specify the total number of servers and VMs that you want to register with AWS. The maximum i

10

Maximum number is 1000.

3. This will give you an Activation code and Activation ID that you will use for the next step.

✔ You have successfully created a new activation. Your activation code is listed below. **Copy**

Activation Code iHxX/SJlH3Llw

Activation ID 9d5d5359-9281-4fdf-91e0-

You can now install amazon-ssm-agent and manage your instance using Run Command.

4. Install the SSM agent on the on-premises server and associate them using the Activation Code and Activation ID. For example, install the SSM agent on an Ubuntu OS:

```
sudo snap install amazon-ssm-agent --classic
sudo systemctl stop snap.amazon-ssm-agent.amazon-ssm-agent.service
sudo /snap/amazon-ssm-agent/current/amazon-ssm-agent -register -code
"activation-code" -id "activation-id" -region "region"
sudo systemctl start snap.amazon-ssm-agent.amazon-ssm-agent.service
```

More detailed instructions for installing on Linux OS can be found [here](#), and Windows OS can be found [here](#).

Managed instances for hybrid activation will have a prefix of **mi-xxxxx** under Fleet Manager on the Systems Manager page.

To configure standardized patching, you can configure **Maintenance Windows** under the AWS Systems Manager page. Maintenance Windows helps you define a schedule for when to perform potentially disruptive actions on your nodes such as patching an operating system, updating drivers, or installing software or patches. It also allows you to set an **Automation** task that will run during the specified maintenance window.

To create an automated patching task under a specific maintenance window, follow the below steps.

1. Create a Maintenance window on the AWS Systems Manager page.



AWS Systems Manager > Maintenance Windows > Create maintenance window

Create maintenance window

A maintenance windows lets you specify when a target set of managed instances should install maintenance activities. Specify the details below to create a new maintenance window:

Provide maintenance window details

Name

Type a name for this maintenance window.

TD-maintenance-window

It has to be between 3 and 128 characters. Valid characters contain the following: a-z, A-Z, 0-9, and . _ -

2. Specify the time for your maintenance window, which is preferably out-of-business hours. Click [Create maintenance window](#).

Schedule

Specify with

- ☒ Cron schedule builder
- ☐ Rate schedule builder
- ☐ CRON/Rate expression

☐ Default
Window starts every 30 minutes

☐ Hourly
Window starts on custom hourly rate

☒ Daily
Window starts on custom daily rate

Every Day ▼ at 02:00

Duration

Maintenance window duration

4 hours

Value from 1 to 24.

Stop initiating tasks

Time to stop starting scheduled task before maintenance window ends

1 hour before the window closes

3. Select the created maintenance window and click > Actions > Register targets to set which instances will be associated with this maintenance window. You can choose to specify targets using tags, resource groups, or selecting instances manually.

Targets

Targets are the AWS resources, such as instances, that you register with the maintenance window.

Target selection

Choose a method for selecting targets.

- ☐ Specify instance tags
Specify one or more tag key-value pairs to select instances that share those tags.
- ☒ Choose instances manually
Manually select the instances you want to register as targets.
- ☐ Choose a resource group
Choose a resource group that includes the resources you want to target.

i-0aa7aac85e18fff0d X

Instances

<1>

☒	Node ID	Source type	Source ID	Name	Ping sta
☒	i-0aa7aac85e18fff0d	AWS::EC2::Instance	i-0aa7aac85e18fff0d		Online

4. Select the created maintenance window and click > Actions > Register Automation task to set an OS patching task for this window time.

Select an Automation Document to patch the instance.

Automation document

Document: AWS-PatchInstanceWithRollback

Q			< 1 >
Document name prefix: Equals: AWS-PatchInstanceWithRollback X			
Clear filters			
	Name	Owner	Platform types
☒	AWS-PatchInstanceWithRollback	Amazon	Windows, Linux, MacOS



Set the proper targets

Targets

Targets are the instances you would like to associate with this document. You can choose to target by both mar

Strict Targets

- ☒ Selecting registered target groups
- ☐ Task target not required

6f55aaf7-4be0-4b5e-af57-9b4a273ccc23 X

☒	Window target ID	Name	On
☒	6f55aaf7-4be0-4b5e-af57-9b4a273ccc23	TD-target-nodes	-

5. Click Register Automation task to apply the changes.

This Automation task will apply the OS patches during the specified maintenance window.

Sources:

<https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-managedinstances.html>

<https://docs.aws.amazon.com/systems-manager/latest/userguide/sysman-managed-instance-activation.html>

<https://docs.aws.amazon.com/systems-manager/latest/userguide/sysman-install-managed-linux.html>

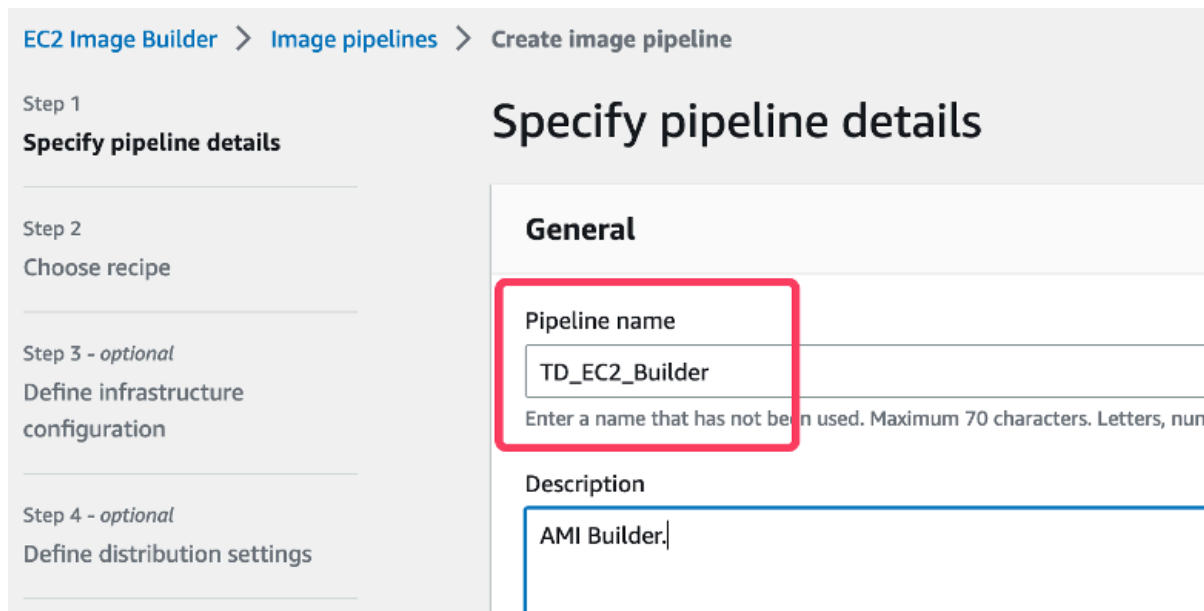
Amazon EC2 Image Builder

Most organizations that deploy virtual machines create a golden image used to host different applications in their data center. The golden image acts as a base template for a virtual machine and generally contains the latest OS patches, proper security configurations, and monitoring or automation agents that the company requires. Creating and updating these golden images can be time-consuming, especially for large organizations where the golden image must be shared across multiple AWS accounts.

AWS provides an automated way to create, manage, customize, and deploy EC2 server images using the **EC2 Image Builder service**. With EC2 Image Builder, you can configure pipelines to automate the installation of patches and updates, as well as configure specific settings on the EC2 instance before generating an Amazon Machine Image (AMI). You can distribute this AMI to your AWS regions, or you can authorize other AWS accounts, organizations, and OUs to launch it from your account.

The following procedure outlines how to create an AMI using EC2 Image Builder and share the AMI to AWS accounts under your AWS Organizations using AWS Resource Access Manager (RAM).

1. Go to the EC2 Image Builder page on the AWS Management Console.
2. Click Create image pipeline button and specify the details of your pipeline.



EC2 Image Builder > Image pipelines > Create image pipeline

Step 1
Specify pipeline details

Step 2
Choose recipe

Step 3 - optional
Define infrastructure configuration

Step 4 - optional
Define distribution settings

Specify pipeline details

General

Pipeline name

TD_EC2_Builder

Enter a name that has not been used. Maximum 70 characters. Letters, numbers, and hyphens are allowed.

Description

AMI Builder.

3. You can set a schedule to run the pipeline at specified intervals, depending on your requirements.

Build schedule [Info](#)

Schedule options



Schedule builder

Automatically run the pipeline using a job schedule. The default schedule is every Monday at 9:00 UTC.



CRON expression

Automatically run the pipeline using a syntax that specifies the time and intervals to run it.



Manual

The pipeline will run when you initiate it.

Run pipeline every **Frequency** **Week** ▼ on **Day** **Monday** ▼ at **Time** **09:00** **Time zone** **UTC**

- Choose a recipe to generate the AMI or Docker image you want to create.

Image type

Choose the image type

Output type



Amazon Machine Image (AMI)



Docker image





General

Enter a name and description for this configuration.

Name

TD_Golden_Image

Maximum of 128 characters. Letters, numbers, spaces, -, and _ are allowed

Version

1.0.0

Use the format:
major.minor.patch

Description - *optional*

TD_Golden_Image

5. Select a base image to use – Amazon Linux.

Base image [Info](#)

Select image

You can select the base image from a list of Image Builder managed images or Amazon Machine has access to or import a virtual image.

☒ **Select managed images**

Image Builder managed images created by you, shared with you, or provided by AWS.

☐ **AWS Marketplace images [New](#)**

Choose from images you subscribed to in AWS Marketplace.

[Learn more](#) 

☐ **Enter custom AMI ID**

The AWS Systems Manager Agent (SSM Agent) must be pre-installed in the selected AMI.

☐ **Import base image**

Import from your VM into Image Builder and use the converted image as the base image in your recipe.

Image Operating System (OS)

Image Builder supports Amazon Linux, Windows, Ubuntu, CentOS, RHEL, and SLES.

☒ **Amazon Linux**

Amazon Linux 2



☐ **Windows**

Windows Server 2012R2, 2016, 2019, 2004, 20H2, and 2022



- Under the Components section, choose which additional packages you want to install.



Components [Info](#)

Components are software scripts that define the custom configuration for an image. Components cannot be modified or deleted after a recipe is created. Automatic version choices are provided for each component. A maximum of 20 components (including build and test) can be applied to a recipe.

Step 1: Choose build components to produce the desired output AMI

Build components are software scripts that define a sequence of steps for downloading, installing, and configuring software packages. They also define validation steps.

Build components - Amazon Linux (32)  [Create build component](#)

 Amazon-managed ▼

 1 2 3 4 

☒	Name ▲	Description				
☒	amazon-cloudwatch-agent-linux	Installs the latest version of the Amazon CloudWatch agent. This component installs only the agent. You must take additional steps to configure and use the Amazon CloudWatch agent. For more information, see the documentation at https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/install-CloudWatch-Agent-on-EC2-Instance.html. <table><tr><td>Owner</td><td>ARN</td></tr><tr><td>Amazon</td><td>arn:aws:imagebuilder:ap-southeast-1:aws:component/amazon-</td></tr></table>	Owner	ARN	Amazon	arn:aws:imagebuilder:ap-southeast-1:aws:component/amazon-
Owner	ARN					
Amazon	arn:aws:imagebuilder:ap-southeast-1:aws:component/amazon-					

- Click Next to select the infrastructure on where you want EC2 Image Builder to run your pipeline. This is an optional step.



Region settings [Info](#)

The AMI will be distributed to the current Region by default.

Region 1

Refer to the help content for guidance on Region settings input parameters.

Region	KMS key	Target accounts Info	Output AMI name Info
ap-southeast-1 (...	Do not enable	Enter an AWS account ID	golden_image
		123123123123	Max 100 characters

► [AMI sharing](#) [Info](#)

► [License configuration](#) [Info](#)

► [Launch template configuration](#) [Info](#)

► [Faster launch configuration \(for Windows AMI only\)](#) [Info](#)

Region 2

Refer to the help content for guidance on Region settings input parameters.

Remove

Region	KMS key	Target accounts Info	Output AMI name Info
us-east-1	Do not enable	Enter an AWS account ID	golden_image
		456456456456	Max 100 characters

- Click Next to select the Distribution setting in order to share the AMI on AWS Accounts or regions. Specify the region and the AWS account IDs that you want to share your AMI.
- Once populated, click Next to Review and Create pipeline.

Another way to automate sharing of the AMI to the AWS Organization is by using **AWS Resource Access Manager (RAM)**. Share the AWS resources that you create in one AWS account with all of the roles and users in that same account or with other AWS accounts. If you manage your account using AWS Organizations, you can share resources with all the other accounts in the organization or only those accounts contained by one or more specified organizational units (OUs).



Follow the steps below to share the AMI with other accounts in the organization.

1. Go to Resource Access Manager in the AWS Management Console and click Create resource share.
2. Specify the details of the share and select the Image Builder Images under the Resources that you want to share.

Specify resource share details

Enter a name for the resource share and select the resources that you want to share.

Resource share name

Name
Provide a descriptive name for the resource share.

Resources - optional

Choose the resources to add to the resource share.

Select resource type

☐	ARN	Name	Version	Platform	Owner
No resources found					

3. Click Next to proceed and associate permissions to your AWS accounts within your organization.



Grant access to principals

Specify the principals that are allowed access to the shared resources. A principal can be any of the following: An entire organization, an AWS account, IAM role, or IAM user.

Principals - optional

☐ Allow sharing with anyone

You can share resources with any AWS accounts, roles, and users. If you are in an organization, you can also share with the entire organization or organizational units in that organization.

☒ Allow sharing only within your organization

You can share resources with the entire organization, organizational units, or AWS accounts, roles, and users in that organization.

Principals

You can add multiple principals of different types.

AWS account

Enter an AWS account ID

An AWS account ID is a 12-digit number.

4. Click Next to Review the details and click Create resource share to save the changes.

The newly created Amazon Machine Images (AMI) from EC2 Image Builder should now be available for use to other AWS Accounts within your organization.

Sources:

<https://docs.aws.amazon.com/imagebuilder/latest/userguide/getting-started-image-builder.html>

<https://docs.aws.amazon.com/imagebuilder/latest/userguide/start-build-image-pipeline.html>

<https://docs.aws.amazon.com/imagebuilder/latest/userguide/what-is-image-builder.html>

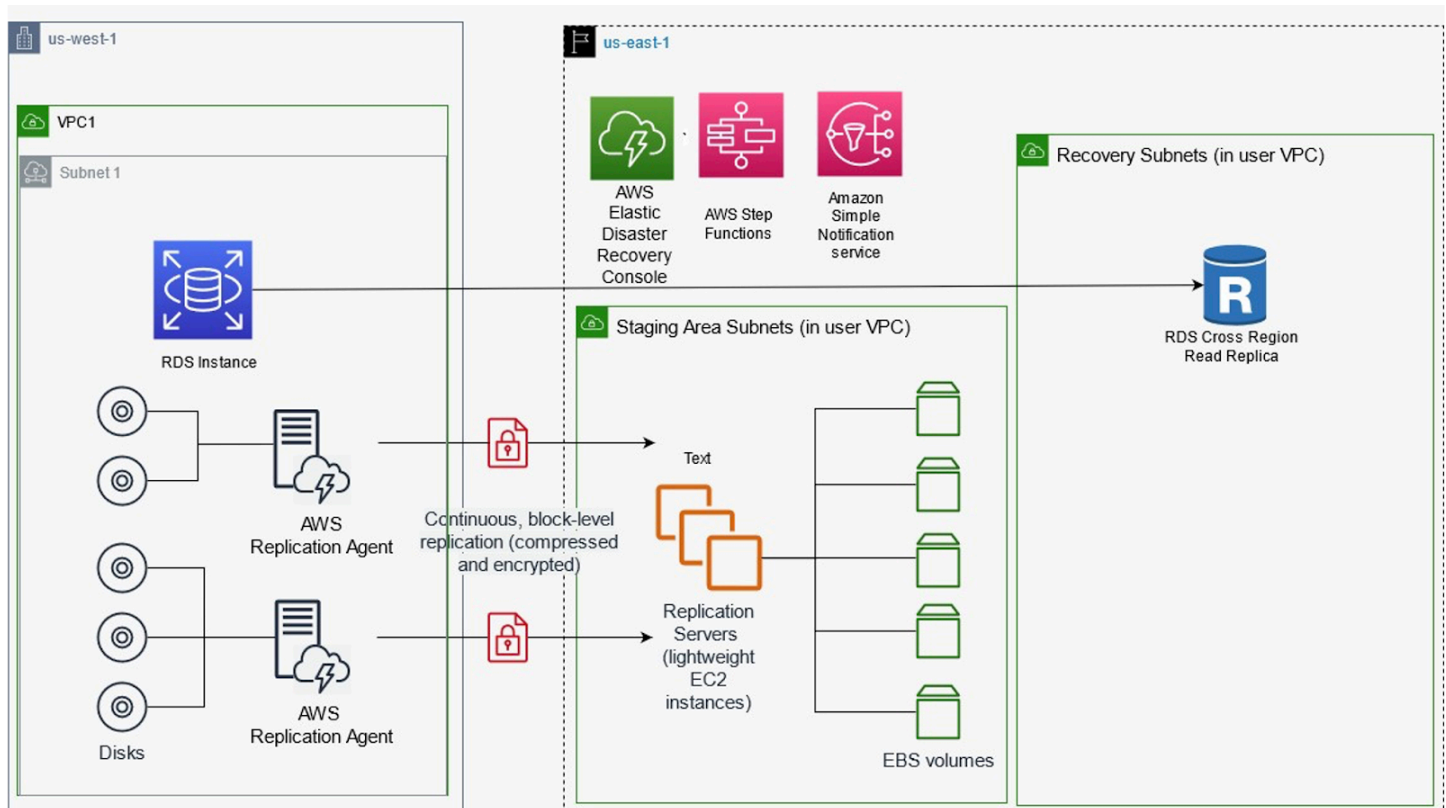
<https://docs.aws.amazon.com/ram/latest/userguide/what-is.html>

Automated RDS Read Replica Promotion

Disaster recovery strategies generally involve copying data to another region and a way to continuously update the data on those backups. The backups can either be stored on cold storage or are asynchronously updated depending on how fast the recovery requirements are in the event of a disaster.

Amazon Aurora MySQL has a built-in solution to create **Cross Region Read Replicas**, which asynchronously replicate data from the RDS instance on the primary region. In the event of a disaster in the primary region, a Read Replica in the secondary region can be promoted to a primary instance. This strategy allows for a swift recovery of your RDS instances, which helps meet the business continuity goals or compliance requirements. Promoting an RDS Read Replica to a primary database requires manual intervention. However, this can be automated by using **AWS Lambda** functions invoked by **AWS Step Functions**.

As a reference architecture, please refer to the diagram below:

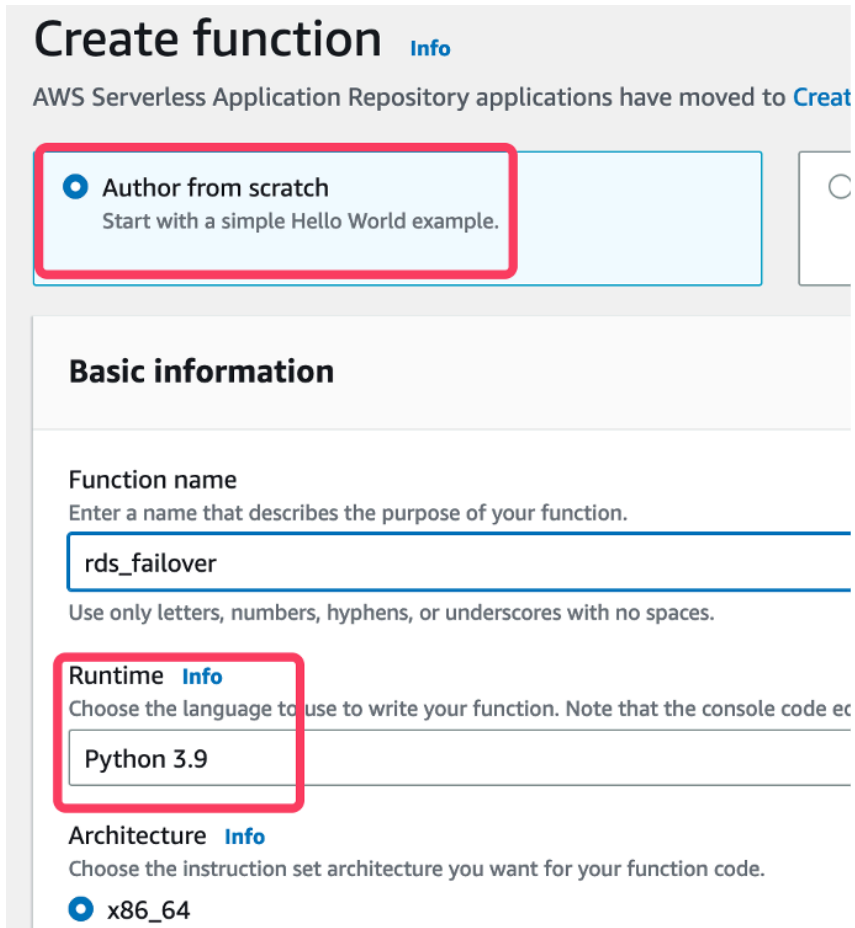


On the diagram, the **AWS Elastic Disaster Recovery Console** is used to replicate the **Amazon EC2 instances** to the secondary region. An Amazon RDS primary instance is being replicated to the secondary region by creating a Cross Regions Read Replica.

In the event of a disaster on the primary region, the **AWS Step Functions State Machine** can be executed to invoke AWS Functions that - promote the Read Replica instance to the primary instance, check if promotion to primary is successful, and deploy Amazon EC2 instances on the secondary region. **Amazon EventBridge** can be used to check for events when the Cross Region Read Replica sends notifications on an **Amazon SNS** Topic.

The following solution creates three AWS Lambda functions.

1. Navigate to the AWS Lambda console and click **Create a function**.
2. Choose **Author from scratch** with the runtime Python 3.9.



Create function [Info](#)

AWS Serverless Application Repository applications have moved to [Create](#)

☒ **Author from scratch**
Start with a simple Hello World example.

Basic information

Function name
Enter a name that describes the purpose of your function.

Use only letters, numbers, hyphens, or underscores with no spaces.

Runtime [Info](#)
Choose the language to use to write your function. Note that the console code ec

Architecture [Info](#)
Choose the instruction set architecture you want for your function code.
☒ **x86_64**

3. Click **Create function** to edit the function. Add the below snippet to your Python code that can promote the Cross Region Read Replica as a primary instance. Click **Deploy** to save the changes.

```
import boto3

rds = boto3.client('rds')

secondary = "rds-drs-crrr-cluster-1"
```




```
def lambda_handler(event, context):
    failover = rds.promote_read_replica_db_cluster(
        DBClusterIdentifier= secondary
    )
    response = "Promoting {} to primary".format(secondary)
    return response
```

4. Repeat Step 1 and 2 to create another function named **rds_status_check**. Add the below snippet to your Python code. This checks if the read replica promotion is successful.

```
import boto3
import json

rds = boto3.client('rds')

def lambda_handler(event, context):
    response = rds.describe_db_clusters(
        DBClusterIdentifier='rds-drs-crrr-cluster-1'
    )
    status = response['DBClusters'][0]['Status']
    responseJSON = {"Status": status}
    return responseJSON
```

5. Repeat Step 1 and 2 to create another function named **drs_failover**. This function will trigger the creation of Amazon EC2 instances leveraging the setup from the AWS Elastic Disaster Recovery console.

```
.....
#Make a list of all source server IDs
serverItems = []
for i in response_iterator:
    serverItems += i.get('items')
serverList = []
for i in serverItems:
    serverList.append(i['sourceServerID'])

#Failover all the source servers
for i in serverList:
    failover = drs.start_recovery(
        isDrill=False,
        sourceServers=[
            {
                'sourceServerID': i
            },
        ]
    )
.....
```

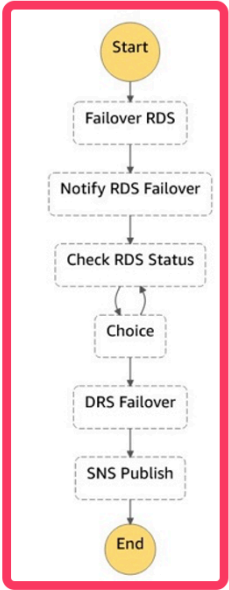
6. Create an AWS Step Function State Machine to invoke the Lambda functions.

Definition
 Define your workflow using [Amazon States Language](#). Test your data flow with the new [Data Flow Simulator](#).

Generate code snippet Format JSON

```

84         "Lambda.AWSLambdaException",
85         "Lambda.SdkClientException"
86     ],
87     "IntervalSeconds": 2,
88     "MaxAttempts": 6,
89     "BackoffRate": 2
90 }
91 },
92 "Next": "SNS Publish"
93 },
94 "SNS Publish": {
95     "Type": "Task",
96     "Resource": "arn:aws:states:::sns:publish",
97     "Parameters": {
98         "Message.$": "$",
99         "TopicArn": "arn:aws:sns:          :drs-invoked"
100     },
101     "End": true
102 }
103 }
104 }
```



```

graph TD
    Start((Start)) --> FailoverRDS[Failover RDS]
    FailoverRDS --> NotifyRDS[Notify RDS Failover]
    NotifyRDS --> CheckRDS[Check RDS Status]
    CheckRDS --> Choice{Choice}
    Choice --> DRSFailover[DRS Failover]
    DRSFailover --> SNSPublish[SNS Publish]
    SNSPublish --> End((End))
    
```

By configuring the State Machine, you can orchestrate the Disaster Recovery flow with a single button. Upon execution, the first Lambda function is invoked, and the Cross Region Read Replica instance is promoted to a primary instance. An Amazon EventBridge rule will detect the RDS event, it will send a message to an Amazon SNS Topic to will notify the subscribers about the disaster recovery event. The second Lambda function will check for the RDS promotion status, and if successful, the third Lambda function will be invoked. It will provision the Amazon EC2 instances on the secondary region to prepare them to accept traffic.

Sources:

<https://lifesciences-resources.awscloud.com/aws-storage-blog/automating-disaster-recovery-of-amazon-rds-and-amazon-ec2-instances>
<https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/AuroraMySQL.Replication.CrossRegion.html>
<https://docs.aws.amazon.com/step-functions/latest/dg/tutorial-creating-lambda-state-machine.html>

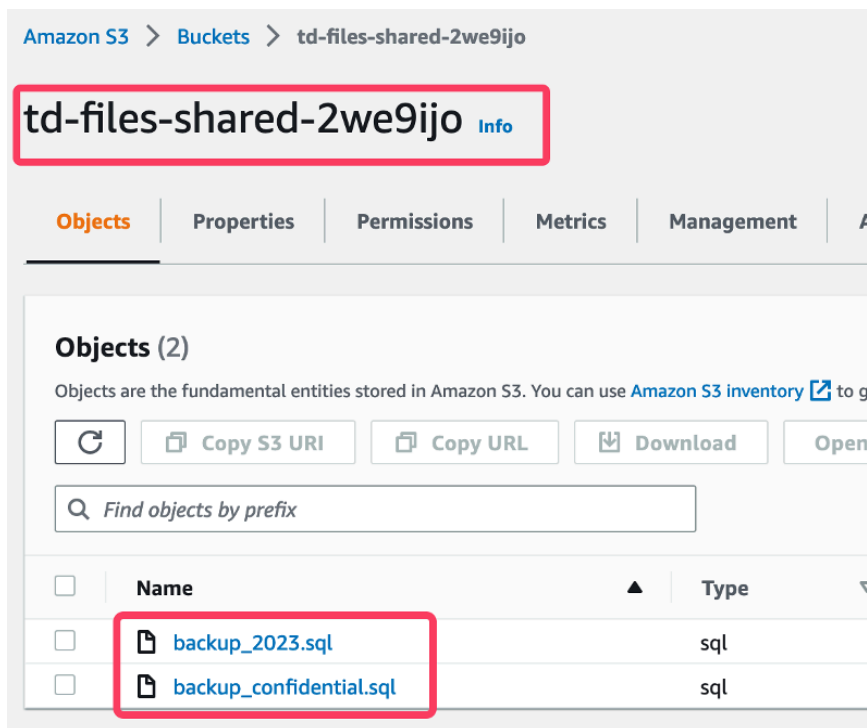
Amazon S3 Object tagging and Access control

Amazon Simple Storage Service (Amazon S3) provides object storage with a high level of availability, scalability, durability, and performance. It is designed to handle large amounts of data for data lakes, web applications, backups, big data analytics, etc. It can also be used to securely store data and control if you want to share data publicly or only with specific users. Tags play an important role in managing and identifying resources in the AWS cloud. Amazon S3 has tagging support for both the bucket itself and the objects contained in it. Tags allow you not only to identify the resources but can also be used to control who can access the resources.

For example, by adding object tags, you can control the IAM users that can access specific objects on an Amazon S3 bucket. You can create an object tag called **DataClassification** that can have a value of either confidential, private, or public. If you add an **Owner** tag to each object to identify who owns that object, then you can use the S3 bucket policy to enforce policies that only allow access to the owner of an object. This is helpful if you have an object that is classified as “confidential” and you want only the owner to have access to it.

The following steps outline how to implement an S3 bucket policy and IAM user policy to demonstrate the above example:

1. On the Amazon S3 management console, create an S3 bucket and upload files to it.



- Click the object you want to tag under the Tags section, and add proper **DataClassification** and **Owner** tags to the objects. Ensure proper tagging in order to enforce access control on the next steps.

Edit tags [Info](#)

⚠ To enable replication of object tags, IAM policies used for Replication must be updated. Object tags were introduced.

Tags

You can use object tags to analyze, manage, and specify permissions for objects. [Learn more](#)

Key	Value - optional
DataClassification	confidential
Owner	tduser1

Specified objects

Name ▲	Type ▼	Last modified
 backup_confidential.sql	sql	



3. Once the objects are tagged, go to the S3 bucket Permissions tab, and edit the Bucket policy. Add the below bucket policy to ensure that s3:GetObject action is allowed only when the object has aws:ResourceTag/DataClassification equals confidential and s3:ExistingObjectTag/Owner equals \${aws:userid}.

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "AccessTags",
    "Principal": "*",
    "Effect": "Allow",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::td-files-shared-2we9ijo/*",
    "Condition": {
      "StringEquals": {
        "aws:ResourceTag/DataClassification": "confidential",
        "s3:ExistingObjectTag/Owner": "${aws:userid}"
      }
    }
  }]
}
```

4. Once the bucket policy is saved. You must add permissions to all IAM users that will need access to the objects on the S3 bucket. Create an IAM policy and attach it to each IAM user or to an IAM group.
5. Go to the IAM user management console > Policies and create the following IAM policy which allows s3:GetObject permission.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "Allow S3 bucket access",
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::td-files-shared-2we9ijo/*"
    }
  ]
}
```

6. Attach the policy to the IAM user – tduser1.



tduser1

Summary

Permissions

Groups

Tags

Security credentials

Access Advisor

Permissions policies (1)

Permissions are defined by policies attached to the user directly or through groups.

☐	Policy name	Type
☐	s3-bucket-access	Customer inline

Upon applying the above Amazon S3 bucket policy and attaching the IAM policy to the IAM user, the “confidential” objects on the S3 bucket can only be accessed by their respective “owner”.

Sources:

<https://docs.aws.amazon.com/AmazonS3/latest/userguide/tagging-and-policies.html>

<https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-with-s3-actions.html#using-with-s3-actions-related-to-objects>

https://docs.aws.amazon.com/service-authorization/latest/reference/list_amazons3.html

https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_condition-keys.html#condition-keys-re-sourcetag



Amazon Storage Gateway - S3 File Gateway

Amazon Storage Gateway is a service that allows an appliance to run on your on-premises network and connect to the storage infrastructure of AWS. This is designed for a hybrid setup with seamless and secure integration of on-premises storage solutions to AWS. You can extend your storage solutions to the cloud, create file shares, or you can use the cloud as a backup solution.

Amazon Storage Gateway provides four types of gateways for different use cases:

- Amazon S3 File Gateway
- Amazon FSx File Gateway
- Tape Gateway
- Volume Gateway

Amazon S3 File Gateway – designed to be deployed for storage solutions that require a file system share using the Network File System (NFS) and Server Message Block (SMB) protocols for data center applications. The objects stored on the file shares retain ownership, permissions, and timestamps. These file shares directly store the objects in your configured Amazon S3 bucket, and they can be managed as native S3 objects. You can also use this to store backups of database backups, log files, or metrics data that you want to use for machine learning or big data analytics.

Amazon FSx File Gateway – designed for Windows-based infrastructures. It provides low-latency, scalable file shares using the SMB protocol. Amazon FSx for Windows File Server offers a scalable shared file system that integrates with your existing environment. With the HDD option, Amazon FSx can present a file storage with full Windows-native compatibilities, such as NTFS support and ACLs.

Tape Gateway – designed for data backups and archival requirements that uses iSCSI-based virtual tape libraries (VTL). As most on-premises data centers use cheap tape backups for data, Tape Gateway offers a fast and durable cloud-based solution. Backups will be automatically stored in Amazon S3. It can save costs for long-term archival and reduces maintenance overhead needed for off-site media storage requirements.

Volume Gateway – designed for creating iSCSI-based block storage devices that can be used by on-premises applications to store data. The data stored in these volumes are asynchronously backed up on Amazon S3. Volume Gateway has two configurations – stored volumes and cached volumes. Stored volumes provide low latency access by storing all your data locally, which are then asynchronously backed up to Amazon S3. Cached volumes only store frequently accessed data locally, while all data are stored on Amazon S3.

S3 File Gateway Refresh Cache

Amazon S3 File Gateway maintains an inventory of objects stored on the destination Amazon S3 bucket. This inventory is cached to improve latency and reduce the frequency of requests to Amazon S3. However, if there

are a lot of new objects uploaded to the S3 bucket, the contents of the inventory cache may get outdated and not show these new objects. S3 File Gateway gives you an automated way to schedule this inventory cache refresh. See the following overview steps on how to enable this.

1. First, we need to create a gateway. On the Storage Gateway management console, go to Gateways, and click Create gateway.
2. Give a name to the gateway, select a gateway time zone, and select Amazon S3 File Gateway.

Gateway settings

Gateway name

td-gateway

The name must be between 2 and 255 characters and cannot include a slash (\ or /).

Gateway time zone

Choose the local time zone based on where you are deploying your gateway.

GMT +8:00 Beijing, Perth, Singapore, Hong Kong

Gateway options [Info](#)

Gateway type

☒ **Amazon S3 File Gateway**

Store and access objects in Amazon S3 from NFS or SMB file data with local caching.



☐ **Amazon FSx File Gateway**

Access fully managed file shares in Amazon Windows File Server using SMB.



☐ **Tape Gateway**

Store virtual tapes in Amazon S3 using iSCSI-VTL, and store archived tapes in Amazon S3 Glacier Flexible Retrieval or Amazon S3 Glacier Deep Archive.



☐ **Volume Gateway**

Store and access iSCSI block storage volume S3.



3. Under the platform options, choose the platform on which you would host the storage gateway appliance. Click Download image and deploy it to your on-premises infrastructure. Once the appliance is deployed, click Next.

- Input the IP address or the activation from the storage gateway appliance.

Connection options

Use the gateway's IP address if it's accessible from your web browser. If the IP address isn't accessible, try using an activation key.

☐ IP address

Your gateway's IP address must be public or accessible from within your current network. Your web browser must be able to connect to this IP address.

☒ Activation key

Enter the activation key of your gateway's virtual machine (VM).

Activation key

To get the activation key, log in to your gateway's local console from the hardware appliance and get the activation key.

12345-12345-12345-12345-12345

- Click Next to review and activate the gateway.
- After activation of the storage gateway, go to the **File shares** section to create a file share.
- Choose the gateway you created and select NFS or SMB for the share type. Select the Destination S3 bucket that will store all objects from the file share. Click Create file share.

Create file share [Info](#)

Basics

Gateway

Choose a gateway ▼

File share type

☒ NFS

Standard for Linux environments.

☐ SMB

Standard for Windows environments.

S3 bucket

td-s3-file-gateway

3/22/2023 Asia Pacific (Singapore) ap-southeast-1 ▼

Create a new S3

- Once the file share is created, you can configure automated cache refresh. Click the File share you just created, and click Actions > Edit file share settings.
- For **Automated cache refresh from S3 after**, select the check box and set the time in days, hours, and minutes to refresh the file share's cache using Time To Live (TTL). Once the TTL has elapsed, the file gateway will automatically run RefreshCache to re-populate its inventory.



This procedure can also be done by scheduling an AWS Lambda function using Amazon Event Bridge. You can write a Lambda function that will call the RefreshCache API for the S3 File Gateway. You can then create an Amazon EventBridge rule that is scheduled to run at regular intervals and set the target to trigger the Lambda function that will run the RefreshCache API.

Sources

<https://aws.amazon.com/storagegateway/features/>

https://docs.aws.amazon.com/storagegateway/latest/APIReference/API_RefreshCache.html

<https://docs.aws.amazon.com/filegateway/latest/files3/refresh-cache.html>

<https://docs.aws.amazon.com/filegateway/latest/files3/file-gateway-concepts.html>



Amazon CodeGuru Reviewer

Amazon CodeGuru uses machine learning to provide developers tools that scan their code for potential security issues, recommendations to improve code quality and identify possible inefficiencies in the source code.

Amazon CodeGuru Reviewer has the ability to connect to your source code repositories and perform automated code reviews to flag potential defects or bugs in the code. It can also provide suggestions on how to improve your code. Amazon CodeGuru Reviewer supports Java and Python languages. CodeGuru Reviewer has been trained with a large data set so it can provide actionable recommendations with a low rate of false positives, and you can improve its ability to analyze code by providing user feedback.

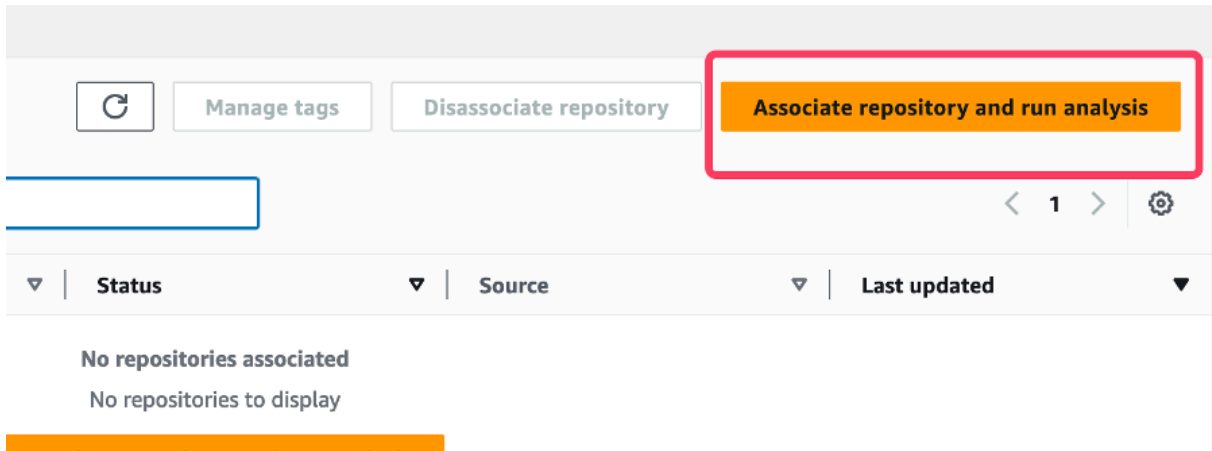
It continuously scans your repositories and can view the recommendations from the analysis and code reviews directly in the CodeGuru Reviewer console or as pull request comments in your repository.

Amazon CodeGuru Reviewer comes with **Secrets Detector**, which can scan your code to find and detect secrets in source code such as passwords, git keys, access keys, API keys, SSH keys, or access tokens, etc. It can integrate with **AWS Secrets Manager** to find your unprotected secrets. Once CodeGuru generates the recommendation and displays the code review recommendation, you can quickly protect the secrets by clicking the “Protect your credential” in the code review, going to Secrets Manager, and creating new secret values.

The following steps demonstrate how Amazon CodeGuru integrates with source code repositories, generates code recommendations, scans for secrets, and helps protect the secret with AWS Secrets Manager.

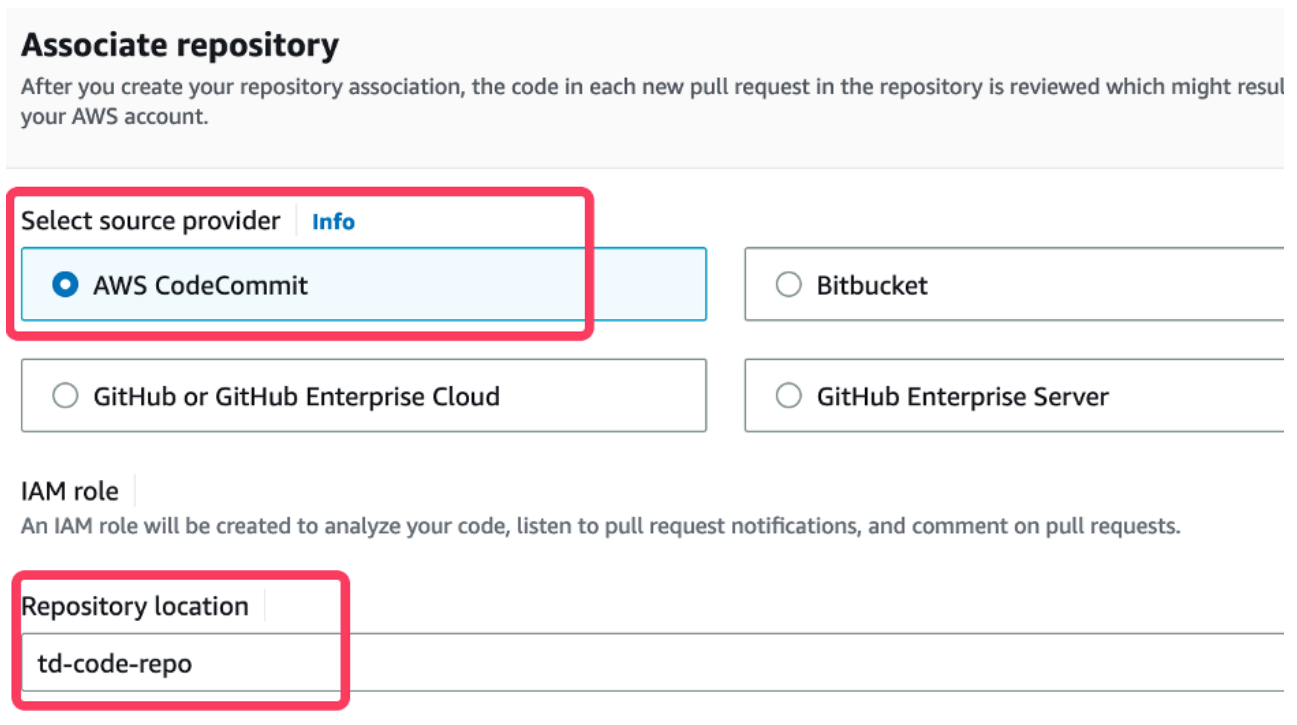
1. There should be an existing Java or Python repository that is supported by CodeGuru Reviewer – Github, Bitbucket, Amazon S3.

2. Navigate to the CodeGuru management console, under Reviewer, click Repositories. Click **Associate repository and run analysis**.



The screenshot shows the AWS CodeGuru management console interface. At the top, there are four buttons: a refresh icon, 'Manage tags', 'Disassociate repository', and 'Associate repository and run analysis'. The 'Associate repository and run analysis' button is highlighted with a red rectangular box. Below the buttons is a search bar and a table with columns for Status, Source, and Last updated. The table is currently empty, displaying 'No repositories associated' and 'No repositories to display'.

3. Provide the details of the repository you want to associate with. Select the provider and the repository location you want to associate with.



The screenshot shows the 'Associate repository' form in the AWS CodeGuru management console. The form has a title 'Associate repository' and a subtitle 'After you create your repository association, the code in each new pull request in the repository is reviewed which might result in your AWS account being compromised.' Below the subtitle, there are four radio button options for 'Select source provider': 'AWS CodeCommit' (selected and highlighted with a red box), 'Bitbucket', 'GitHub or GitHub Enterprise Cloud', and 'GitHub Enterprise Server'. Below the provider selection, there is a section for 'IAM role' with a subtitle 'An IAM role will be created to analyze your code, listen to pull request notifications, and comment on pull requests.' Below the IAM role section, there is a section for 'Repository location' with a text input field containing 'td-code-repo' (highlighted with a red box).

- Under the source branch, select the repository branch you want to scan and the code review name. Click Associate repository and run analysis to start the scanning process.

Run a repository analysis [Info](#)

When you associate a repository, CodeGuru Reviewer performs a full code review on the repository. After the repository association is created, code in all pull requests are reviewed and you can create new repository code reviews.

Source branch

Select a source branch for CodeGuru Reviewer to scan.

main

Code review name - optional

Enter your code review name.

td-code-repo-main-268457b2-6f05-4b07-8fc5-20d2b1688433

► Analysis configuration file - optional

Suppress recommendations from CodeGuru Reviewer.

► Tags - optional

A tag is a label that you assign to an associated repository. Each tag consists of a key and an optional value. You can use tags to search and filter your associated repositories or track your AWS costs.

i It may take 30 seconds to associate a repository, and 5 to 10 minutes to create full repository analysis.

Cancel

Associate repository and run analysis

- Go to Reviewer > Code reviews to view the generated recommendations and security analysis.

Full repository analysis (2)

List of full repository analysis code reviews on associated repositories for the last 90 days. Create a repository analysis to get a code review for all the code in one branch.

Create full repository analysis

Find code reviews

Name	Status	Reposit...	Branch	Recommendation
td-code-repo-main-268457b2-6f05-4b07-8fc	Completed	td-code-repo	main	1

6. On this sample repository, you can see the recommendation to protect secret keys that are hardcoded on the source code. Click **Protect your credentials** to open AWS Secrets Manager.

Recommendations (1)

< 1 >
⚙️

dev.env Line: 3 [🔗](#) Security Secrets

It appears your code contains a hardcoded AWS Access Key ID. Hardcoded secrets or credentials can allow attackers to bypass authentication methods and perform malicious actions. We recommend revoking access to resources that use this access key and using IAM roles instead of hardcoded access keys for IAM users. If you must use IAM users, secure their secrets outside your code.

[Learn more about the use of hardcoded credentials](#) [🔗](#)

Hardcoded credentials can be intercepted by malicious actors. Even after removing them from the code they may still pose a risk because an attacker might have recorded them to use them at a later point in time.

Protect your credentials

7. On AWS Secrets Manager, create a new secret to protect the AWS credentials from the code.



Secret type [Info](#)

☐ Credentials for Amazon RDS database

☐ Credentials for Amazon DocumentDB database

☐ Credentials for Amazon Redshift cluster

☐ Credentials for other database

☒ Other type of secret
API key, OAuth token, other.

Key/value pairs [Info](#)

Key/value

Plaintext

REACT_APP_AWS_ACCESS_KEY_ID	AKIATBV[REDACTED]	Re
REACT_APP_AWS_SECRET_ACCESS_KEY	sj6Z4hDpgRk[REDACTED]	Re
+ Add row		

Sources:

<https://aws.amazon.com/blogs/aws/codeguru-reviewer-secrets-detector-identify-hardcoded-secrets/>

<https://docs.aws.amazon.com/codeguru/latest/reviewer-ug/welcome.html>

<https://docs.aws.amazon.com/codeguru/latest/reviewer-ug/how-codeguru-reviewer-works.html>

<https://docs.aws.amazon.com/codeguru/latest/reviewer-ug/recommendations.html#secrets-detection>



Amazon CodeGuru Profiler

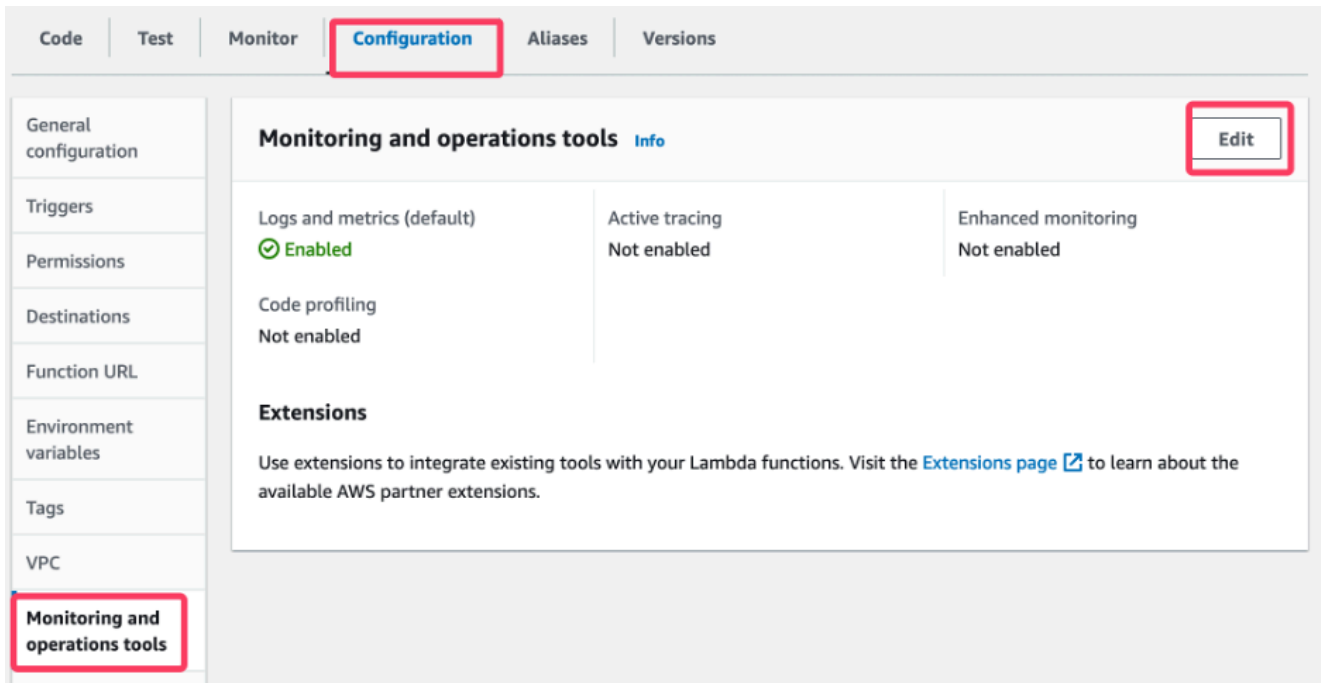
Amazon CodeGuru Profiler is another tool that uses machine learning to create a performance profile of your live applications. It continuously collects runtime performance data to identify expensive lines of code. And it can generate suggestions to improve the performance and efficiency of your source code. It supports applications written in all Java virtual machine (JVM) languages and Python. Following are the features of CodeGuru Profiler based on supported languages.

Feature	Java/JVM	Python
CPU profiling	Yes	Yes
Support for AWS Lambda and other AWS compute platforms	Yes	Yes
Anomalies and recommendation reports	Yes	Yes
Colored thread states	Yes	Yes
Heap summary visualization	Yes	No

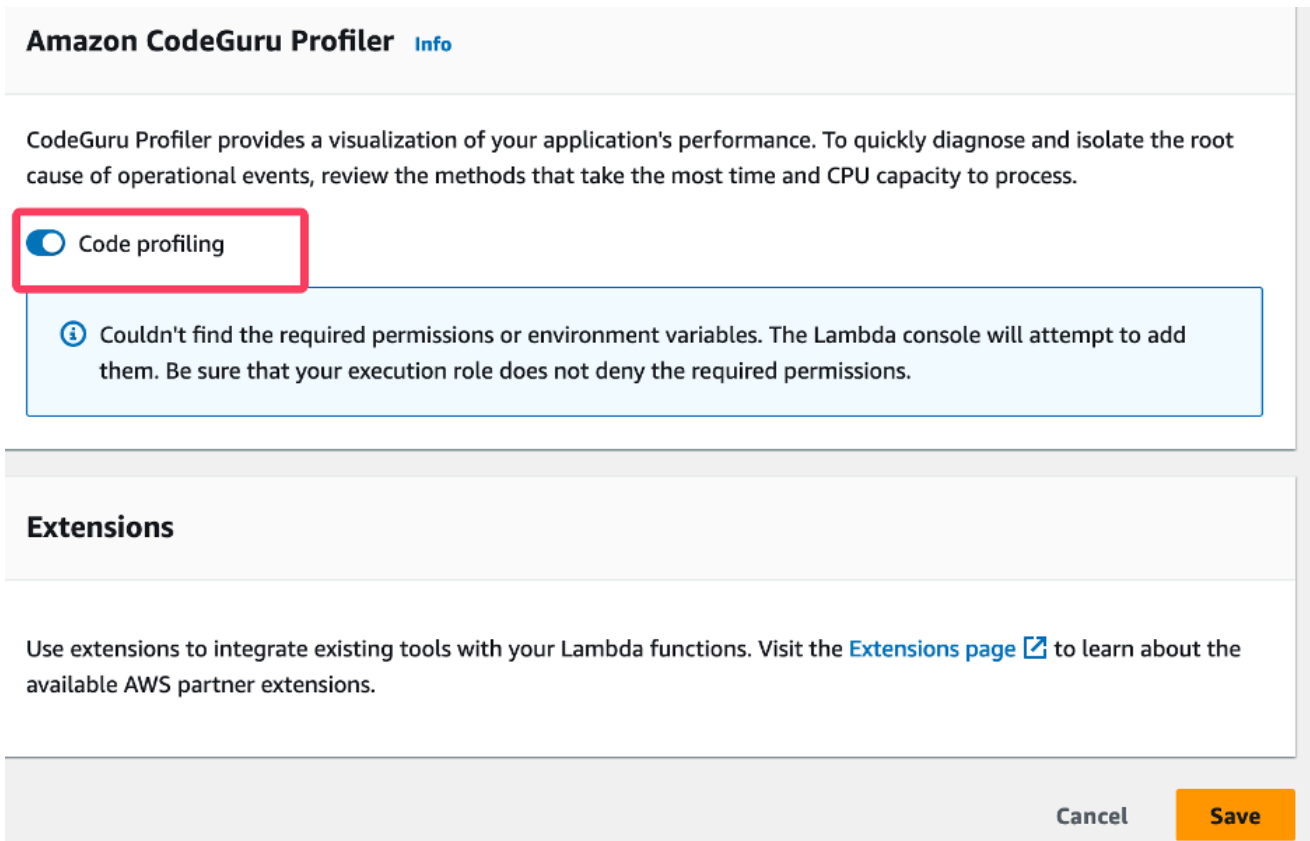
Once enabled on the application code, Amazon CodeGuru Profiler creates a dashboard of profiling data for your application where you can visualize the performance of your application. You can view which specific calls or routines in the code are consuming the most resources so you can analyze and reduce bottlenecks in your source code.

Amazon CodeGuru supports profiling AWS Lambda functions, and the only requirement is to start the agent is the profiling group name and add the `@with_lambda_profiler()` decorator to your handler function. The following steps demonstrate how you can enable **Amazon CodeGuru Profiler** on an **AWS Lambda** function by adding the `@with_lambda_profiler()` decorator to your handler function.

1. To enable Amazon CodeGuru Profiler, go to the AWS Lambda console and open your Lambda function.
2. Click the **Configuration** tab, and click **Monitoring and operations tools**. Click **Edit**.



3. Enable Amazon CodeGuru Profiler and click Save. This creates a profiling group when a profile is available to submit.





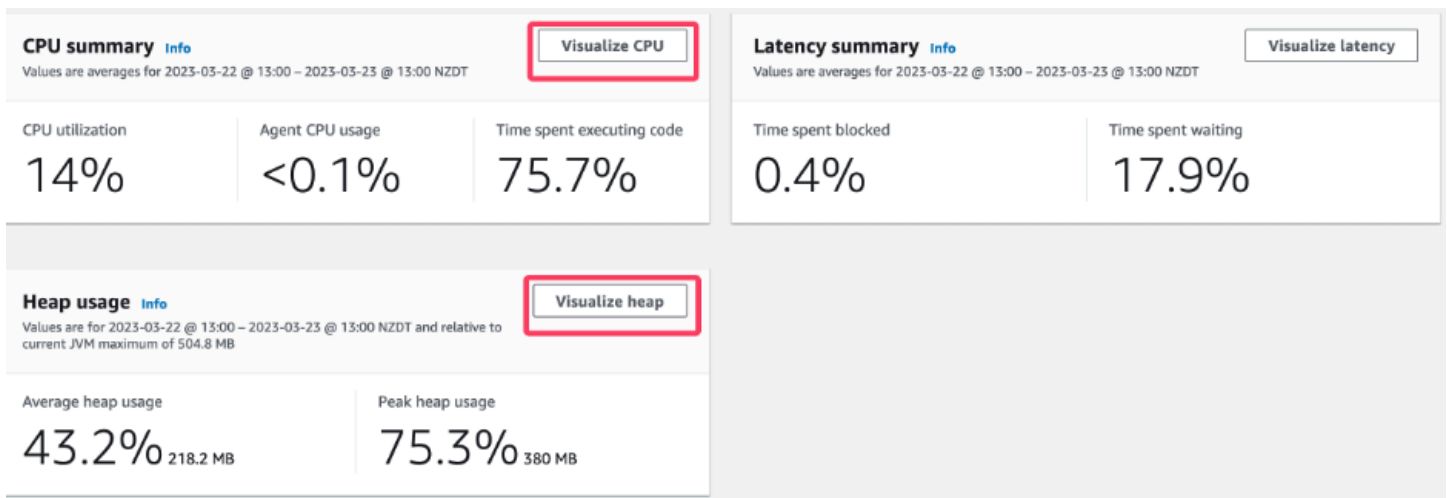
4. You can also manually add CodeGuru Profiler on your Lambda source code by decorating your handler function with `@with_lambda_profiler()`. See the example code below:

```
from codeguru_profiler_agent import with_lambda_profiler

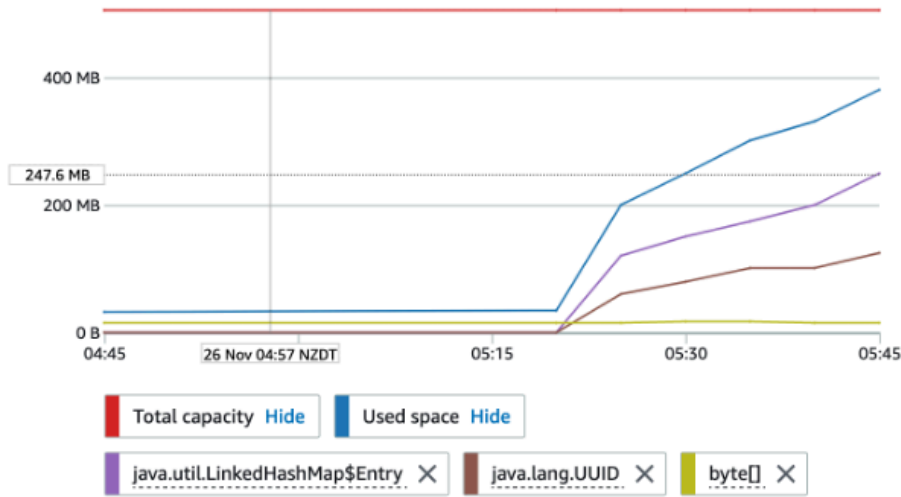
@with_lambda_profiler(profiling_group_name="MyGroupName")
def handler_name(event, context):
    return "Hello World"
```

It is important to note that the decorator should only be added in the handler function and not in other internal functions. You can pass the profiling group name directly in the decorator or with environment variables.

The findings of CodeGuru Profiler can be visualized on a dashboard. Go to the CodeGuru management console, and click Profiling groups. Click on one of the generated profiling to view the dashboard. For example, below are the findings for a sample Java application.



You can view the heap size, analyze it and take actions to improve the efficiency of your application.



Object types on heap (3/18) [Info](#)

🔍 Filter types by name, or code type ☐ Only show selected types

☐	Type name	Average usage by type	Average n
☒	java.util.LinkedHashMap\$Entry	127.9 MB	12.8m
☒	java.lang.UUID	66.5 MB	6.6m
☒	byte[]	15.7 MB	62k

Sources:

- <https://docs.aws.amazon.com/codeguru/latest/profiler-ug/what-is-codeguru-profiler.html>
- <https://docs.aws.amazon.com/codeguru/latest/profiler-ug/python-lambda.html>
- <https://docs.aws.amazon.com/codeguru/latest/profiler-ug/python-lambda-command-line.html>
- <https://docs.aws.amazon.com/codeguru/latest/profiler-ug/setting-up-short.html>



Domain 4: Monitoring and Logging



Overview

The fourth exam domain of the AWS Certified DevOps Engineer Professional test is all about the process of monitoring and logging resources in AWS. You must learn how to set up advanced monitoring configurations using Amazon CloudWatch, AWS Systems Manager, AWS CloudTrail, and other related services. The task of aggregating logs and metrics across multiple AWS accounts and regions is also covered. Roughly 15% of questions in the actual DevOps exam revolve around these topics.

This domain will challenge your know-how in doing the following:

- Securely storing and managing logs
- Creating CloudWatch metrics from log events by using metric filters
- Creating CloudWatch metric streams using Amazon S3 or AmazonData Firehose
- Collecting custom metrics using the CloudWatch agent
- Managing log storage lifecycles such as S3 lifecycles and CloudWatch log group retention
- Processing log data by using CloudWatch log subscriptions and its integrations with Kinesis, Lambda and Amazon OpenSearch Service
- Searching log data by using filter and pattern syntax or CloudWatch Logs Insights
- Configuring encryption of log data via AWS KMS and other encryption services
- Building CloudWatch dashboards and Amazon QuickSight visualizations
- Associating CloudWatch alarms with standard or custom metrics in Amazon CloudWatch
- Configuring AWS X-Ray for different services such as in AWS Lambda, containers, API Gateway and other external services
- Analyzing real-time log streams using Amazon Kinesis Data Streams
- Analyzing logs with AWS services via Amazon Athena, Amazon QuickSight & CloudWatch Logs Insights
- Configuring solutions for auto-scaling using DynamoDB, EC2 Auto Scaling groups, RDS storage auto-scaling, ECS capacity provider and the likes.
- Creating CloudWatch custom metrics and metric filters, alarms, and notifications using Amazon SNS, Amazon EventBridge, AWS Lambda, and AWS Step Functions
- Configuring S3 events to process log files using Lambda and deliver log files to another destination such as OpenSearch Service, CloudWatch Logs et cetera
- Configuring EventBridge to send notifications based on a particular event pattern
- Installing and configuring agents on EC2 instances such as the AWS Systems Manager Agent (SSM Agent) and CloudWatch agent
- Configuring AWS Config rules to remediate issues
- Configuring health checks in Amazon Route 53 and Application Load Balancer

In this chapter, we will cover all of the related topics for monitoring and logging in AWS that will likely show up in your DevOps exam.



AWS Config Multi-Account Multi-Region Data Aggregation

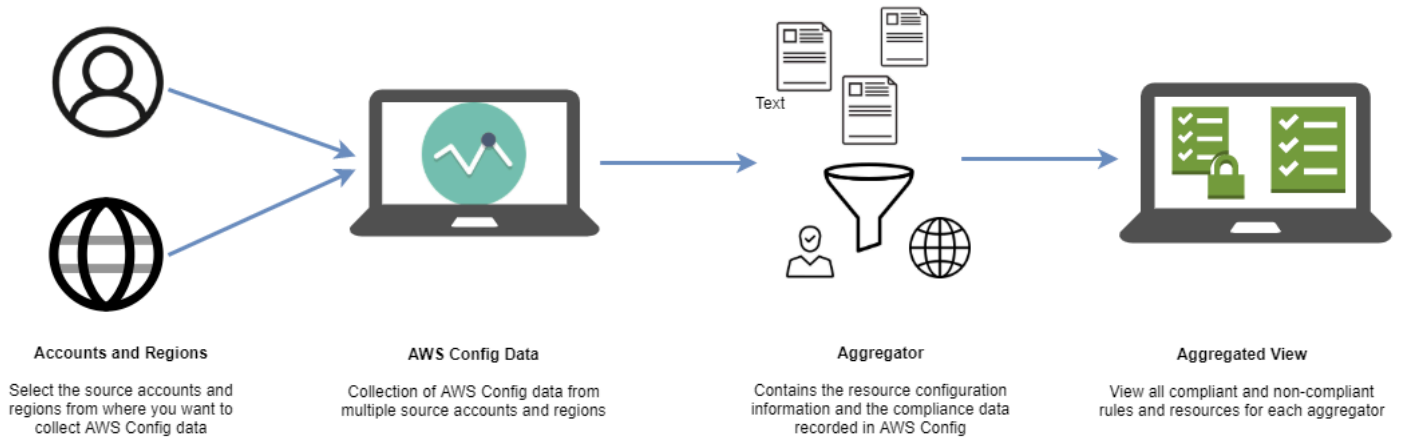
AWS Config enables you to monitor, audit, and evaluate the configurations of your AWS resources. It allows you to track and review the configuration changes of your resources as well as determine your overall compliance against your internal IT policies. You can also use AWS Systems Manager Automation to automatically remediate noncompliant resources that were evaluated by your AWS Config Rules.

This service is a great monitoring and automatic remediation tool for your AWS resources. However, there are certain limitations that you should know about AWS Config. The scope of this service is regional which means that it can only monitor the AWS resources in a specific region. It is usually enabled on a per-region basis on your AWS account. This poses a problem if your organization is using multiple AWS regions and accounts.

You can use the multi-Account, multi-Region data aggregation capability of AWS Config if you want to centralize the auditing and governance of your ubiquitous cloud resources. This functionality reduces the time and overhead required to collect an enterprise-wide view of your compliance status. It provides you with a single, aggregated view of your AWS resources across regions, accounts, and even your AWS Organizations. To do this, you have to create an *Aggregator* first and specify the regions where you want to collect data from.

An *Aggregator*, as its name suggests, is a resource of AWS Config that collects or groups data together. It replicates data from the specified source accounts into the aggregator AWS account where the aggregated view will be used. The aggregator account has access to the resource configuration and compliance details for multiple accounts and regions. This is a type of AWS Config resource that gathers AWS Config configuration and compliance data from the following:

- A single AWS account with resources from multiple AWS Regions.
- Multiple AWS accounts where each account uses multiple AWS Regions.
- The master and member accounts of an organization entity in AWS Organizations.



You can use an aggregator to view the resource configuration and compliance data recorded in AWS Config.

DevOps Exam Notes:

AWS Config is a regional resource. If you want to implement a centralized monitoring system of your resources across various AWS Regions and AWS accounts, you have to set up data aggregation using an **Aggregator** in AWS Config.

Remember that **AWS AppConfig** and **AWS Config** are two different services! The former is a capability of AWS Systems Manager that assists you in managing, storing, and deploying your application configurations to your Amazon EC2 instances at runtime. The latter is a configuration management service that helps you monitor, audit, and evaluate the configurations of your AWS resources.

Consolidating VPC Flow Logs From Multiple Sources

VPC Flow Logs is a feature in AWS that allows you to capture information about the incoming and outgoing IP traffic of the network interfaces in your Amazon VPC. Flow logs can assist you in properly monitoring and logging the activities in your VPC. It can diagnose overly restrictive security groups or network ACL rules, monitor the incoming traffic to your EC2 instances, and determine the flow of traffic to and from the network interfaces. After you've created a flow log, you can retrieve and view its data in the chosen destination.

A flow log data can be published to these destinations:

- Amazon CloudWatch Logs
- Amazon S3

Large companies often have multiple AWS accounts and use multiple VPCs for their cloud architecture. Monitoring the IP traffic flow could be difficult for a complex and extensive network architecture since the scope of the flow logs is only within a single VPC. You can enable flow logs for the VPCs that are peered with



your VPC as long as the peer VPC is in your AWS account. However, VPC Peering is still not enough to build centralized logging for multi-account environments with different types of network configurations.

Storing all the log data to Amazon S3 is a strategy that you can adopt to consolidate every flow log from across all VPCs that you own. The flow logs of your VPC can be published to an Amazon S3 bucket that you specify. The collected flow log records for all of the monitored network interfaces are sent to a series of log file objects stored in the S3 bucket. In this way, all of your logs are in one place which lessens the management overhead.

The buckets and the objects in Amazon S3 are private by default and only the bucket owner can access the data stored in it. You must grant the proper access and modify the bucket policy to allow the `delivery.logs.amazonaws.com` service to send and store the logs to the centralized S3 bucket.

You can use Amazon Athena to efficiently query flow log records stored in a centralized Amazon S3 bucket. Amazon Athena is an interactive query service that simplifies analyzing data in Amazon S3 using standard SQL. Amazon Athena is the recommended service for analyzing unified datasets across multiple objects stored in Amazon S3.

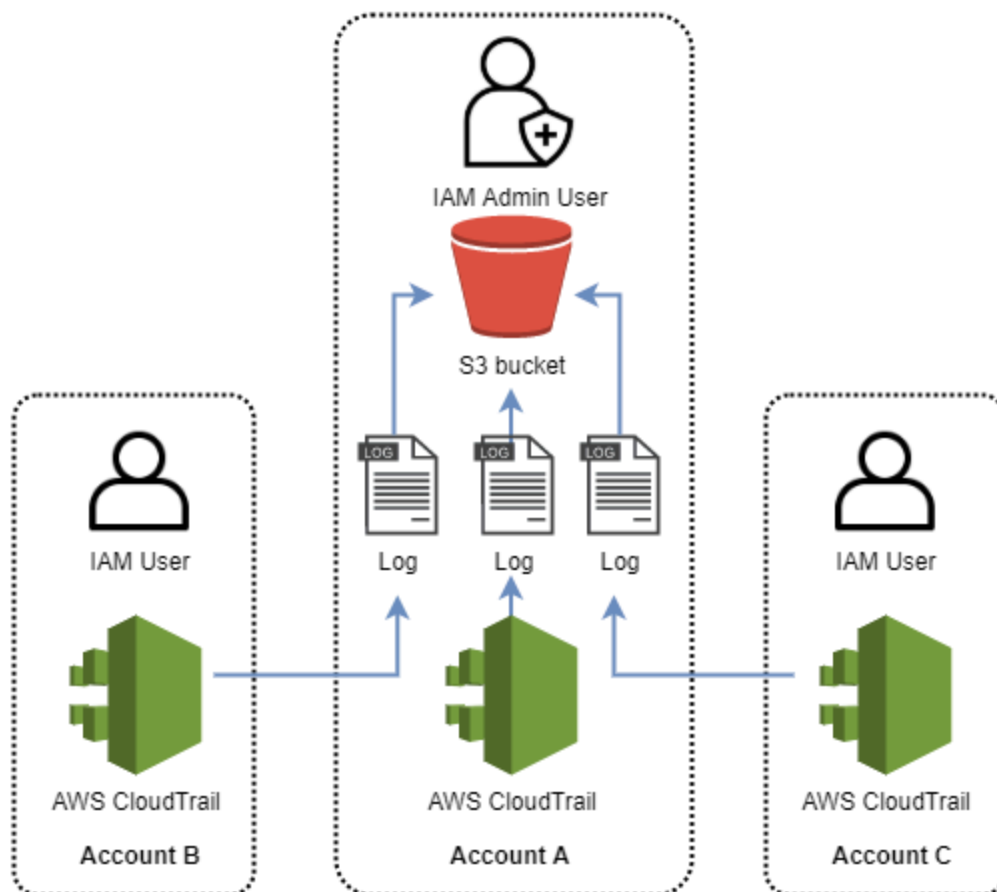
Remember that VPC Flow logs do not capture each and every IP traffic. The following items are not logged:

- Traffic to and from the instance metadata (169.254.169.254)
- Traffic to and from the Amazon Time Sync Service (169.254.169.123)
- Dynamic Host Configuration Protocol (DHCP) traffic.
- For the default VPC router, the traffic to the Reserved IP address is not logged.
- Traffic between a Network Load Balancer (ELB) network interface and an endpoint network interface (ENI).
- Traffic generated by an Amazon EC2 Windows instance for the Windows license activation.
- Traffic generated by the Amazon EC2 instances when they connect to the Amazon DNS server.
However, if you use your own BIND DNS server, all traffic to that DNS server is logged by the VPC Flow Logs.

Consolidating CloudTrail Log Files from Multiple Sources

AWS CloudTrail is the primary service used for auditing your AWS resources. It provides an event history of all of your AWS account activity, including the actions taken through the AWS SDKs, AWS CLI, AWS Management Console, and other AWS services. However, it can only track the API calls made on a single AWS account. If your company has multiple AWS accounts, you can consolidate them into a single organizational unit (OU) using AWS Organizations. With this, you can create a trail that will collect all events for all AWS accounts in that organization. This is often referred to as an “organization trail” in AWS CloudTrail. The organization trail logs events for the master account and all member accounts in the organization.

However, some companies have complex cloud architectures that hinder them from using AWS Organizations. Businesses may have two or more external AWS accounts that belong to their subsidiaries or partners. To support this use case, you can configure AWS CloudTrail to send the log files from multiple AWS accounts into a single Amazon S3 bucket for centralized logging.





For example, there are four AWS accounts owned by your company: the Manila account, New York account, Singapore account, and London account, that you want to effectively monitor. These AWS accounts are separate business units that handle their own billing. Using AWS Organizations to consolidate billing and trail logs is not applicable due to organizational constraints. But alternatively, you can configure CloudTrail to deliver log files from all four accounts to an S3 bucket that belongs to a central AWS Account that you specify.

This can be done with these simple steps:

1. Enable AWS CloudTrail in the AWS account where the destination bucket will belong (e.g. *tutorialsdojo-trail S3 bucket in the Manila AWS account*). You may refer to this as your “central” or top-level account. Make sure that CloudTrail is disabled on the other accounts.
2. Modify the S3 bucket policy on the destination bucket to grant cross-account permissions to AWS CloudTrail.
3. Enable AWS CloudTrail in the other accounts that you want to include. Configure CloudTrail in these AWS accounts to use the same S3 bucket, which belongs to the AWS account that you specified in the first step.

Sources:

<https://docs.aws.amazon.com/vpc/latest/userguide/flow-logs-s3.html>

<https://aws.amazon.com/blogs/architecture/stream-amazon-cloudwatch-logs-to-a-centralized-account-for-audit-and-analysis/>



Ensuring the Integrity of the CloudTrail Log Files

The trail logs produced by AWS CloudTrail are invaluable for conducting security and compliance audits, including forensic investigations. It can attest that a particular user credential performed a specific API activity. For IT audit activities, the trail logs in CloudTrail can be used as proof that your AWS infrastructure complies with the specified set of operational guidelines. But what if these logs were modified and deleted? How can you secure and validate the integrity of your trail logs?

To protect your trail data, you can enable the 'log file integrity validation' feature in the CloudTrail via the AWS Management Console, CloudTrail API, or AWS CLI. This feature verifies whether a trail log file was modified, removed, or kept unchanged after CloudTrail sent it to the S3 bucket. AWS CloudTrail tracks the changes in each trail log using a separate digest file, which will also be stored in the S3 bucket. This digest file contains the digital signatures and hashes used to validate the integrity of the log files.

Storage location

Create a new S3 bucket ☒ Yes ☐ No

S3 bucket* ⓘ

▼ Advanced

Log file prefix ⓘ

Location: Trail/AWSLogs/[REDACTED]CloudTrail/ap-northeast-1

Encrypt log files with SSE-KMS ☐ Yes ☒ No ⓘ

Enable log file validation ☒ Yes ☐ No ⓘ

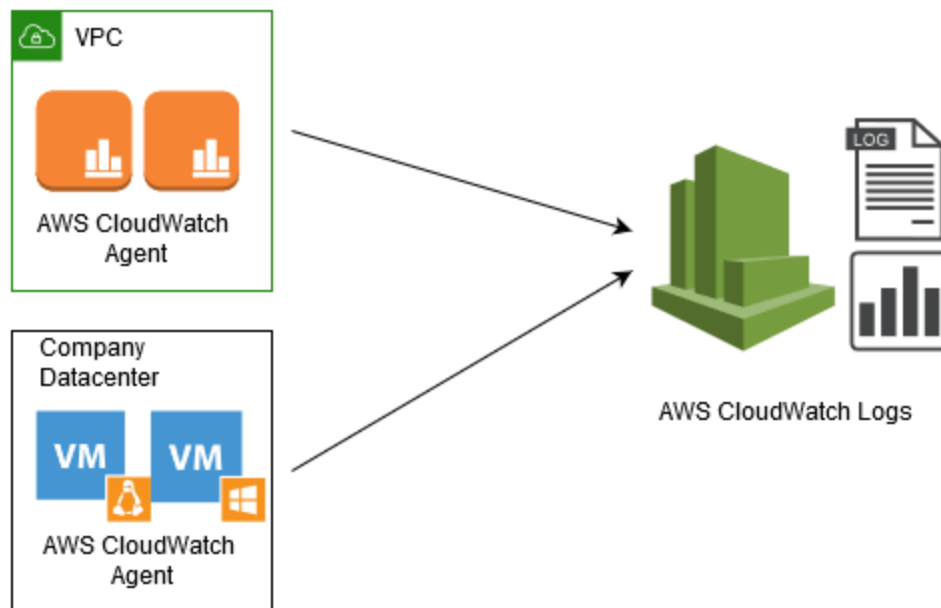
Send SNS notification for every log file delivery ☐ Yes ☒ No ⓘ

AWS CloudTrail uses SHA-256 for hashing and SHA-256 with RSA for digital signing for log file integrity validation. These industry-standard algorithms make it computationally infeasible to modify and delete the CloudTrail log files. The digest file is signed by AWS CloudTrail using the private key of a public and private key pair. The public key can be used to validate the digest file for a specific AWS Region.

Fetching Application Logs from Amazon EC2, ECS and On-premises Servers

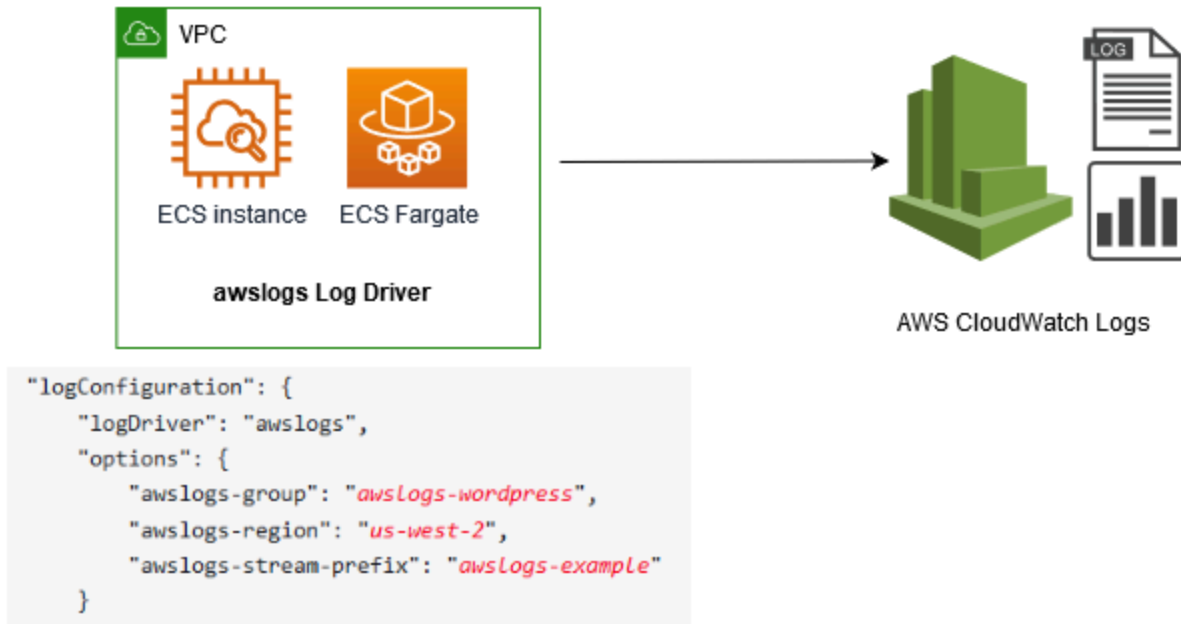
Application logs are vital for monitoring, troubleshooting, and regulatory compliance of every enterprise system. Without it, your team will waste a lot of time trying to find the root cause of an issue that can be easily detected by simply checking the logs. These files often live inside the server or a container. Usually, you have to connect to the application server via SSH or RDP before you can view the logs. This manual process seems to be inefficient, especially for high-performance organizations with hybrid network architectures.

Using the Amazon CloudWatch Logs agent, you can collect system metrics and logs from your Amazon EC2 instances and on-premises application servers. Gone are the days of spending several minutes connecting to your server and manually retrieving the application logs. For Linux servers, you don't need to issue a `tail -f` command anymore since you can view the logs on the CloudWatch dashboard on your browser in near real-time. It also collects both system-level and custom metrics from your EC2 instances and on-premises servers, making your monitoring tasks a breeze.



You have to manually download and install the Amazon CloudWatch Logs agent to your EC2 instances or on-premises servers using the command line. Alternatively, you can use AWS Systems Manager to automate the installation process. For your EC2 instances, it is preferable to attach an IAM Role to allow the application to send data to CloudWatch. For your on-premises servers, you have to create a separate IAM User to integrate your server into CloudWatch. Of course, you should first establish a connection between your on-premises data center and VPC using a VPN or a Direct Connect connection. You have to use a named profile in your local server that contains the credentials of the IAM user that you created.

If you are running your containerized application in Amazon Elastic Container Service (ECS), you can view the different logs from your containers in one convenient location by integrating Amazon CloudWatch. You can configure your Docker containers' tasks to send log data to CloudWatch Logs by using the *awslogs* log driver.



If your ECS task is using a Fargate launch type, you can enable the *awslogs* log driver and add the required *logConfiguration* parameters to your task definition. For EC2 launch types, you have to ensure that your Amazon ECS container instances have an attached IAM role that contains *logs:CreateLogStream* and *logs:PutLogEvents* permissions. Storing the log files to Amazon CloudWatch prevents your application logs from taking up disk space on your container instances.

Sources:

<https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/UseCloudWatchUnifiedAgent.html>

https://docs.aws.amazon.com/AmazonECS/latest/userguide/using_awslogs.html

CloudWatch Logs Agent to CloudWatch Logs Subscription

You can install CloudWatch Agent on your on-premises instances or EC2 instances to allow them to send detailed metrics to CloudWatch or send application logs to CloudWatch Logs. The logs will be sent to your configured CloudWatch log group for viewing and searching.

DevOps Exam Notes:

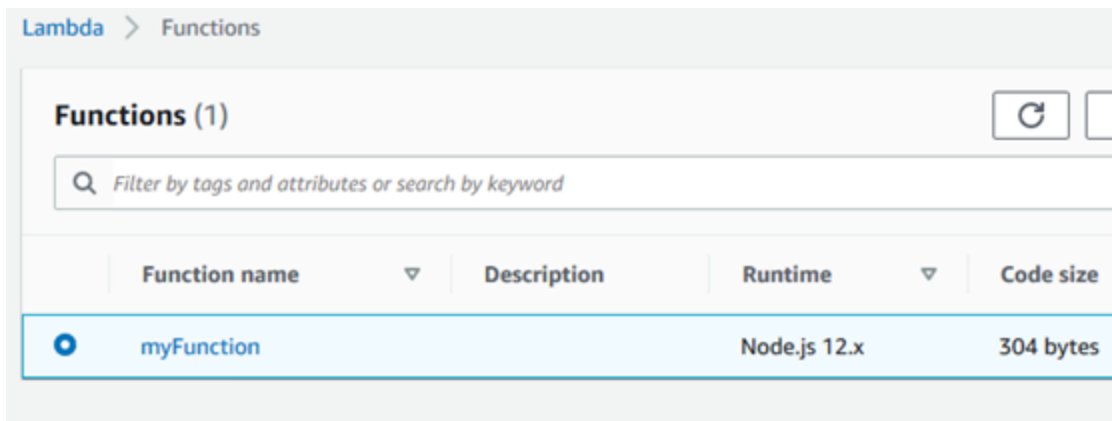
Additionally, you can use CloudWatch Logs subscriptions to get access to a real-time feed of log events from CloudWatch Logs and have it delivered to other services such as an Amazon Kinesis stream, an Amazon Data Firehose stream, or AWS Lambda for custom processing, analysis, or loading to other systems. This way, you can perform near real-time analysis of the logs, further log processing using AWS Lambda, or have advanced searching capability using OpenSearch (ElasticSearch).

To begin subscribing to log events, create the receiving source such as a Kinesis stream or Lambda function where the events will be delivered.

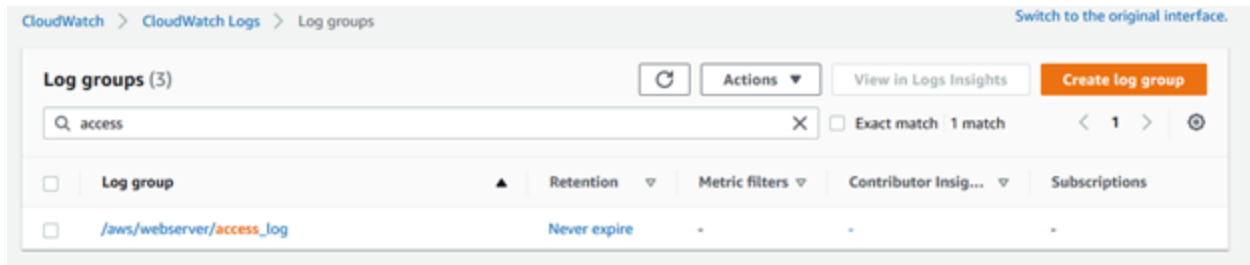
A subscription filter defines the filter pattern to use to sort out which log events get delivered to your AWS resource, as well as information about where to send matching log events to.

Here are the steps to setup CloudWatch logs subscription:

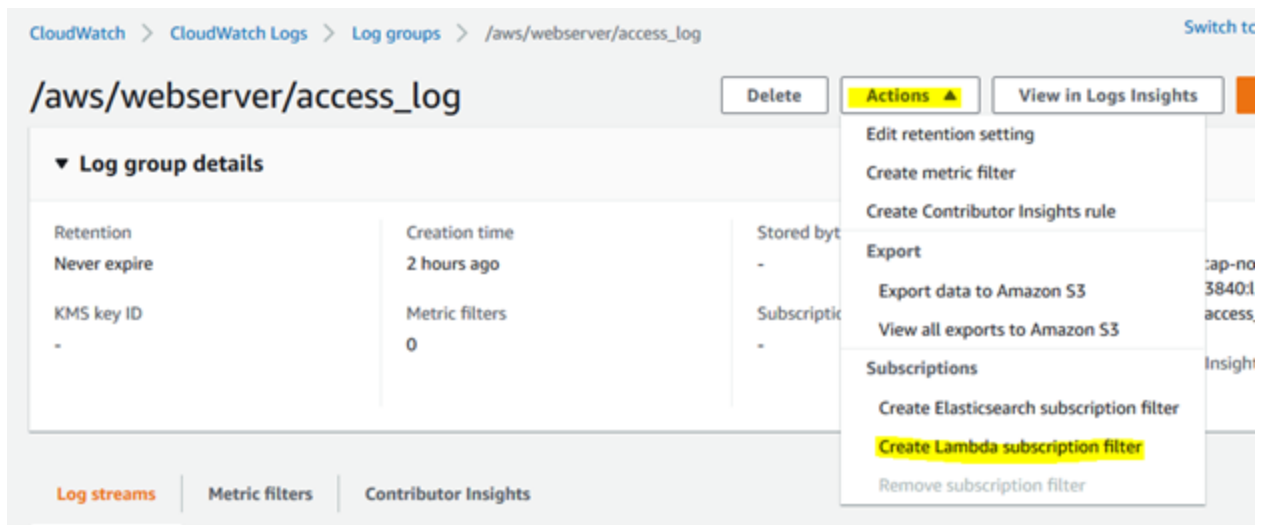
1. Create a receiving source, such as a Kinesis stream, OpenSearch cluster, or Lambda function.



2. Install CloudWatch Unified Agent on the EC2 instance (Linux or Windows) and configure it to send application logs to CloudWatch log group.
3. Create the CloudWatch log group that will access the logs.



4. Create a subscription filter for the log group and select the Lambda function or OpenSearch cluster that you created. If you need to set a Data Firehose stream as the subscription filter, you will need to use AWS CLI as the web console does not support it yet.



Sources:

<https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/QuickStartEC2Instance.html>
<https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/Subscriptions.html>
<https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/SubscriptionFilters.html>



Monitoring Service Limits with Trusted Advisor

AWS Trusted Advisor provides you real-time guidance to help you provision your resources following AWS best practices. It provides recommendations on 5 categories – Cost Optimization, Performance, Security, Fault Tolerance, and Service Limits.

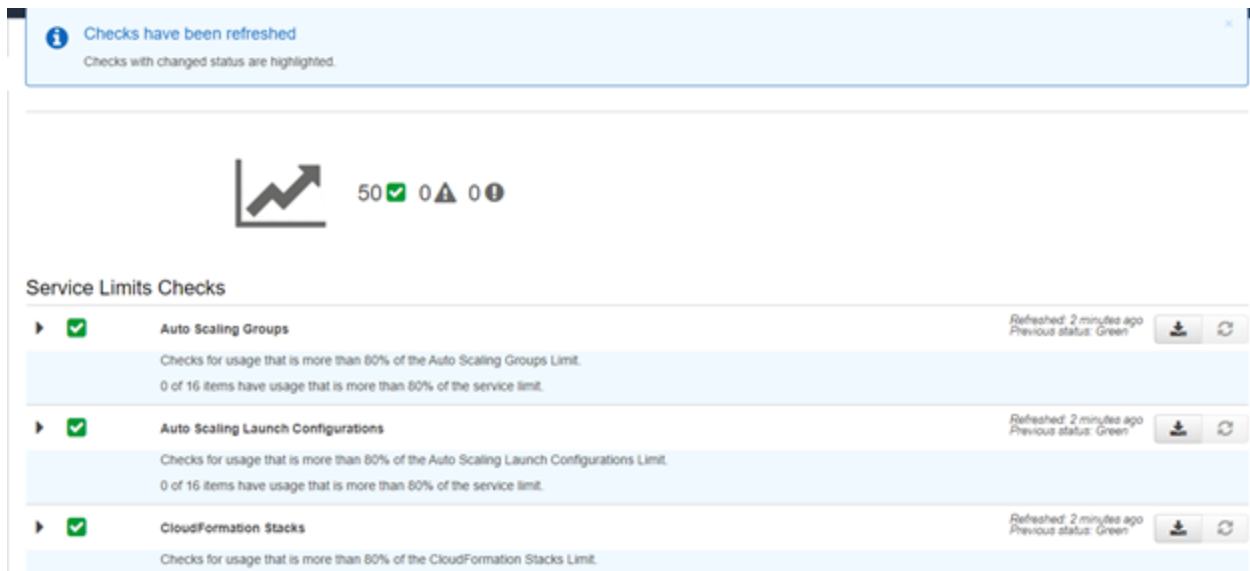
With Trusted Advisor's Service Limit Dashboard, you can view, refresh, and export utilization and limit data on a per-limit basis. These metrics are published on Amazon CloudWatch in which you can create custom alarms based on a percentage of service utilization against limits, understand the number of resources by each check, or view time-aggregate views of check results across service categories.

DevOps Exam Notes:

You need to understand that service limits are important when managing your AWS resources. In the exam, you can be given a scenario in which you have several Auto Scaling groups in your AWS account and you need to make sure that you are not reaching the service limit when you perform your blue/green deployments for your application. You can track service limits with Trusted Advisor and CloudWatch Alarms. The ServiceLimitUsage metric on CloudWatch Alarms is only visible for Business and Enterprise support customers.

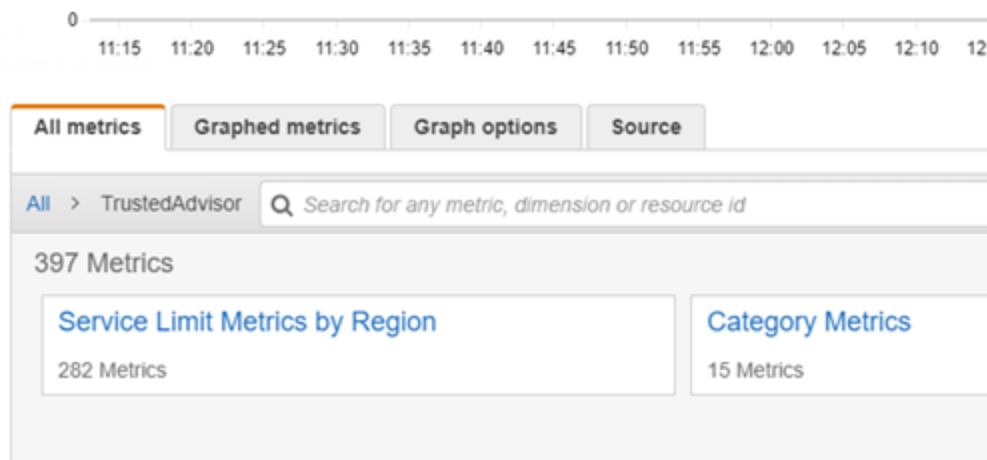
Here's how you can create a CloudWatch Alarm to detect if you are nearing your auto-scaling service limit and send a notification so you can request a service limit increase to AWS support.

1. First, head over to AWS Trusted Advisor > Service Limits and click the refresh button. This will refresh the service limit status for your account.



2. Go to CloudWatch > Alarms. Make sure that you are in the North Virginia region. Click the “Create Alarm” button and click “Select Metric”.

3. In the “All metrics” tab, click “Trusted Advisor” category and you will see “Service Limits by Region”.



4. Search for Auto Scaling groups on your desired region and click Select Metric.

All metrics

Graphed metrics (1)

Graph options

Source

All > TrustedAdvisor > Service Limit Metrics by Region

auto

scaling

groups

Search

Graph search

☐	Region (16)	ServiceLimit	ServiceName	Metric Name
☒	ap-northeast-1	Auto Scaling groups	AutoScaling	ServiceLimitUsage
☐	ap-northeast-2	Auto Scaling groups	AutoScaling	ServiceLimitUsage
☐	ap-south-1	Auto Scaling groups	AutoScaling	ServiceLimitUsage

5. We can set the condition for this Alarm so that when your Auto Scaling group reaches 80 for that particular region, the alarm is triggered.

Conditions

Threshold type

☒ Static
Use a value as a threshold

☐ Anomaly detection
Use a band as a threshold

Whenever ServiceLimitUsage is...

Define the alarm condition.

☒ Greater
> threshold

☐ Greater/Equal
>= threshold

☐ Lower/Equal
<= threshold

☐ Lower
< threshold

than...

Define the threshold value.

Must be a number

► Additional configuration

6. You can then configure an SNS topic to receive a notification for this alarm.

Notification

Alarm state trigger
Define the alarm state that will trigger this action.

☒ **In alarm**
The metric or expression is outside of the defined threshold.

☐ **OK**
The metric or expression is within the defined threshold.

☐ **Insufficient data**
The alarm has just started or not enough data is available.

Select an SNS topic
Define the SNS (Simple Notification Service) topic that will receive the notification.

☒ **Select an existing SNS topic**
☐ Create new topic
 ☐ Use topic ARN

Send a notification to...

Only email lists for this account are available.

Email (endpoints)

Topic has no endpoints - [View in SNS Console](#)

Add notification

Remove

7. Click Next to Preview the alarm and click “Create Alarm” to create the alarm.

Sources:

<https://aws.amazon.com/about-aws/whats-new/2017/11/aws-trusted-advisor-adds-service-limit-dashboard-and-cloudwatch-metrics/>
<https://aws.amazon.com/blogs/mt/monitoring-service-limits-with-trusted-advisor-and-amazon-cloudwatch/>



Domain 5: Incident and Event Response



Overview

The fifth exam domain of the AWS Certified DevOps Engineer Professional test focuses on incident and event management in the AWS infrastructure. It has the least share among all the exam domains of only 14% in the overall test coverage so you have to limit the time you spend in this domain.

This exam domain will challenge your knowledge and understanding in various topics on Incident Management and Event Response Management. Make sure that you are familiar with the different AWS services that generate, capture, and process events such as AWS Health, EventBridge, and CloudTrail to name a few. Event-driven architectures namely fan out, event streaming, and queuing are also included. Fleet management services including Systems Manager and AWS Auto Scaling must be reviewed as well. Ensure that you have experience in using the configuration management services in the AWS platform; especially the AWS Config service, plus other AWS metrics and logging services (e.g. CloudWatch, X-Ray). The various AWS service health services are also covered like AWS Health, CloudWatch, and Systems Manager OpsCenter. Having the knowledge of doing Root Cause Analysis after a production incident is also helpful.

This domain will test skills on the following tasks:

- Integrating AWS event sources using AWS Health, Amazon EventBridge, and AWS CloudTrail.
- Building event processing workflows via Amazon Simple Queue Service (Amazon SQS), Amazon Kinesis, Amazon SNS, Lambda, and Step Functions
- Applying configuration changes to systems
- Modifying infrastructure configurations in response to events
- Remediating a non-desired system state
- Analyzing failed deployments using AWS CodePipeline, CodeBuild, CodeDeploy, CloudFormation, and CloudWatch synthetic monitoring
- Analyzing incidents regarding failed processes



Incident and Event Response Management on AWS

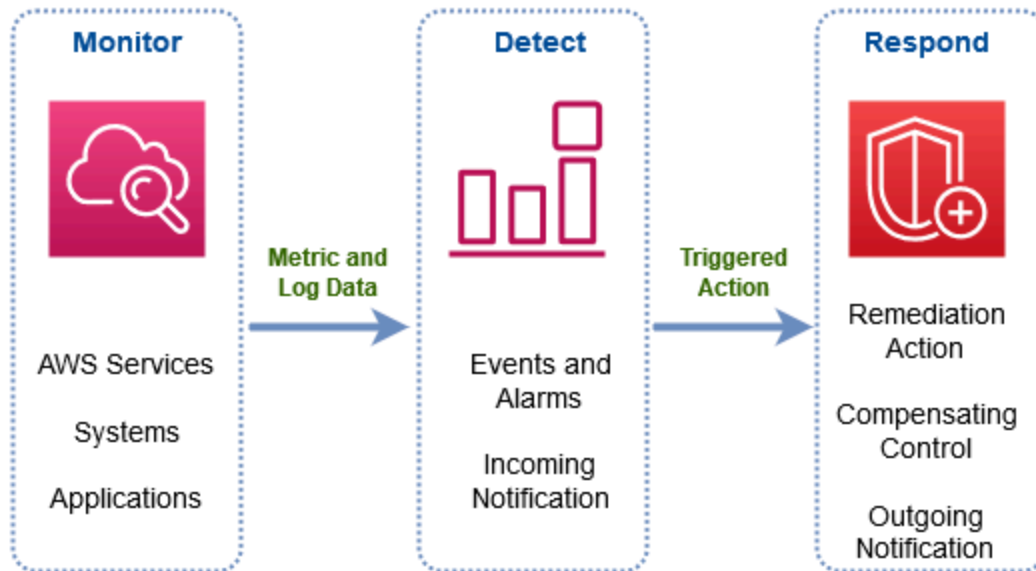
There is always a lot of action in a typical IT environment. The development team builds the new features within a given sprint and rectifies any software defects along the way. The operations team continuously monitors the underlying resources used by the enterprise applications and troubleshoots technical problems throughout the day. Critical incidents might suddenly happen that could adversely affect your production environments and many other system events that could potentially bring your business operations to a screeching halt! Thus, automating the incident and event management of your infrastructure is of utmost importance to maintain maximum efficiency and reduce unnecessary interruptions.

AWS provides a myriad of services and features to automate manual and repetitive IT tasks. Gone are the days of receiving outraged emails, calls, or tweets from your customers because your production server was down and you're not even aware of it. By using Amazon EventBridge and CloudWatch Alarms, your teams can immediately be notified of any system events or breaches of a specified threshold. Deployment issues can quickly be resolved or prevented through deployment monitoring and automatic rollbacks using AWS CodeDeploy, and Amazon EventBridge. S3 Events enables you to monitor unauthorized actions in your S3 buckets continuously, and RDS Events keeps you in the know for any failover, configuration change, or backup-related events that affect your database tier. Amazon EventBridge can also track all the changes in your AWS services, your custom applications, and external Software-as-a-Service (SaaS) applications in real time. These AWS features and services complement your existing security information and event management (SIEM) solutions to manage your entire cloud infrastructure properly.

DevOps Exam Notes:

An event indicates a change in a resource that is routed by an 'event bus' to its associated rule. A custom event bus can receive rules from AWS services, custom applications, and SaaS partner services. Amazon EventBridge is the ideal service to manage your events.

Automated Remediation Flow



The process of auditing your applications, systems, and infrastructure services in AWS is also simplified as all events and activities are appropriately tracked. Within minutes, you can identify the root cause of a recent production incident by checking the event history in AWS CloudTrail. Real-time logs feed on CloudWatch can be delivered to an Amazon Kinesis stream, an Amazon Data Firehose stream, or AWS Lambda for processing, analysis, or integration to other systems through CloudWatch Subscription Filters. Security incidents can be remediated immediately by setting up custom responses to Amazon GuardDuty findings using Amazon EventBridge. In this way, any security vulnerability in your AWS resources, such as an SSH brute force attack on one of your EC2 instances, can immediately be identified.



Amazon S3 Event Notifications

The Amazon Simple Storage System (Amazon S3) service provides scalable and durable object storage for storing a variety of data. Companies can store static content, media files, data archives, and even confidential information on their S3 buckets. Many AWS Services also use it for storing CloudTrail logs, CloudFormation templates, EBS Snapshots, ELB access logs, and many others. You can also utilize Amazon S3 as a data lake for Amazon Redshift Spectrum, AWS Glue, Amazon Athena, or Amazon EMR. With this vast amount of stored information, organizations must ensure that their Amazon S3 buckets are secured and monitored for unexpected changes.

The Amazon S3 Event Notification feature enables teams to be notified when specific events happen in their S3 buckets. You can choose the particular S3 Events that you want to monitor, and Amazon S3 will publish the notifications to your desired destinations. This feature enables you to have more visibility of your data and promptly remediate any potential data leaks.

Amazon S3 can publish notifications for these events:

- Object creation
- Object deletion
- Object restoration from the S3 Glacier Flexible Retrieval (formerly S3 Glacier) storage class
- Reduced Redundancy Storage (RRS) object lost events
- Objects eligible for replication using Amazon S3 Replication Time Control

You can configure Amazon S3 to publish events to these destinations:

- Amazon SNS Topic
- Amazon SQS queue
- AWS Lambda



Tutorials Dojo

Events

+ Add notification

Delete

Edit

Name	Events	Filter	Type
New event			

Name

TutorialsDojo_S3_Events

Events

☒ PUT

☒ POST

☐ COPY

☐ Multipart upload completed

☒ All object create events

☐ Object in RRS lost

☐ Permanently deleted

☐ Delete marker created

☒ All object delete events

☐ Restore initiated

☐ Restore completed

☐ Replication time missed threshold

☐ Replication time completed after threshold

☐ Replication time not tracked

☐ Replication failed

Prefix

contracts/

Suffix

.pdf

Send to

SNS Topic

SNS Topic

SQS Queue

Lambda Function

0 Active notifications

Cancel

Save

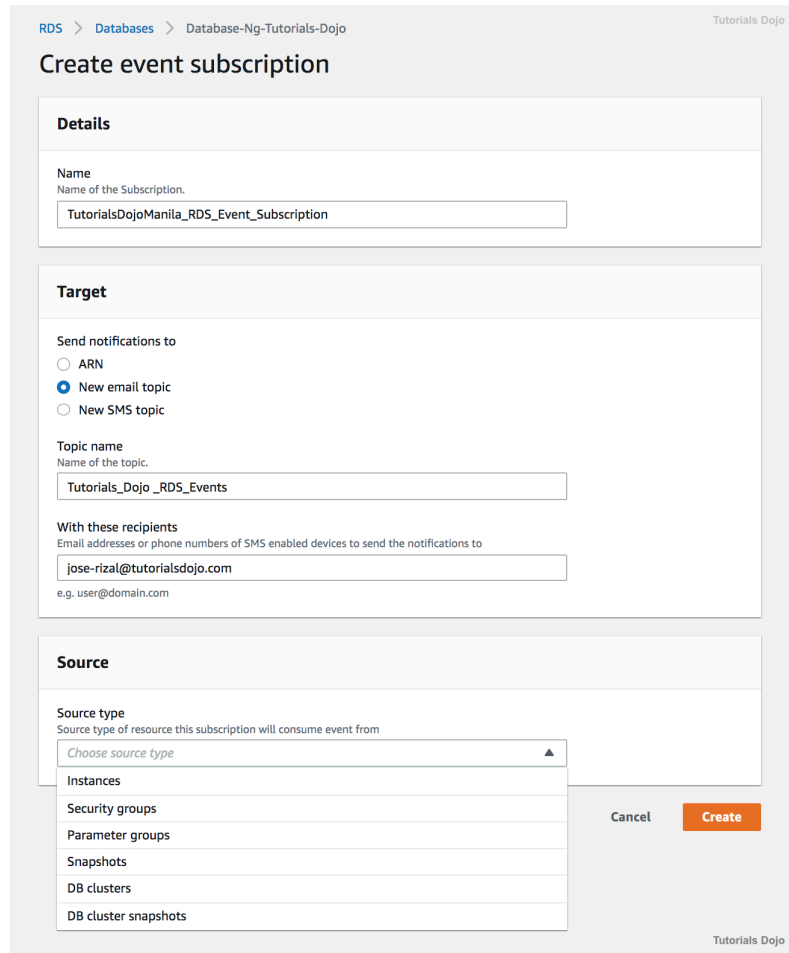
The S3 event notifications are usually transmitted within seconds and are designed to be delivered at least once. You can enable object versioning on your S3 bucket to ensure that an event notification is always sent for every successful object upload. With versioning, every successful write operation will produce a new version of your S3 object and send the corresponding event notification. Versioning averts the event notification issue where only one notification is sent when multiple operations are executed to a non-versioned object.

Source:

<https://docs.aws.amazon.com/AmazonS3/latest/user-guide/enable-event-notifications.html>

Amazon RDS Event Notifications

A database is a critical component of any enterprise system; therefore, it should be appropriately monitored for any potential issues. Like Amazon S3, there is also an event notification feature in Amazon RDS that notifies you of occurrences in your database resources. You can configure RDS to send you updates if an RDS DB instance has been created, restarted, or deleted. The RDS Event notifications can also detect low storage, configuration changes, Multi-AZ failover events, and many more.



RDS > Databases > Database-Ng-Tutorials-Dojo

Create event subscription

Details

Name
Name of the Subscription.
TutorialsDojoManila_RDS_Event_Subscription

Target

Send notifications to
☐ ARN
☒ New email topic
☐ New SMS topic

Topic name
Name of the topic.
Tutorials_Dojo_RDS_Events

With these recipients
Email addresses or phone numbers of SMS enabled devices to send the notifications to
jose-rizal@tutorialsdojo.com
e.g. user@domain.com

Source

Source type
Source type of resource this subscription will consume event from
Choose source type ▲

- Instances
- Security groups
- Parameter groups
- Snapshots
- DB clusters
- DB cluster snapshots

Cancel Create

Amazon RDS produces numerous events in specific categories that you can subscribe to using various tools such as the AWS CLI, Amazon RDS Console, or the RDS API. Each event category can refer to the parameter group, snapshot, or security group of your DB instance. Moreover, you can automatically process your RDS event notifications by using an AWS Lambda function or set an alarm threshold that tracks specific metrics by creating a CloudWatch Alarm.

Source:

https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_Events.html



AWS_RISK_CREDENTIALS_EXPOSED Event

Step Functions make it easy to develop and orchestrate components of operational response automation using visual workflows and it can be integrated with an Amazon EventBridge rule.

For example, your Development team is storing code on GitHub, and the developers sometimes forget to remove their personal IAM keys on their code before pushing it to the repository. Since an exposed IAM key is a security issue, you want to make sure that you are notified of this event and the security issue is automatically remediated.

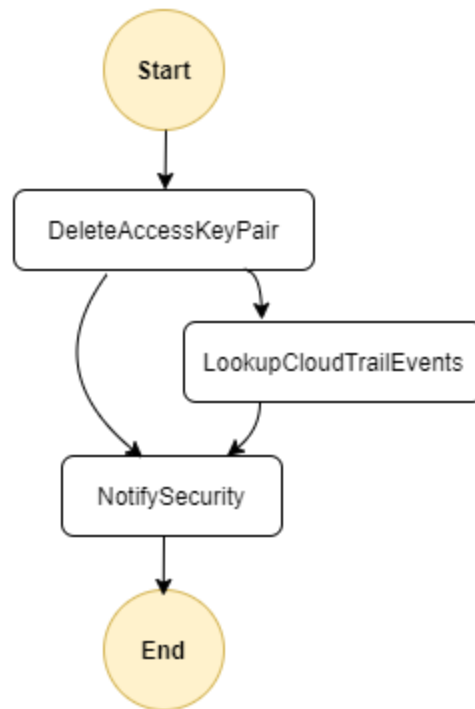
DevOps Exam Notes:

AWS can monitor popular code repository sites for IAM access keys that have been publicly exposed. Upon detection of an exposed IAM access key, AWS Health generates an **AWS_RISK_CREDENTIALS_EXPOSED** event in the AWS account related to the exposed key.

A configured Amazon EventBridge rule detects this **RISK** service event and invokes a **Step Functions state machine**. The state machine then orchestrates the automated workflow that deletes the exposed IAM access key, summarizes the recent API activity for the exposed key, and sends the summary message to an Amazon SNS topic to notify the subscribers.

Here are the steps for this setup:

1. Create a State function state machine that does the following:
 - a. Deletes the exposed IAM access key (to ensure that the exposed key can no longer be used)
 - b. Summarizes the recent API activity for the exposed key from AWS CloudTrail (to know what changes were made using the exposed key)
 - c. Send a summary message to an Amazon SNS topic to notify the subscribers.



2. Create an Amazon EventBridge rule for detecting the **AWS_RISK_CREDENTIALS_EXPOSED** event from the Personal Health Dashboard Service.



Event Source

Build or customize an Event Pattern or set a Schedule to invoke Targets.

☒ Event Pattern ⓘ ☐ Schedule ⓘ

Build event pattern to match events by service

Service Name

Event Type

This builder helps to build an event pattern to get events from AWS Health regarding health status of other AWS services.

☐ Any service ☒ Specific service(s)

☐ Any event type category ☒ Specific event type category(s)

☐ Any event type code ☒ Specific event type code(s)

☒ Any resource ☐ Specific resource(s)

Event Pattern Preview

[Copy to clipboard](#) [Edit](#)

```
{
  "source": [
    "aws.health"
  ],
  "detail-type": [
    "AWS Health Event"
  ],
  "detail": {
    "service": [
      "RISK"
    ],
    "eventTypeCategory": [
      "issue"
    ]
  }
}
```

- Set the Step function as Target for this Amazon EventBridge rule.

Targets

Select Target to invoke when an event matches your Event Pattern or when schedule is triggered.

Step Functions state machine

State machine*

Select state machine

Configure input

CloudWatch Events needs permission to send events to your Step Functions state machine. By continuing, you are allowing us to do so.

☒ Create a new role for this specific resource

AWS_Events_Invoke_Step_Functions_1203584006

☐ Use existing role ⓘ

[Learn more](#) about CloudWatch Events identity-based policies.

Add target*

Source:

<https://aws.amazon.com/blogs/compute/automate-your-it-operations-using-aws-step-functions-and-amazon-cloudwatch-events/>



AWS-Scheduled Maintenance Notification to Slack Channel

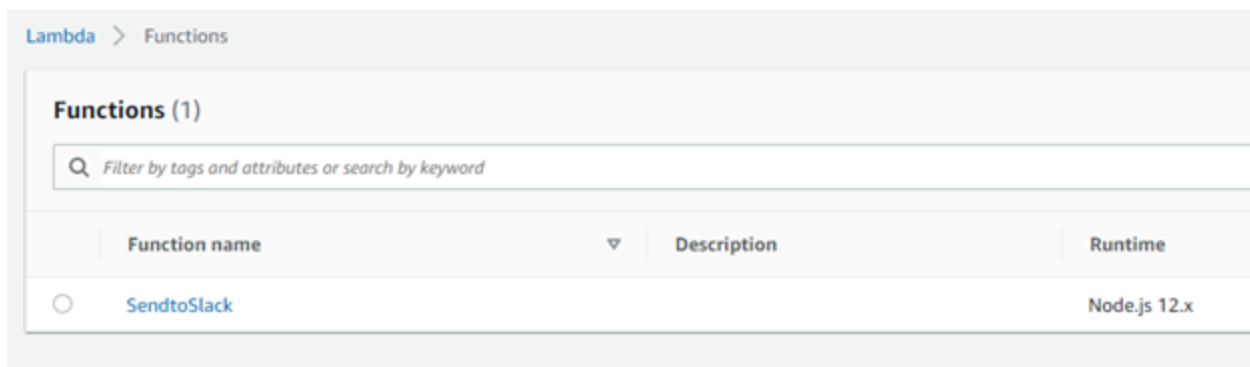
AWS Scheduled maintenance events are listed on AWS Health Dashboard. For example, if AWS needs to perform maintenance on the underlying host of your EC2 instance in which the EC2 instance usually needs to shut down, you will see an event on your AWS Health Dashboard for it. You can use Amazon EventBridge to detect these changes and you trigger notifications so you will be notified of these events. You can then perform the needed actions based on the events.

You can choose the following types of targets when use Amazon EventBridge as part of your AWS Health workflow:

- AWS Lambda functions
- Amazon Kinesis Data Streams
- Amazon Simple Queue Service (Amazon SQS) queues
- Built-in targets (CloudWatch alarm actions)
- Amazon Simple Notification Service (Amazon SNS) topics

For example, you can use a Lambda function to pass a notification to a Slack channel when an AWS Health event occurs. Here are the steps to do this.

1. Create a Lambda function that will send a message to your Slack Channel. A Nodejs or Python script will suffice. The function will call an API URL for the Slack channel passing along the message.



2. Go to Amazon EventBridge and create a rule.
3. Set a rule for the AWS Health Service, and EC2 Service Event Type.

Define rule detail Info

Rule detail

Name

Maximum of 64 characters consisting of numbers, lower/upper case letters, -, _.

Description - optional

Event bus Info

Select the event bus this rule applies to, either the default event bus or a custom or partner event bus.

default

☒ **Enable the rule on the selected event bus**

Rule type Info

☒ **Rule with an event pattern**
A rule that runs when an event matches the defined event pattern. EventBridge sends the event to the specified target.

☐ **Schedule**
A rule that runs on a schedule

Creation method

Method

☐ **Use schema**
Use an Amazon EventBridge schema to generate the event pattern.

☒ **Use pattern form**
Use a template provided by EventBridge to create an event pattern.

☐ **Custom pattern (JSON editor)**
Write an event pattern in JSON.

Event pattern Info

Event source
AWS service or EventBridge partner as source

AWS services

AWS service
The name of the AWS service as the event source

Health

Event type
The type of events as the source of the matching pattern

Specific Health events

ⓘ This builder helps to build an event pattern to get events from AWS Health regarding health status of other AWS services.

Event Type Specification 1

☐ Any service
 ☒ Specific service(s)

Specific service(s)

EC2

Event pattern
Event pattern, or filter to match the events

```

1 {
2   "source": ["aws.health"],
3   "detail-type": ["AWS Health Event"],
4   "detail": {
5     "service": ["EC2"]
6   }
7 }
            
```


4. Add a Target for this event to run the Lambda function and save the EventBridge Rule.

Amazon EventBridge > Rules > Create rule

Step 1
[Define rule detail](#)

Step 2
[Build event pattern](#)

Step 3
Select target(s)

Step 4 - optional
[Configure tags](#)

Step 5
[Review and create](#)

Select target(s)

Permissions
Note: When using the EventBridge console, EventBridge will automatically configure the proper permissions for the selected targets. If you're using the AWS CLI, SDK, or CloudFormation, you'll need to configure the proper permissions.

Target 1

Target types
Select an EventBridge event bus, EventBridge API destination (SaaS partner), or another AWS service as a target.

☐ EventBridge event bus
☐ EventBridge API destination
☒ AWS service

Select a target [Info](#)
Select target(s) to invoke when an event matches your event pattern or when schedule is triggered (limit of 5 targets per rule)

Lambda function ▼

Function
SendtoSlack ▼ 

► **Configure version/alias**

► **Additional settings**

Sources:

<https://aws.amazon.com/premiumsupport/knowledge-center/cloudwatch-notification-scheduled-events/>
<https://docs.aws.amazon.com/health/latest/ug/cloudwatch-events-health.html>

Using AWS Health API and Amazon EventBridge for Monitoring AWS-Scheduled Deployments/Changes

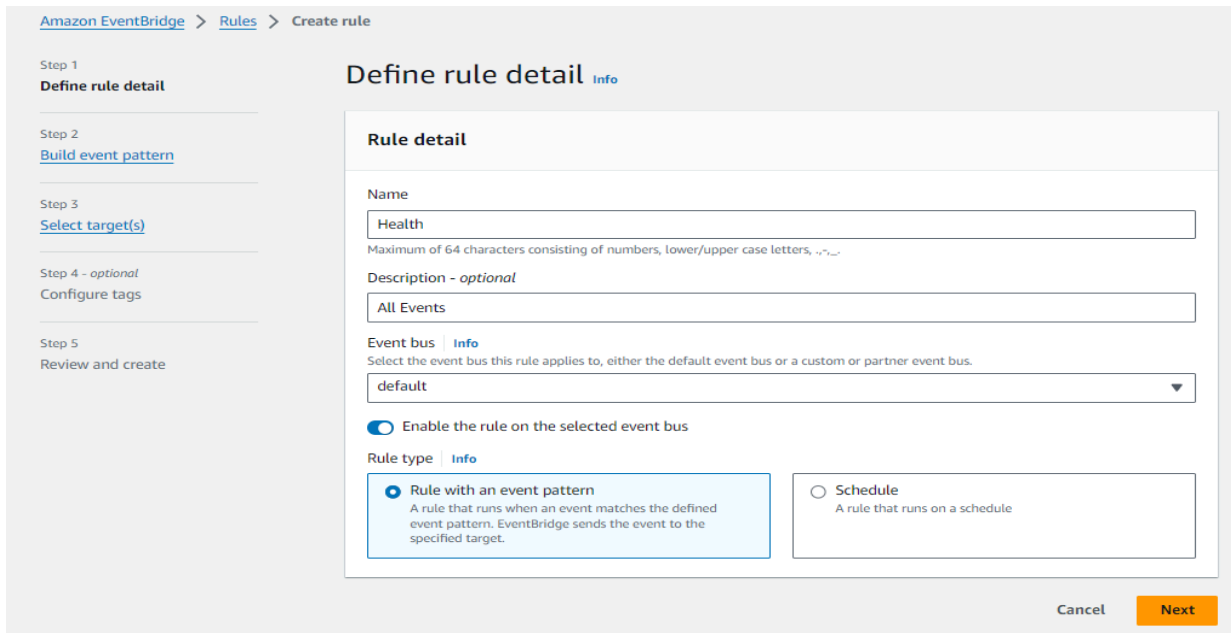
AWS Personal Health Dashboard provides you with alerts and remediation status when AWS is experiencing Amazon EventBridge events that may impact your resources. These events may be scheduled or unscheduled. For example, scheduled events such as changes on your underlying EC2 hosts may shut down or terminate your instances, or AWS RDS scheduled upgrades that may reboot your RDS instance.

DevOps Exam Notes:

You can monitor these AWS Health events using Amazon EventBridge by calling the AWS Health API. Then, you can set a target to an SNS topic to inform you of that Event, or you can trigger a Lambda function to perform a custom action based on the Event.

Here are the steps on to set this up:

1. Go to EventBridge > Rules: Navigate to the EventBridge service in the AWS Management Console and click on "Rules" to create a new rule.
2. Create a Rule for AWS Health Events: Click on "Create rule" and define your rule pattern. You can specify the event source as "aws.health" to detect AWS Health Events. Optionally, you can filter events based on services by specifying the service names in the event pattern.



The screenshot shows the 'Define rule detail' page in the Amazon EventBridge console. The breadcrumb trail is 'Amazon EventBridge > Rules > Create rule'. On the left, a sidebar lists five steps: Step 1 (Define rule detail), Step 2 (Build event pattern), Step 3 (Select target(s)), Step 4 (optional, Configure tags), and Step 5 (Review and create). The main content area is titled 'Define rule detail' with an 'Info' icon. It contains the following fields and options:

- Rule detail** section:
 - Name:** A text input field containing 'Health'. Below it, a note states: 'Maximum of 64 characters consisting of numbers, lower/upper case letters, "-","_","."'.
 - Description - optional:** A text input field containing 'All Events'.
 - Event bus:** A dropdown menu with 'default' selected. Above it, a note says: 'Select the event bus this rule applies to, either the default event bus or a custom or partner event bus.'
 - Enable the rule on the selected event bus:** A toggle switch that is currently turned on.
 - Rule type:** Two radio button options:
 - Rule with an event pattern:** Selected. Description: 'A rule that runs when an event matches the defined event pattern. EventBridge sends the event to the specified target.'
 - Schedule:** Description: 'A rule that runs on a schedule.'

At the bottom right, there are 'Cancel' and 'Next' buttons.

Event pattern [Info](#)

Event source
AWS service or EventBridge partner as source

AWS services ▼

AWS service
The name of the AWS service as the event source

Health ▼

Event type
The type of events as the source of the matching pattern

All Events ▼

Build an event pattern to match all events from this service

Event pattern
Event pattern, or filter to match the events

```

1 {
2   "source": ["aws.health"]
3 }

```

Copy

Test pattern

Edit pattern

3. Define a Target: After setting up the rule pattern, define a target for the events. You can choose an SNS Topic to receive notifications or a Lambda function to perform custom actions. Select "SNS Topic" or "Lambda function" as the target based on your requirements.
4. Review Rule Details: Finally, Review the details of the rule, such as its name, description, event pattern, target and tags. Once reviewed, create and activate the EventBridge rule.

Monitoring Amazon EC2 Auto Scaling Events

Auto Scaling provides fault tolerance, high availability, and cost management to your computing resources. It automatically scales in (terminates existing EC2 instances) if the demand is low and scales out (launches new EC2 instances) if the incoming traffic exceeds the specified threshold. AWS offers various ways of monitoring your fleet of Amazon EC2 instances as well as responding to Auto Scaling events.

You can use Amazon EventBridge to track the Auto Scaling Events and run a corresponding custom action using AWS Lambda. Amazon EventBridge integrates internal and external services. It allows you to track the changes of your Auto Scaling group in near real-time, including your custom applications, Software-as-a-Service (SaaS) partner apps, and other AWS services.

Amazon EventBridge

Developer resources
Learn
Sandbox
Quick starts
Buses
Event buses
Rules
Global endpoints
Archives
Replays
Pipes
Pipes
Scheduler
Schedules
Schedule groups
Integration
Partner event sources
API destinations
Schema registry
Schemas
Documentation

Creation method

☐ Use schema
Use an Amazon EventBridge schema to generate the event pattern.

☒ Use pattern form
Use a template provided by EventBridge to create an event pattern.

☐ Custom pattern (JSON editor)
Write an event pattern in JSON.

Event pattern Info

Event source
AWS service or EventBridge partner as source

AWS services

AWS service
The name of the AWS service as the event source

Auto Scaling

Event type
The type of events as the source of the matching pattern

Instance Launch and Terminate

Event Type Specification 1
☒ Any instance event
☐ Specific instance event(s)

Specific instance event(s)

Event Type Specification 2
☒ Any group name
☐ Specific group name(s)

Specific group name(s)

Event pattern
Event pattern, or filter to match the events

```

1 {
2   "source": ["aws.autoscaling"],
3   "detail-type": ["EC2 Instance Launch Successful",
4     "EC2 Instance Terminate Successful",
5     "EC2 Instance Launch Unsuccessful",
6     "EC2 Instance Terminate Unsuccessful",
7     "EC2 Instance-launch Lifecycle Action",
8     "EC2 Instance-terminate Lifecycle Action"]
9 }

```

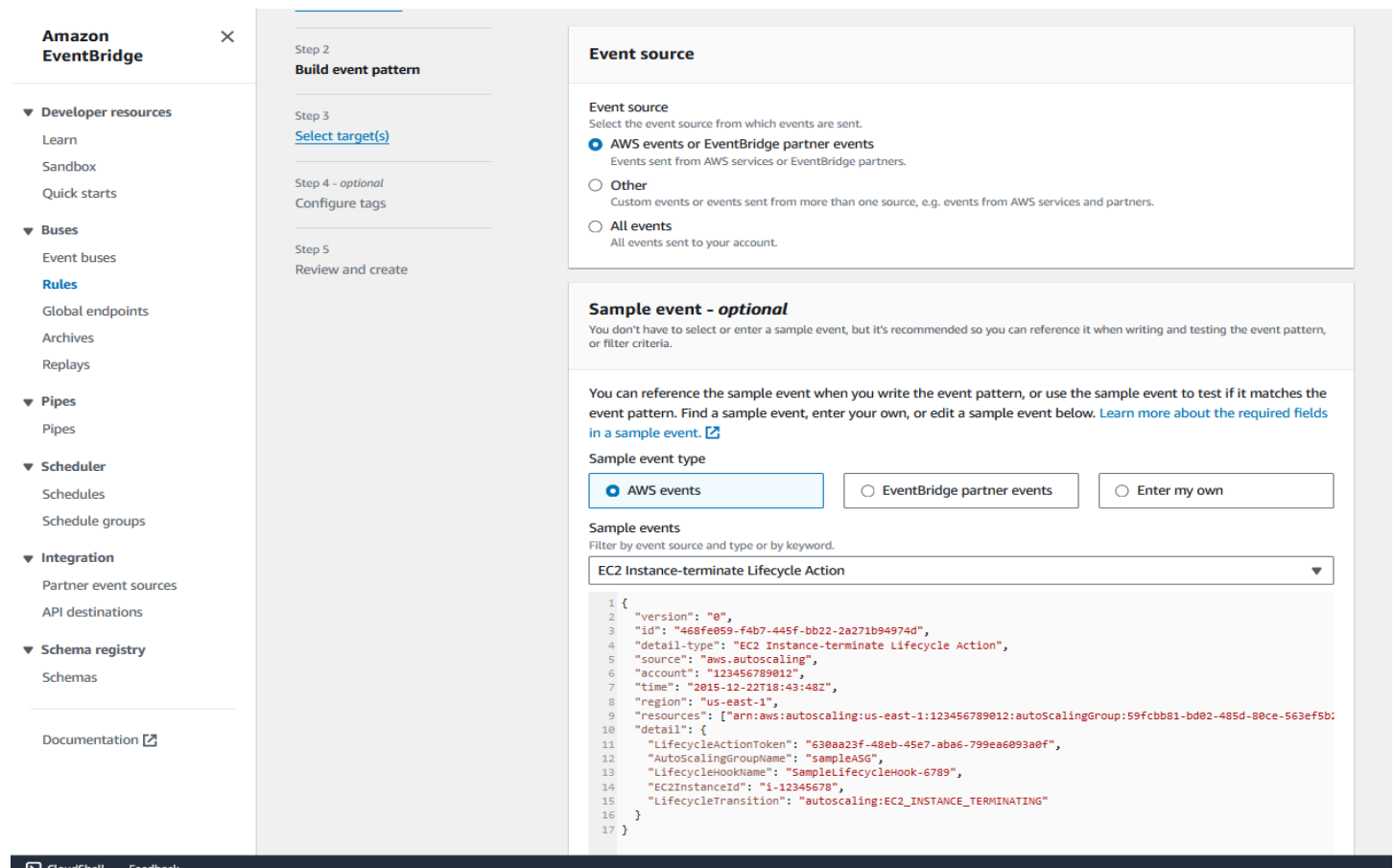
Copy
Test pattern
Edit pattern

Cancel
Previous
Next

Amazon EventBridge can be used to send events to the specified target when the following events occur:

- EC2 Instance-launch Lifecycle Action
- EC2 Instance Launch Successful
- EC2 Instance Launch Unsuccessful
- EC2 Instance-terminate Lifecycle Action
- EC2 Instance Terminate Successful
- EC2 Instance Terminate Unsuccessful

The **EC2 Instance-launch Lifecycle Action** is a scale-out event in which the Amazon EC2 Auto Scaling moved an EC2 instance to a **Pending:Wait** state due to a lifecycle hook. Conversely, the **EC2 Instance-terminate Lifecycle Action** is a scale-in event in which EC2 Auto Scaling updates an instance to a **Terminating:Wait** state.



Amazon EventBridge

Developer resources

- Learn
- Sandbox
- Quick starts

Buses

- Event buses
- Rules**
- Global endpoints
- Archives
- Replays

Pipes

- Pipes

Scheduler

- Schedules
- Schedule groups

Integration

- Partner event sources
- API destinations

Schema registry

- Schemas

Documentation

Step 2: Build event pattern

Step 3: [Select target\(s\)](#)

Step 4 - optional: Configure tags

Step 5: Review and create

Event source

Event source
Select the event source from which events are sent.

☒ **AWS events or EventBridge partner events**
Events sent from AWS services or EventBridge partners.

☐ **Other**
Custom events or events sent from more than one source, e.g. events from AWS services and partners.

☐ **All events**
All events sent to your account.

Sample event - optional

You don't have to select or enter a sample event, but it's recommended so you can reference it when writing and testing the event pattern, or filter criteria.

You can reference the sample event when you write the event pattern, or use the sample event to test if it matches the event pattern. Find a sample event, enter your own, or edit a sample event below. [Learn more about the required fields in a sample event.](#)

Sample event type

☒ **AWS events** ☐ EventBridge partner events ☐ Enter my own

Sample events

Filter by event source and type or by keyword.

EC2 Instance-terminate Lifecycle Action

```

1 {
2   "version": "0",
3   "id": "468fe059-f4b7-445f-bb22-2a271b94974d",
4   "detail-type": "EC2 Instance-terminate Lifecycle Action",
5   "source": "aws.autoscaling",
6   "account": "123456789012",
7   "time": "2015-12-22T18:43:48Z",
8   "region": "us-east-1",
9   "resources": ["arn:aws:autoscaling:us-east-1:123456789012:autoScalingGroup:59fcb81-bd02-485d-80ce-563ef5b"],
10  "detail": {
11    "LifecycleActionToken": "630aa23f-48eb-45e7-aba6-799ea6093a0f",
12    "AutoScalingGroupName": "sampleASG",
13    "LifecycleHookName": "SampleLifecycleHook-6789",
14    "EC2InstanceId": "i-12345678",
15    "LifecycleTransition": "autoscaling:EC2_INSTANCE_TERMINATING"
16  }
17 }

```

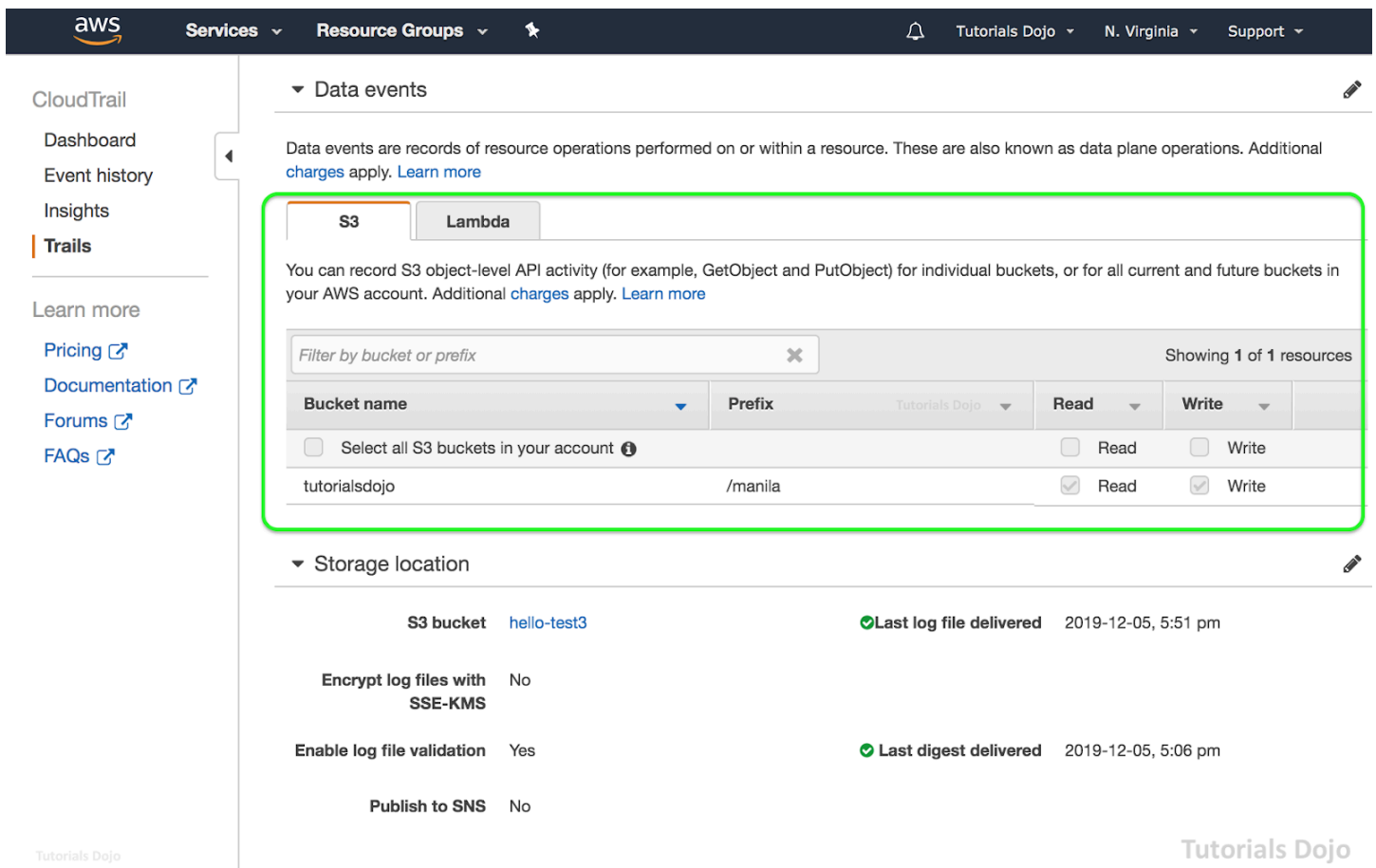
Source:

<https://docs.aws.amazon.com/autoscaling/ec2/userguide/cloud-watch-events.html>

Monitoring Amazon S3 Data Events in AWS CloudTrail

AWS CloudTrail only logs bucket-level actions in your Amazon S3 buckets by default. If you want to record all object-level API activity in your S3 bucket, you can set up data events in CloudTrail. You also have the option to configure data event logging for your individual Lambda functions.

Every data event or log entry in CloudTrail contains essential information about who generated the request to your S3 buckets. This capability allows you to determine whether the S3 request was made by another AWS service, including the IAM User and temporary security credentials used. Amazon S3 Data Events duly records all S3 object changes and updates in your production S3 buckets.



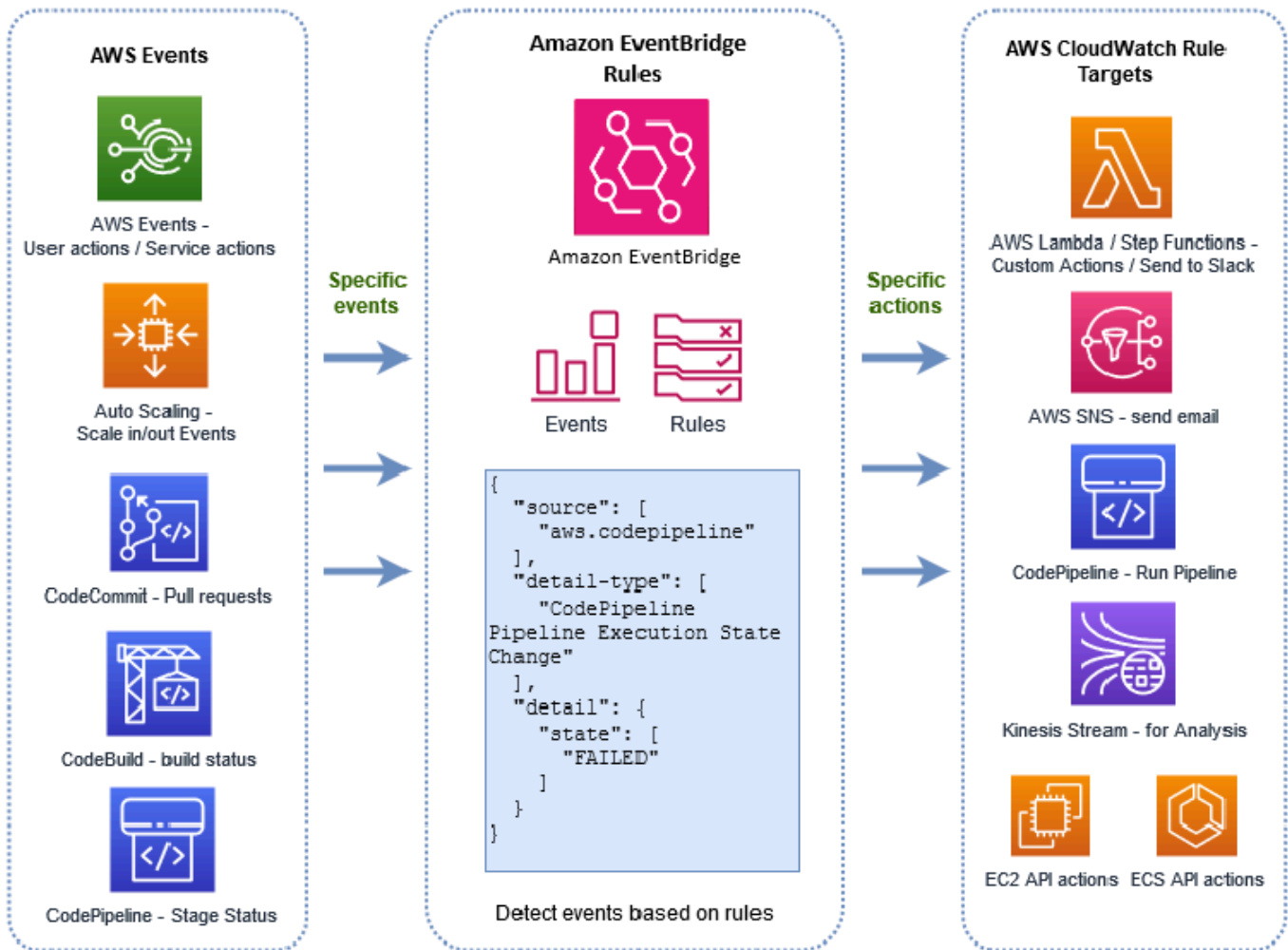
The screenshot shows the AWS CloudTrail console interface. The left sidebar contains navigation links for CloudTrail, Dashboard, Event history, Insights, Trails, and Learn more (Pricing, Documentation, Forums, FAQs). The main content area is titled 'Data events' and includes a description: 'Data events are records of resource operations performed on or within a resource. These are also known as data plane operations. Additional charges apply. [Learn more](#)'. Below this, there are tabs for 'S3' and 'Lambda'. The 'S3' tab is active, showing a configuration for recording S3 object-level API activity. A filter box 'Filter by bucket or prefix' is present, and a table shows 'Showing 1 of 1 resources'. The table has columns for 'Bucket name', 'Prefix', 'Read', and 'Write'. The row shows 'tutorialsdojo' as the bucket name and '/manila' as the prefix, with both 'Read' and 'Write' checkboxes checked. Below the table, the 'Storage location' section shows configuration for the S3 bucket 'hello-test3', including 'Last log file delivered' at 2019-12-05, 5:51 pm, 'Encrypt log files with SSE-KMS' set to No, 'Enable log file validation' set to Yes, and 'Last digest delivered' at 2019-12-05, 5:06 pm. The 'Publish to SNS' option is also set to No.

Source:

<https://docs.aws.amazon.com/AmazonS3/latest/dev/cloudtrail-logging.html>

AWS CodePipeline Event Patterns

You can detect and respond to certain changes in your pipeline state in AWS CodePipeline using Amazon EventBridge. You can use AWS CodePipeline (aws.codepipeline) as the event source of your Amazon EventBridge rule and then associate an Amazon SNS topic to send a notification or a Lambda function to execute a custom action. Amazon EventBridge rule can automatically detect the state changes of your pipelines, stages, or actions in AWS CodePipeline which improves incident and event management of your CI/CD processes.





You can specify both the state and type of CodePipeline execution that you want to monitor. An item can be in a **STARTED**, **SUCCEEDED**, **RESUMED**, **FAILED**, **CANCELED** or **SUPERSEDED** state. Refer to the table below for the list of available detail types that you can use.

Entity	Detail Type
Pipeline	CodePipeline Pipeline Execution State Change
Stage	CodePipeline Stage Execution State Change
Action	CodePipeline Action Execution State Change

You can use this sample event pattern to capture failed **deploy** and **build** actions across all your pipelines in AWS CodePipeline:

```
{
  "source": [
    "aws.codepipeline"
  ],
  "detail-type": [
    "CodePipeline Action Execution State Change"
  ],
  "detail": {
    "state": [
      "FAILED"
    ],
    "type": {
      "category": ["Deploy", "Build"]
    }
  }
}
```




The sample event pattern below captures all **rejected** or **failed** approval actions across all the pipelines:

```
{
  "source": [
    "aws.codepipeline"
  ],
  "detail-type": [
    "CodePipeline Action Execution State Change"
  ],
  "detail": {
    "state": [
      "FAILED"
    ],
    "type": {
      "category": ["Approval"]
    }
  }
}
```

The following sample event pattern tracks all the events from the specified pipelines (**TD-Pipeline-Manila**, **TD-Pipeline-Frankfurt** and **TD-Pipeline-New-York**)

```
{
  "source": [
    "aws.codepipeline"
  ],
  "detail-type": [
    "CodePipeline Pipeline Execution State Change",
    "CodePipeline Action Execution State Change",
    "CodePipeline Stage Execution State Change"
  ],
  "detail": {
    "pipeline": ["TD-Pipeline-Manila",
      "TD-Pipeline-Frankfurt",
      "TD-Pipeline-New-York"]
  }
}
```

Source:

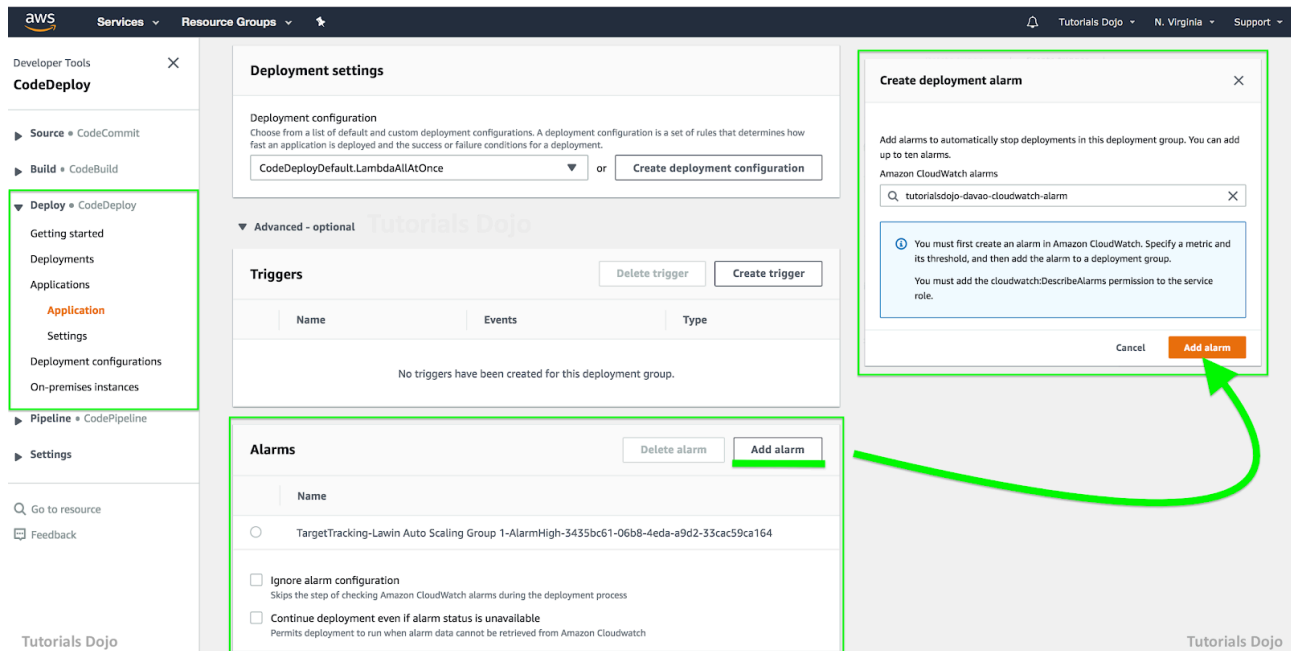
<https://docs.aws.amazon.com/codepipeline/latest/userguide/detect-state-changes-cloudwatch-events.html>

Monitoring Deployments in AWS CodeDeploy

Amazon EventBridge can help you detect any changes in the state of an instance or deployment in AWS CodeDeploy. You can also send notifications, collect state information, rectify issues, initiate events, or execute other actions using CloudWatch Alarms. This type of monitoring is useful if you want to be notified via Slack (or in other channels) whenever your deployments fail or push deployment data to a Kinesis stream for real-time status monitoring. If you integrated Amazon EventBridge in your AWS CodeDeploy operations, you can specify the following as targets to monitor your deployments:

- AWS Lambda functions
- Kinesis streams
- Amazon SQS queues
- Built-in targets (CloudWatch alarm actions)
- Amazon SNS topics

Integrating AWS CodeDeploy and CloudWatch Alarms provides you an automated way to roll back your release when your deployment fails or if certain thresholds are not met. You can easily track the minimum number of healthy instances (**MinimumHealthyHosts**) that should be available at any time during the deployment. The **HOST_COUNT** or **FLEET_PERCENT** deployment configuration parameters can also be utilized to monitor the absolute number or just the relative percentage of healthy hosts respectively.



The screenshot shows the AWS CodeDeploy console. On the left, the 'Deploy' section is expanded, showing 'CodeDeploy' as the selected service. The 'Deployment settings' panel is visible, showing the 'Deployment configuration' dropdown set to 'CodeDeployDefault.LambdaAllAtOnce'. The 'Triggers' section is empty, with a 'Create trigger' button. The 'Alarms' section is also empty, with an 'Add alarm' button. A green box highlights the 'Add alarm' button in the 'Alarms' section. A green arrow points from this button to the 'Create deployment alarm' dialog box on the right. The dialog box has a title bar with a close button. It contains a text area for 'Add alarms to automatically stop deployments in this deployment group. You can add up to ten alarms.' Below this is a search bar for 'Amazon CloudWatch alarms' with the text 'tutorialsdjo-davao-cloudwatch-alarm' entered. A blue information box states: 'You must first create an alarm in Amazon CloudWatch. Specify a metric and its threshold, and then add the alarm to a deployment group. You must add the cloudwatch:DescribeAlarms permission to the service role.' At the bottom of the dialog are 'Cancel' and 'Add alarm' buttons.

Source:

<https://docs.aws.amazon.com/codedeploy/latest/userguide/monitoring-create-alarms.html>



Orchestrating Events in AWS CodePipeline

In AWS CodePipeline, you can control your pipeline's actions and stages to optimize the performance of your CI/CD processes. The `runOrder` and `PollForSourceChanges` parameters can assist you in orchestrating the various activities in your pipeline. For example, you have a serverless application with independent AWS Lambda functions, and you want to expedite the pipeline execution time. You can modify your CodePipeline configuration to execute actions for each Lambda function in parallel by specifying the same `runOrder` value. The `PollForSourceChanges` parameter automates the pipeline creation by tracking source changes from Amazon S3, or Github. In this way, you will have more control over the various stages of your pipeline.

You can also add a manual approval action in CodePipeline before any change is deployed to your production environment. This provides a final checkpoint for your release process after all your unit and integration tests were successfully completed.

Edit action

Tutorials Dojo

Action name
Choose a name for your action

No more than 100 characters

Action provider

Manual approval

Approval

Manual approval

Build
AWS CodeBuild
Add Jenkins

Deploy
AWS AppConfig
AWS CloudFormation
AWS CodeDeploy
AWS Elastic Beanstalk
AWS OpsWorks Stacks
AWS Service Catalog

roduces in your configuration. [Learn more](#)

Cancel Done



By adding a manual approval step in your deployment, you can:

- Perform a code review before a revision is pushed to the next stage of a pipeline.
- Perform manual quality assurance testing on the latest version of a feature or application
- Review any updated feature or application before it is published to the production environment

AWS CodePipeline offers a way to publish approval notifications for your release managers and other authorized staff. A pipeline can be configured with a manual approval action to push a message to an Amazon SNS topic when the approval action was invoked. Amazon SNS delivers the message to every endpoint subscribed to the SNS topic that you specified. Amazon SNS lets the approvers know, via email or SMS, that a new update is ready to be deployed. You can also forward these notifications to SQS queues or HTTP/HTTPS endpoints and execute a custom action using a Lambda function.

Sources:

<https://docs.aws.amazon.com/codepipeline/latest/userguide/reference-pipeline-structure.html#action-requirements>

<https://docs.aws.amazon.com/codepipeline/latest/userguide/approvals.html>



Domain 6: Security and Compliance



Overview

The sixth exam domain of the AWS Certified DevOps Engineer Professional exam deals with the automation of policies and standards in your architecture to enforce logging, metrics, monitoring, testing, and security. Since it covers policies and governance, the related AWS services that you have to review are AWS Organizations, AWS Control Tower, AWS Config, AWS Security Hub, AWS Service Catalog, and AWS Identity and Access Management (IAM).

This domain will test your know-how on the following:

- Designing policies to enforce least privilege access
- Implementing role-based and attribute-based access control patterns
- Automating credential rotation for machine identities via AWS Secrets Manager
- Managing permissions to control access to human and machine identities such as enabling multi-factor authentication (MFA), AWS Security Token Service (AWS STS), IAM profiles and the likes.
- Automating the application of security controls in multi-account and multi-Region environments using AWS Security Hub, AWS Organizations, AWS Control Tower, and the various AWS Systems Manager modules.
- Combining security controls to apply defense in depth with the help of AWS Certificate Manager (ACM), AWS WAF, AWS Config, AWS Config rules, Security Hub, GuardDuty, security groups, network ACLs, Amazon Detective, Network Firewall and other security services.
- Automating the discovery of sensitive data at scale using Amazon Macie.
- Encrypting data in transit and data at rest on AWS KMS, AWS CloudHSM, and AWS ACM.
- Implementing robust security auditing
- Configuring alerting based on unexpected or anomalous security events
- Configuring service and application logging through AWS CloudTrail and Amazon CloudWatch Logs
- Analyzing logs, metrics, and security findings

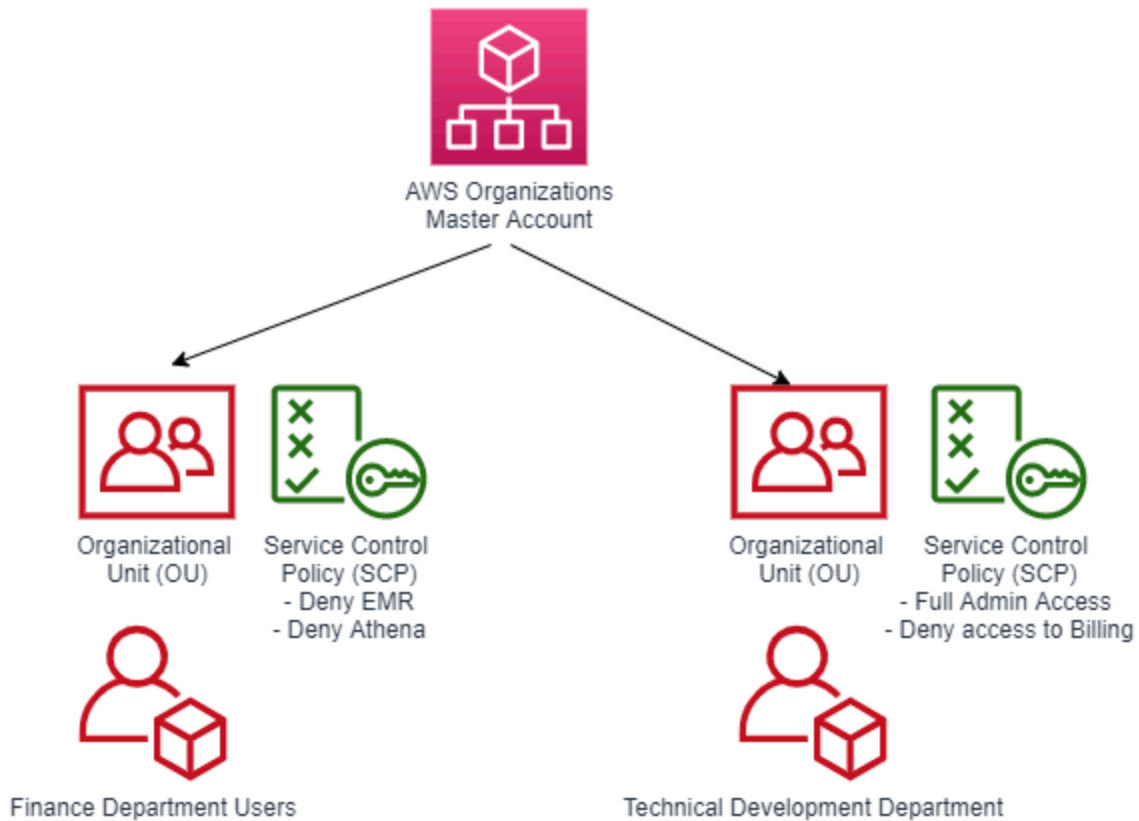


Management and Governance on AWS

Every company has its own standard operating procedures and policies that need to be enforced to its infrastructure to ensure efficient operations and comply with industry regulations. For example, healthcare organizations in the US must abide by the privacy and security standards of the Health Insurance Portability and Accountability Act of 1996 (HIPAA). Other companies also follow the data protection and privacy regulations covered in the General Data Protection Regulation (GDPR), which addresses the transfer of personal data outside the European Union (EU) and other European Economic Area (EEA). Failure to conform to these policies entails millions of dollars in fines and potential loss of revenue. Implementing strict security standards and procedures within your organization could also safeguard your business from unnecessary system failures and data leaks.

AWS provides various management and governance services to provision, manage, govern, and operate their cloud environments more effectively. You can use AWS Organizations, AWS Config, AWS Service Catalog, AWS Systems Manager, and other services to enforce operational standards across all your AWS resources and comply with your corporate IT policies.

The **AWS Organizations** service enables you to govern your AWS accounts and resources centrally. It provides consolidated billing, access control, compliance, and security, as well as the ability to share resources across your AWS accounts. You can use Service Control Policies (SCPs) to ensure that only authorized users can execute actions that meet your policy requirements. Central logging can be implemented to monitor all activities performed across your organization using AWS CloudTrail. You can also aggregate data from all your AWS Config rules to quickly audit your environment for compliance.



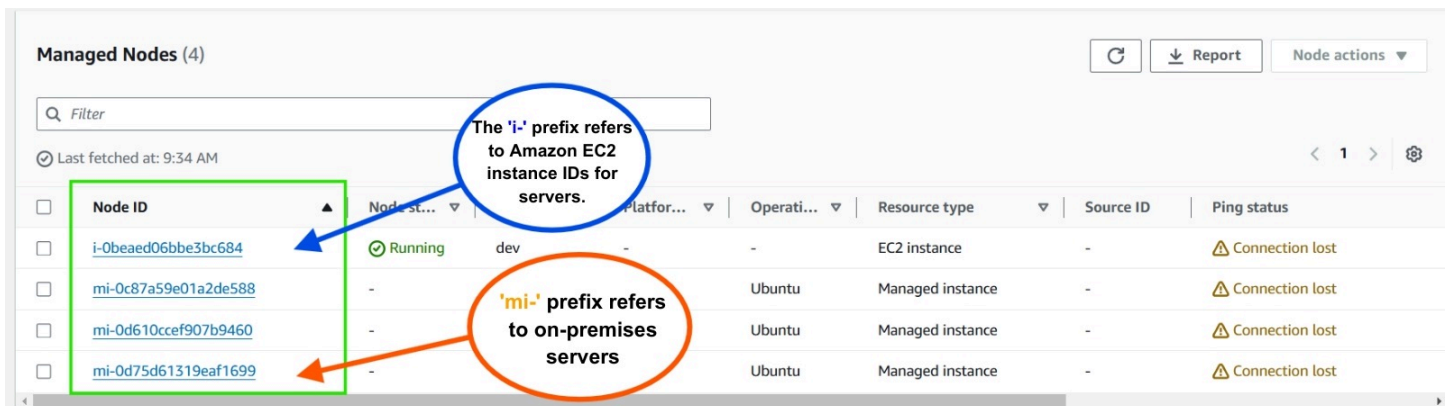
AWS Service Catalog empowers you to set up and centrally manage catalogs of approved IT services that you specify on AWS. You can manage various IT services, referred to as "products" in Service Catalog then group them in a portfolio. In AWS Service Catalog, a product could be a machine image, application server, program, tool, database, or other services that you use for your cloud architecture. AWS Service Catalog assists you in meeting your compliance requirements and enforces granular access control that allows the deployment of only approved IT services to your AWS Cloud.

AWS Config automates the compliance assessment of your internal policies and regulatory standards by giving you visibility on the existing configurations of your various AWS and third-party resources. It continuously assesses changes in your resource configuration and compares them against your specified criteria. You can create rules that detect an EC2 instance running on an unapproved AMI, publicly accessible S3 buckets, and many more. The evaluation can either be triggered periodically or by an actual configuration change of your AWS resource (e.g., CloudTrail was disabled in one of your accounts).

You can integrate AWS Config with Amazon EventBridge, and AWS Lambda to keep you updated for any resource changes in near real-time and to execute custom actions. Remediating noncompliant AWS resources can be automated by just creating AWS Config Rules and AWS Systems Manager Automation documents.

AWS Systems Manager (SSM) is a suite of services that provides you visibility and control of your cloud and on-premises infrastructure. SSM has several features that you can leverage such as the Session Manager, State Manager, Patch Manager, Automation, Maintenance Window, Run Command, Parameter Store, and many other sub-modules.

Through the **SSM agent**, the AWS Systems Manager service can manage both Amazon EC2 instances and on-premises servers in which the former is prefixed with an "i-" and the latter with "mi-" in your AWS Management Console. The **Patch Manager** automates the process of patching the OS of your EC2 instances and on-premises servers using predefined and custom patch baselines. You can set a scheduled maintenance window to execute the patching activities to reduce any operational impact. With the **State Manager**, you can control the configuration details or the "state" of your resources, such as server configurations, virtualized hardware, and firewall settings. You can even associate Ansible playbooks, Chef recipes, PowerShell modules, and other SSM Documents to your resources. The **Systems Manager Parameter Store** provides centralized storage and management of your "parameters" such as passwords, database strings, Amazon Machine Image (AMI) IDs, license codes, environment variables, et cetera. Store the parameter as a *SecureString* datatype to instruct SSM to automatically encrypt it using a KMS key in AWS KMS.



The 'i-' prefix refers to Amazon EC2 instance IDs for servers.

'mi-' prefix refers to on-premises servers

Node ID	Node status	Platform	Operating system	Resource type	Source ID	Ping status
i-0beaed06bbe3bc684	Running	dev	-	EC2 instance	-	Connection lost
mi-0c87a59e01a2de588	-	-	Ubuntu	Managed instance	-	Connection lost
mi-0d610cce907b9460	-	-	Ubuntu	Managed instance	-	Connection lost
mi-0d75d61319eaf1699	-	-	Ubuntu	Managed instance	-	Connection lost

Amazon Inspector is an automated security assessment service that allows you to identify security issues and enforce standards in your AWS environment. You can install an Amazon Inspector agent using the Systems Manager Run Command and run security vulnerability assessments throughout your EC2 instances. Agentless network reachability assessments are also possible. You can run Common Vulnerabilities and Exposures (CVE), Center for Internet Security (CIS) Operating System configuration benchmarks, Network Reachability, and other assessments. It can also assess programs in your instances installed via apt, yum, or Microsoft Installer.



The screenshot displays the Amazon Inspector console for 'Assessment Templates'. The left sidebar shows navigation options: Dashboard, Assessment targets, Assessment templates (selected), Assessment runs, and Findings. The main content area is titled 'Amazon Inspector - Assessment Templates' and includes a description: 'An assessment template allows you to specify various properties for an assessment run, including rules packages, duration, SNS notifications, and how to label any findings.' Below this are buttons for 'Create', 'Run', 'Delete', 'Clone', and 'Create Assessment Events'. A table lists the templates, with 'Assessment-Template-Default-All-Rules' selected. The details for this template are shown below the table, including its Name, ARN, Target name, Rules packages (Common Vulnerabilities and Exposures-1.1, CIS Operating System Security Configuration Benchmarks-1.0, Network Reachability-1.1, Security Best Practices-1.0), Duration (1 Hour), and Assessment Events (7 days). A green arrow points to the 'Rules packages' section.

Amazon GuardDuty fortifies the security of your AWS architecture with intelligent threat detection and continuous monitoring across your AWS accounts. It aggregates and analyzes the data collected from your AWS CloudTrail, VPC Flow Logs, and DNS logs to detect various threats such as intra-VPC port scanning, cryptocurrency mining, malware, backdoor command, control (C&C) activities, and many other vulnerabilities. You can consolidate your security finding by setting up a master account for GuardDuty and associating other AWS accounts as member accounts. This integration can be done via AWS Organizations or by manually sending an invitation to the target member account.

Amazon Macie helps your organizations comply with the Health Insurance Portability and Accountability Act (HIPAA) and General Data Privacy Regulation (GDPR) regulations by detecting personally identifiable information (PII) in your Amazon S3 buckets. It also comes with native multi-account support to efficiently confirm your data security posture across your multiple AWS accounts from a single Macie administrator account.



AWS Trusted Advisor provides a set of best practice checks and recommendations for your AWS infrastructure covering cost optimization, security, fault tolerance, performance, and service limits. You can also fetch the various Trusted Advisor checks programmatically via web service using the AWS Support API. You can even use an EventBridge rule to monitor your Trusted Advisor checks or create a CloudWatch Alarm to notify you of any status changes in your resources. These integrations to other AWS services make it easy to track underutilized Amazon EC2 instances in your account or detect any exposed IAM access keys on public code repositories such as GitHub.

Sources:

<https://aws.amazon.com/blogs/mt/monitor-changes-and-auto-enable-logging-in-aws-cloudtrail/>
<https://docs.aws.amazon.com/config/latest/developerguide/remediation.html>
<https://docs.aws.amazon.com/awssupport/latest/user/trustedadvisor.html>

AWS CodeBuild Configuration Best Practices

Configuration files are essential in your Continuous Integration / Continuous Delivery (CI/CD) pipeline. It contains vital information on how to build your application and the required permissions, passwords or other credentials to enable it to connect to its external dependencies such as a database or other AWS services. The sensitive information in these configuration files could pose a security risk. Part of configuration management is to ensure that these files comply with your organization's security standards and policies.

A build specification file (buildspec.yaml) contains the commands and related settings to instruct AWS CodeBuild on how to build your source code. Adding IAM access keys and database passwords in plaintext in your specification file is discouraged as these could be easily seen by unauthorized personnel. A better approach is to create an IAM Role and attach a permissions policy that grants permissions on your resources. You can store passwords and other sensitive credentials in AWS Systems Manager Parameter Store or AWS Secrets Manager. You can also leverage using AWS Systems Manager Run Command instead of using scp and ssh commands that could potentially expose the SSH keys, IP addresses, and root access of your production servers.

Below is an example of a poorly designed `buildspec.yaml` file. What do you notice?



```
1  version: 0.2
2  run-as: tutorialsdojo-cebu-admin
3  env:
4    variables:
5      AWS_DEFAULT_REGION: "ap-southeast-2"
6      AWS_ACCESS_KEY_ID: "AKIAIOSFOD4NTUTSDOJO"
7      AWS_SECRET_ACCESS_KEY: "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"
8      DATABASE_PASSWORD: "tut0rialsd0j0DBpass"
9  git-credential-helper: yes
10
11 proxy:
12   upload-artifacts: yes
13   logs: yes
14
15 phases:
16   build:
17     run-as: tutorialsdojo-cebu-admin
18     commands:
19       - echo Starting Installation...
20       - cd /tmp
21       - aws s3 cp s3://tutorialsdojo-db/my.cnf.template my.cnf
22       - sed -i 's/MYSQL_PW/${DATABASE_PASSWORD}/' my.cnf
23       - aws s3 cp s3://tutorialsdojo-db/ec2-instance.key ec2-instance.key
24       - chmod 600 ec2-instance.key
25       - scp -i ec2-instance.key my.cnf ubuntu@10.10.5.243:/etc/my.cnf
26       - ssh -i ec2-instance.key ubuntu@10.10.5.243 /etc/init.d/mysqld restart
27   finally:
28     - echo Installation Complete!
```

The above configuration file has the following security issues:

- The AWS access and secret keys are stored in the `buildspec.yaml` file.
- The database credential (`DATABASE_PASSWORD`) is stored as an environment variable in plaintext.
- Contains embedded `scp` and `ssh` commands that expose the SSH keys and server IP addresses.

Source:

<https://docs.aws.amazon.com/codebuild/latest/userguide/data-protection.html>



S3 Bucket Policy to Only Allow HTTPS Requests

By default, Amazon S3 allows both HTTP and HTTPS requests. As part of security compliance, you may be required to ensure that all data in transit to and from your S3 buckets are encrypted using the HTTPS protocol.

If you have a lot of S3 buckets, you can enable the AWS Config rule “**s3-bucket-ssl-requests-only**”, which checks whether S3 buckets have policies that require requests to use Secure Socket Layer (SSL).

Set up AWS Config

Step 1: Settings

Step 2: Rules

Step 3: Review

AWS Config rules

AWS Config can check the configuration of your resources against rules own rules.

s3-bucket-ssl

s3-bucket-ssl-requests-only

Checks whether S3 buckets have policies that require requests to use Secure Socket Layer (SSL).

S3 . Zelkova

DevOps Exam Notes:

To be compliant on the AWS Config rule, your S3 bucket will need to have the proper bucket policy to explicitly deny HTTP requests and only allow HTTPS requests. To determine HTTP or HTTPS requests in a bucket policy, use a condition that checks for the key "aws:SecureTransport". You can set the action to "Deny" any requests if the condition “**aws:SecureTransport**” is **false**.



Here's an example S3 bucket policy to deny HTTP requests on the bucket, thus forcing all connections to be HTTPS-only.:

```
"Id": "ExamplePolicy",
"Version": "2012-10-17",
"Statement": [
  {
    "Sid": "AllowSSLRequestsOnly",
    "Action": "s3:*",
    "Effect": "Deny",
    "Resource": [
      "arn:aws:s3:::exampletutorialsdojobucket",
      "arn:aws:s3:::exampletutorialsdojobucket/*"
    ],
    "Condition": {
      "Bool": {
        "aws:SecureTransport": "false"
      }
    },
    "Principal": "*"
  }
]
```

Sources:

<https://aws.amazon.com/premiumsupport/knowledge-center/s3-bucket-policy-for-config-rule/>
<https://docs.aws.amazon.com/config/latest/developerguide/s3-bucket-ssl-requests-only.html>



Secrets Manager vs. Systems Manager Parameter Store

AWS Secrets Manager and Systems Manager Parameter Store offer similar functionalities that allow you to centrally manage and secure your secret information that can then be retrieved by your applications and resources in AWS.

Both services offer similar web interfaces on which you can declare key-value pairs for your parameters and secrets. So it is important to know the similarities and differences they have in order to choose the right service for a given situation in the exam.

Here's a summary of features of SSM Parameter Store and AWS Secrets Manager:

	SSM Parameter Store	AWS Secrets Manager
Store values up to 4096 Characters	Yes	Yes
Values can be encrypted with KMS	Yes	Yes
Can be referenced in CloudFormation	Yes	Yes
Built-in password generator		Yes
Automated secret rotation		Yes
Cross-account access		Yes
Additional Cost	Free	Yes

DevOps Exam Notes:

In the exam, the usual situation is when you need to store a database password, you should choose SSM Parameter Store if you don't have to automatically rotate the secret regularly, plus it doesn't cost anything.



As an additional note, Parameter Store is now integrated with Secrets Manager so that you can retrieve Secrets Manager secrets when using other AWS services that already support references to Parameter Store parameters. This is helpful if your application is configured to use Parameter Store APIs, but you want your secrets to be stored in Secrets Manager.

Sources:

<https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-parameter-store.html>

<https://aws.amazon.com/about-aws/whats-new/2018/07/aws-systems-manager-parameter-store-integrates-with-aws-secrets-manager-and-adds-parameter-version-labeling/>

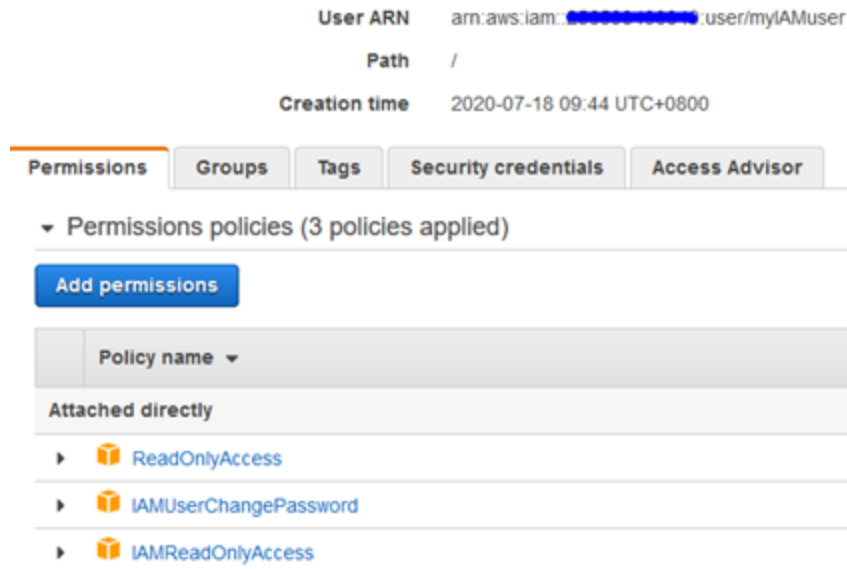


AWS Managed Policy

AWS provides you with three types of policies that you can attach when managing permissions for your IAM Users, Groups or Roles – **AWS managed policy**, **customer managed policy** or an **inline policy**.

An **AWS managed policy** is a standalone policy that is created and administered by AWS which can be used to provide permissions for many common use cases or specific job functions. You cannot change the permissions defined in AWS managed policies. AWS occasionally updates the permissions defined in an AWS managed policy. For example, `arn:aws:iam::aws:policy/IAMReadOnlyAccess` is an AWS managed policy as well as, `AmazonDynamoDBFullAccess`, `IAMFullAccess`, or `AmazonEC2ReadOnlyAccess`.

Example of attached AWS managed policies:



Customer managed policies are standalone policies that you manage on your own AWS Account. You can attach these policies to users, groups, or roles the same way as managed policies. You can copy an AWS managed policy and modify its contents to apply it as a customer managed policy. This gives you much better control of the permissions you grant to your IAM entities.

Example of customer managed policy:

User ARN `arn:aws:iam::[REDACTED]:user/myIAMuser`
 Path `/`
 Creation time 2020-07-18 09:44 UTC+0800

Permissions Groups Tags Security credentials Access Advisor

▼ Permissions policies (1 policy applied)

Add permissions

Policy name ▼
Attached directly
▶ myCustomPolicy

An **inline policy** is a policy that's embedded in an IAM identity (a user, group, or role). When you create an identity in IAM, you can directly attach an inline policy. An inline policy is a strict one-to-one relationship between a policy and the identity that it's applied to. It can't be re-used to be attached to other IAM identities. For example, you want to be sure that the permissions in a policy are not inadvertently assigned to an identity other than the one they're intended for.

User ARN `arn:aws:iam::[REDACTED]:user/myIAMuser` 
 Path `/`
 Creation time 2020-07-18 09:44 UTC+0800

Permissions Groups Tags Security credentials

▼ Permissions policies (1 policy applied)

Add permissions

Policy name ▼	Policy type ▼
Attached directly	
▶ myInlinePolicy	Inline policy

Managed Policy vs Inline Policy

Managed Policies is more flexible than Inline policies. Managed policies are recommended for the following reasons:

- **Reusability** - Can be reused by attaching to other identities
- **Central change management** – one policy change can be applied to all identities.



- **Versioning and rollback** – can create multiple versions and rollback changes if needed.
- **Delegating permissions management** – users can attach/detach policies on their own, while you control the permissions on those policies.
- **Automatic updates for AWS managed policies** – AWS updates managed policies when necessary

Source:

https://docs.aws.amazon.com/IAM/latest/UserGuide/access_policies_managed-vs-inline.html

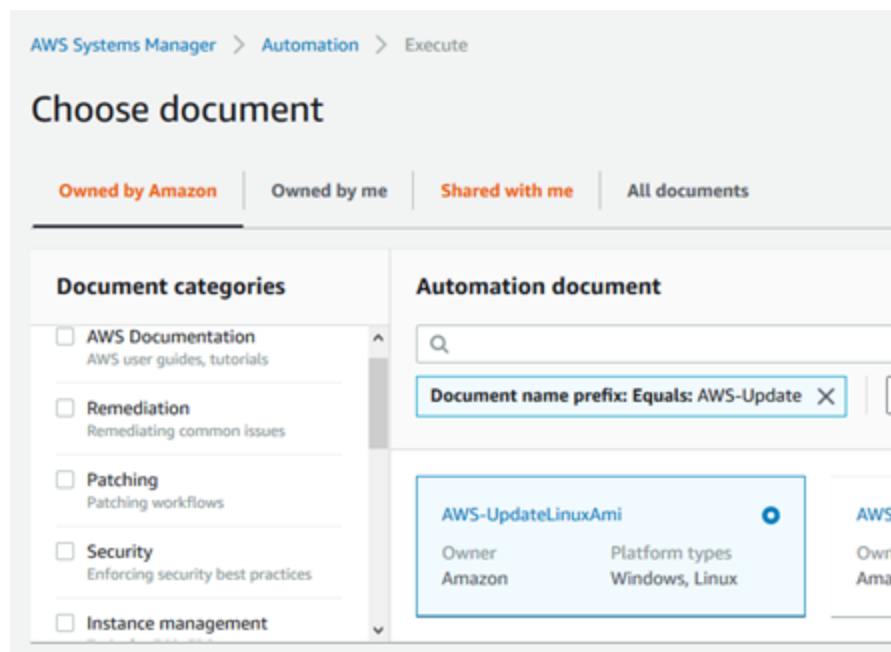
Using Systems Manager Automation to create AMIs

Systems Manager Automation allows you to perform maintenance and deployment tasks for EC2 instances. There are several predefined AWS Automation documents that you provide for common use cases but you can also upload your own Automation documents or share documents with other AWS accounts. Some use cases for SSM Automation are the following:

- Build Automation workflows to configure and manage instances and AWS resources.
- Create custom workflows or use pre-defined workflows maintained by AWS.
- Receive notifications about Automation tasks and workflows by using Amazon EventBridge.
- Monitor Automation progress and execution details by using the Amazon EC2 or the AWS Systems Manager console.

Among the automation workflows is the ability to create updated AMIs. This is helpful if for example, you want to apply the latest system patches to your EC2 instances and create an updated AMI so that all new instances that you will create will have the latest patch applied.

On AWS Systems Manager Automation, select "Execute", and you can choose the Automation Document **AWS-UpdateLinuxAMI**.



The next page will present you with an option to fill out the input parameters for this document. The important part here is the **SourceAmiId**. This value should be your current AMI ID that you want to use as a base and

where the patches will be applied. It's also important to have an **AutomationAssumeRole** present in order to allow SSM Automation to perform the needed actions.

Execute automation document

☒ **Simple execution**
Execute on targets.

☐ **Rate control**
Execute safely on multiple targets by defining concurrency and error thresholds.

☐ **Multi-account and Region**
Execute in multiple accounts and Regions.

☐ **Manual execution**
Step-by-step runbook mode.

Document details

Document name AWS-UpdateWindowsAmi	Document version \$DEFAULT
--	--------------------------------------

▼ Document description

Updates a Microsoft Windows AMI. By default it will install all Windows updates, Amazon software, and Amazon drivers. It will then sysprep and create a new AMI. Supports Windows Server 2008 R2 and greater.

Input parameters

SourceAmiId (Required) The source Amazon Machine Image ID. ami-06ad9296e6cf1e3cf	IamInstanceProfileName (Required) The name of the role that enables Systems Manager to manage the instance. ManagedInstanceProfile
AutomationAssumeRole (Required) The ARN of the role that allows Automation to perform the actions on your behalf. arn:aws:iam::{{global:ACCOUNT_ID}}:role/AutomationServiceR...	TargetAmiName (Optional) The name of the new AMI that will be created. Default is a system-generated string including the source AMI id, and the creation time and date. UpdateWindowsAmi_from_{{SourceAmiId}}_on_{{global:DATE_TIM
TargetImageDescription (Optional) The description of the new AMI that will be created. Updated Windows Ami from {{SourceAmiId}} on {{global:DATE_TIM	InstanceType (Optional) Type of instance to launch as the workspace host. Instance types vary by region. Default is t2.medium. t2.medium

Upon clicking the Execute button, SSM Automation will take the following actions based on the automation document:

- Take the AMI and create a new EC2 instance with it
- Execute the document runbook that applies patches to the OS
- Shutdown the instance
- Create an AMI of the EC2 instance
- Terminate the EC2 instance
- Output the new AMI ID



The output AMI ID can now be stored on SSM Parameter Store or be used by a Launch Template. You can also register this Automation task on the SSM Maintenance Windows console page to allow you to set a schedule for this automation to run regularly.

Sources:

<https://aws.amazon.com/premiumsupport/knowledge-center/ec2-systems-manager-ami-automation/>
<https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-automation.html>

AWS SSM Session Manager to Record Sessions on your Instances

AWS Systems Manager includes the Session Manager service which allows you to manage your EC2 instances, on-premises instances and virtual machines (VMs) through an interactive one-click browser-based shell or through the AWS CLI. With Session Manager, you don't need to open inbound ports or use bastion hosts when you want to have Shell access or RDP access to your EC2 instances.

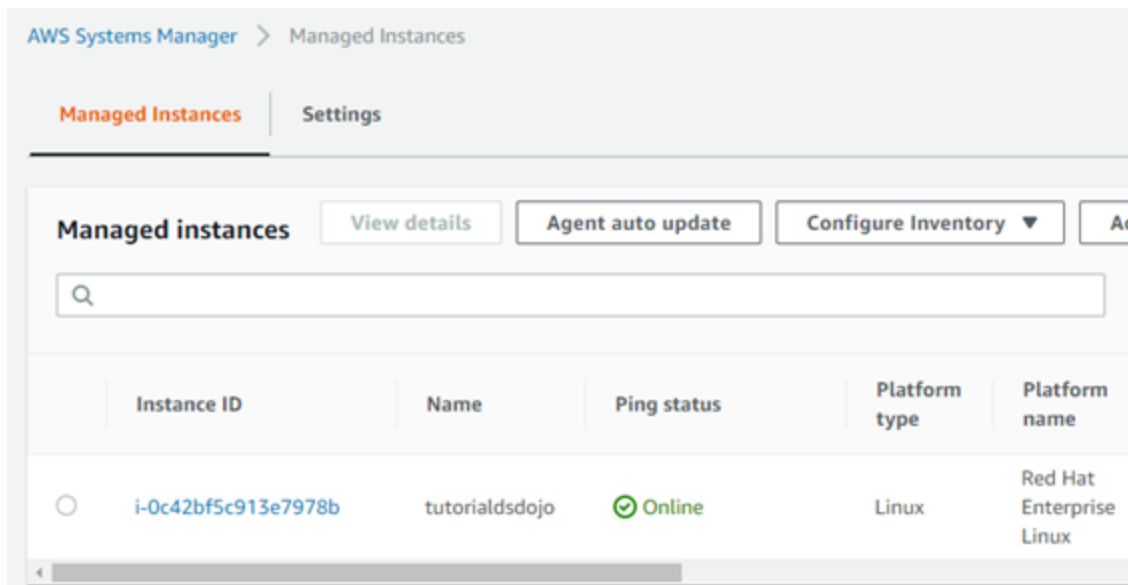
Session Manager also makes it easy to comply with corporate policies that require controlled access to instances, strict security practices, and fully auditable logs with instance access details. For example, you can view all sessions performed by users on the Session Manager History page. Or you can send the whole session log to an S3 bucket or CloudWatch Logs for recording purposes.

DevOps Exam Notes:

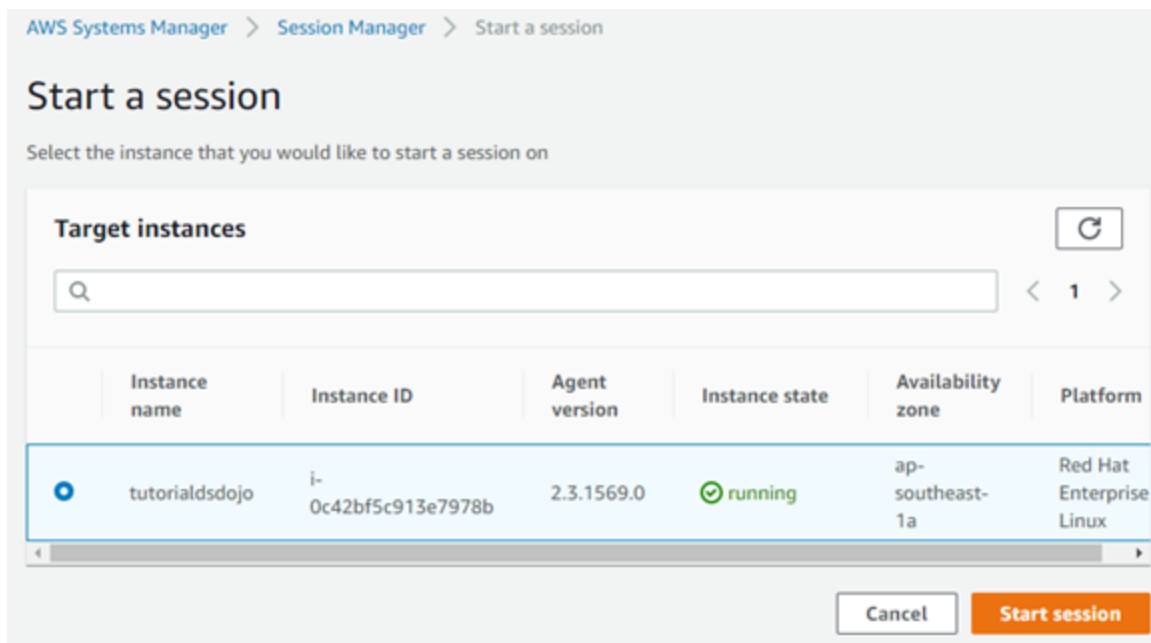
Remember that SSM Session manager provides a centralized location where users can SSH to EC2 instances or on-premises service configured with the SSM agent. For example, you are required by the company to provide a solution for Developers to SSH on both on-premises instances and EC2 instances and save all sessions to an S3 bucket (or CloudWatch Logs) which will be available for the security team to perform an audit. Once you installed and configured the SSM agent on your instances, they will show up on the Session Manager page.

Here's how you can use Session Manager and send your session logs to an S3 bucket.

1. Be sure that your instance has the SSM agent installed and is registered on the Managed Instances section of Systems Manager.



2. Go to Systems Manager > Session Manager and click the “Start session” button. Select your EC2 instance and click “Start Session”.



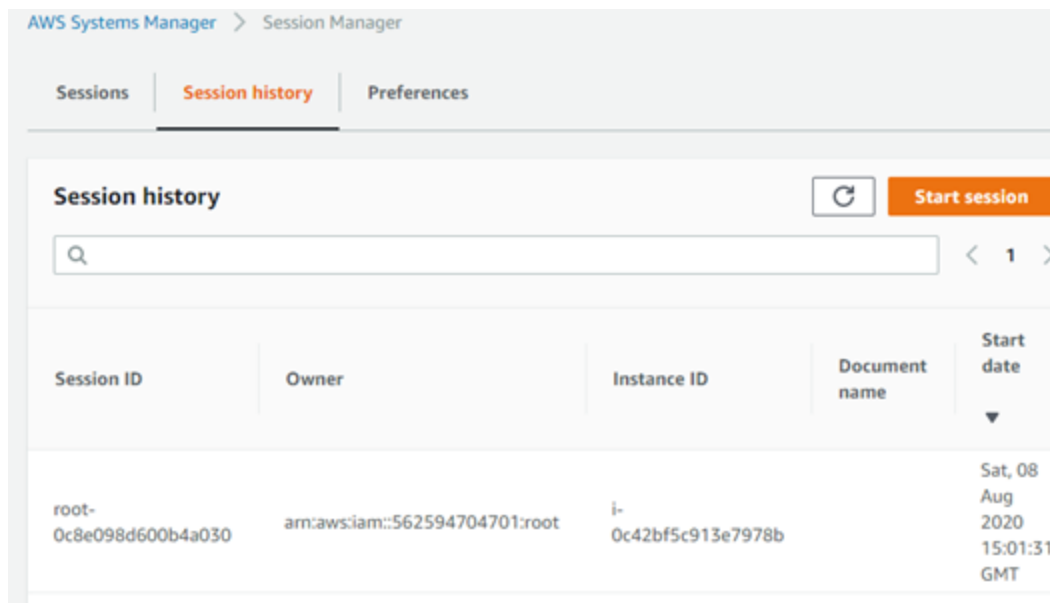
3. You will have a browser tab that provides Shell access to your instance. You can click the “Terminate” button on the upper-right side once you are done with your session.



```
Session ID: root-0c8e098d600b4a030      Instance ID: i-0c42bf5c913e7978b

sh-4.4$ echo "hello tutorials Dojo"
hello tutorials Dojo
sh-4.4$ uptime
 23:01:57 up   5:42,  1 user,  load average: 0.00, 0.00, 0.00
sh-4.4$
```

4. You can view this session on the Session Manager history page.



5. On the preferences page, you can configure Session Manager to send logs to an S3 bucket or CloudWatch Logs. Click the “Edit” button on the Preferences page.



Write session output to an Amazon S3 bucket

You can store and encrypt log data for all sessions in your account in an S3 bucket that you choose. [Learn more](#)

- ☒ S3 bucket
- ☒ Encrypt log data

S3 bucket name

Choose the bucket to store session logs

- ☐ Choose a bucket name from the list
- ☒ Enter a bucket name in the text box

my-session-manager-logs

The bucket is not encrypted

S3 key prefix - optional

To write output to a sub-folder, enter a sub-folder name.

Send session output to CloudWatch Logs

You can stream and encrypt log data for all sessions in your account to a CloudWatch Logs log group in your account. [Learn more](#)

- ☒ CloudWatch logs
- ☒ Encrypt log data

CloudWatch log group

Enter the name of the log group to upload session logs to.

- ☒ Choose a log group name from the list
- ☐ Enter a log group in the text box

CloudWatch log groups

< 1 >

Sources:

<https://docs.aws.amazon.com/systems-manager/latest/userguide/session-manager.html>

<https://docs.aws.amazon.com/systems-manager/latest/userguide/session-manager-getting-started.html>



AWS Systems Manager Inventory

AWS SSM Inventory provides visibility for your Amazon EC2 instances or on-premises servers. You can use SSM Inventory to collect metadata from your managed instances such as installed applications, OS versions, Windows Updates, Network configuration, running services, etc. You can store this metadata in a central Amazon S3 bucket, and then use built-in tools to query the data and quickly determine which instances are running the software and configurations required by your software policy, and which instances need to be updated.

DevOps Exam Notes:

AWS SSM inventory is helpful if you want to make sure that your managed instances have the correct application versions and patches installed. You even filter which instances have outdated system files. On the exam, you need to remember that SSM Inventory is also helpful if you want to identify details of your on-premises instances that you want to migrate to AWS. It will help you view and gather details of those instances to sync the data collected on an S3 bucket. This helps ensure that your new instances on AWS will have similar configurations to those on your current on-premises environment.

The following steps show how to set up Systems Manager Inventory:

1. Ensure that your instance is registered as a Managed Instance on Systems Manager.
2. Go to Systems Manager > Inventory and click "Setup Inventory". Set a name for this inventory association.

Setup Inventory

Create an inventory association to collect information about software and settings for a target set of managed instances.

Provide inventory details

Name - *Optional*

Provide a name for your Inventory.



3. Select targets for your inventory. You can choose to select all instances, or use a specific Tag identifier, or manually select your desired instances.

Targets

Specify targets by

☐ Selecting all managed instances in this account

☐ Specifying a tag

☒ Manually selecting instances

X

< 1 >

☒	Name	Instance ID	Instance state	Availability zone	Ping status
☒	TutorialsDojo	i-0c42bf5c913e7978b	running	ap-southeast-1a	Online

4. You can set the schedule on how often the inventory is updated as well as the parameters it will collect on the instances.



Schedule

(Requires SSMAgent version 2.0.790.0 and above)

Collect inventory data every Minute(s) ▼

Parameters

- ☒ **Applications**
(Optional) Collect data for installed applications.
- ☐ **AWS Components**
(Optional) Collect data for AWS Components like amazon-ssm-agent.
- ☒ **Network Config**
(Optional) Collect data for Network configurations.
- ☐ **Windows Updates**
(Optional, Windows OS only) Collect data for all Windows Updates.
- ☐ **Instance Detailed Information**
(Optional) Collect additional information about the instance, including the CPU model, speed, and the number of cores, to name a few.

5. You can also specify an S3 bucket on which the inventory will sync the data it collects.



Advanced

☒ Sync inventory execution logs to an S3 bucket

S3 bucket name
Type the name of a bucket in Amazon S3.

S3 bucket prefix - optional
Type a prefix for the bucket above that receives the output; for example, mycommands/domainjoin.

[Cancel](#) [Setup Inventory](#)

6. After clicking the “Setup Inventory” button, the details of your instances will be shown on the SSM Inventory page.

Sources:

<https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-inventory.html>

<https://docs.aws.amazon.com/systems-manager/latest/userguide/sysman-inventory-configuring.html>



Systems Manager Patch Manager and Maintenance Windows

AWS SSM Patch Manager automates the process of patching managed instances with both security-related and other types of updates. Patch Manager supports several OS types such as Windows Server, Amazon Linux, Ubuntu, RHEL, Debian, Centos, SLES, etc. It allows you to select patch baselines which are a list of approved patches that will be applied to your instances. You can choose from AWS pre-defined patch baselines or you can create your own patch baseline if you have your own custom repository.

AWS SSM Maintenance Windows allows you to define schedules for when to perform potentially disruptive actions on your instances such as patching an operating system or installing software updates. Each maintenance window has a schedule, a duration, a set of registered targets, and a set of registered tasks. When you create a Patch Manager task, it will be shown on the Maintenance Windows Tasks page.

DevOps Exam Notes:

On the exam, you need to remember that Patch Manager allows you to define custom patch baselines. This is helpful if your company has a custom repository of software updates that need to be installed on your managed instances. You can schedule the application of the patches by creating a Maintenance Window and assigning your Patching task to that window.

Below are the steps to set up Maintenance Windows and Patch Manager for your managed instances.

1. Ensure that your managed instances are registered on Systems Manager.
2. Go to Systems Manager > Maintenance Windows and click "Create Maintenance Window".
3. Provide a name and description for your Maintenance window.



Create maintenance window

A maintenance windows lets you specify when a target set of managed instances should install updates or perform maintenance activities. Specify the details below to create a new maintenance window:

Provide maintenance window details

Name

Type a name for this maintenance window.

It has to be between 3 and 128 characters. Valid characters contain the following: a-z, A-Z, 0-9, and . _ -

Description - optional

Type description for this maintenance window.

It has to be between 1 and 128 characters.

Unregistered targets

Allow maintenance tasks scheduled for this maintenance window to run on targets that are not currently registered with this maintenance window.

☐ Allow unregistered targets

4. Provide a schedule for this maintenance. We'll set this to run every Sunday morning at 1:00 AM with a 3-hour window time. And one hour before the window closes, SSM will stop initiating tasks.

Schedule

Specify with

- ☒ Cron schedule builder
- ☐ Rate schedule builder
- ☐ CRON/Rate expression

Window starts

- ☐ Every 30 minutes
- ☐ Every hours
- ☒ Every at

Duration

Maintenance window duration

hours

Value from 1 to 24.

Stop initiating tasks

Time to stop starting scheduled task before maintenance window ends

hour before the window closes

Value from 0 to 23.

5. Click "Create maintenance window". You will then see your created maintenance window.

AWS Systems Manager > Maintenance Windows

Maintenance windows

< 1 >

	Window ID	Name	State	Next execution time
☐	mw-0beaf6011b344a198	tutorialDojo-sunday-maintenance-window	Enabled	Sun, Aug 9, 2020, 1:00:00 AM UTC

6. Now go to Patch Manager and click "Configure patching". Select your instances to patch. You can select by instance tags, patch groups, or by selecting the instances manually.

Instances to patch

How do you want to select instances?

☐ Enter instance tags
☐ Select a patch group
☒ Select instances manually

Select one or more instances you want to patch.

i-0c42bf5c913e7978b X

<
1
>

☒	Name	Instance ID	Platform Type
☒	TutorialsDojo	i-0c42bf5c913e7978b	Linux

7. Select your Patching schedule then select the previously created maintenance window.

Patching schedule

How do you want to specify a patching schedule?

☒ Select an existing Maintenance Window
☐ Schedule in a new Maintenance Window
☐ Skip scheduling and patch instances now

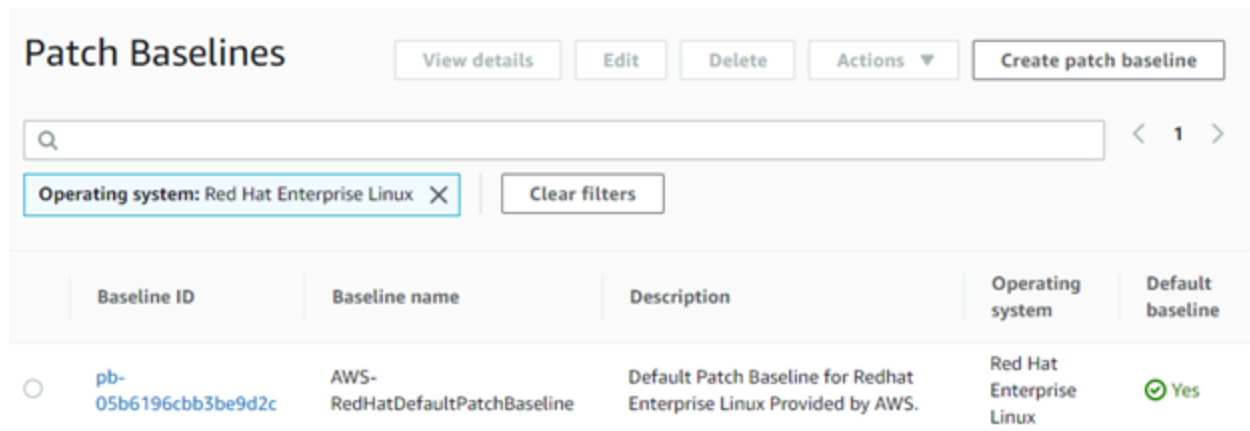
Maintenance Window

Select a [Maintenance Window](#) .

tutorialDojo-sunday-maintenance-window (ID: mw-0beaf601... ▼

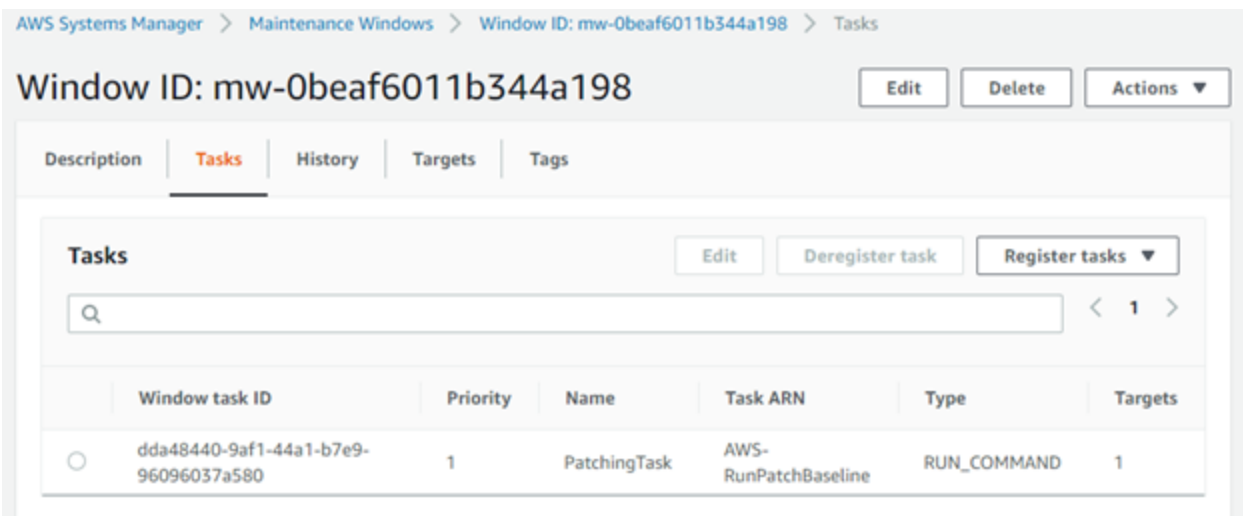
↺

8. Click “Configure patching” to create this patching task. You should see the “AWS-RedHatDefaultPatchBaseline” for your instance as the default baseline.



	Baseline ID	Baseline name	Description	Operating system	Default baseline
☐	pb-05b6196cbb3be9d2c	AWS-RedHatDefaultPatchBaseline	Default Patch Baseline for Redhat Enterprise Linux Provided by AWS.	Red Hat Enterprise Linux	✔ Yes

9. Go to the Maintenance Windows page again and select your maintenance window. Click the Tasks section and you should see your configured Patching task.



	Window task ID	Priority	Name	Task ARN	Type	Targets
☐	dda48440-9af1-44a1-b7e9-96096037a580	1	PatchingTask	AWS-RunPatchBaseline	RUN_COMMAND	1

10. SSM Maintenance Windows will now run this task on the specified window time.

Sources:

<https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-patch.html>

<https://docs.aws.amazon.com/systems-manager/latest/userguide/systems-manager-maintenance.html>



AWS IAM Identity Center

AWS IAM Identity Center allows you to centrally manage access to multiple AWS accounts and applications using identity federation. Some ways you can implement identity federation using IAM Identity Center are:

1. Connect your existing identity provider, like Active Directory, to IAM Identity Center. This allows you to continue managing your users in AD and sync the user identities to the IAM Identity Center.
2. Define policies in the IAM Identity Center to control what AWS accounts and resources individual users or groups can access.
3. Create IAM roles in the AWS accounts and attach the required permissions policies to allow federated users to assume roles.
4. Users can then access AWS accounts and applications using a single set of credentials through the IAM Identity Center user portal after signing in using their AD credentials.
5. You can also federate user identities from external identity providers like Okta, PingFederate etc using SAML 2.0 or OpenID Connect.
6. IAM Identity Center also supports just-in-time access to AWS accounts by allowing users to directly assume roles without needing long term credentials.

Sources:

<https://docs.aws.amazon.com/singlesignon/latest/userguide/what-is.html>

<https://repost.aws/knowledge-center/iam-identity-center-federation>



RELATED AWS SERVICES TO FOCUS



Amazon Elastic MapReduce (EMR)

The Amazon Elastic MapReduce (EMR) service allows you to run different types of big data frameworks in AWS. This is a managed big data platform for processing vast amounts of data using open-source tools such as Apache Hadoop, Flink, HBase, HCatalog, Hive, Hudi, Hue, Jupyter, Livy, MXNet, Oozie, Phoenix, Pig, Presto, Spark, Sqoop, TensorFlow, Tex, Zeppelin, ZooKeeper and many more. It is quite obvious that you can run a lot of open-source data analytics workloads with Amazon EMR.

Technically, Amazon EMR runs your big data framework on Amazon EC2 instances, Amazon Elastic Kubernetes Service clusters, or in your on-premises EMR cluster via AWS Outposts. These compute resources are deployed in your VPC and then grouped as an Amazon EMR cluster. You can directly access and control the underlying EC2 instances of your EMR cluster. Take note that this service is not serverless since it is using Amazon EC2 and EKS. The Amazon EMR service just automates the server provisioning and management process for you. The data in your EMR cluster can also interact with other AWS data stores such as Amazon S3 and Amazon DynamoDB.

This service also has a more cost-efficient feature called Amazon EMR Serverless, which is a serverless option in Amazon EMR that simplifies the process of running open-source big data analytics frameworks for data analysts and engineers to run without configuring, managing, and scaling clusters or any virtual servers. Amazon EMR Serverless provides all the features and benefits of the standard Amazon EMR but without the management overhead of planning, managing, and maintaining any computing clusters.

Amazon QuickSight

Amazon QuickSight is a scalable, serverless, embeddable, machine-learning-powered business intelligence service. It lets you create and publish interactive dashboards that can be accessed from different browsers or mobile devices. You can also embed dashboards into your applications. Amazon QuickSight is highly scalable and can easily scale up to thousands of users without any software to install, servers to deploy, or infrastructure to manage.

Amazon Kinesis Data Streams

Amazon Kinesis Data Streams is a powerful AWS service designed for real-time processing and analysis of large data streams. It allows you to easily collect, process, and analyze streaming data, enabling applications to respond quickly to new information.

Features

- Amazon Kinesis Data Streams is ideal for real-time applications like log and event data processing, real-time analytics, and complex stream processing.
- The fundamental data unit stored in Kinesis Data Streams consists of a sequence number, partition key, and data blob, which can be up to 1MB.



- Shards are the basic throughput unit of Kinesis Data Streams, capable of handling up to 1MB/s of data for writing or 1,000 records/s and 2MB/s of data for reading.
- Offers two modes, on-demand and provisioned, to manage and scale the service according to your needs.
- Data records are stored for a default of 24 hours, with the option to extend storage to 365 days for long-term analysis (additional charges may apply).
- Producers input data into streams, while consumers process and analyze the data in real-time.

Use Cases

- Ideal for collecting and analyzing infrastructure performance and operational data in real-time.
- Perform analytics on data as it arrives, such as monitoring application activity or user interactions on a website.
- Support for advanced data processing models, including multi-stage processing and parallel processing of streams.

Amazon AppFlow

Amazon AppFlow is a fully managed integration service that enables you to securely transfer data between Software-as-a-Service (SaaS) applications and your AWS Services. It supports different SaaS apps such as Salesforce, Marketo, Slack, ServiceNow, and many more. You can also integrate other AWS services like Amazon S3 and Amazon Redshift in just a few clicks. With AppFlow, you can run your data flows on-demand or by setting up a schedule. You can also run it as a response to a business event. Amazon AppFlow provides you with powerful data transformation capabilities like filtering and validation to easily generate rich and ready-to-use data for your custom applications.

Amazon Eventbridge

Amazon Eventbridge is a serverless event bus that makes it easy to connect applications together using data from your own applications, Software-as-a-Service (SaaS) applications, and other AWS services. Amazon EventBridge is meant to be used for your own applications, SaaS apps, and other external sources to complement the data provided by AWS services. It is used for building event-driven applications that take care of event ingestion and delivery, security, authorization, and error handling for the user.

AWS App Runner

AWS App Runner allows you to easily run your web application, API services, backend web services, and websites on AWS without any infrastructure or container orchestration required. This service allows you to directly use your existing container image, container registry, source code repository, or existing CI/CD workflow to a fully running containerized web application on the AWS platform and run it in a matter of minutes.



It seamlessly integrates with your development and CI/CD workflow to provide the appropriate level of automation to launch your code, application, or container image faster. AWS App Runner automates all the required dependencies which eliminate the need for you to understand, provision, scale, or manage any AWS compute resources. In addition, this service also manages all the related networking, and routing resources for your application. App Runner empowers you to run thousands of applications that automatically scale while providing security and compliance best practices.

AWS App2Container

AWS App2Container or A2C is a command-line tool for modernizing .NET and Java applications into containerized applications. A2C analyzes and builds an inventory of all applications running in virtual machines, on-premises, or in AWS. You just have to choose the application you want to containerize, and A2C will package the application artifact and dependencies into container images. It also configures the network ports and generates the ECS task and Kubernetes pod definitions.

Amazon EKS Deployment Options

Amazon Elastic Kubernetes Service or Amazon EKS is a managed service that you can use to run Kubernetes on AWS. It's like Amazon ECS, but instead of Docker containers, this service is used for running Kubernetes clusters. Amazon EKS automates the installation, operation, and maintenance of your own Kubernetes control plane, pods, and nodes.

You can deploy your Kubernetes cluster in various ways in AWS and can include additional networking add-ons to improve your containerized architecture. A Kubernetes container can be deployed via

- Amazon EKS cluster in your AWS account
- Amazon EKS on AWS Outposts
- Amazon EKS Anywhere
- Amazon EKS Distro

The first option allows you to launch a Kubernetes cluster using managed or self-managed Amazon EC2 nodes that you can customize and control. You can also choose to deploy your Kubernetes pods on AWS Fargate to make the cluster serverless and extremely cost-effective.

Amazon EKS on AWS Outposts

Amazon EKS on AWS Outposts is a deployment option that uses a physical AWS Outpost rack on your on-premises network to run your Kubernetes workloads. The data plane is also located on-premises, so you can have more control compared with running it exclusively in AWS. Using Amazon EKS Anywhere is another way to deploy your containers on-premises. It works like Amazon ECS Anywhere, which allows you to run your Kubernetes cluster entirely on your own. This means that the hardware, app deployment location, control plane,



and data plane are all controlled on your own physical network. This gives you extensive control over all the components of your containerized application suite while maintaining official support from AWS.

Amazon EKS Distro

The other deployment option that you can choose is to deploy your Kubernetes container is through the Amazon EKS Distro option. The word “distro” simply refers to the distribution of the same open-source Kubernetes software deployed by Amazon EKS in the AWS cloud. Amazon EKS Distro follows the same Kubernetes version release cycle as Amazon EKS and is provided to you as an open-source project that you can deploy on your own computer or on-site environment. It’s similar to the Amazon EKS Anywhere option, except that it does not include support services offered by AWS.



Red Hat OpenShift Service on AWS (ROSA)

Red Hat OpenShift Service on AWS (ROSA) is a service in the AWS Cloud that is operated by Red Hat and jointly supported with AWS. It provides a fully managed Red Hat OpenShift platform on a pay-as-you-go billing option. This allows enterprise developers who are familiar with deploying their applications with OpenShift on-premises to quickly build and deploy applications. ROSA is also integrated with AWS Security Token Service (STS) support for a more integrated experience. This makes deploying cloud applications with dependencies on AWS cloud-native services much easier.

AWS Database Migration Service (AWS DMS)

AWS Database Migration Service or AWS DMS helps you migrate your databases to AWS quickly and securely. The source database remains fully operational during the migration, minimizing downtime to applications that rely on the database. The AWS Database Migration Service can migrate your data to and from the most widely used commercial and open-source databases, and it can also be used for continuous data replication with high availability. You can use AWS Schema Conversion Tool or AWS SCT to make heterogeneous database migration. You can transform the schema of your source database to a different one. You can convert PostgreSQL database to MySQL, Oracle to Amazon Aurora, Apache Cassandra to DynamoDB, and many more.

Amazon DocumentDB (with MongoDB compatibility)

Amazon DocumentDB is a fast, scalable, highly available MongoDB-compatible database service. MongoDB is a document-oriented database program that is cross-platform and is also a type of a NoSQL database. In a MongoDB database, you call a table a “collection”; a row a “document” and a column a “field”. Each document contains fields and values in JSON format with no rigid schema enforced, unlike in traditional SQL databases. This is also the same concept in Amazon DocumentDB – it stores, queries, and index JSON documents. That’s how DocumentDB got its name!

Amazon MemoryDB for Redis

As the name suggests, Amazon MemoryDB for Redis is a Redis-compatible, durable, in-memory database service in the AWS cloud. It delivers ultra-fast cache performance with microsecond read latency, single-digit millisecond write latency, high throughput, and Multi-AZ durability for modern applications. Amazon MemoryDB for Redis is perfect for microservices architectures that require distributed caching and for web applications that need low latency, high scalability, and flexible data structures. It also has APIs to make your development experience more agile and simplified.

Amazon MemoryDB for Redis stores your entire dataset in memory for faster access compared to storing it on disk drives. It leverages a distributed transactional log to provide both in-memory speed and data durability, consistency, and recoverability in the event of system outages.



Amazon ElastiCache

Let's now discuss in-memory databases in AWS that are primarily used for caching.

As its name implies, Amazon ElastiCache is a caching service that allows you to set up, run, and scale open-source in-memory databases like Memcached or Redis. By storing data in-memory, they can read data more quickly than disk-based databases. If your application experiences performance slowdowns by having frequent calls to return identical datasets, then you must apply database caching to remove this bottleneck. You can refactor your application to use Amazon ElastiCache and fetch the data in-memory, instead of fetching the same exact datasets again and again. Aside from caching, you can use this service in real-time analytics, distributed session management, geospatial services, and many more. In ElastiCache, there are two types that you can launch: Amazon ElastiCache for Memcached or Amazon ElastiCache for Redis. Both of these types provide sub-millisecond latency, data partitioning and require a minimal amount of code to integrate into your application.

Amazon ElastiCache for Memcached is based on the open-source Memcached in-memory data store. This is suitable for building a simple, scalable caching layer for your data-intensive apps. Memcached is multithreaded, which means it can utilize multiple processing cores. It lets you handle more operations by scaling up compute capacity. The downside of using Memcached is its lack of data replication capability, which can affect the availability of your application.

Amazon ElastiCache for Redis is based on the open-source Redis in-memory data store. It provides advanced data structures, pub/sub messaging, geospatial, and point-in-time snapshot support. In addition, it also has a replication feature that is not available in Memcached. So if you need an in-memory database storage solution that provides high availability using data replication, then this type is the one to use. You just have to enable the "Cluster Mode" in Redis to have multiple primary nodes and replicas across two or more Availability Zones.

AWS Command Line Interface (AWS CLI)

AWS Command Line Interface (AWS CLI) is a unified tool for managing AWS services. It enables users to manage multiple AWS services via the command line and automate them using scripts. The latest version, AWS CLI v2, introduces several enhancements, including improved installers, new configuration options like AWS IAM Identity Center, and various interactive features. It is designed to simplify the management and deployment of AWS resources, offering commands for a wide range of AWS services. Users can perform actions such as launching and managing EC2 instances, publishing messages on SNS topics, or syncing files to S3 buckets directly from the command line. The AWS CLI also supports file commands for efficiently managing Amazon S3 objects and providing commands for listing, uploading, and syncing files. The AWS CLI makes it easier for developers and system administrators to interact with AWS services through scripting and direct command line access.



AWS Cloud Development Kit (AWS CDK)

The AWS Cloud Development Kit, or AWS CDK for short, is an open-source software development kit for Amazon Web Services. You can use this to programmatically model your AWS infrastructure using TypeScript, Python, Java, .NET, or any other programming languages that you prefer.

The AWS CDK Command Line Interface (CLI) can be used to interact with your CDK applications in AWS. CDK CLI is capable of listing the stacks defined in your CDK app, synthesizing the stacks into AWS CloudFormation templates, determining the differences between running stack instances and the stacks defined in your AWS CDK code, and deploying stacks to any public AWS Region that you choose.

The CDK framework is primarily used to author AWS CDK projects that are executed to generate AWS CloudFormation templates. Various projects made using AWS CDK can be executed using the AWS CDK command line or in a continuous delivery system that you own.

AWS CloudShell

AWS CloudShell is a browser-based shell that is available in the AWS Management Console. This service makes it easier to securely manage, explore, and interact with your AWS resources via a cloud-based command line interface. This is pre-authenticated with your IAM user or AWS Management Console credentials.

Various common development and operations tools are pre-installed on CloudShell, so there's no need to install or set up any software on your local machine. You can easily run scripts with the AWS Command Line Interface (AWS CLI), experiment with AWS service APIs using the AWS SDKs, or use a plethora of other tools to fast-track your production process.

AWS CloudShell empowers you to automate tasks, manage infrastructure, and interact with various AWS resources. This service can be used to clone repositories containing commonly used scripts, make edits to those scripts, and store them for future reference. The Amazon Elastic Container Service (Amazon ECS) CLI and the AWS Serverless Application Model (AWS SAM) CLI can be integrated with AWS SDKs to develop applications and use common CLIs, such as the person to manage your AWS resources.

AWS CodeArtifact

Basically, AWS CodeArtifact is a fully managed artifact repository service that can securely store, publish, and distribute software packages. This is beneficial for companies in simplifying their software development process and application deployment. The AWS CodeArtifact service works with commonly used package managers and builds tools like Maven and Gradle, Node Package Manager (NPM), yarn, pip, twine, NuGet, and others.



AWS CodeArtifact can be integrated with AWS CodeBuild to improve your CI/CD workflow. The CodeArtifact repositories can be specified as a source/target for consuming and publishing packages in your AWS CodeBuild project configuration. The CodeBuild images have client tools for all the package types, which makes the integration to CodeArtifact faster.

You can provision the build's IAM role in AWS CodeBuild and configure the build tool or package manager to use the target repository and fetch a CodeArtifact AUTH token at the launch of the build. Once the build completes, the artifacts can be published to your AWS CodeArtifact repository. The AWS CodeBuild builds can also be triggered using Amazon EventBridge emitted by a CodeArtifact repository when one of its contents changes.



Amazon CodeGuru

Amazon CodeGuru is a suite of development services in AWS. It contains different tools and features such as Amazon CodeGuru Reviewer, Amazon CodeGuru Profiler, BugBust, and many more. The primary function of Amazon CodeGuru Reviewer is to provide intelligent recommendations for improving your application performance, efficiency, and code quality. It can scan your code and detect a plethora of code defects like bad exception handling, insecure CORS policy, path traversal, hardcoded credentials, and many more. You can also integrate this with your CI/CD workflow so you can run code reviews and recommendations to improve your codebase.

The other module for this service is called the Amazon CodeGuru Profiler. A profiler is basically a component that collects your CPU data and analyzes the runtime performance data from your live applications. This is helpful in identifying expensive lines of codes that inefficiently use the CPU, which causes CPU bottlenecks.

AWS Fault Injection Simulator (AWS FIS)

AWS Fault Injection Simulator is a managed service that provides you the capability to perform fault injection experiments or simulations on your AWS workloads. Fault injection is based on the principles of chaos engineering, which is basically the process of testing a distributed computing system to verify that it can withstand unexpected disruptions or faults. In AWS Fault Injection, you run “experiments” in your AWS workloads to stress the applications or the underlying resources by creating disruptive events, which allows you to observe how your enterprise application responds. The information you gathered here can improve the performance and resiliency of your applications. These experiments help you create the real-world scenarios needed to uncover rare application issues that can be quite difficult to see or spot. AWS FIS provides templates that generate disruptions as well as the needed controls and guardrails to run experiments in production. It also provides an option to automatically roll back or stop an experiment if certain conditions are met.

AWS Control Tower

AWS Control Tower is a service that helps you set up and govern a secure multi-account AWS environment. It automates the setup of your multi-account AWS environment with just a few clicks. The setup uses blueprints that follow AWS best practices for security and management. Control Tower provides mandatory high-level rules, called guardrails, that help enforce your policies using service control policies or detect policy violations using AWS Config rules.

You can use the AWS Control Tower service to automate the manual process of setting up a new landing zone. Each landing zone that is launched by AWS Control Tower includes all the relevant best practices, identity blueprints, federated access, and account structure. The blueprints implemented on AWS Control Tower include, but are not limited to, the following:

- A multi-account environment via AWS Organizations
- Cross-account security audits using AWS IAM and AWS IAM Identity Center



- Identity management via the Identity Center default directory
- Centralized logging from AWS CloudTrail, and AWS Config stored in Amazon S3

There are particular aspects of your AWS Control Tower landing zone are configurable in the AWS Management Console like the selection of Regions and optional controls while other modifications may be made outside the console through automation. You can create more extensive customizations of your landing zone with the **Customizations for AWS Control Tower** capability, which is a GitOps-style customization framework that works with AWS Control Tower lifecycle events as well as AWS CloudFormation templates.

Customizations for AWS Control Tower (CfCT)

The Customizations for AWS Control Tower (CfCT) feature helps you customize your AWS Control Tower landing zone while maintaining compliant with the best practices in AWS. The different customizations are implemented via AWS CloudFormation templates and service control policies (SCPs) for a more automated and granular approach. The AWS Control Tower lifecycle events are fully integrated with CfCT, which allows your resource deployments to be kept synchronized with your landing zone.

When a new AWS account is created via an Account Factory, all cloud resources that you configured and attached to the account are deployed automatically. Custom templates and policies can also be deployed to individual AWS accounts and organizational units (OUs) within your organization.



Amazon Compute Optimizer

AWS Compute Optimizer is a service that optimizes your computing capacity in AWS by helping you right-size your Amazon Elastic Compute Cloud (EC2) instance types, Amazon Elastic Block Store (EBS) volume configurations, and AWS Lambda function memory sizes. It uses machine learning to identify the optimal AWS resource configurations by analyzing historical utilization metrics. AWS Compute Optimizer comes with a set of APIs and an intuitive console experience to reduce your OPEX and efficiently increase workload performance by recommending the appropriate AWS resources for your AWS workloads.

For example, the AWS Compute Optimizer can recommend that you resize your current Amazon EC2 instance to a smaller instance type if the CPU Utilization of your virtual machine is constantly at 20%, even during peak hours. Conversely, it can optimize your workloads by suggesting that you vertically scale up your instance if it keeps breaching its burst capacity.

Amazon Managed Grafana

Amazon Managed Grafana is a fully managed service for Grafana. Grafana is an open-source analytics platform that is commonly used to query, visualize, observe, and make use of your system data that are gathered from multiple sources. When you hear the phrase “Amazon Managed”, that means AWS is managing the underlying infrastructure required to run an open-source program or a particular tool. For Amazon Managed Grafana, AWS is the one that provisions and manages the required resources to run your Grafana dashboards, along with its other dependencies.

Amazon Managed Grafana can collect system metrics from multiple data sources in your observability stack, such as Amazon Managed Service for Prometheus, Amazon CloudWatch, and Amazon OpenSearch Service. System alerts can also be automated by using different notification services in AWS. You can also integrate this with third-party vendors like Datadog, Splunk et cetera. In addition, you can set up your own self-managed data source like InfluxDB and integrate it with your Grafana workspace. You also don't have to worry about the infrastructure required to run your Grafana visualizations and dashboards since the necessary resources are all provisioned and managed by AWS itself.

Amazon Managed Service for Prometheus

Amazon Managed Service for Prometheus is a fully managed service for the open-source monitoring tool called Prometheus. This is commonly used for monitoring modern cloud-native applications and Kubernetes clusters. Prometheus can enable you to securely ingest, store, and query metrics from different container environments. In Amazon Managed Service for Prometheus, the resources required to run the open-source Prometheus tool are all provisioned and managed by the AWS team themselves. Scaling the underlying architecture is also handled by Amazon. You have the option to collect system metrics from your container clusters running in AWS, running on-premises, or even both.



Amazon Managed Service for Prometheus allows you to use the open-source Prometheus query language or PromQL. This query language helps you to monitor the performance of your containerized workloads that are running on the AWS Cloud or on-site. It can also scale automatically as your workloads increase or shrink and uses AWS security services to enable fast and secure access to data.

Amazon FSx

Amazon FSx is a storage service in AWS. It has various types namely:

- Amazon FSx for Lustre
- Amazon FSx for Windows Server
- Amazon FSx for NetApp ONTAP
- Amazon FSx for OpenZFS

Amazon FSx for Lustre is quite similar to Amazon EFS. It is also a POSIX-compliant shared file system that only supports Linux servers. The word “Lustre” actually refers to the open-source file system that this service is using. Lustre is a parallel file system used for large-scale cluster computing. The name Lustre is basically a combination of the word Linux and cluster. FSx for Lustre is primarily used for High-Performance Computing, machine learning, or HPC applications that need high-performance parallel storage for frequently accessed ‘hot’ data. This service can provide a throughput of hundreds of gigabytes per second and millions of IOPS to support your demanding workloads. You can mount an Amazon FSx for Lustre file share to your EC2 instances or your containers. You can use the Container Storage Interface (CSI) to connect to your Amazon EKS cluster.

Amazon FSx for Windows Server is essentially a fully managed Microsoft Windows file server. Unlike Lustre, which is Linux-based, this service is backed by a fully native Windows file system. There are a lot of Microsoft-based technologies that you can integrate with this service. You can access this file share using the Server Message Block protocol or SMB. SMB is a protocol commonly used by Windows servers. You can also integrate your existing Microsoft Active Directory to provision file system access to your users. The Amazon FSx for Windows Server can be used as shared file storage for your Microsoft SharePoint, Microsoft SQL Server database, Windows Container, or any other Windows-based applications.

Amazon FSx for NetApp ONTAP is a storage service in the AWS Cloud that enables customers to launch and run fully managed ONTAP file systems in the cloud. Basically, ONTAP is NetApp’s file system technology that provides a widely adopted set of data access and data management capabilities for various organizations. Amazon FSx for NetApp ONTAP provides the same features, performance, and APIs of on-premises NetApp file systems but is bestowed with greater agility, scalability, and simplicity of a fully managed AWS service.

Amazon FSx for OpenZFS is a fully managed file storage service that enables customers to easily launch, run, and scale fully managed file systems that are built on the open-source OpenZFS file system. This service helps you migrate your on-premises file servers without changing your enterprise applications or even on how you



manage data. You can use FSx for OpenZFS in building new high-performance, data-intensive enterprise applications on the AWS cloud.

AWS Backup

The function of the AWS Backup service is quite obvious based on its name. AWS Backup is a fully managed backup service that makes it easy to automate your server and database backups. By default, the automated backups of your RDS databases only have a 7-day retention period. The maximum retention period is 35 days. By using AWS Backup, you can perform a daily snapshot of the database with the retention set to 90 days, a year, or even longer! You can also create a lifecycle policy to automatically move your backups to cold storage.

AWS Elastic Disaster Recovery

The AWS Elastic Disaster Recovery (AWS DRS) service helps companies and organizations minimize downtime and data loss with fast, reliable recovery of on-premises and cloud-based applications. It uses affordable storage, minimal compute, and point-in-time recovery to meet your RTO/RPO in a cost-effective manner.

The resiliency of your AWS workloads will further be improved when you use AWS Elastic Disaster Recovery to replicate on-premises or cloud-based applications running on supported OS to various AZs and regions. The AWS Management Console can be used to configure replication and launch settings, monitor data replication, and launch instances for Business Continuity Process (BCP) drills or even for recovery procedures. The AWS Elastic Disaster Recovery can be configured on your source servers to initiate secure data replication to specific targets you specified. Your custom data is replicated to a staging area subnet of your Amazon VPC that is located in the AWS Region, and AWS account you select. Replicating the data into a staging area reduces costs through the use of affordable cloud storage options and minimal compute resources to maintain the ongoing replication.

AWS Proton

AWS Proton is a service that automates container and serverless deployments in AWS. It empowers your platform teams and developers to have consistent development standards and best practices. This service is very useful if you have a large number of developers in your organization. AWS Proton enables your developers to deploy container and serverless applications using pre-approved stacks that your platform team manages. It balances control and flexibility in your organization by allowing developers to innovate within the set guardrails that you implement.

It also offers a self-service portal for your developers, which contains AWS Proton templates that they can use and deploy. A Proton template contains all the information required to deploy your custom environments and services. You can create an AWS Proton Component as well, which provides flexibility to your service



templates. These components in AWS Proton provide platform teams with a way to extend core infrastructure patterns and define guardrails for your developers.

AWS CloudHSM

AWS CloudHSM is a fully managed, cloud-based hardware security module or HSM. Obviously, the word: “HSM” in CloudHSM means “hardware security module”, which enables you to easily generate and use your own encryption keys. These encryption keys can be 128-bit or 256-bit, which are used to encrypt your custom data or other encryption keys. An HSM is just a physical hardware device that performs cryptographic operations and securely stores cryptographic key material. This key material is basically a random, Base64 or hexadecimal string in a binary format that is used by your encryption key.

In CloudHSM, the cluster can be accessed or managed using CloudHSM clients, which is installed and hosted in your Amazon EC2 instances. The CloudHSM cluster is deployed in your Amazon VPC. Your clients can communicate with your HSM cluster using the elastic network interfaces of your HSMs. Since all of these resources are in your Amazon VPC and under your control, the CloudHSM cluster only has one user or tenant – which is you.

This is what single-tenant access means in CloudHSM. This service can be used to offload the SSL Processing for your web servers, enabling Transparent Data Encryption (TDE) for Oracle Databases and protecting the private keys for an Issuing Certificate Authority (CA). You can also integrate CloudHSM and AWS KMS to create a custom key store.

AWS Network Firewall

AWS Network Firewall is a managed network firewall service for your Amazon Virtual Private Clouds. This network security service is a managed network firewall that comes with intrusion prevention and detection capabilities. The AWS Network Firewall service allows you to filter traffic within the perimeter of your Amazon VPCs. This service is commonly used in various network security use cases such as inspecting VPC-to-VPC traffic, filtering outbound traffic, securing both AWS Direct Connect connection and VPN traffic as well as filtering the Internet traffic. AWS Network Firewall also offers fine-grained network security controls for interconnected VPCs via the AWS Transit Gateway.

You can also use this to filter your outbound traffic to prevent unwanted data loss, block malware, and satisfy your strict network security compliance requirement. A single AWS Network Firewall can be configured with



thousands of rules that can filter out network traffic routed to known bad IP addresses or suspicious domain names. It can also protect the AWS Direct Connect or VPN traffic that originates from client devices and your on-premises environments. The AWS Network Firewall can ensure that only authorized sources of traffic are granted access to your mission-critical VPC resources. It is also capable of performing the same activities as your Intrusion Detection Systems and Intrusion Prevention Systems or IDP/IPS. This is achieved by inspecting all inbound Internet traffic using features such as ACL rules, stateful inspection, protocol detection, intrusion prevention, et cetera.

Amazon Detective

Amazon Detective makes it easy to analyze, investigate, and quickly identify the root cause of potential security issues or suspicious activities. Amazon Detective automatically collects log data from your AWS resources. What it basically does is collect logs from AWS CloudTrail, Amazon VPC Flow Logs, Amazon GuardDuty findings, and other AWS services, then use machine learning to analyze and conduct security investigations.

Amazon Cognito

Amazon Cognito is a security service for managing user identities, allowing developers to easily add user sign-in, sign-up, and access control to their web and mobile apps. The service consists of two main components: User Pools and Identity Pools.

User Pools function as a user directory managing sign-up and sign-in processes for app users. They enable users to sign in directly with a user pool or through various third-party identity providers (IdPs) like Google, Facebook, Amazon, and Apple. Amazon Cognito User Pools comply with standards such as OpenID Connect (OIDC) for authentication and OAuth 2.0 for authorization, offering a secure and scalable user directory that can be easily integrated into your apps.

Identity Pools (Federated Identities) enable developers to grant authenticated users temporary access to AWS services. By integrating Identity Pools into your application, you can authenticate users through an external IdP and then provide them with credentials to access AWS resources like S3 buckets or DynamoDB tables, adhering to fine-grained access controls based on their identity.

Key Features:

- Enhance security by enabling MFA, including SMS-based verification and Time-based One-Time Passwords (TOTPs).
- Customize user sign-up and sign-in flows with AWS Lambda triggers and adjust authentication flows according to your business needs.
- Identity providers supported by AWS Cognito include
 - Social identity providers such as Google, Facebook and Amazon



- Enterprise identity providers like Microsoft Active Directory, LDAP, and SAML 2.0-compatible providers
- Custom authentication solutions using OpenID Connect (OIDC) or OAuth 2.0 protocols
- Automatically handle millions of users without requiring upfront infrastructure investment or management overhead
- Create precise access control policies, assigning roles and permissions based on user attributes, groups, or custom conditions, ensuring secure resource access for authorized users.

AWS Virtual Private Network (AWS VPN)

AWS Virtual Private Network, or AWS VPN, is a service that enables you to securely connect your on-premises network to AWS. This is basically just a regular VPN, which is an encrypted connection that passes through the public Internet. It uses the IPsec protocol to authenticate and encrypt your data in transit.

It is comprised of two services: AWS Site-to-Site VPN and AWS Client VPN. The Site-to-Site VPN creates encrypted tunnels between your network and your Amazon VPCs or AWS Transit Gateways. On the other hand, AWS Client VPN is simply a software that allows your users to connect to AWS or to on-premises resources. Both of these types have a corresponding endpoint to your VPC. You can create a site-to-site VPN endpoint or a client VPN endpoint.

AWS PrivateLink

AWS PrivateLink provides companies and customers to privately access their AWS services hosted in a highly available and scalable way while keeping all the network traffic within the internal AWS network. Customers can access services powered by PrivateLink in a private connection from their Amazon Virtual Private Cloud (VPC) or their on-premises, without using public IPs or requiring traffic to traverse across the public Internet. Network Load Balancers can also use PrivateLink services to grant access to their services to other AWS customers.

You have to create Interface-type VPC endpoints for certain AWS services to take advantage of the PrivateLink feature. These service endpoints will be shown as Elastic Network Interfaces (ENIs) with private IPs in your Amazon VPCs. Any traffic destined to these IPs will get privately routed to the corresponding AWS services internally without traversing the public Internet.

AWS License Manager

AWS License Manager simplifies the process of managing your software licenses from various vendors such as IBM, SAP, Oracle, and Microsoft across your AWS Cloud and on-premises environments. This service lets administrators craft customized licensing rules that mirror the terms of their licensing agreements. System Administrators can use these custom rules to avoid licensing violations like using more licenses than their service level agreement stipulates. These rules in AWS License Manager prevent companies from inadvertently



having licensing breach by stopping the instance from launching or by sending a notification to system administrators about a potential license infringement. You can gain control and visibility of all your software licenses with the AWS License Manager dashboard, reducing the risk of non-compliance, misreporting, and unnecessary costs due to licensing overages.

AWS Service Health Dashboard

The AWS Service Health Dashboard is basically just a public dashboard that shows the status of all AWS Services across various regions. The AWS Service Health Dashboard contains the most up-to-the-minute information on the service availability of each and every AWS service. You can be notified of any service interruptions by subscribing to an RSS feed.

AWS Health API

AWS Health API provides programmatic access to the AWS Health information that appears in your AWS Personal Health Dashboard. The AWS Health API is basically a RESTful web service that you can access via HTTPS and return a response in JSON format. This service is not available by default. You must have a Business or Enterprise support plan in order to use this service.

AWS Resilient Hub

AWS Resilience Hub can assist companies and organizations to proactively prepare and protect their AWS applications from unexpected service disruptions. This service offers resiliency assessment and validation that integrate into your software development lifecycle (SDLC) to discover the weak points of your cloud architecture. AWS Resilience Hub provides the capability to estimate whether or not the recovery time objective (RTO) and recovery point objective (RPO) for your applications and cloud solution can be met. It also helps resolve resiliency issues even before they are released into your production environment. You can continue to use the AWS Resilience Hub even after you deploy your solutions into production to track the resiliency posture of your enterprise application. AWS Resilience Hub will send a notification to your team to launch the associated recovery process in the event of a service outage.



AWS CHEAT SHEETS



AWS Compute Services

Amazon Elastic Compute Cloud (EC2)

- A Linux-based/Windows-based/Mac-based virtual server that you can provision.

Features

- Server environments called **instances**.
- Package OS and additional installations in a reusable template called **Amazon Machine Images**
- Secure login information for your instances using **key pairs**
- Storage volumes for temporary data that are deleted when you STOP or TERMINATE your instance, known as **instance store volumes**.
- Persistent storage volumes for your data using **Elastic Block Store volumes** (see aws storage services).
- Multiple physical locations for deploying your resources, such as instances and EBS volumes, known as **regions** and **Availability Zones** (see AWS overview).
- A firewall that enables you to specify the protocols, ports, and source IP ranges that can reach your instances using **security groups** (see aws networking and content delivery).
- Static IPv4 addresses for dynamic cloud computing, known as **Elastic IP addresses** (see aws networking and content delivery).
- Metadata, known as **tags**, that you can create and assign to your EC2 resources
- Virtual networks you can create that are logically isolated from the rest of the AWS cloud, and that you can optionally connect to your own network, known as **virtual private clouds** or **VPCs** (see aws networking and content delivery).
- Add a script that will be run on instance boot called **user-data**.
- **Host Recovery for Amazon EC2** automatically restarts your instances on a new host in the event of an unexpected hardware failure on a Dedicated Host.

Instance states

- To prevent accidental termination, enable termination protection.

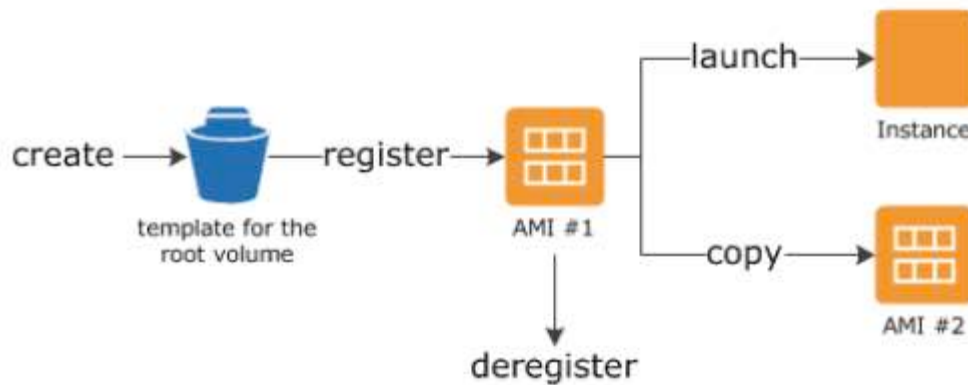
Root Device Volumes

- The root device volume contains the image used to boot the instance.
- Instance Store-backed Instances
 - Any data on the instance store volumes is deleted when the instance is terminated (instance store-backed instances do not support the Stop action) or if it fails (such as if an underlying drive has issues).
- Amazon EBS-backed Instances

- An Amazon EBS-backed instance can be stopped and later restarted without affecting data stored in the attached volumes.
- By default, the root device volume for an AMI backed by Amazon EBS is deleted when the instance terminates.

AMI

- Includes the following:
 - A template for the root volume for the instance (OS, application server, and applications)
 - Launch permissions that control which AWS accounts can use the AMI to launch instances
 - A block device mapping that specifies the volumes to attach to the instance when it's launched



- You can copy AMIs to different regions.

Pricing

- On-Demand - pay for the instances that you use by the second, with no long-term commitments or upfront payments.
- Reserved - make a low, one-time, up-front payment for an instance, reserve it for a *one-* or *three-* year term, and pay a significantly lower hourly rate for these instances.
- Spot - request unused EC2 instances, which can lower your costs significantly. Spot Instances are available at up to a 90% discount compared to On-Demand prices.

Security

- Use IAM to control access to your instances (see AWS Security and Identity Service).
 - IAM policies
 - IAM roles



- Restrict access by only allowing trusted hosts or networks to access ports on your instance.
- A **security group** acts as a virtual firewall that controls the traffic for one or more instances.
 - Evaluates all the rules from all the security groups that are associated with an instance to decide whether to allow traffic or not.
 - By default, security groups allow **all outbound traffic**.
 - Security group rules are **always permissive**; you can't create rules that deny access.
 - Security groups are **stateful**
- You can replicate the network traffic from an EC2 instance within your Amazon VPC and forward that traffic to security and monitoring appliances for content inspection, threat monitoring, and troubleshooting.

Networking

- An **Elastic IP address** is a static IPv4 address designed for dynamic cloud computing. With it, you can mask the failure of an instance or software by rapidly remapping the address to another instance in your account.
- If you have not enabled auto-assign public IP address for your instance, you need to associate an Elastic IP address with your instance to enable communication with the internet.
- An elastic **network interface** is a logical networking component in a VPC that represents a virtual network card, which directs traffic to your instance
- Scale with **EC2 Scaling Groups** and distributes traffic among instances using **Elastic Load Balancer**.

Monitoring

- EC2 items to monitor
 - CPU utilization, Network utilization, Disk performance, Disk Reads/Writes using EC2 metrics
 - Memory utilization, disk swap utilization, disk space utilization, page file utilization, log collection using a monitoring agent/CloudWatch Logs
- Automated monitoring tools include:
 - System Status Checks - monitor the AWS systems required to use your instance to ensure they are working properly. These checks detect problems with your instance that require AWS involvement to repair.
 - Instance Status Checks - monitor the software and network configuration of your individual instance. These checks detect problems that require your involvement to repair.
 - Amazon CloudWatch Alarms - watch a single metric over a time period you specify and perform one or more actions based on the value of the metric relative to a given threshold over a number of time periods.
 - Amazon EventBridge - automate your AWS services and respond automatically to system events.
 - Amazon CloudWatch Logs - monitor, store, and access your log files from Amazon EC2 instances, AWS CloudTrail, or other sources.



- Monitor your EC2 instances with CloudWatch. By default, EC2 sends metric data to CloudWatch in 5-minute periods.
- You can also enable detailed monitoring to collect data in 1-minute periods.

Instance Metadata and User Data

- **Instance metadata** is data about your instance that you can use to configure or manage the running instance.
- View all categories of instance metadata from within a running instance at <http://169.254.169.254/latest/meta-data/>
- Retrieve user data from within a running instance at <http://169.254.169.254/latest/user-data>



Amazon Elastic Container Registry (ECR)

- A managed AWS Docker registry service. Stores your Docker images that you can use to deploy on your EC2, ECS, or Fargate deployments.

Features

- ECR supports Docker Registry HTTP API V2 allowing you to use Docker CLI commands or your preferred Docker tools in maintaining your existing development workflow.
- You can transfer your container images to and from Amazon ECR via HTTPS.

Components

- **Registry**
 - A registry is provided to each AWS account; you can create image repositories in your registry and store images in them.
 - The URL for your default registry is `https://aws_account_id.dkr.ecr.region.amazonaws.com`.
- **Authorization token**
 - Your Docker client needs to authenticate to ECR registries as an AWS user before it can push and pull images. The AWS CLI `get-login` command provides you with authentication credentials to pass to Docker.
- **Repository**
 - An image repository contains your Docker images.
 - **ECR lifecycle policies** enable you to specify the lifecycle management of images in a repository.
- **Repository policy**
 - You can control access to your repositories and the images within them with repository policies.
- **Image**
 - You can push and pull Docker images to your repositories. You can use these images locally on your development system, or you can use them in ECS task definitions.
 - You can replicate images in your private repositories across AWS regions.



Amazon Elastic Container Service (ECS)

- A container management service to run, stop, and manage Docker containers on a cluster.
- ECS can be used to create a consistent deployment and build experience, manage, and scale batch and **Extract-Transform-Load (ETL)** workloads, and build sophisticated application architectures on a microservices model.

Features

- You can create ECS clusters within a new or existing VPC.
- After a cluster is up and running, you can define task definitions and services that specify which Docker container images to run across your clusters.

Components

- Containers and Images
 - Your application components must be architected to run in **containers** — containing everything that your software application needs to run: code, runtime, system tools, system libraries, etc.
 - Containers are created from a read-only template called an **image**.
- Task Components
 - **Task definitions** specify various parameters for your application. It is a text file, in JSON format, that describes one or more containers, up to a maximum of ten, that form your application.
 - Task definitions are split into separate parts:
 - Task family - the name of the task, and each family can have multiple revisions.
 - IAM task role - specifies the permissions that containers in the task should have.
 - Network mode - determines how the networking is configured for your containers.
 - Container definitions - specify which image to use, how much CPU and memory the container is allocated, and many more options.
- Tasks and Scheduling
 - A **task** is the instantiation of a task definition within a cluster. After you have created a task definition for your application, you can specify the number of tasks that will run on your cluster.
 - Each task that uses the Fargate launch type has its own isolation boundary and does not share the underlying kernel, CPU resources, memory resources, or elastic network interface with another task.
 - You can upload a new version of your application task definition, and the ECS scheduler automatically starts new containers using the updated image and stop containers running the previous version.
- Clusters
 - When you run tasks using ECS, you place them in a **cluster**, which is a logical grouping of resources.
 - Clusters can contain tasks using both the Fargate and EC2 launch types.



- When using the Fargate launch type with tasks within your cluster, ECS manages your cluster resources.
- Enabling managed Amazon ECS cluster auto-scaling allows ECS to manage the scale-in and scale-out actions of the Auto Scaling group.
- Services
 - ECS allows you to run and maintain a specified number of instances of a task definition simultaneously in a cluster.
 - In addition to maintaining the desired count of tasks in your service, you can optionally run your service behind a load balancer.
 - There are two deployment strategies in ECS:
 - **Rolling Update**
 - This involves the service scheduler replacing the current running version of the container with the latest version.
 - **Blue/Green Deployment with AWS CodeDeploy**
 - This deployment type allows you to verify a new deployment of a service before sending production traffic to it.
 - The service must be configured to use either an Application Load Balancer or Network Load Balancer.
- Container Agent (AWS ECS Agent)
 - The **container agent** runs on each infrastructure resource within an ECS cluster.
 - It sends information about the resource's current running tasks and resource utilization to ECS, and starts and stops tasks whenever it receives a request from ECS.
 - Container agent is only supported on Amazon EC2 instances.

AWS Fargate

- You can use Fargate with ECS to run containers without having to manage servers or clusters of EC2 instances.
- You no longer have to provision, configure, or scale clusters of virtual machines to run containers.
- Fargate only supports container images hosted on Elastic Container Registry (ECR) or Docker Hub.

Task Definitions for Fargate Launch Type

- Fargate task definitions require that the network mode is set to *awsvpc*. The *awsvpc* network mode provides each task with its own elastic network interface.
- Fargate task definitions only support the *awslogs* log driver for the log configuration. This configures your Fargate tasks to send log information to Amazon CloudWatch Logs.
- Task storage is **ephemeral**. After a Fargate task stops, the storage is deleted.

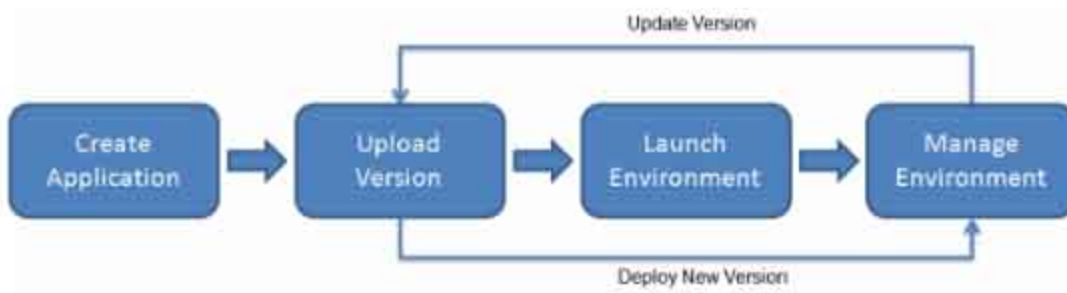


Monitoring

- You can configure your container instances to send log information to CloudWatch Logs. This enables you to view different logs from your container instances in one convenient location.
- With CloudWatch Alarms, watch a single metric over a time period that you specify, and perform one or more actions based on the value of the metric relative to a given threshold over a number of time periods.
- Share log files between accounts, and monitor CloudTrail log files in real-time by sending them to CloudWatch Logs.

AWS Elastic Beanstalk

- Allows you to quickly deploy and manage applications in the AWS Cloud without worrying about the infrastructure that runs those applications.
- Elastic Beanstalk automatically handles the details of capacity provisioning, load balancing, scaling, and application health monitoring for your applications.
- Elastic Beanstalk supports Docker containers.
- Elastic Beanstalk Workflow:



- Your application's domain name is in the format: `subdomain.region.elasticbeanstalk.com`

Elastic Beanstalk Concepts

- **Application** - a logical collection of Elastic Beanstalk components, including environments, versions, and environment configurations. It is conceptually similar to a folder.
- **Application Version** - refers to a specific, labeled iteration of deployable code for a web application. An application version points to an Amazon S3 object that contains the deployable code. Applications can have many versions, and each application version is unique.
- **Environment** - a version that is deployed on to AWS resources. Each environment runs only a single application version at a time, however, you can run the same version or different versions in many environments at the same time.
- **Environment Tier** - determines whether Elastic Beanstalk provisions resources to support an application that handles HTTP requests or an application that pulls tasks from a queue. An application that serves HTTP requests runs in a **web server environment**. An environment that pulls tasks from an Amazon SQS queue runs in a **worker environment**.
- **Environment Configuration** - identifies a collection of parameters and settings that define how an environment and its associated resources behave.
- **Configuration Template** - a starting point for creating unique environment configurations.



AWS Lambda

- A serverless compute service. Function-as-a-Service.
- Lambda executes your code only when needed and scales automatically.
- Lambda functions are stateless - no affinity to the underlying infrastructure.
- You choose the amount of memory you want to allocate to your functions, and AWS Lambda allocates proportional CPU power, network bandwidth, and disk I/O.

Components of a Lambda Application

- **Function** – a script or program that runs in Lambda. Lambda passes invocation events to your function. The function processes an event and returns a response.
- **Runtimes** – Lambda runtimes allow functions in different languages to run in the same base execution environment. The runtime sits in between the Lambda service and your function code, relaying invocation events, context information, and responses between the two.
- **Layers** – Lambda layers are a distribution mechanism for libraries, custom runtimes, and other function dependencies. Layers let you manage your in-development function code independently from the unchanging code and resources that it uses.
- **Event source** – an AWS service or a custom service that triggers your function and executes its logic.
- **Downstream resources** – an AWS service that your Lambda function calls once it is triggered.
- **Log streams** – While Lambda automatically monitors your function invocations and reports metrics to CloudWatch, you can annotate your function code with custom logging statements that allow you to analyze the execution flow and performance of your Lambda function.
- AWS Serverless Application Model

Lambda Functions

- You upload your application code in the form of one or more *Lambda functions*. Lambda stores code in Amazon S3 and encrypt it at rest.
- To create a Lambda function, you first package your code and dependencies in a deployment package. Then, you upload the deployment package to create your Lambda function.
- After your Lambda function is in production, Lambda automatically monitors functions on your behalf, reporting metrics through Amazon CloudWatch.
- Configure **basic function settings**, including the description, memory usage, execution timeout, and role that the function will use to execute your code.
- **Environment variables** are always encrypted at rest and can be encrypted in transit as well.
- **Versions and aliases** are secondary resources that you can create to manage function deployment and invocation.
- A **layer** is a ZIP archive that contains libraries, a custom runtime, or other dependencies. Use layers to manage your function's dependencies independently and keep your deployment package small.



AWS Serverless Application Model (SAM)

- An open-source framework for building serverless applications.
- It provides shorthand syntax to express functions, APIs, databases, and event source mappings.
- You create a **JSON** or **YAML** configuration template to model your applications.
- During deployment, SAM transforms and expands the SAM syntax into **AWS CloudFormation syntax**. Any resource that you can declare in an AWS CloudFormation template, you can also declare in an AWS SAM template.
- The **SAM CLI** provides a Lambda-like execution environment that lets you locally build, test, and debug applications defined by SAM templates. You can also use the SAM CLI to deploy your applications to AWS.
- You can use AWS SAM to build serverless applications that use **any runtime supported by AWS Lambda**. You can also use SAM CLI to locally debug Lambda functions written in Node.js, Java, Python, and Go.
- Commonly used SAM CLI commands
 - The **sam init** command generates pre-configured AWS SAM templates.
 - The **sam local** command supports local invocation and testing of your Lambda functions and SAM-based serverless applications by executing your function code locally in a Lambda-like execution environment.
 - The **sam package** and **sam deploy** commands let you bundle your application code and dependencies into a "deployment package" and then deploy your serverless application to the AWS Cloud.
 - The **sam logs** command enables you to fetch, tail, and filter logs for Lambda functions.
 - The output of the **sam publish** command includes a link to the AWS Serverless Application Repository directly to your application.
 - Use **sam validate** to validate your SAM template.

AWS Serverless Application Repository

- The AWS Serverless Application Repository is a central location where users can easily discover, deploy, and publish serverless applications in the AWS Cloud.
- Deeply integrated with AWS Lambda console for easy serverless computing integration.

Publishing Applications

- Publishing applications in the AWS Serverless Application Repository allows developers to share their serverless applications with the broader community.

Key steps:

- Users define serverless applications using an AWS Serverless Application Model (AWS SAM) template. This template describes the application, its resources, and permissions.



- Users can publish applications using the AWS Management Console, AWS SAM CLI, or an AWS SDK, which will upload the user's code and SAM template.
- Users' applications are initially private, only visible to their AWS accounts. They can choose to share them privately with specific accounts or publicly with all users.
- When public, the application is available in all AWS regions. The repository copies deployment artifacts like code to S3 buckets in other regions for easy deployment globally.
- Applications can be licensed openly under common licenses.
- Nested applications containing multiple services can also be published like standalone apps.

Deploying Applications

- Deploying applications from the AWS Serverless Application Repository is straightforward. Users can browse, search, and filter applications. Once they find an application, they can configure any parameters and deploy it with a few clicks from the AWS Lambda console.
 - Applications are tested and reviewed by AWS and partners to ensure they work as described. Some may have a verified author badge linking to the publisher profile.
 - Before deploying, check the application documentation and permissions needed. Make sure it meets your use case and security requirements.
 - Deploying provisions the necessary AWS resources like Lambda functions, APIs, etc., and handles all the plumbing. Users don't need serverless expertise.
 - Standard AWS pricing applies to the underlying services. There is no additional cost for deploying applications from the repository.
 - Post deployment, you can manage and monitor the application like any other AWS resources using the management console or AWS CLI/SDKs.

References:

<https://aws.amazon.com/serverless/serverlessrepo/faqs/>

<https://docs.aws.amazon.com/serverlessrepo/latest/devguide/what-is-serverlessrepo.html>

<https://docs.aws.amazon.com/serverlessrepo/latest/devguide/serverlessrepo-consuming-applications.html>

<https://docs.aws.amazon.com/serverlessrepo/latest/devguide/serverlessrepo-publishing-applications.html>



AWS Storage Services

Amazon EBS

- **Block level storage** volumes for use with EC2 instances.
- Well-suited for use as the primary storage for file systems, databases, or for any applications that require fine granular updates and access to raw, unformatted, block-level storage.
- Well-suited to both database-style applications (random reads and writes), and to throughput-intensive applications (long, continuous reads and writes).

Amazon EFS

- A fully-managed **file storage service** that makes it easy to set up and scale file storage in the Amazon Cloud.

Features

- The service manages all the file storage infrastructure for you, avoiding the complexity of deploying, patching, and maintaining complex file system configurations.
- EFS supports the Network File System version 4 protocol.
- Multiple Amazon EC2 instances can access an EFS file system at the same time, providing a common data source for workloads and applications running on more than one instance or server.
- Moving your EFS file data can be managed simply with AWS DataSync - a managed data transfer service that makes it faster and simpler to move data between on-premises storage and Amazon EFS.

Amazon S3

- S3 stores data as objects within **buckets**.
- An **object** consists of a file and optionally any metadata that describes that file.
- A **key** is the unique identifier for an object within a bucket.
- Storage capacity is virtually unlimited.
- Good for storing static web content or media. Can be used to host static websites.

Buckets

- For each bucket, you can:
 - Control access to it (create, delete, and list objects in the bucket)
 - View access logs for it and its objects
 - Choose the geographical region where to store the bucket and its contents.
- **Bucket name** must be a unique DNS-compliant name.
 - The name must be unique across all existing bucket names in Amazon S3.



- After you create the bucket you cannot change the name.
- The bucket name is visible in the URL that points to the objects that you're going to put in your bucket.
- You can host static websites by configuring your bucket for website hosting.

Security

- Policies contain the following:
 - **Resources** – buckets and objects
 - **Actions** – set of operations
 - **Effect** – can be either allow or deny. Need to explicitly grant allow to a resource.
 - **Principal** – the account service, or user who is allowed access to the actions and resources in the statement.
- Resource Based Policies
 - Bucket Policies
 - Provides **centralized access control** to buckets and objects based on a variety of conditions, including S3 operations, requesters, resources, and aspects of the request (e.g., IP address).
 - Can either **add or deny permissions** across all (or a subset) of objects within a bucket.
 - IAM users need additional permissions from the root account to perform bucket operations.
 - Bucket policies are limited to 20 KB in size.
 - User Policies
 - AWS IAM (see AWS Security and Identity Services)
 - IAM User Access Keys
 - Temporary Security Credentials
- Versioning
 - Use versioning to keep multiple versions of an object in one bucket.
 - Versioning protects you from the consequences of unintended overwrites and deletions.
 - You can also use versioning to archive objects so you have access to previous versions.
 - Since versioning is disabled by default, you need to EXPLICITLY enable it.
- Encryption
 - Server-side Encryption using
 - **Amazon S3-Managed Keys (SSE-S3)**
 - **AWS KMS Keys (SSE-KMS)**
 - **Customer-Provided Keys (SSE-C)**
 - Client-side Encryption using
 - AWS KMS key
 - client-side master key
- MFA Delete
 - MFA delete grants additional authentication for either of the following operations:



- Change the versioning state of your bucket
 - Permanently delete an object version
- Cross-Account Access
 - You can provide another AWS account access to an object that is stored in an Amazon Simple Storage Service (Amazon S3) bucket. These are the methods on how to grant cross-account access to objects that are stored in your own Amazon S3 bucket:
- Monitoring
 - Automated monitoring tools to watch S3:
 - Amazon CloudWatch Alarms – Watch a single metric over a time period that you specify, and perform one or more actions based on the value of the metric relative to a given threshold over a number of time periods.
 - AWS CloudTrail Log Monitoring – Share log files between accounts, monitor CloudTrail log files in real-time by sending them to CloudWatch Logs, write log processing applications in Java, and validate that your log files have not changed after delivery by CloudTrail.
 - Only certain S3 events are recorded on CloudTrail Event History. For Object level events you **enable server access logging for an S3 bucket**
- S3 Events Notification
 - To enable notifications, add a *notification configuration* identifying the events to be published, and the destinations where to send the event notifications.
 - Can publish the following events:
 - A new object created event
 - An object removal event
 - A Reduced Redundancy Storage (RRS) object lost event
 - Supports the following destinations for your events:
 - Amazon Simple Notification Service (Amazon SNS) topic
 - Amazon Simple Queue Service (Amazon SQS) queue
 - AWS Lambda



Amazon S3 Bucket Policies for VPC Endpoints

What are VPC endpoints?

A VPC endpoint is what you use to privately connect your VPC to supported AWS services, such as Amazon S3. It adds a gateway entry in your VPC's route table so that communication between your AWS resources, such as Amazon EC2 instances, and your S3 bucket pass through the gateway instead of the public internet. As a result, the VPC endpoint is a regional service. You should create the endpoint in the same region as the VPC you want to link it to.

VPC endpoints are best used when you have compliance requirements or sensitive information stored in S3 that should not leave the Amazon network. A VPC endpoint is also a better option for private network connections in AWS, as compared to using a VPN solution or a NAT solution since it is easier to set up and offers you more network bandwidth at your disposal.

Amazon S3 Glacier Flexible Retrieval (formerly S3 Glacier)

Amazon S3 Glacier Flexible Retrieval (formerly S3 Glacier) is a durable, highly secure, cost-effective cloud storage service created for data archiving and long-term backup. It is renowned for its exceptional durability of 99.999999999%, and it offers comprehensive security and compliance features to meet stringent regulatory requirements. Additionally, it enables query-in-place functionality, allowing you to run analytics directly on your archived data.

Storage Classes

S3 Glacier Flexible Retrieval (formerly S3 Glacier) offers three main storage classes to cater to different retrieval needs and cost efficiencies:

- **S3 Glacier Instant Retrieval** is ideal for archiving data that is accessed no more than once per quarter and requires rapid retrieval in milliseconds. It offers significant savings over S3 Standard-Infrequent Access (S3 Standard-IA), albeit with higher data access costs.
- **S3 Glacier Flexible Retrieval** is suited for archives where data might occasionally need to be accessed. It offers expedited retrievals in minutes. Objects have a minimum storage duration of 90 days. This class allows for free bulk retrievals that can be completed in 5-12 hours.
- **S3 Glacier Deep Archive** is the most cost-effective solution for long-term archiving data that rarely needs to be accessed. Its minimum storage duration is 180 days, and the default retrieval time is 12 hours.



Key Features

- Amazon S3 Glacier Flexible Retrieval (formerly S3 Glacier) offers 99.999999999% durability, providing comprehensive security and compliance capabilities to meet stringent regulatory requirements.
- It provides flexible retrieval options from a few minutes to several hours, catering to different needs and cost considerations.
- Amazon S3 Glacier Flexible Retrieval (formerly S3 Glacier) is designed to be an economical choice for data archiving. Pricing varies based on the storage class, data retrieval time, and amount of data stored. Data transfer into Amazon S3 is free, while data transfer out is priced by region.
- Amazon S3 Glacier Flexible Retrieval (formerly S3 Glacier) can easily manage archives through the AWS Management Console, AWS CLI, and AWS SDKs. The service supports backup and restore process automation and includes features like S3 Object Lock for additional data protection.

Usage Tips

- When using S3 Glacier Flexible Retrieval (formerly S3 Glacier), remember that your objects remain in Amazon S3. Direct access through the separate Amazon S3 Glacier Flexible Retrieval (formerly S3 Glacier) service is not available.
- To avoid unexpected costs, consider the minimum storage duration charges before deleting or transitioning objects to a different storage class.
- Utilize Amazon S3 Lifecycle policies to automate the transition of S3 objects to Glacier storage classes, helping manage lifecycle and costs efficiently.
- Consider aggregating files into a single archive for bulk data or numerous small files to minimize storage overhead and costs.

Pricing

Amazon S3 Glacier Flexible Retrieval (formerly S3 Glacier) pricing is tailored for cost-effectiveness in data archiving and long-term backup, influenced by storage, retrieval, and data transfer costs. Storage pricing varies with S3 Glacier Instant Retrieval being suitable for quick access data but at higher costs, S3 Glacier Flexible Retrieval designed for infrequent access with a minimum 90-day storage duration, and S3 Glacier Deep Archive as the lowest-cost option for rarely accessed data with a minimum 180-day storage duration. Retrieval costs depend on the storage class. Instant Retrieval provides the quickest access at higher costs. Flexible Retrieval balances speed and cost, and Deep Archive is the most economical for infrequent access. Data transfer into S3 Glacier Flexible Retrieval (formerly S3 Glacier) is free, but transferring data out incurs costs based on the AWS region and amount of data. Additionally, operations such as PUT, COPY, POST requests, and data retrievals contribute to the overall cost, with considerations for minimum storage duration charges for early deletions or transitions and minimum object size charges in some storage classes.

References:

<https://docs.aws.amazon.com/whitepapers/latest/how-aws-pricing-works/amazon-s3-glacier.html>



<https://aws.amazon.com/s3/storage-classes/glacier/>

<https://docs.aws.amazon.com/prescriptive-guidance/latest/backup-recovery/amazon-s3-glacier.html>

AWS Storage Gateway

AWS Storage Gateway is a service that enables **hybrid storage** between your on-premises environment and AWS's cloud storage. It provides seamless and secure access to virtually unlimited cloud storage. The service is designed for easy integration with existing applications and systems while leveraging AWS's scalability, security, and cost-efficiency.

AWS Storage Gateway supports various storage solutions tailored to different needs:

- **File Gateway** offers SMB or NFS-based access to data in Amazon S3, which is suitable for integrating existing file-based applications with cloud storage without modification.
 - File gateway supports:
 - S3 Standard
 - S3 Standard – Infrequent Access
 - S3 One Zone – IA
 - File Gateway now supports Amazon S3 Object Lock, enabling write-once-read-many (WORM) file-based systems to store and access objects in Amazon S3.
 - File Gateway's local cache can support up to 64TB of data.
- **Tape Gateway**: Virtualizes tape backup and archive workflows, is compatible with leading backup applications, and stores virtual tapes in Amazon S3 or Amazon S3 Glacier Flexible Retrieval (formerly S3 Glacier).
 - Tape Gateway can transfer your virtual tapes from Amazon S3 Glacier Flexible Retrieval (formerly S3 Glacier) to the Amazon S3 Glacier Deep Archive storage class, allowing you to decrease the monthly cost of storing long-term data in the cloud by up to 75%.
- **Volume Gateway**: Provides block storage volumes via the iSCSI protocol, with data backed up as Amazon EBS snapshots, suitable for on-premises applications requiring scalable storage with cloud recovery options.
- **Cached Volumes**: This option allows you to store your primary data on Amazon S3 while maintaining a local cache of your data's most frequently accessed parts for quicker access. These cached volumes can vary from 1 GiB to 32 TiB, and sizes are adjusted to the nearest GiB. Each gateway has the capacity to support up to 32 volumes with cached volumes.
- **Stored Volumes**: For those requiring immediate access to their complete data set, this approach involves storing all your data on the local premises first. Subsequently, snapshots of this data are taken at specific points in time and then asynchronously backed up to Amazon S3 for safekeeping. Stored volumes also have a size range from 1 GiB to 16 TiB, with the requirement to round to the nearest GiB. A single gateway configured for stored volumes can accommodate up to 32 volumes.

Security



AWS Storage Gateway ensures secure data transfer and storage by using SSL encryption for data in transit between your gateway appliance and AWS. It also supports FIPS 140-2 compliant endpoints in specific regions and integrates with AWS IAM for access management. Data at rest can be encrypted using AWS KMS integration with either default or custom keys. The service is designed to operate within AWS's high-security standards, following the shared responsibility model, which divides security responsibilities between AWS and the customer.

Pricing

AWS Storage Gateway uses a pay-as-you-go pricing model. The costs depend on the specific services used (file, tape, or volume gateway), data storage, transfer volumes, and any additional features such as snapshots or tape storage. AWS provides a Free Tier for new users, allowing them to ingest up to 100 GB of data through the Storage Gateway service. This enables businesses to try out the service at no initial cost.

References:

<https://aws.amazon.com/storagegateway/features/>

<https://docs.aws.amazon.com/storagegateway/latest/vgw/security.html>



AWS Database Services

Amazon Aurora

- A fully managed relational database engine that's compatible with MySQL and PostgreSQL.
- With some workloads, Aurora can deliver up to five times the throughput of MySQL and up to three times the throughput of PostgreSQL.
- Aurora includes a high-performance storage subsystem. The underlying storage grows automatically as needed, up to 64 terabytes. The minimum storage is 10GB.
- **DB Clusters**
 - An Aurora DB cluster consists of one or more DB instances and a cluster volume that manages the data for those DB instances.
 - An Aurora cluster volume is a virtual database storage volume that spans multiple AZs, with each AZ having a copy of the DB cluster data.
 - Cluster Types:
 - Primary DB instance – Supports read and write operations, and performs all of the data modifications to the cluster volume. Each Aurora DB cluster has one primary DB instance.
 - Aurora Replica – Connects to the same storage volume as the primary DB instance and supports only read operations. Each Aurora DB cluster can have up to 15 Aurora Replicas in addition to the primary DB instance. Aurora automatically fails over to an Aurora Replica in case the primary DB instance becomes unavailable. You can specify the failover priority for Aurora Replicas. Aurora Replicas can also offload read workloads from the primary DB instance.
- **Monitoring**
 - Subscribe to Amazon RDS events to be notified when changes occur with a DB instance, DB cluster, DB cluster snapshot, DB parameter group, or DB security group.
 - Database log files
 - RDS Enhanced Monitoring – Look at metrics in real-time for the operating system.
 - RDS Performance Insights monitors your Amazon RDS DB instance load so that you can analyze and troubleshoot your database performance.
 - Use CloudWatch Metrics, Alarms, and Logs

Aurora Serverless v2

- Aurora Serverless v2 is an auto-scaling configuration for Amazon Aurora that allows databases to automatically scale capacity up or down based on real-time usage.
- It supports all features of provisioned Aurora including read replicas, multi-AZ configuration, Global Database, RDS proxy and Performance Insights.
- Aurora Serverless v2 scales capacity incrementally in small units of 0.5 Aurora Capacity Units (ACUs), allowing the capacity to closely match the application's needs.



- With reader instances, it separates read and write workloads, improving performance and allowing more flexible scaling of each.
- Some of the advantages of Aurora Serverless v2:
 - Allows all types of database workloads from simple development/test to mission critical applications.
 - Provides high scalability even during long transactions or table locks.
 - Almost all configuration parameters can be modified as with provisioned clusters.
 - It provides simpler capacity management than provisioned Aurora as it automatically scales capacity up or down based on usage, reducing the effort needed for manual resizing.
 - It allows faster and easier scaling during periods of high activity with no disruption to the database.
 - It is more cost-effective during periods of low activity as you only pay for the resources consumed.
- Some common use cases of Aurora Serverless v2:
 - Development and test environments that have intermittent usage.
 - Applications with unpredictable traffic patterns like websites facing sudden spikes.
 - Mission critical databases that require high availability and scale on demand to handle traffic.
 - Data warehousing workloads where analysis or reporting jobs are run on the database during certain periods.
 - Serverless applications where the underlying database also needs to scale automatically based on incoming requests to the application.

References:

<https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/aurora-serverless-v2.html>

<https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/aurora-serverless-v2.how-it-works.html>

Amazon DynamoDB

- NoSQL database service that provides fast and predictable performance with seamless scalability.
- Offers encryption at rest.
- You can create database tables that can store and retrieve any amount of data, and serve any level of request traffic.
- You can scale up or scale down your tables' throughput capacity without downtime or performance degradation, and use the AWS Management Console to monitor resource utilization and performance metrics.

Core Components

- **Tables** - a collection of items
 - DynamoDB stores data in a table, which is a collection of data.
 - Are schemaless.



- There is an initial limit of 256 tables per region.
- **Items** - a collection of attributes
 - DynamoDB uses **primary keys** to uniquely identify each item in a table and **secondary indexes** to provide more querying flexibility.
 - Each table contains zero or more items.
- **Attributes** - a fundamental data element
 - DynamoDB supports nested attributes up to 32 levels deep.
- **Primary Key** - uniquely identifies each item in the table, so that no two items can have the same key. Must be scalar.
 - **Partition key** - a simple primary key, composed of one attribute.
 - **Partition key and sort key** (*composite primary key*) - composed of two attributes.
 - DynamoDB uses the partition key value as input to an internal hash function. The output from the hash function determines the partition in which the item will be stored. All items with the same partition key are stored together, in sorted order by sort key value. If no sort key is used, no two items can have the same partition key value.
- **Secondary Indexes** - lets you query the data in the table using an alternate key, in addition to queries against the primary key.
 - You can create one or more secondary indexes on a table.
 - Two kinds of indexes:
 - **Global secondary index** – An index with a partition key and sort key that can be different from those on the table.
 - **Local secondary index** – An index that has the same partition key as the table, but a different sort key.
 - You can define up to 20 global secondary indexes and 5 local secondary indexes per table.

DynamoDB Accelerator (DAX)

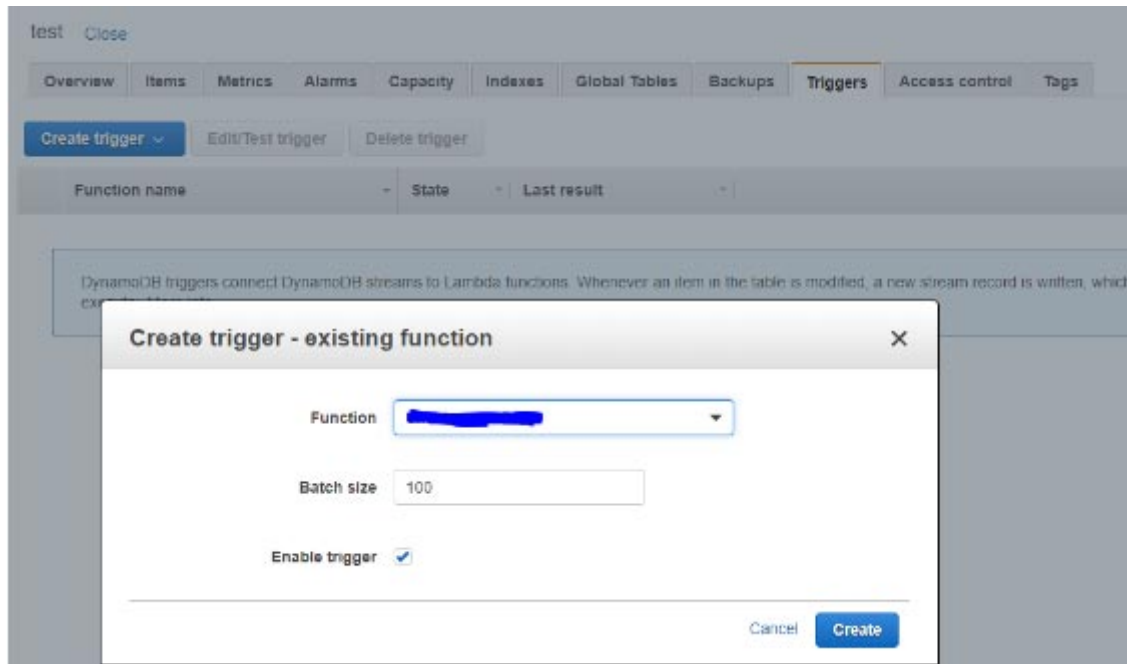
- DAX is a fully managed, highly available, in-memory cache for DynamoDB.
- **DynamoDB Accelerator (DAX)** delivers microsecond response times for accessing eventually consistent data.
- It requires only minimal functional changes to use DAX with an existing application since it is API-compatible with DynamoDB.
- For read-heavy or bursty workloads, DAX provides increased throughput and potential cost savings by reducing the need to overprovision read capacity units.
- DAX lets you scale on demand.
- DAX is fully managed. You no longer need to do hardware or software provisioning, setup, and configuration, software patching, operating a reliable, distributed cache cluster, or replicating data over multiple instances as you scale.



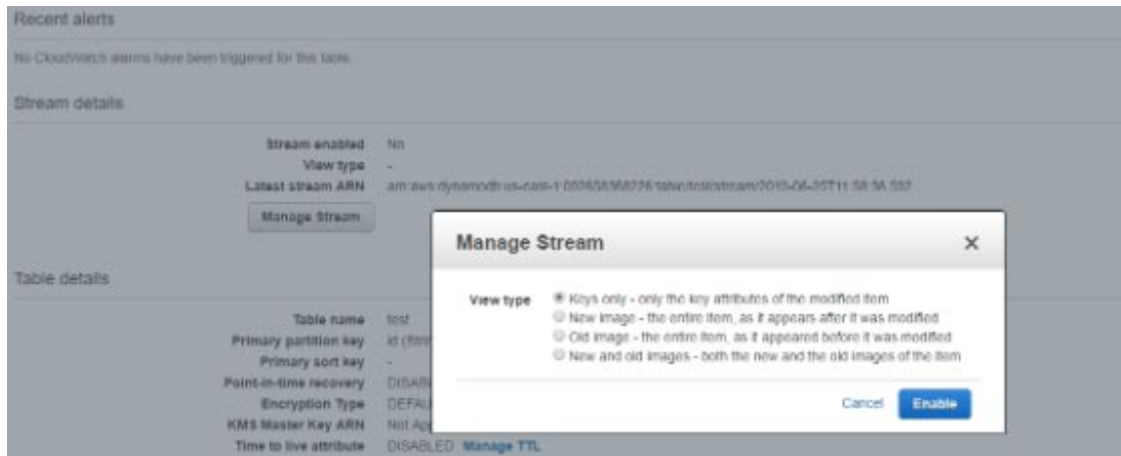
- Amazon DynamoDB is integrated with AWS Lambda so that you can create triggers, which are pieces of code that automatically respond to events in DynamoDB Streams. With triggers, you can build applications that react to data modifications in DynamoDB tables.

- After you enable DynamoDB Streams on a table, associate the DynamoDB table with a Lambda function. AWS Lambda polls the stream and invokes your Lambda function synchronously when it detects new stream records.

<https://portal.tutorialsdojo.com/>



- Configure the StreamSpecification you want for your DynamoDB Streams:
 - StreamEnabled (Boolean) - indicates whether DynamoDB Streams is enabled (true) or disabled (false) on the table.
 - StreamViewType (string) - when an item in the table is modified, StreamViewType determines what information is written to the stream for this table. Valid values for StreamViewType are:
 - KEYS_ONLY - Only the key attributes of the modified items are written to the stream.
 - NEW_IMAGE - The entire item, as it appears after it was modified, is written to the stream.
 - OLD_IMAGE - The entire item, as it appeared before it was modified, is written to the stream.
 - NEW_AND_OLD_IMAGES - Both the new and the old item images of the items are written to the stream.



Sources:

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Streams.Lambda.html>
https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_StreamSpecification.html

Amazon RDS

- Supports **Aurora, MySQL, MariaDB, PostgreSQL, Oracle, Microsoft SQL Server**.
- You can get high availability with a primary instance and a synchronous secondary instance that you can fail over to when problems occur. You can also use MySQL, MariaDB, or PostgreSQL Read Replicas to increase read scaling.
- You can select the computation and memory capacity of a DB instance, determined by its **DB instance class**. If your needs change over time, you can change DB instances.
- Each DB instance has minimum and maximum storage requirements depending on the storage type and the database engine it supports.
- You can run your DB instance in several AZs, an option called a **Multi-AZ deployment**. Amazon automatically provisions and maintains a secondary standby DB instance in a different AZ. Your primary DB instance is synchronously replicated across AZs to the secondary instance to provide data redundancy, and failover support, eliminate I/O freezes, and minimize latency spikes during system backups.

Security

- Security Groups
 - **DB Security Groups** - controls access to a DB instance that is not in a VPC. By default, network access is turned off to a DB instance. This SG is for the EC2-Classical platform.
 - **VPC Security Groups** - controls access to a DB instance inside a VPC. This SG is for the EC2-VPC platform.
 - **EC2 Security Groups** - controls access to an EC2 instance and can be used with a DB instance.



- A *resource owner* is the AWS account that created a resource. That is, the resource owner is the AWS account of the *principal entity* (the root account, an IAM user, or an IAM role) that authenticates the request that creates the resource.
- A *permissions policy* describes who has access to what. Policies attached to an IAM identity are *identity-based policies* (IAM policies), and policies attached to a resource are *resource-based policies*. Amazon RDS supports only identity-based policies (IAM policies).
- MySQL and PostgreSQL both support **IAM database authentication**.

High Availability using Multi-AZ

- Multi-AZ deployments for **Oracle, PostgreSQL, MySQL, and MariaDB** DB instances use **Amazon's failover technology**. **SQL Server DB** instances use **SQL Server Mirroring**.
- The primary DB instance switches over automatically to the standby replica if any of the following conditions occur:
 - An Availability Zone outage
 - The primary DB instance fails
 - The DB instance's server type is changed
 - The operating system of the DB instance is undergoing software patching
 - A manual failover of the DB instance was initiated using **Reboot with failover**

Read Replicas

- Updates made to the source DB instance are asynchronously copied to the Read Replica.
- You can reduce the load on your source DB instance by routing read queries from your applications to the Read Replica.
- You can elastically scale out beyond the capacity constraints of a single DB instance for read-heavy database workloads.
- You can create a Read Replica that has a different storage type from the source DB instance.

Backups and Restores

- Your DB instance must be in the **ACTIVE state** for automated backups to occur. Automated backups and automated snapshots don't occur while a copy is executing in the same region for the same DB instance.
- The first snapshot of a DB instance contains the data for the full DB instance. Subsequent snapshots of the same DB instance are incremental.
- The default backup retention period is one day if you create the DB instance using the RDS API or the AWS CLI, or seven days if you use the AWS Console.
- Manual snapshot limits are limited to 100 per region.
- You can copy a snapshot within the same AWS Region, you can copy a snapshot across AWS Regions, and you can copy a snapshot across AWS accounts.



- When you restore a DB instance to a point in time, the default DB parameter and default DB security group is applied to the new DB instance.

Amazon Redshift

- Amazon Redshift is a fully managed AWS data warehouse service that make it easy and affordable to analyze data using SQL and various business intelligence tools.
- With Amazon Redshift, you don't need to worry about provisioning, patching, backup or monitoring the underlying infrastructure. It offers automatic scaling of storage and compute capacity.
- Some key features of Amazon Redshift includes Redshift Advisor which provides recommendations for optimizing queries and cluster configuration, integration with other AWS services like S3 and Glue for data loading and management, and security features like encryption and IAM roles.
- Amazon Redshift enables you to analyze petabytes of structured and semi-structured data using existing SQL skills, with support for popular business intelligence tools through high-performance ODBC/JDBC drivers and SQL queries.
- It provides fast performance for both analytical and operational workloads through massively parallel processing architecture and columnar data storage. Pricing is on a per-second usage basis so you only pay for what you use.

The main components of Amazon Redshift are:

- **Cluster** - This is the fundamental unit of compute and storage in Amazon Redshift. A cluster consists of one or more nodes that work together to handle queries and store data.
- **Nodes** - Each cluster contains leader and compute nodes. The leader node coordinates queries and manages the cluster, while compute nodes store data and perform query processing.
- **Databases** - Redshift organizes data into databases which can contain multiple schemas and tables.
- **Tables** - Data is stored in columns and rows within tables in a database. Redshift uses columnar storage, which improves query performance.
- **Storage** - Each node contains disks that store columnar data and can independently scale storage capacity and throughput.
- **Query engine** - Redshift uses a massively parallel processing (MPP) architecture and a distributed query engine to process queries across all nodes very fast.
- **Security** - Authentication is managed through IAM roles and policies. Data is encrypted at rest and in transit using KMS and SSL. Network access is governed by security groups and VPC endpoints.

References:

<https://docs.aws.amazon.com/redshift/latest/mgmt/welcome.html>



AWS Networking & Content Delivery

Amazon API Gateway

- Enables developers to create, publish, maintain, monitor, and secure APIs at any scale.
- Allows creating, deploying, and managing a RESTful API to expose backend HTTP endpoints, Lambda functions, or other AWS services.
- Together with Lambda, API Gateway forms the app-facing part of the AWS serverless infrastructure.
- **Concepts**
 - API deployment - a point-in-time snapshot of your API Gateway API resources and methods. To be available for clients to use, the deployment must be associated with one or more API stages.
 - API endpoints - host names APIs in API Gateway, which are deployed to a specific region and of the format: rest-api-id.execute-api.region.amazonaws.com
 - API key - An alphanumeric string that API Gateway uses to identify an app developer who uses your API.
 - API stage - A logical reference to the lifecycle state of your API. API stages are identified by API ID and stage name.
 - Model - Data schema specifying the data structure of a request or response payload.
 - Private API - An API that is exposed through interface VPC endpoints and isolated from the public internet
 - Private integration - An API Gateway integration type for a client to access resources inside a customer's VPC through a private API endpoint without exposing the resources to the public internet.
 - Proxy integration - You can set up a proxy integration as an HTTP proxy integration type or a Lambda proxy integration type.
 - For the HTTP proxy integration, API Gateway passes the entire request and response between the front and an HTTP backend.
 - For the Lambda proxy integration, API Gateway sends the entire request as input to a backend Lambda function.
 - Usage plan - Provides selected API clients with access to one or more deployed APIs. You can use a usage plan to configure throttling and quota limits, which are enforced on individual client API keys.
- **Features**
 - API Gateway can execute Lambda code in your account, start Step Functions state machines, or make calls to Elastic Beanstalk, EC2, or web services outside of AWS with publicly accessible HTTP endpoints.
 - API Gateway helps you define plans that meter and restrict third-party developer access to your APIs.
 - API Gateway helps you manage traffic to your backend systems by allowing you to set throttling rules based on the number of requests per second for each HTTP method in your APIs.



- You can set up a cache with customizable keys and time-to-live in seconds for your API data to avoid hitting your backend services for each request.
- API Gateway lets you run multiple versions of the same API simultaneously with **API Lifecycle**.
- After you build, test, and deploy your APIs, you can package them in an API Gateway usage plan and sell the plan as a Software as a Service (SaaS) product through AWS Marketplace.
- API Gateway offers the ability to create, update, and delete documentation associated with each portion of your API, such as methods and resources.
- Amazon API Gateway offers general availability of HTTP APIs, which gives you the ability to route requests to private ELBs AWS AppConfig, Amazon EventBridge, Amazon Kinesis Data Streams, Amazon SQS, AWS Step Functions, and IP-based services registered in AWS CloudMap such as ECS tasks. Previously, HTTP APIs enabled customers to only build APIs for their serverless applications or to proxy requests to HTTP endpoints.
- You can create data mapping definitions from an HTTP API's method request data (e.g. path parameters, query string, and headers) to the corresponding integration request parameters and from the integration response data (e.g. headers) to the HTTP API method response parameters.
- Use wildcard custom domain names (*.example.com) to create multiple URLs that route to one API Gateway HTTP API.
- You can configure your custom domain name to route requests to different APIs. Using multi-level base path mappings, you can implement path-based API versioning and migrate API traffic between APIs according to request paths with many segments.
- All of the APIs created expose HTTPS endpoints only. API Gateway does not support unencrypted (HTTP) endpoints.



Amazon Route 53

- A highly available and scalable Domain Name System (DNS) web service used for domain registration, DNS routing, and health checking.

Routing Internet Traffic to your Website or Web Application

- Use the Route 53 console to register a domain name and configure Route 53 to route internet traffic to your website or web application.
- After you register your domain name, Route 53 automatically creates a **public-hosted zone** that has the same name as the domain.
- To route traffic to your resources, you create **records**, also known as *resource record sets*, in your hosted zone.
- You can create special Route 53 records, called **alias records**, that route traffic to S3 buckets, CloudFront distributions, and other AWS resources.
- Each record includes information about how you want to route traffic for your domain, such as:
 - Name - the name of the record corresponds with the domain name or subdomain name that you want Route 53 to route traffic for.
 - Type - determines the type of resource that you want traffic to be routed to.
 - Value

Route 53 Health Checks

- Create a health check and specify values that define how you want the health check to work, such as:
 - The IP address or domain name of the endpoint that you want Route 53 to monitor.
 - The protocol that you want Route 53 to use to perform the check: HTTP, HTTPS, or TCP.
 - The **request interval** you want Route 53 to send a request to the endpoint.
 - How many consecutive times the endpoint must fail to respond to requests before Route 53 considers it unhealthy. This is the **failure threshold**.
- You can configure a health check to check the health of one or more other health checks.
- You can configure a health check to check the status of a CloudWatch alarm so that you can be notified on the basis of a broad range of criteria.

Know the following Concepts

- Domain Registration Concepts - domain name, domain registrar, domain registry, domain reseller, top-level domain
- DNS Concepts
 - **Alias record** - a type of record that you can create to route traffic to AWS resources.



- **Hosted zone** - a container for records, which includes information about how to route traffic for a domain and all of its subdomains.
- **Name servers** - servers in the DNS that help to translate domain names into the IP addresses that computers use to communicate with one another.
- **Record** (DNS record) - an object in a hosted zone that you use to define how you want to route traffic for the domain or a subdomain.
- **Routing policy** - policy on how to redirect users based on configured routing policy
- **Subdomain** - name below the zone apex. Example: portal.tutorialsdojo.com
- Time to live (TTL) - time that the DNS record is cached by querying servers.
- Health Checking Concepts
 - **DNS failover** - a method for routing traffic away from unhealthy resources and to healthy resources.
 - Endpoint - the URL or endpoint on which the health check will be performed.
 - Health check - the metric on which to determine if an endpoint is healthy or not.

Records

- Create records in a hosted zone. Records define where you want to route traffic for each domain name or subdomain name. The name of each record in a hosted zone must end with the name of the hosted zone.
- Alias Records
 - Route 53 **alias records** provide a Route 53–specific extension to DNS functionality. Alias records let you route traffic to selected AWS resources. They also let you route traffic from one record in a hosted zone to another record.
 - You can create an alias record at the top node of a DNS namespace, also known as the zone apex.
- CNAME Record
 - You cannot create an alias record at the top node (zone apex) of a DNS namespace using a CNAME record.

Amazon CloudFront

Amazon CloudFront is a content delivery network (CDN) service that aims to accelerate the distribution of dynamic and static web content to users worldwide, including HTML, CSS, JavaScript, and image files. It achieves this by utilizing a global network of data centers known as **edge locations**. CloudFront ensures that user requests are handled from the nearest location, reducing latency and improving content delivery speed. If a user requests content served with CloudFront, the service intelligently directs the request to the edge location with the lowest latency. If the content is available at that location, it's delivered immediately; if not,



CloudFront retrieves it from the specified origin, such as an Amazon S3 bucket or a web server, and delivers it to the user. This process ensures an efficient and rapid content delivery experience.

Key features:

- CloudFront's infrastructure is strategically located worldwide, including regional edge caches within AWS regions, over 600 Points of Presence (PoPs) in more than 100 cities across over 50 countries, and additional embedded PoPs within ISP networks. This extensive network ensures high-performance and low-latency content delivery to end users globally.
- CloudFront provides strong security features to protect content and applications. It seamlessly integrates with AWS Shield for DDoS protection, AWS WAF for application layer defense, and offers SSL/TLS encryption for secure and protected content delivery. Access control mechanisms such as signed URLs and cookies, token authentication, and geo-restriction capabilities further ensure that content delivery is secure and compliant with global standards.
- Amazon CloudFront offers customizable content delivery and network response through CloudFront Functions and AWS Lambda@Edge. This includes manipulation of HTTP headers, URL rewrites, cache-key normalizations, and more directly at the edge locations. These features support high-scale, latency-sensitive operations, enabling instant scalability and minimal latency for millions of requests per second.
- Integration with Amazon CloudWatch provides real-time metrics and logging, offering insights into distributions' operation. CloudFront supports standard and real-time logging for detailed analysis and content delivery performance monitoring.
- CloudFront provides flexible pricing options, including pay-as-you-go and the CloudFront Security Savings Bundle, which offers discounts in exchange for a monthly spend commitment. This makes it a cost-effective solution for company of all sizes looking to deliver content efficiently and at scale.

Reference:

<https://aws.amazon.com/cloudfront/features/?nc=sn&loc=2&whats-new-cloudfront.sort-by=item.additionalFields.postDateTime&whats-new-cloudfront.sort-order=desc>



AWS Elastic Load Balancing (ELB)

- Distributes incoming application or network traffic across multiple targets, such as **EC2 instances**, **containers (ECS)**, **Lambda functions**, and **IP addresses**, in multiple Availability Zones.
- When you create a load balancer, you must specify one public subnet from at least two Availability Zones. You can specify only one public subnet per Availability Zone.

General features

- Accepts incoming traffic from clients and routes requests to its registered targets.
- Monitors the health of its registered targets and routes traffic only to healthy targets.
- **Cross Zone Load Balancing** - when enabled, each load balancer node distributes traffic across the registered targets in all enabled AZs.

Types of Load Balancers

- **Application Load Balancer**
- **Network Load Balancer**
- **Gateway Load Balancer**

Features

- **Slow Start Mode** gives targets time to warm up before the load balancer sends them a full share of requests.
- **Sticky sessions** route requests to the same target in a target group. You enable sticky sessions at the target group level. You can also set the duration for the stickiness of the load balancer-generated cookie in seconds. Useful if you have stateful applications.
- **Health checks** verify the status of your targets. The statuses for a registered target are:

VALUE	DESCRIPTION
initial	The load balancer is in the process of registering the target or performing the initial health checks on the target.
healthy	The target is healthy.
unhealthy	The target did not respond to a health check or failed the health check.
unused	The target is not registered with a target group, the target group is not used in a listener rule, the target is in an Availability Zone that is not enabled, or the target is in the stopped or terminated state.



draining	The target is deregistering and connection draining is in process.
unavailable	Target health is unavailable.

AWS Transit Gateway

- A networking service that uses a hub and spoke model to enable customers to connect their on-premises data centers and their Amazon Virtual Private Clouds (VPCs) to a single gateway.
- With this service, customers only have to create and manage a single connection from the central gateway into each on-premises data center, remote office, or VPC across your network.
- If a new VPC is created, it is automatically connected to the Transit Gateway and will also be available to every other network that is also connected to the Transit Gateway.

Features:

- **Inter-region peering**
 - Transit Gateway leverages the AWS global network to allow customers to route traffic across AWS Regions.
 - Inter-region peering provides an easy and cost-effective way to replicate data for geographic redundancy or to share resources between AWS Regions.
- **Multicast**
 - Enables customers to have fine-grain control on who can consume and produce multicast traffic.
 - It allows you to easily create and manage multicast groups in the cloud instead of the time-consuming task of deploying and managing legacy hardware on-premises.
 - This multicast solution is also scalable so the customers can simultaneously distribute a stream of content to multiple subscribers.
- **Automated Provisioning**
 - Customers can automatically identify the Site-to-Site VPN connections and the on-premises resources with which they are associated using AWS Transit Gateway.
 - Using the Transit Gateway Network Manager, you can also manually define your on-premises network.

Reference:

<https://aws.amazon.com/transit-gateway/>



AWS Security & Identity Services

Amazon GuardDuty

- An intelligent threat detection service. It analyzes billions of events across your AWS accounts from AWS CloudTrail (AWS user and API activity in your accounts), Amazon VPC Flow Logs (network traffic data), and DNS Logs (name query patterns). Take note that GuardDuty is a regional service.
- Threat detection categories
 - **Reconnaissance** – Activity suggesting reconnaissance by an attacker, such as unusual API activity, intra-VPC port scanning, unusual patterns of failed login requests, or unblocked port probing from a known bad IP.
 - **Instance compromise** – Activity indicating an instance compromises, such as cryptocurrency mining, backdoor command and control activity, malware using domain generation algorithms, outbound denial of service activity, unusually high volume of network traffic, unusual network protocols, outbound instance communication with a known malicious IP, temporary Amazon EC2 credentials used by an external IP address, and data exfiltration using DNS.
 - **Account compromise** – Common patterns indicative of account compromise include API calls from unusual geolocation or anonymizing proxy, attempts to disable AWS CloudTrail logging, changes that weaken the account password policy, unusual instance or infrastructure launches, infrastructure deployments in an unusual region, and API calls from known malicious IP addresses.
- CloudTrail Event Source
 - Currently, GuardDuty only analyzes CloudTrail management events. (Read about types of CloudTrail trails for more information)
 - GuardDuty processes all CloudTrail events that come into a region, including global events that CloudTrail sends to all regions, such as AWS IAM, AWS STS, Amazon CloudFront, and Route 53.
- VPC Flow Logs Event Source
 - VPC Flow Logs capture information about the IP traffic going to and from Amazon EC2 network interfaces in your VPC.
- DNS Logs Event Source
 - If you use AWS DNS resolvers for your EC2 instances (the default setting), then GuardDuty can access and process your request and response DNS logs through the internal AWS DNS resolvers. Using other DNS resolvers will not provide GuardDuty access to its DNS logs.
- GuardDuty vs Macie
 - Amazon GuardDuty provides broad protection of your AWS accounts, workloads, and data by helping to identify threats such as attacker reconnaissance, instance compromise, and account compromise. Amazon Macie helps you protect your data in Amazon S3 by helping you classify what data you have, the value that data has to the business, and the behavior associated with access to that data.



Amazon Inspector

- An automated security assessment service that helps you test the network accessibility of your EC2 instances and the security state of your applications running on the instances.
- Inspector uses IAM *service-linked roles*.

Features

- Inspector provides an engine that analyzes system and resource configuration and monitors activity to determine what an assessment target looks like, how it behaves, and its dependent components. The combination of this telemetry provides a complete picture of the assessment target and its potential security or compliance issues.
- Inspector incorporates a built-in library of rules and reports. These include checks against best practices, common compliance standards and vulnerabilities.
- Automate security vulnerability assessments throughout your development and deployment pipeline or against static production systems.
- Inspector is an API-driven service that uses an optional *agent*, making it easy to deploy, manage, and automate.

Concepts

- **Inspector Agent** - A software agent that you can install on all EC2 instances that are included in the assessment target, the security of which you want to evaluate with Inspector.
- **Assessment run** - The process of discovering potential security issues through the analysis of your assessment target's configuration and behavior against specified rules packages.
- **Assessment target** - A collection of AWS resources that work together as a unit to help you accomplish your business goals. Inspector assessment targets can consist only of EC2 instances.
- **Assessment template** - A configuration that is used during your assessment run, which includes
 - Rules packages against which you want Inspector to evaluate your assessment target,
 - The duration of the assessment run,
 - Amazon SNS topics to which you want the Inspector to send notifications about assessment run states and findings,
 - Inspector-specific attributes (key-value pairs) that you can assign to findings generated by the assessment run that uses this assessment template.
 - After you create an assessment template, you can't modify it.
- **Finding** - A potential security issue was discovered during the assessment run of the specified target.
- **Rule** - A security check performed during an assessment run. When a rule detects a potential security issue, Inspector generates a finding that describes the issue.
- **Rules package** - A collection of rules that corresponds to a security goal that you might have.
- **Telemetry** - EC2 instance data collected by Inspector during an assessment run and passed to the Inspector service for analysis.



- The telemetry data generated by the Inspector Agent during assessment runs is formatted in JSON files and delivered in near-real-time over TLS to Inspector, where it is encrypted with a per-assessment-run, ephemeral KMS-derived key and securely stored in an S3 bucket dedicated to the service.

Assessment Reports

- A document that details what is tested in the assessment run and the results of the assessment.
- You can view the following types of assessment reports:
 - **Findings report** - this report contains the following information:
 - Executive summary of the assessment
 - EC2 instances evaluated during the assessment run
 - Rules packages included in the assessment run
 - Detailed information about each finding, including all EC2 instances that had the finding
 - **Full report** - this report contains all the information that is included in a findings report, and additionally provides the list of rules that passed on all instances in the assessment target.



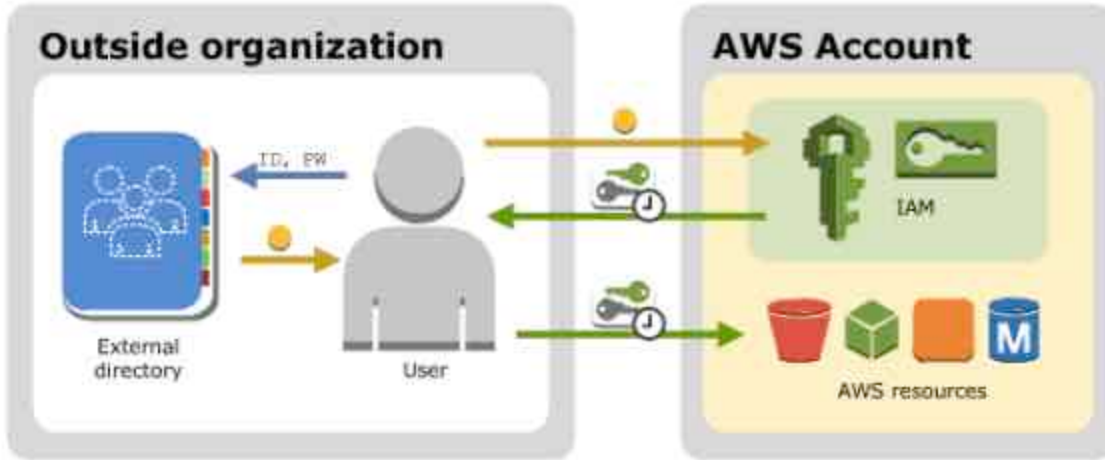
Amazon Macie

- A security service that uses machine learning to automatically discover, classify, and protect sensitive data in AWS. Macie recognizes sensitive data such as personally identifiable information (PII) or intellectual property.
- Amazon Macie allows you to achieve the following:
 - Identify and protect various data types, including PII, PHI, regulatory documents, API keys, and secret keys
 - Verify compliance with automated logs that allow for instant auditing
 - Identify changes to policies and access control lists
 - Observe changes in user behavior and receive actionable alerts
 - Receive notifications when data and account credentials leave protected zones
 - Detect when large quantities of business-critical documents are shared internally and externally
- **Concepts**
 - An **Alert** is a notification about a potential security issue that Macie discovers. Alerts appear on the Macie console and provide a comprehensive narrative about all activity that occurred over the last 24 hours.
 - **Basic alerts** – Alerts that are generated by the security checks that Macie performs. There are two types of basic alerts in Macie:
 - **Managed** (curated by Macie) basic alerts that you can't modify. You can only enable or disable the existing managed basic alerts.
 - **Custom** basic alerts that you can create and modify to your exact specifications.
 - **Predictive alerts** – Automatic alerts based on activity in your AWS infrastructure that deviates from the established normal activity baseline. More specifically, Macie continuously monitors IAM user and role activity in your AWS infrastructure and builds a model of normal behavior. It then looks for deviations from that normal baseline, and when it detects such activity, it generates automatic predictive alerts.
 - **Data source** is the origin or location of a set of data.
 - **AWS CloudTrail** event logs and errors, including Amazon S3 object-level API activity. You can't modify existing or add new CloudTrail events to the list that Macie manages. You can enable or disable the supported CloudTrail events, thus instructing Macie to either include or exclude them in its data security process.
 - **Amazon S3** objects. You can integrate Macie with your S3 buckets and/or specify S3 prefixes
 - **User**, in the context of Macie, a user is the AWS Identity and Access Management (IAM) identity that makes the request.



AWS Identity & Access Management (IAM)

- Control who is authenticated (signed in) and authorized (has permissions) to use resources.
- AWS account root user is a single sign-in identity that has complete access to all AWS services and resources in the account.
- **Features**
 - You can grant other people permission to administer and use resources in your AWS account without having to share your password or access key.
 - You can grant different permissions to different people for different resources.
 - You can use IAM features to securely provide credentials for applications that run on EC2 instances which provide permissions for your applications to access other AWS resources.
 - You can add two-factor authentication to your account and to individual users for extra security.
 - You can allow users to use identity federation to get temporary access to your AWS account.
 - You receive AWS CloudTrail log records that include information about IAM identities who made requests for resources in your account.
 - You use an access key (an access key ID and secret access key) to make programmatic requests to AWS. An Access Key ID and Secret Access Key can only be uniquely generated once and must be regenerated if lost.
 - You can generate and download a credential report that lists all users on your AWS account. The report also shows the status of passwords, access keys, and MFA devices.
- **Users**
 - IAM Users
 - Instead of sharing your root user credentials with others, you can create individual IAM users within your account that correspond to users in your organization. IAM users are not separate accounts; they are users within your account.
 - Each user can have their own password for access to the AWS Management Console. You can also create an individual access key for each user so that the user can make programmatic requests to work with resources in your account.
 - By default, a brand new IAM user has NO permissions to do anything.
 - Users are global entities.
 - Federated Users
 - If the users in your organization already have a way to be authenticated, you can federate those user identities into AWS.



- IAM Groups
 - An IAM group is a collection of IAM users.
 - You can organize IAM users into IAM groups and attach access control policies to a group.
 - A user can belong to multiple groups.
 - Groups cannot belong to other groups.
 - Groups do not have security credentials, and cannot access web services directly.
- IAM Role
 - A role does not have any credentials associated with it.
 - An IAM user can assume a role to temporarily take on different permissions for a specific task. A role can be assigned to a federated user who signs in by using an external identity provider instead of IAM.
 - AWS service role is a role that a service assumes to perform actions in your account on your behalf. This service role must include all the permissions required for the service to access the AWS resources that it needs.
 - AWS service role for an EC2 instance is a special type of service role that a service assumes to launch an EC2 instance that runs your application. This role is assigned to the EC2 instance when it is launched.
 - AWS service-linked role is a unique type of service role that is linked directly to an AWS service. Service-linked roles are predefined by the service and include all the permissions that the service requires to call other AWS services on your behalf.
 - An instance profile is a container for an IAM role that you can use to pass role information to an EC2 instance when the instance starts.
- Users or groups can have multiple policies attached to them that grant different permissions.



When to Create IAM User	When to Create an IAM Role
You create an AWS account and you're the only person who works in your account.	You're creating an application that runs on an Amazon EC2 instance and that application makes requests to AWS.
Other people in your group need to work in your AWS account, and your group is using no other identity mechanism.	You're creating an app that runs on a mobile phone that makes requests to AWS.
You want to use the command-line interface to work with AWS.	Users in your company are authenticated in your corporate network and want to be able to use AWS without having to sign in again (federate into AWS).

AWS Key Management Service

- A managed service that enables you to easily encrypt your data. KMS provides a highly available key storage, management, and auditing solution for you to encrypt data within your own applications and control the encryption of stored data across AWS services.
- It works almost like CloudHSM since, under the hood, KMS also uses hardware security modules that make it easy for you to create and control your encryption keys. But unlike CloudHSM, this service has multi-tenant access, which means you share the HSM with other tenants or AWS customers. You also cannot launch an HSM to Amazon VPC or EC2 instances that you own. The HSM is fully managed by the Amazon Web Services team themselves. AWS KMS can be integrated with other AWS services to help you protect the data you store with these services. For example, encrypting volumes or snapshots in Amazon EBS is powered by AWS KMS as well as Server-Side encryption (SSE-KMS) in Amazon S3 and database encryption in Amazon RDS.
- AWS KMS uses envelope encryption, which is the practice of encrypting your plaintext data with a data key; and then encrypting that data key using another key, called the master key. The primary resources in KMS are called KMS keys. A KMS key is basically a representation of the master key that encrypts your data key. With AWS KMS, you can store your KMS keys and automatically rotate them to meet your encryption requirements. You can also create a custom key store in AWS KMS with CloudHSM. This custom key store provides complete control over your encryption key lifecycle management and allows you to remove the key material of your encryption keys. You can also audit key usage independently of AWS CloudTrail or KMS itself.



Features

- KMS is integrated with CloudTrail, which provides you the ability to audit who used which keys, on which resources, and when.
- KMS key are used to control access to data encryption keys that encrypt and decrypt your data.
- You can choose to have KMS automatically rotate master keys created within KMS once per year without the need to re-encrypt data that has already been encrypted with your master key.

Concepts

- **KMS key** - You can use a KMS key to encrypt and decrypt up to 4 KB of data. Typically, you use KMS keys to generate, encrypt, and decrypt the data keys that you use outside of KMS to encrypt your data. Master keys are 256 bits in length.
- There are three types of KMS keys:

Type of KMS key	Can view	Can manage	Used only for my AWS account
Customer managed key	Yes	Yes	Yes
AWS managed key	Yes	No	Yes
AWS owned key	No	No	No

- **Customer managed keys** are KMS keys that you create, own, and manage. You have full control over these KMS keys, including establishing and maintaining their key policies, IAM policies, and grants, enabling and disabling them, rotating their cryptographic material, adding tags, creating aliases that refer to the KMS key, and scheduling the KMS key for deletion.
- **AWS managed keys** are KMS keys in your account that are created, managed, and used on your behalf by an AWS service that integrates with KMS. You can view the AWS managed keys in your account, view their key policies, and audit their use in CloudTrail logs. However, you cannot manage these KMS keys or change their permissions. And, you cannot use AWS managed keys in cryptographic operations directly; the service that creates them uses them on your behalf.
- **AWS owned keys** are not in your AWS account. They are part of a collection of KMS keys that AWS owns and manages for use in multiple AWS accounts. AWS services can use AWS owned keys to protect your data. You cannot view, manage, or use AWS owned keys, or audit their use.



AWS Secrets Manager

- A secret management service that enables you to easily rotate, manage, and retrieve database credentials, API keys, and other secrets throughout their lifecycle.
- **Features**
 - AWS Secrets Manager encrypts secrets at rest using encryption keys that you own and store in AWS Key Management Service [customer managed keys]. When you retrieve a secret, Secrets Manager decrypts the secret and transmits it securely over TLS to your local environment.
 - You can rotate secrets on a schedule or on demand by using the Secrets Manager console, AWS SDK, or AWS CLI.
 - Secrets Manager natively supports rotating credentials for databases hosted on Amazon RDS and Amazon DocumentDB and clusters hosted on Amazon Redshift.
 - You can extend Secrets Manager to rotate other secrets, such as credentials for Oracle databases hosted on EC2 or OAuth refresh tokens, by using custom AWS Lambda functions.
- A secret consists of a set of credentials (username and password), and the connection details used to access a secured service.
- A secret can contain versions:
 - Although you typically only have one version of the secret active at a time, multiple versions can exist while you rotate a secret on the database or service. Whenever you change the secret, Secrets Manager creates a new version.
 - Each version holds a copy of the encrypted secret value.
 - Each version can have one or more staging labels attached identifying the stage of the secret rotation cycle.
- **Supported Secrets**
 - Database credentials, on-premises resource credentials, SaaS application credentials, third-party API keys, and SSH keys.
 - You can also store JSON documents.
- To retrieve secrets, you simply replace secrets in plain text in your applications with code to pull in those secrets programmatically using the Secrets Manager APIs.
- Secrets can be cached on the client side and updated only during a secret rotation.
- During the secret rotation process, Secrets Manager tracks the older credentials, as well as the new credentials you want to start using until the rotation completes. It tracks these different versions by using staging labels.



AWS Certificate Manager

- A service that lets you easily provision, manage, and deploy public and private SSL/TLS certificates for use with AWS services and your internal connected resources. SSL/TLS certificates are used to secure network communications and establish the identity of websites over the Internet as well as resources on private networks.
- ACM is integrated with the following services:
 - Elastic Load Balancing
 - Amazon CloudFront - To use an ACM certificate with CloudFront, you must request or import the certificate in the US East (N. Virginia) region.
 - AWS Elastic Beanstalk
 - Amazon API Gateway
 - AWS CloudFormation
- AWS Certificate Manager manages the renewal process for the certificates managed in ACM and used with ACM-integrated services.
- You can import your own certificates into ACM, however, you have to renew these yourself.
- Concepts
 - ACM Certificate are X.509 version 3 certificates. Each is valid for **13 months**.
 - When you request an ACM certificate, you must validate that you own or control all of the domains that you specify in your request.
 - **Each ACM Certificate must include at least one fully qualified domain name (FQDN)**. You can add additional names if you want to.
 - You can create an ACM Certificate containing a wildcard name (*.example.com) that can protect several sites in the same domain (subdomains).
 - You cannot download the private key for an ACM Certificate.
 - The first time you request or import a certificate in an AWS region, ACM creates an AWS-managed key (KMS key) in AWS KMS with the alias aws/acm. This KMS key is unique in each AWS account and each AWS region. ACM uses this KMS key to encrypt the certificate's private key.
 - You cannot add or remove domain names from an existing ACM Certificate. Instead, you must request a new certificate with the revised list of domain names.
 - You cannot delete an ACM Certificate that is being used by another AWS service. To delete a certificate that is in use, you must first remove the certificate association.
 - Applications and browsers trust public certificates automatically by default, whereas an administrator must explicitly configure applications to trust private certificates.
- Types of Certificates For Use With ACM
 - **Public certificates**
 - ACM manages the renewal and deployment of public certificates used with ACM-integrated services.
 - You cannot install public ACM certificates directly on your website or application, only for integrated services.



- **Private certificates**
 - ACM Private CA provides three ways to create and manage private certificates. 1) You can choose to delegate private certificate management to ACM. When used in this way, ACM can automatically renew and deploy private certificates used with ACM-integrated services. 2) You can export private certificates from ACM and use them with EC2 instances, containers, on-premises servers, and IoT devices. ACM Private CA automatically renews these certificates and sends an Amazon CloudWatch notification when the renewal is completed. You can write client-side code to download renewed certificates and private keys and deploy them with your application. 3) ACM Private CA gives you the flexibility to create your own private keys, generate a certificate signing request (CSR), issue private certificates from your ACM Private CA, and manage the keys and certificates yourself. You are responsible for renewing and deploying these private certificates.
- **Imported certificates**
 - If you want to use a third-party certificate with ACM-integrated services, you may import it into ACM using the AWS Management Console, AWS CLI, or ACM APIs. ACM does not manage the renewal process for imported certificates. You are responsible for monitoring the expiration date of your imported certificates and for renewing them before they expire. You can use the AWS Management Console to monitor the expiration dates of imported certificates and import a new third-party certificate to replace an expiring one.
- **CA certificates**
 - ACM private CA can issue certificates to identify private certificate authorities. These certificates allow CA administrators to create a private CA hierarchy, which provides strong security and restrictive access controls for the most-trusted root CA at the top of the trust chain, while allowing more permissive access and bulk certificate issuance for subordinate CAs lower in the chain.

ACM Private Certificate Authority

- ACM PCA allows you to create a private certificate authority (CA) and then use ACM to issue private certificates.
- With ACM Private CA, you can create complete CA hierarchies, including root and subordinate CAs. A CA hierarchy provides strong security and restrictive access controls for the most-trusted root CA at the top of the trust chain while allowing more permissive access and bulk certificate issuance for subordinate CAs lower in the chain.
- A private CA handles the issuance, validation, and revocation of private certificates within a private network. It is comprised of two major components: The first is the **CA certificate**, a cryptographic building block upon which certificates can be issued. The second is a **set of run-time services** for maintaining revocation information through the **Certificate Revocation List (CRL)**.
- Benefits of a Private CA
 - Create certificates with any subject name you want.



- Create certificates with any expiration date you want.
- Use any supported private key algorithm and key length.
- Use any supported signing algorithm.
- Configure certificates in bulk using templates.
- Automatic renewal is not available for ACM Private CA certificates for which ACM does not create the private key and certificate signing request (CSR).
- You cannot copy private CAs between Regions. To use private CAs in more than one Region, you must create your CAs in those Regions.

Domain Verification for Certificates

- Before the Amazon certificate authority can issue a certificate for your site, AWS Certificate Manager must verify that you own or control all of the domain names that you specified in your request. You can choose either **email validation** or **DNS validation** when you request a certificate.
- For DNS validation, ACM uses **CNAME (Canonical Name) records to validate** that you own or control a domain.
- In the DNS validation console page, ACM will provide you a CNAME record that you must add to your DNS database, whether it be Route 53 or other hosts.
- For email validation, AWS Certificate Manager (ACM) sends emails to five predefined common system addresses for each domain you specify: **admin@**, **administrator@**, **hostmaster@**, **postmaster@**, and **webmaster@**. Previously, emails were also sent to the three contact addresses listed in the WHOIS database, but this lookup has been discontinued. To complete the validation process, one of the recipients must click the approval link in the email.

Pricing

- There is no additional charge for provisioning public or private SSL/TLS certificates you use with ACM-integrated services, such as Elastic Load Balancing and API Gateway.
- You are billed for each active ACM Private CA per month pro-rated
- For private certificates, ACM Private CA allows you to pay monthly for the service and certificates you create. You pay less per certificate as you create more private certificates.

References:

<https://aws.amazon.com/certificate-manager/>

<https://aws.amazon.com/certificate-manager/faqs/>

<https://docs.aws.amazon.com/acm/latest/userguide/acm-overview.html>

<https://docs.aws.amazon.com/acm-pca/latest/userguide/PcaWelcome.html>



AWS Directory Service

For Microsoft Active Directory

- Also known as **AWS Managed Microsoft AD**, the service enables your directory-aware workloads and AWS resources to use **managed Active Directory** in the AWS Cloud.
- The service is built on the actual Microsoft Active Directory and powered by Windows Server 2012 R2.
- AWS Managed Microsoft AD is your best choice if you need actual Active Directory features to support AWS applications or Windows workloads, including Amazon RDS for Microsoft SQL Server. It's also best if you want a standalone AD in the Cloud that supports Office 365 or you need an LDAP directory to support your Linux applications.
- Concepts
 - AWS Managed Microsoft AD provides multiple directory choices for customers who want to use existing Microsoft AD or Lightweight Directory Access Protocol (LDAP)–aware applications in the cloud.
 - When you create a directory, AWS Directory Service creates two domain controllers and adds the DNS service on your behalf. The domain controllers are created in different subnets in a VPC
 - When creating a directory, you need to provide some basic information such as a fully qualified domain name (FQDN) for your directory, the Administrator account name and password, and the VPC you want the directory to be attached to.
 - AWS does not provide Windows PowerShell access to directory instances, and it restricts access to directory objects, roles, and groups that require elevated privileges.
 - AWS Managed Microsoft AD does not allow direct host access to domain controllers via Telnet, Secure Shell (SSH), or Windows Remote Desktop Connection.
 - When you create an AWS Managed Microsoft AD directory, you are assigned an organizational unit (OU) and an administrative account with delegated administrative rights for the OU.
 - AWS Managed Microsoft AD directories are deployed across **two Availability Zones in a region** by default and connected to your Amazon VPC.
 - You cannot configure the storage, CPU, or memory parameters of your AWS Managed Microsoft AD directory.
- Active Directory Schema
 - A **schema** is the definition of attributes and classes that are part of a distributed directory and is similar to fields and tables in a database. Schemas include a set of rules which determine the type and format of data that can be added or included in the database.
 - Attributes, classes, and objects are the basic elements that are used to build object definitions in the schema.
 - Each schema attribute, which is similar to a field in a database, has several properties that define the characteristics of the attribute.
 - The classes are analogous to tables in a database and also have several properties to be defined.



- Each class and attribute must have an Object ID that is unique for all of your objects. Software vendors must obtain their own Object ID to ensure uniqueness.
- Some attributes are linked between two classes with forward and backlinks, such as groups. A group shows you the members of the group, while a member shows what groups it belongs to.
- Features
 - AWS Managed Microsoft AD is deployed in HA and across multiple Availability Zones. You can also scale out your directory by deploying additional domain controllers.
 - AWS Managed Microsoft AD runs on AWS-managed infrastructure with monitoring that automatically detects and replaces domain controllers that fail.
 - Data replication and automated daily snapshots are configured for you.
 - You can integrate AWS Managed Microsoft AD easily with your existing Active Directory by using **Active Directory trust relationships**.
 - Allows seamless domain join for new and existing Amazon EC2 for Windows Server instances.
 - AWS Managed Microsoft AD can also provide a single directory for all kinds of workloads (EC2, RDS, WorkSpaces, etc).
 - The service supports schema extensions that you submit to the service in the form of a LDAP Data Interchange Format (LDIF) file.
 - You can configure Amazon SNS to receive email and text messages when the status of your AWS Directory Service changes.
 - You can configure SAML 2.0–based authentication with cloud applications using AWS Directory Service.
 - You can use AWS Managed Microsoft AD as a resource forest that contains primarily computers and groups with trust relationships to your on-premises directory. This enables your users to access AWS applications and resources with their on-premises AD credentials.
- Microsoft AD Prerequisites
 - A VPC with at least two subnets. Each of the subnets must be in a different Availability Zone.
 - The necessary ports for the domain controllers that AWS Directory Service creates for you should be open to allow them to communicate with each other.
 - The VPC must have default hardware tenancy.
 - AWS Directory Service does not support using NAT with Active Directory.
- Two Editions of AWS Managed Microsoft AD
 - Both Standard Edition and Enterprise Edition can be used as your organization's primary directory to manage users, devices, and computers.
 - You also can use both editions to create resource forests and extend your on-premises AD to the AWS Cloud. **Resource forests** use a trust relationship with your on-premises AD to enable you to access AWS applications and resources with your on-premises AD credentials.
 - Both editions also support the creation of additional domain controllers to improve the redundancy and performance of your managed directory.
 - Unique to Standard Edition



- Optimized to be a primary directory for small and midsize businesses with up to 5,000 employees.
 - Provides you with enough storage capacity to support up to approximately 30,000 directory objects, such as users, groups, and computers.
- Unique to Enterprise Edition
 - Designed to support enterprise organizations with up to approximately 500,000 directory objects.
- Seamless Domain Joins
 - **Seamless domain join** is a feature that allows you to join your Amazon EC2 for Windows Server instances seamlessly to a domain, at the time of launch and from the AWS Management Console. You can join instances to AWS Managed Microsoft AD that you launch in the AWS Cloud.
 - You cannot use the seamless domain join feature from the AWS Management Console for **existing EC2 for Windows Server** instances, but you can join existing instances to a domain using the EC2 API or by using PowerShell on the instance.
- Security and Monitoring
 - AWS Managed Microsoft AD is both HIPAA and PCI DSS compliant.
 - Manage users and devices by using native Active Directory Group Policy objects (GPOs).
 - AWS Managed Microsoft AD uses the same Kerberos-based authentication as Active Directory to deliver IAM Identity Center.
 - AWS Managed Microsoft AD supports federation access for users and groups to the AWS Management Console.
 - Amazon EBS volumes used in the directory service are encrypted.
- Pricing
 - You pay only for the type and size of the managed directory that you use.
 - AWS Managed Microsoft AD allows you to use a directory in one account and share it with multiple accounts and VPCs. There is an hourly sharing charge for each additional account to which you share a directory.

Active Directory Connector

- A **proxy service** that provides an easy way to connect compatible AWS applications, such as Amazon WorkSpaces, Amazon QuickSight, and Amazon EC2 for Windows Server instances, to your existing on-premises Microsoft Active Directory.
- AD Connector is your best choice when you want to use your existing on-premises directory with compatible AWS services.
- Features
 - When users log in to the AWS applications, AD Connector forwards sign-in requests to your on-premises Active Directory domain controllers for authentication.



- You can also join your EC2 Windows instances to your on-premises Active Directory domain through AD Connector using seamless domain join.
- AD Connector is NOT compatible with RDS SQL Server.
- AD Connector comes in two sizes, small and large.
- You can spread application loads across multiple AD Connectors to scale to your performance needs. There are no enforced user or connection limits.
- AD Connector Prerequisites
 - You need to have a VPC with at least two subnets. Each of the subnets must be in a different Availability Zone.
 - The VPC must be connected to your existing network through a virtual private network (VPN) connection or AWS Direct Connect.
 - The VPC must have default hardware tenancy.
 - Your user accounts must have Kerberos pre-authentication enabled.

Simple AD

- A **standalone Microsoft Active Directory-compatible** directory from AWS Directory Service that is powered by **Samba 4**.
- You can use Simple AD as a standalone directory in the cloud to support Windows workloads that need basic AD features, compatible AWS applications, or to support Linux workloads that need LDAP service.
- Features
 - Simple AD supports basic Active Directory features such as user accounts, group memberships, joining a Linux domain or Windows-based EC2 instances, Kerberos-based SSO, and group policies.
 - AWS provides monitoring, daily snapshots, and recovery as part of the service.
 - Simple AD is compatible with the following AWS applications: Amazon WorkSpaces, Amazon WorkDocs, Amazon QuickSight, and Amazon WorkMail.
 - You can also sign in to the AWS Management Console with Simple AD user accounts.
 - Simple AD does NOT support multi-factor authentication, trust relationships, DNS dynamic update, schema extensions, communication over LDAPS, PowerShell AD cmdlets, or FSMO role transfer.
 - Simple AD is NOT compatible with RDS SQL Server.
 - Simple AD is available in two sizes:
 - Small - Supports up to 500 users
 - Large - Supports up to 5,000 users
- Simple AD Prerequisites
 - Your VPC should have at least two subnets. For Simple AD to install correctly, you must install your two domain controllers in separate subnets that must be in a different Availability Zone. In addition, the subnets must be in the same Classless Inter-Domain Routing (CIDR) range.



- The necessary ports for the domain controllers that AWS Directory Service creates for you should be open to allow them to communicate with each other.
 - The VPC must have default hardware tenancy.
- When you create a directory with Simple AD, AWS Directory Service performs the following tasks on your behalf:
 - Set up a Samba-based directory within the VPC.
 - Creates a directory administrator account with the user name 'Administrator' and the specified password. You use this account to manage your directory.
 - Creates a security group for the directory controllers.
 - Creates an account that has domain admin privileges.
- Simple AD forwards DNS requests to the IP address of the Amazon-provided DNS servers for your VPC. These DNS servers will resolve names configured in your Route 53 private hosted zones

Amazon Cloud Directory

- A **cloud-native directory** that can store hundreds of millions of application-specific objects with multiple relationships and schemas. Use Amazon Cloud Directory if you need a **highly scalable directory store** for your application's **hierarchical data**.
- You can organize directory objects into multiple hierarchies to support many organizational pivots and relationships across directory information.
- Concepts
 - A schema is a collection of facets that define what objects can be created in a directory and how they are organized.
 - A schema also enforces data integrity and interoperability.
 - A single schema can be applied to more than one directory at a time.
 - Amazon Cloud Directory supports uploading a compliant **JSON file for schema creation**.
 - A directory is a schema-based data store that contains specific types of objects organized in a multi-hierarchical structure.
 - Before you can create a directory in Amazon Cloud Directory, AWS Directory Service requires that you first apply a schema to it. A directory cannot be created without a schema and typically has one schema applied to it.

References:

<https://aws.amazon.com/directoryservice/features/?nc=sn&loc=2>

https://docs.aws.amazon.com/clouddirectory/latest/developerguide/what_is_cloud_directory.html

https://docs.aws.amazon.com/directoryservice/latest/admin-guide/what_is.html

https://docs.aws.amazon.com/clouddirectory/latest/developerguide/what_is_cloud_directory.html



AWS Resource Access Manager

- A service that enables you to easily and securely share AWS resources with any AWS account or, if you are part of AWS Organizations, with Organizational Units (OUs) or your entire Organization. If you share resources with accounts that are outside of your Organization, then those accounts will receive an invitation to the Resource Share and can start using the shared resources upon accepting the invitation.
 - Only the master account can enable sharing with AWS Organizations.
 - The organization must be enabled for all features.
- RAM eliminates the need to create duplicate resources in multiple accounts. You can create resources centrally in a multi-account environment and use RAM to share those resources across accounts in three simple steps:
 1. Create a Resource Share
 2. Specify resources
 3. Specify accounts
- You can stop sharing a resource by deleting the share in AWS RAM.
- Services you can share with AWS RAM

Service	Resource
Amazon Aurora	DB Clusters
AWS CodeBuild	Project, Report Groups
Amazon EC2	Capacity Reservations, Dedicated Hosts, Subnets, Traffic mirror targets, Transit Gateways
Amazon EC2 Image Builder	Components, Images (AMI), Image recipes
AWS License Manager	License configurations
AWS Resource Groups	Resource groups
Amazon Route 53	Forwarding rules

- Security
 - Use IAM policies to secure who can access resources that you shared or received from another account.
- Pricing
 - There is no additional charge for using AWS RAM.



References:

<https://aws.amazon.com/ram/>

<https://aws.amazon.com/ram/faqs/>



AWS Security Hub

- AWS Security Hub provides a **comprehensive view** of your **security state** within AWS and your **compliance** with security industry standards and best practices.
- Features
 - You now have a single place that **aggregates, organizes, and prioritizes your security alerts**, or findings, across multiple accounts, AWS partner tools, and AWS services such as Amazon GuardDuty, Amazon Inspector, Amazon Macie, AWS IAM Access Analyzer, AWS Firewall Manager, and AWS Audit Manager.
 - AWS Security Hub works with AWS Organizations to simplify security posture management across all of your existing and future AWS accounts in an organization.
 - You can run automated, continuous account-level configuration and compliance checks based on industry standards and best practices, such as the Center for Internet Security (CIS) AWS Foundations Benchmark. These checks provide a compliance score and identify specific accounts and resources that require attention.
 - AWS Security Hub compliance checks also leverage configuration items recorded by AWS Config.
 - Integrated dashboards consolidate your security findings across accounts to show you their current security and compliance status.
 - You can send security findings to ticketing, chat, email, or automated remediation systems through integration with Amazon EventBridge.
 - All findings are stored for at least 90 days within AWS Security Hub.
- Security Hub receives and processes only those findings from the same Region where you enabled Security Hub in your account.
- Concepts
 - AWS Security Finding Format - A standardized format for the contents of findings that Security Hub aggregates or generates.
 - Control - A safeguard or countermeasure prescribed for an information system or an organization designed to protect the confidentiality, integrity, and availability of its information and to meet a set of defined security requirements. A security standard consists of controls.
 - Custom action - A Security Hub mechanism for sending selected findings to Amazon EventBridge.
 - Finding - The observable record of a compliance check or security-related detection.
 - Insight - A collection of related findings defined by an aggregation statement and optional filters. An insight identifies a security area that requires attention and intervention.
 - Compliance standards - Sets of controls that are based on regulatory requirements or best practices.
 - You can disable specific compliance controls that are not relevant to your workloads.
- Compliance standard vs. Control vs. Compliance check
 - A compliance standard is a collection of controls based on regulatory frameworks or industry best practices. Security Hub conducts automated compliance checks against controls. Each



compliance check consists of an evaluation of a rule against a single resource. A single control may involve multiple resources, and a compliance check is performed against each resource.

- AWS Security Hub uses a **service-linked role** that includes the permissions and trust policy that Security Hub requires to detect and aggregate findings and to configure the requisite AWS Config infrastructure needed to run compliance checks. In order for Security Hub to run **compliance checks** in an account, you must have **AWS Config enabled** in that account.
- Pricing
 - AWS Security Hub is priced based on the **quantity of compliance checks** and the **quantity of finding ingestion events**.
 - Pricing is on a monthly per-account, per-region basis.

References:

<https://aws.amazon.com/security-hub/>

<https://aws.amazon.com/security-hub/faqs/>

AWS Security Token Service (AWS STS)

AWS Security Token Service is a service that enables the management of temporary, limited-privilege credentials for AWS resources. By using AWS STS, you can avoid long-term credential distribution, allowing for secure temporary access through roles or identity federation. This service is particularly useful in scenarios requiring short-term access to AWS resources, thereby enhancing security by minimizing the exposure of more permanent credentials.

Key Features:

- AWS STS provides short-term credentials that can last from a few minutes to several hours, expiring automatically after the set duration. This mitigates risks associated with long-term credential exposure.
- By default, STS operates globally with a central endpoint. However, AWS recommends utilizing regional endpoints to reduce latency, enhance redundancy, and potentially increase session token validity.
- AWS STS is compatible with AWS CloudTrail, allowing for detailed logging of STS API calls. This feature aids in auditing and monitoring the use of temporary credentials across AWS resources.
- AWS STS supports identity federation, allowing users from external systems to access AWS resources without AWS-specific credentials. It also supports cross-account roles, enabling resource access across different AWS accounts without direct identity provisioning within each account.
- You can specify the desired validity period for the temporary credentials, tailored to the needs of specific tasks or operations within AWS environments.

AWS Security Token Service supports a range of commands for various operations, including:

- `assume-role` : To assume an IAM role.
- `assume-role-with-saml` : For assuming a role with SAML authentication.



- `assume-role-with-web-identity` : To assume a role with web identity federation.
- `decode-authorization-message` : To decode additional information about authorization status.
- `get-access-key-info` : For retrieving information about the access key used in a request.
- `get-caller-identity` : To retrieve details about the entity making the call.
- `get-federation-token` : To get a federation token for a federated user.
- `get-session-token` : For getting a session token for MFA or in cases where none is required.

References:

<https://docs.aws.amazon.com/STS/latest/APIReference/welcome.html>

<https://awscli.amazonaws.com/v2/documentation/api/2.0.33/reference/sts/index.html>



AWS Shield

- A managed Distributed Denial of Service (DDoS) protection service that safeguards applications running on AWS.

Shield Tiers and Features

- **Standard**
 - All AWS customers benefit from the automatic protections of Shield Standard.
 - Shield Standard provides always-on network flow monitoring, which inspects incoming traffic to AWS and detects malicious traffic in real-time.
 - Uses several techniques like deterministic packet filtering and priority-based traffic shaping to automatically mitigate attacks without impact to your applications.
 - When you use Shield Standard with CloudFront and Route 53, you receive comprehensive availability protection against all known infrastructure attacks.
 - You can also view all the events detected and mitigated by AWS Shield in your account.
- **Advanced**
 - Shield Advanced provides enhanced detection, inspecting network flows, and also monitoring application layer traffic to your Elastic IP address, Elastic Load Balancing, CloudFront, or Route 53 resources.
 - It handles the majority of DDoS protection and mitigation responsibilities for **layer 3**, **layer 4**, and **layer 7** attacks.
 - You have 24x7 access to the AWS DDoS Response Team. To contact the DDoS Response Team, customers will need the Enterprise or Business Support levels of AWS Premium Support.
 - It automatically provides additional mitigation capacity to protect against larger DDoS attacks. The DDoS Response Team also applies manual mitigations for more complex and sophisticated DDoS attacks.
 - It gives you complete visibility into DDoS attacks with near real-time notification via CloudWatch and detailed diagnostics on the “AWS WAF and AWS Shield” Management Console.
 - Shield Advanced comes with “DDoS cost protection”, a safeguard from scaling charges as a result of a DDoS attack that causes usage spikes on your AWS services. It does so by providing service credits for charges due to usage spikes.
 - It is available globally on all CloudFront and Route 53 edge locations.
 - With Shield Advanced, you will be able to see the history of all incidents in the trailing 13 months.

Pricing

- **Shield Standard** provides protection at no additional charge.



- **Shield Advanced**, however, is a paid service. It requires a 1-year subscription commitment and charges a monthly fee, plus a usage fee based on data transfer out from CloudFront, ELB, EC2, and AWS Global Accelerator.

References:

<https://aws.amazon.com/shield/features/>

<https://aws.amazon.com/shield/pricing/>

<https://aws.amazon.com/shield/faqs/>



AWS WAF

- A web application firewall that helps protect web applications from attacks by allowing you to configure rules that **allow, block, or monitor (count) web requests** based on conditions that you define.
- These conditions include:
 - IP addresses
 - HTTP headers
 - HTTP body
 - URI strings
 - SQL injection
 - cross-site scripting.

Features

- WAF lets you create rules to filter web traffic based on conditions that include IP addresses, HTTP headers, and body, or custom URIs.
- You can also create rules that block common web exploits like SQL injection and cross-site scripting.
- For application layer attacks, you can use WAF to respond to incidents. You can set up proactive rules like *Rate-Based Blacklisting* to automatically block bad traffic or respond immediately to incidents as they happen.
- WAF provides real-time metrics and captures raw requests that include details about IP addresses, geo-locations, URIs, User-Agent, and Referers.
- **AWS WAF Security Automations** is a solution that automatically deploys a single web access control list (web ACL) with a set of AWS WAF rules designed to filter common web-based attacks. The solution supports log analysis using Amazon Athena and AWS WAF full logs.

Conditions, Rules, and Web ACLs

- You define your conditions, combine your conditions into rules, and combine the rules into a web ACL.
- **Conditions** define the basic characteristics that you want WAF to watch for in web requests.
- You combine conditions into **rules** to precisely target the requests that you want to allow, block, or count. WAF provides two types of rules:
 - **Regular rules** - use only conditions to target specific requests.
 - **Rate-based rules** - are similar to regular rules, with a rate limit. Rate-based rules count the requests that arrive from a specified IP address every five minutes. The rule can trigger an action if the number of requests exceeds the rate limit.
- **WAF Managed Rules** are an easy way to deploy pre-configured rules to protect your applications' common threats like application vulnerabilities. All Managed Rules are automatically updated by AWS Marketplace security Sellers.



- After you combine your conditions into rules, you combine the rules into a **web ACL**. This is where you define an action for each rule—allow, block, or count—and a default action, which determines whether to allow or block a request that doesn't match all the conditions in any of the rules in the web ACL.

Pricing

- WAF charges are based on the number of web access control lists (web ACLs) that you create, the number of rules that you add per web ACL, and the number of web requests that you receive.

References:

<https://aws.amazon.com/waf/features/>

<https://aws.amazon.com/waf/pricing/>

<https://aws.amazon.com/waf/faqs/>



AWS Management Tools

AWS Organizations

- It offers policy-based management for multiple AWS accounts.

Features

- With Organizations, you can create groups of accounts and then apply policies to those groups.
- Organizations provide you with a policy framework for multiple AWS accounts. You can apply policies to a group of accounts or all the accounts in your organization.
- AWS Organizations enables you to set up a single payment method for all the AWS accounts in your organization through **consolidated billing**. With consolidated billing, you can see a combined view of charges incurred by all your accounts, as well as take advantage of pricing benefits from aggregated usage, such as volume discounts for EC2 and S3.
- AWS Organizations, like many other AWS services, is **eventually consistent**. It achieves high availability by replicating data across multiple servers in AWS data centers within its region.

Administrative Actions in Organizations

- Create an AWS account and add it to your organization, or add an existing AWS account to your organization.
- Organize your AWS accounts into groups called *organizational units* (OUs).
- Organize your OUs into a hierarchy that reflects your company's structure.
- Centrally manage and attach policies to the entire organization, OUs, or individual AWS accounts.

Concepts

- An **organization** is a collection of AWS accounts that you can organize into a hierarchy and manage centrally.
- A **management account** is the AWS account you use to create your organization. You cannot change which account in your organization is the management account.
 - From the management account, you can create other accounts in your organization, invite and manage invitations for other accounts to join your organization, and remove accounts from your organization.
 - You can also attach policies to entities such as administrative roots, organizational units (OUs), or accounts within your organization.
 - The management account has the role of a payer account and is responsible for paying all charges accrued by the accounts in its organization.
- A **member account** is an AWS account, other than the management account, that is part of an organization. A member account can belong to only one organization at a time. The management



account has the responsibilities of a payer account and is responsible for paying all charges that are accrued by the member accounts.

- An **administrative root** is the starting point for organizing your AWS accounts. The administrative root is the topmost container in your organization's hierarchy. Under this root, you can create OUs to logically group your accounts and organize these OUs into a hierarchy that best matches your business needs.
- An **organizational unit** (OU) is a group of AWS accounts within an organization. An OU can also contain other OUs enabling you to create a hierarchy.
- A **policy** is a "document" with one or more statements that define the controls that you want to apply to a group of AWS accounts.
 - **Service control policy** (SCP) is a policy that specifies the services and actions that users and roles can use in the accounts that the SCP affects. SCPs are similar to IAM permission policies except that they don't grant any permissions. Instead, SCPs are *filters* that allow only the specified services and actions to be used in affected accounts
- AWS Organizations has two available feature sets:
 - All organizations support **consolidated billing**, which provides basic management tools that you can use to centrally manage the accounts in your organization.
 - If you enable **all features**, you continue to get all the consolidated billing features plus a set of advanced features such as service control policies.
- You can remove an AWS account from an organization and make it into a standalone account.
- Organization Hierarchy
 - Including root and AWS accounts created in the lowest OUs, your hierarchy can be five levels deep.
 - Policies inherited through hierarchical connections in an organization.
 - Policies can be assigned at different points in the hierarchy.

Pricing

- This service is free.

References:

<https://docs.aws.amazon.com/organizations/latest/userguide/>

<https://aws.amazon.com/organizations/features/>

<https://aws.amazon.com/organizations/faqs/>



Amazon CloudWatch

- Monitoring tool for your AWS resources and applications.
- Display metrics and create alarms that watch the metrics and send notifications or automatically make changes to the resources you are monitoring when a threshold is breached.
- CloudWatch is basically a metrics repository. An AWS service, such as Amazon EC2, puts metrics into the repository and you retrieve statistics based on those metrics. If you put your own custom metrics into the repository, you can retrieve statistics on these metrics as well.
- CloudWatch does not aggregate data across regions. Therefore, metrics are completely separate between regions.
- **CloudWatch Concepts**
 - **Namespaces** - a container for CloudWatch metrics.
 - There is no default namespace.
 - The AWS namespaces use the following naming convention: `AWS/service`.
 - **Metrics** - represents a time-ordered set of data points that are published to CloudWatch.
 - Exists only in the region in which they are created.
 - By default, several services provide free metrics for resources. You can also enable **detailed monitoring**, or publish your own application metrics.
 - **Metric math** enables you to query multiple CloudWatch metrics and use math expressions to create new time series based on these metrics.
 - **Important note for EC2 metrics:** CloudWatch does not collect memory utilization and disk space usage metrics right from the get go. You need to install CloudWatch Agent in your instances first to retrieve these metrics.
 - **Dimensions** - a name/value pair that uniquely identifies a metric.
 - You can assign up to 10 dimensions to a metric.
 - Whenever you add a unique dimension to one of your metrics, you are creating a new variation of that metric.
 - **Statistics** - metric data aggregations over specified periods of time.
 - Each statistic has a unit of measure. Metric data points that specify a unit of measure are aggregated separately.
 - You can specify a unit when you create a custom metric. If you do not specify a unit, CloudWatch uses *None* as the unit.
 - A *period* is the length of time associated with a specific CloudWatch statistic. The default value is 60 seconds.
 - CloudWatch aggregates statistics according to the period length that you specify when retrieving statistics.
 - For large datasets, you can insert a pre-aggregated dataset called a *statistic set*.
- **Alarms** - watches a single metric over a specified time period, and performs one or more specified actions based on the value of the metric relative to a threshold over time.



- You can create an alarm for monitoring CPU usage and load balancer latency, for managing instances, and for billing alarms.
- When an alarm is on a dashboard, it turns red when it is in the *ALARM* state.
- Alarms invoke actions for sustained state changes only.
- Alarm States
 - **OK**—The metric or expression is within the defined threshold.
 - **ALARM**—The metric or expression is outside of the defined threshold.
 - **INSUFFICIENT_DATA**—The alarm has just started, the metric is not available, or not enough data is available for the metric to determine the alarm state.
- You can also monitor your estimated AWS charges by using Amazon CloudWatch Alarms. However, take note that you can only track the estimated AWS charges in CloudWatch and not the actual utilization of your resources. Remember that you can only set coverage targets for your reserved EC2 instances in AWS Budgets or Cost Explorer, but not in CloudWatch.
- When you create an alarm, you specify three settings:
 - **Period** is the length of time to evaluate the metric or expression to create each individual data point for an alarm. It is expressed in seconds.
 - **Evaluation Period** is the number of the most recent periods or data points to evaluate when determining alarm state.
 - **Datapoints to Alarm** is the number of data points within the evaluation period that must be breached to cause the alarm to go to the *ALARM* state. The breaching data points do not have to be consecutive, they just must all be within the last number of data points equal to the **Evaluation Period**.

CloudWatch Dashboard

- Customizable home pages in the CloudWatch console that you can use to monitor your resources in a single view, even those spread across different regions.
- There is no limit on the number of CloudWatch dashboards you can create.
- All dashboards are **global**, not region-specific.
- You can add, remove, resize, move, edit, or rename a graph. You can metrics manually in a graph.

CloudWatch Logs

- Features
 - Monitor logs from EC2 instances in real-time
 - Monitor CloudTrail logged events
 - By default, logs are kept indefinitely and never expire
 - Archive log data
 - Log Route 53 DNS queries

CloudWatch Agent

- Collect more logs and system-level metrics from EC2 instances and your on-premises servers.
- Needs to be installed.



AWS Auto Scaling

- Configure automatic scaling for the AWS resources quickly through a scaling plan that uses dynamic scaling and predictive scaling.
- Optimize for availability, cost, or a balance of both.
- Scaling in means decreasing the size of a group, while scaling out means increasing the size of a group.
- Useful for:
 - Cyclical traffic, such as high use of resources during regular business hours and low use of resources overnight
 - On and off traffic patterns, such as batch processing, testing, or periodic analysis
 - Variable traffic patterns, such as software for marketing campaigns with periods of spiky growth
- **Features**
 - Launch or terminate EC2 instances in an Auto Scaling group.
 - Launch or terminate instances from an EC2 Spot Fleet request or automatically replace instances that get interrupted for price or capacity reasons.
 - Adjust the ECS service desired count up or down in response to load variations.
 - Use Dynamic Scaling to add and remove capacity for resources to maintain resource utilization at the specified target value.
 - Use Predictive Scaling to forecast your future load demands by analyzing your historical records for a metric. It also allows you to schedule scaling actions that proactively add and remove resource capacity to reflect the load forecast and control maximum capacity behavior. Only available for EC2 Auto Scaling groups.
 - You can suspend and resume any of your AWS Application Auto Scaling actions.
- **Amazon EC2 Auto Scaling**
 - Ensuring you have the correct number of EC2 instances available to handle your application load using Auto Scaling Groups.
 - An Auto Scaling group contains a collection of EC2 instances that share similar characteristics and are treated as a logical grouping for the purposes of instance scaling and management.
 - You specify the minimum, maximum, and desired number of instances in each Auto Scaling group.
 - Key Components



Groups	Your EC2 instances are organized into groups so that they are treated as a logical unit for scaling and management. When you create a group, you can specify its minimum, maximum, and desired number of EC2 instances.
Launch templates	Your group uses a launch template for its EC2 instances. When you create a launch template, you can specify information such as the AMI ID, instance type, key pair, security groups, and block device mapping for your instances.
Scaling options	How to scale your Auto Scaling groups.

- Scaling Options
 - Scale to maintain current instance levels at all times
 - Manual Scaling
 - Scale based on a schedule
 - Scale based on a demand
- The cooldown period is a configurable setting that helps ensure not to launch or terminate an additional instance before the previous scaling activities take effect.
 - EC2 Auto Scaling supports cooldown periods when using simple scaling policies but not when using target tracking policies, step scaling policies, or scheduled scaling.
- Amazon EC2 Auto Scaling marks an instance as unhealthy if the instance is in a state other than running, the system status is impaired, or Elastic Load Balancing reports that the instance failed the health checks.
- Termination of Instances
 - When you configure automatic scale-in, you must decide which instances should terminate first and set up a termination policy. You can also use instance protection to prevent specific instances from being terminated during automatic scale-in.
 - Default Termination Policy
 - Custom Termination Policies
 - OldestInstance - Terminate the oldest instance in the group.
 - NewestInstance - Terminate the newest instance in the group.
 - OldestLaunchTemplate - Terminate instances that have the oldest launch template.
 - ClosestToNextInstanceHour - Terminate instances that are closest to the next billing hour.

You can create launch templates that specify instance configuration information when you launch EC2 instances and allows you to have multiple versions of a template.

A launch template is an instance configuration template that an Auto Scaling group uses to launch EC2 instances, and you specify information for the instances.



- You can specify your launch template with multiple Auto Scaling groups.
- You can only specify one launch template for an Auto Scaling group at a time, and you can't modify a launch template after you've created it.
- **Monitoring**
 - Health checks - identifies any instances that are unhealthy
 - Amazon EC2 status checks (default)
 - Elastic Load Balancing health checks
 - Custom health checks.
 - Autoscaling does not perform health checks on instances in the standby state. Standby state can be used for performing updates/changes/troubleshooting without health checks being performed or replacement instances being launched.
 - CloudWatch metrics - enables you to retrieve statistics about Auto Scaling-published data points as an ordered set of time-series data, known as metrics. You can use these metrics to verify that your system is performing as expected.
 - Amazon EventBridge - Auto Scaling can submit events to Amazon EventBridge when your Auto Scaling groups launch or terminate instances or when a lifecycle action occurs.
 - SNS notifications - Auto Scaling can send Amazon SNS notifications when your Auto Scaling groups launch or terminate instances.
 - CloudTrail logs - enables you to keep track of the calls made to the Auto Scaling API by or on behalf of your AWS account and stores the information in log files in an S3 bucket that you specify.



AWS CloudFormation

- A service that gives developers and businesses an easy way to create a collection of related AWS resources and provision them in an orderly and predictable fashion.

Features

- CloudFormation allows you to model your entire infrastructure in a text file called a **template**. You can use JSON or YAML to describe what AWS resources you want to create and configure. If you want to design visually, you can use *AWS CloudFormation Designer*.
- CloudFormation automates the provisioning and updating of your infrastructure in a safe and controlled manner. You can use **Rollback Triggers** to specify the CloudWatch alarm that CloudFormation should monitor during the stack creation and update process. If any of the alarms are breached, CloudFormation rolls back the entire stack operation to a previously deployed state.
- CloudFormation enables you to build custom extensions to your stack template using AWS Lambda.

Concepts

- **Templates**
 - A JSON or YAML formatted text file.
 - CloudFormation uses these templates as blueprints for building your AWS resources.
- **Stacks**
 - Manage related resources as a single unit.
 - All the resources in a stack are defined by the stack's CloudFormation template.
- **Change Sets**
 - Before updating your stack and making changes to your resources, you can generate a change set, which is a summary of your proposed changes.
 - Change sets allow you to see how your changes might impact your running resources, especially critical resources, before implementing them.
- With AWS CloudFormation and AWS CodePipeline, you can use continuous delivery to automatically build and test changes to your CloudFormation templates before promoting them to production stacks.
- *CloudFormation artifacts* can include a stack template file, a template configuration file, or both. AWS CodePipeline uses these artifacts to work with CloudFormation stacks and change sets.
 - **Stack Template File** - defines the resources that CloudFormation provisions and configures. You can use YAML or JSON-formatted templates.
 - **Template Configuration File** - a JSON-formatted text file that can specify template parameter values, a stack policy, and tags. Use these configuration files to specify parameter values or a stack policy for a stack.



Stacks

- If a resource cannot be created, CloudFormation rolls the stack back and automatically deletes any resources that were created. If a resource cannot be deleted, any remaining resources are retained until the stack can be successfully deleted.
- Stack update methods
 - Direct update
 - Creating and executing change sets
- **Drift detection** enables you to detect whether a stack's actual configuration differs, or has drifted, from its expected configuration. Use CloudFormation to detect drift on an entire stack or on individual resources within the stack.
 - A resource is considered to have drifted, if any, if its actual property values differ from the expected property values.
 - A stack is considered to have drifted if one or more of its resources have drifted.
- To share information between stacks, export a stack's output values. Other stacks that are in the same AWS account and region can import the exported values.
- You can nest stacks.

Templates

- Templates include several major sections. The Resources section is the only required section.
- **CloudFormation Designer** is a graphic tool for creating, viewing, and modifying CloudFormation templates. You can diagram your template resources using a drag-and-drop interface and then edit their details using the integrated JSON and YAML editor.
- Custom resources enable you to write custom provisioning logic in templates that CloudFormation runs anytime you create, update (if you change the custom resource), or delete stacks.
- Template macros enable you to perform custom processing on templates, from simple actions like find-and-replace operations to extensive transformations of entire templates.

StackSets

- CloudFormation StackSets allow you to roll out CloudFormation stacks over multiple AWS accounts and in multiple Regions with just a couple of clicks. StackSets is commonly used together with AWS Organizations to centrally deploy and manage services in different accounts.
- Administrator and target accounts - An *administrator account* is the AWS account in which you create stack sets. A stack set is managed by signing in to the AWS administrator account in which it was created. A *target account* is an account into which you create, update, or delete one or more stacks in your stack set.
- In addition to the organization's management account, you can delegate other administrator accounts in your AWS Organization that can create and manage stack sets with service-managed permissions for the organization.



- Stack sets - A *stack set* lets you create stacks in AWS accounts across regions by using a single CloudFormation template. All the resources included in each stack are defined by the stack set's CloudFormation template. A stack set is a regional resource.
- Stack instances - A *stack instance* is a reference to a stack in a target account within a region. A stack instance can exist without a stack and can be associated with only one stack set.
- Stack set operations - Create stack set, update stack set, delete stacks, and delete stack set.
- Tags - You can add tags during stack set creation and update operations by specifying key and value pairs.
- Drift detection identifies unmanaged changes or changes made to stacks outside of CloudFormation. When CloudFormation performs drift detection on a stack set, it performs drift detection on the stack associated with each stack instance in the stack set. If the current state of a resource varies from its expected state, that resource is considered to have drifted.
- If one or more resources in a stack have drifted, then the stack itself is considered to have drifted, and the stack instances that the stack is associated with are considered to have drifted as well.
- If one or more stack instances in a stack set have drifted, the stack set itself is considered to have drifted.



AWS CloudTrail

- Actions taken by a user, role, or an AWS service in the AWS Management Console, AWS Command Line Interface, and AWS SDKs and APIs are recorded as events.
- CloudTrail is enabled on your AWS account when you create it.
- CloudTrail focuses on auditing API activity.

Trails

- Create a CloudTrail trail to archive, analyze, and respond to changes in your AWS resources.
- Types
 - A trail that applies to all regions - CloudTrail records events in each region and delivers the CloudTrail event log files to an S3 bucket that you specify. This is the default option when you create a trail in the CloudTrail console.
 - A trail that applies to one region - CloudTrail records the events in the region that you specify only. This is the default option when you create a trail using the AWS CLI or the CloudTrail API.
- You can create an organization trail that will log all events for all AWS accounts in an organization created by AWS Organizations. Organization trails must be created in the management account.
- By default, CloudTrail event log files are encrypted using Amazon S3 server-side encryption. You can also choose to encrypt your log files with an AWS Key Management Service key.
- You can store your log files in your S3 bucket for as long as you want and also define S3 lifecycle rules to archive or delete log files automatically. If you want notifications about log file delivery and validation, you can set up Amazon SNS notifications. CloudTrail publishes log files about every five minutes.

Events

- The record of activity in an AWS account. This activity can be an action taken by a user, role, or service that is monitorable by CloudTrail.
- Types
 - Management events
 - Logged by default
 - Management events provide insight into management operations performed on resources in your AWS account, also known as control plane operations.
 - Data events
 - Not logged by default
 - Data events provide insight into the resource operations performed on or in a resource, also known as data plane operations.
 - Data events are often high-volume activities.



AWS Config

- A fully managed service that provides you with an AWS resource inventory, configuration history, and configuration change notifications to enable security and governance.

Features

- Multi-account, multi-region data aggregation gives you an enterprise-wide view of your **Config rule** compliance status, and you can associate your AWS organization to quickly add your accounts.
- Provides you with pre-built rules to evaluate your AWS resource configurations and configuration changes, or create your own custom rules in AWS Lambda that define your internal best practices and guidelines for resource configurations.
- **Config records** details of changes to your AWS resources to provide you with a configuration history and automatically deliver it to an S3 bucket you specify.
- Receive a notification whenever a resource is created, modified, or deleted.
- Config enables you to record software configuration changes within your EC2 instances and servers running on-premises, as well as servers and Virtual Machines in environments provided by other cloud providers. You gain visibility into:
 - operating system configurations
 - system-level updates
 - installed applications
 - network configuration
- Config can provide you with a **configuration snapshot** - a point-in-time capture of all your resources and their configurations.

Concepts

Configuration History

- A collection of the configuration items for a given resource over any time period, containing information such as when the resource was first created, how the resource has been configured over the last month, etc.
- Config automatically delivers a configuration history file for each resource type that is being recorded to an S3 bucket that you specify.
- A configuration history file is sent every six hours for each resource type that Config records.

Configuration item

- A record of the configuration of a resource in your AWS account. Config creates a configuration item whenever it detects a change to a resource type that it is recording.
- The components of a configuration item include metadata, attributes, relationships, current configuration, and related events.

Configuration Recorder

- Stores the configurations of the supported resources in your account as configuration items.



- By default, the configuration recorder records all supported resources in the region where Config is running. You can create a customized configuration recorder that records only the resource types that you specify.
- You can also have Config record supported types of *global resources* which are IAM users, groups, roles, and customer-managed policies.

Configuration Item

- The configuration of a resource at a given point in time. A CI consists of 5 sections:
 - Basic information about the resource that is common across different resource types.
 - Configuration data specific to the resource.
 - Map of relationships with other resources.
 - CloudTrail event IDs that are related to this state.
 - Metadata that helps you identify information about the CI, such as the version of this CI, and when this CI was captured.

Resource Relationship

- Config discovers AWS resources in your account and then creates a map of relationships between AWS resources.

Config rule

- Represents your desired configuration settings for specific AWS resources or for an entire AWS account.
- Provides customizable, predefined rules. If a resource violates a rule, Config flags the resource and the rule as noncompliant and notifies you through Amazon SNS.
- Evaluates your resources either **in response to configuration changes** or **periodically**.
- Multi-Account Multi-Region Data Aggregation
 - An aggregator collects configuration and compliance data from the following:
 - Multiple accounts and multiple regions.
 - Single account and multiple regions.
 - An organization in AWS Organizations and all the accounts in that organization.

Monitoring

- Use Amazon SNS to send you notifications every time a supported AWS resource is created, updated, or otherwise modified as a result of user API activity.
- Use Amazon EventBridge to detect and react to changes in the status of AWS Config events.
- Use AWS CloudTrail to capture API calls to Config.



AWS Health

- Provides ongoing visibility into the state of your AWS resources, services, and accounts.
- The service delivers alerts and notifications triggered by changes in the health of AWS resources.
- The **Personal Health Dashboard**, powered by the AWS Health API, is available to all customers. The dashboard requires no setup, and it is ready to use for authenticated AWS users. The Personal Health Dashboard organizes issues in three groups:
 - Open issues - restricted to issues whose start time is within the last seven days.
 - Scheduled changes - contain items that are ongoing or upcoming.
 - Other notifications - restricted to issues whose start time is within the last seven days.
- You can centrally aggregate your AWS Health events from all accounts in your AWS Organization. The AWS Health Organizational View provides centralized, and real-time access to all AWS Health events posted to individual accounts in your organization, including operational issues, scheduled maintenance, and account notifications.



AWS Systems Manager

- Allows you to centralize operational data from multiple AWS services and automate tasks across your AWS resources.

Features

- Create logical groups of resources such as applications, different layers of an application stack, or production versus development environments.
- You can select a resource group and view its recent API activity, resource configuration changes, related notifications, operational alerts, software inventory, and patch compliance status.
- Collects information about your instances and the software installed on them.
- Allows you to safely automate common and repetitive IT operations and management tasks across AWS resources.
- Provides a browser-based interactive shell and CLI for managing Windows and Linux EC2 instances without the need to open inbound ports, manage SSH keys, or use bastion hosts. Administrators can grant and revoke access to instances through a central location by using IAM policies.
- Helps ensure that your software is up-to-date and meets your compliance policies.
- Lets you schedule windows of time to run administrative and maintenance tasks across your instances.

SSM Agent is the tool that processes Systems Manager requests and configures your machine as specified in the request. SSM Agent must be installed on each instance you want to use with Systems Manager. On newer AMIs and instance types, SSM Agent is installed by default. On older versions, you must install it manually.

Capabilities

- **Automation**
 - Allows you to safely automate common and repetitive IT operations and management tasks across AWS resources
 - A **step** is defined as an initiated action performed in the Automation execution on a per-target basis. You can execute the entire Systems Manager automation document in one action or choose to execute one step at a time.
 - Concepts
 - **Automation document** - defines the Automation workflow.
 - **Automation action** - the Automation workflow includes one or more steps. Each step is associated with a particular action or plugin. The action determines the inputs, behavior, and outputs of the step.
 - **Automation queue** - if you attempt to run more than 25 Automations simultaneously, Systems Manager adds the additional executions to a queue and displays a status of *Pending*. When an Automation reaches a terminal state, the first execution in the queue starts.



- You can schedule Systems Manager automation document execution.
- **Resource Groups**
 - A collection of AWS resources that are all in the same AWS region and that match criteria provided in a query.
 - Use Systems Manager tools such as *Automation* to simplify management tasks on your groups of resources. You can also use groups as the basis for viewing monitoring and configuration *insights* in Systems Manager.
- **Built-in Insights**
 - Show detailed information about a single, selected resource group.
 - Includes recent API calls through CloudTrail, recent configuration changes through Config, Instance software inventory listings, instance patch compliance views, and instance configuration compliance views.
- **Systems Manager Activation**
 - Enable hybrid and cross-cloud management. You can register any server, whether physical or virtual, to be managed by Systems Manager.
- **Inventory Manager**
 - Automates the process of collecting software inventory from managed instances.
 - You specify the type of metadata to collect, the instances from where the metadata should be collected, and a schedule for metadata collection.
- **Configuration Compliance**
 - Scans your fleet of managed instances for patch compliance and configuration inconsistencies.
 - View compliance history and change tracking for Patch Manager patching data and State Manager associations by using AWS Config.
 - Customize Systems Manager Compliance to create your own compliance types.
- **Run Command**
 - Remotely and securely manage the configuration of your managed instances at scale.
 - **Managed Instances** - any EC2 instance or on-premises server or virtual machine in your hybrid environment that is configured for Systems Manager.
- **Session Manager**
 - Manage your EC2 instances through an interactive one-click browser-based shell or through the AWS CLI.
 - Makes it easy to comply with corporate policies that require controlled access to instances, strict security practices, and fully auditable logs with instance access details while still providing end users with simple one-click cross-platform access to your Amazon EC2 instances.
 - You can use AWS Systems Manager Session Manager to tunnel SSH (Secure Shell) and SCP (Secure Copy) traffic between a client and a server.
- **Distributor**
 - Lets you package your own software or find AWS-provided agent software packages to install on Systems Manager managed instances.
 - After you create a package in Distributor, which creates a Systems Manager document, you can install the package in one of the following ways.



- One time by using Systems Manager Run Command.
- On a schedule by using Systems Manager State Manager.
- **Patch Manager**
 - Automate the process of patching your managed instances.
 - Enables you to scan instances for missing patches and apply missing patches individually or to large groups of instances by using EC2 instance tags.
 - For security patches, Patch Manager uses *patch baselines* that include rules for auto-approving patches within days of their release, as well as a list of approved and rejected patches.
 - You can use AWS Systems Manager Patch Manager to select and apply Microsoft application patches automatically across your Amazon EC2 or on-premises instances.
 - AWS Systems Manager Patch Manager includes common vulnerability identifiers (CVE ID). CVE IDs can help you identify security vulnerabilities within your fleet and recommend patches.
 - You can configure actions to be performed on a managed instance before and after installing patches.
- **Maintenance Window**
 - Set up recurring schedules for managed instances to execute administrative tasks like installing patches and updates without interrupting business-critical operations.
 - Supports running four types of tasks:
 - Systems Manager Run Command commands
 - Systems Manager Automation workflows
 - AWS Lambda functions
 - AWS Step Functions tasks
- **Systems Manager Document (SSM)**
 - Defines the actions that Systems Manager performs.
 - Types of SSM Documents

Type	Use with	Details
Command document	Run Command, State Manager	Run Command uses command documents to execute commands. State Manager uses command documents to apply a configuration. These actions can be run on one or more targets at any point during the lifecycle of an instance.
Policy document	State Manager	Policy documents enforce a policy on your targets. If the policy document is removed, the policy action no longer happens.
Automation document	Automation	Use automation documents when performing common maintenance and deployment tasks such as creating or updating an AMI.



Package document	Distributor	In Distributor, a package is represented by a Systems Manager document. A package document includes attached ZIP archive files that contain software or assets to install on managed instances. Creating a package in Distributor creates the package document.
------------------	-------------	---

- Can be in JSON or YAML.
- You can create and save different versions of documents. You can then specify a default version for each document.
- If you want to customize the steps and actions in a document, you can create your own.
- You can tag your documents to help you quickly identify one or more documents based on the tags you've assigned to them.

State Manager

- A service that automates the process of keeping your EC2 and hybrid infrastructure in a state that you define.
- A *State Manager association* is a configuration that is assigned to your managed instances. The configuration defines the state that you want to maintain on your instances. The association also specifies actions to take when applying the configuration.

Parameter Store

- Provides secure, hierarchical storage for configuration data and secrets management.
- You can store values as plain text or encrypted data with *SecureString*.
- Parameters work with Systems Manager capabilities such as Run Command, State Manager, and Automation.

OpsCenter

- OpsCenter helps you view, investigate, and resolve operational issues related to your environment from a central location.
- OpsCenter complements existing case management systems by enabling integrations via Amazon Simple Notification Service (SNS) and public AWS SDKs. By aggregating information from AWS Config, AWS CloudTrail logs, resource descriptions, and Amazon EventBridge, OpsCenter helps you reduce the mean time to resolution (MTTR) of incidents, alarms, and operational tasks.



AWS Trusted Advisor

- Trusted Advisor analyzes your AWS environment and provides best practice recommendations in five categories:
 - Cost Optimization
 - Performance
 - Security
 - Fault Tolerance
 - Service Limits
- Access to the seven core Trusted Advisor checks are available to all AWS users.
- Access to the full set of Trusted Advisor checks is available to Business and Enterprise Support plans.

AWS Trusted Advisor is an indispensable tool for ensuring your account is as cost-effective as possible. The Cost Optimization feature under AWS Trusted Advisor makes use of the well-architected best practices for cost-efficiency, so you have a centralized monitoring solution that continuously reviews your account for any items that can incur unnecessary expenses.

How AWS Trusted Advisor does this is by having multiple checks that scan for underutilized (e.g. idle instances) and unoptimized (e.g. oversized instances) resources that are running in your account. The number of Trusted Advisor checks that will be available to you will depend on your support plan. Nevertheless, you should often review your AWS Trusted Advisor to ensure all your resources are well-utilized and right-sized.



AWS Service Catalog

- Allows you to create, manage, and distribute catalogs of approved products to end-users, who can then access the products they need in a personalized portal.
- Administrators can control which users have access to each product to enforce compliance with organizational business policies. Administrators can also set up adopted roles, so that end users only require IAM access to AWS Service Catalog in order to deploy approved resources.
- This is a regional service.

Features

- Standardization of assets
- Self-service discovery and launch
- Fine-grain access control
- Extensibility and version control

Concepts

- Users
 - Catalog administrators – Manage a catalog of products, organizing them into portfolios and granting access to end users. Catalog administrators prepare AWS CloudFormation templates, configure constraints, and manage IAM roles that are assigned to products to provide for advanced resource management.
 - End users – Use AWS Service Catalog to launch products to which they have been granted access.
- Products
 - Can comprise one or more AWS resources, such as EC2 instances, storage volumes, databases, monitoring configurations, and networking components, or packaged AWS Marketplace products.
 - You create your products by importing AWS CloudFormation templates. The templates define the AWS resources required for the product, the relationships between resources, and the parameters for launching the product to configure security groups, create key pairs, and perform other customizations.
 - You can see the products that you are using and their health state in the AWS Service Catalog console.
- Portfolio
 - A collection of products, together with configuration information. Portfolios help manage product configuration and determine who can use specific products and how they can use them.
 - When you add a new version of a product to a portfolio, that version is automatically available to all current users of that portfolio.



- You can also share your portfolios with other AWS accounts and allow the administrator of those accounts to distribute your portfolios with additional constraints.
- When you add tags to your portfolio, the tags are applied to all instances of resources provisioned from products in the portfolio.
- Versioning
 - Service Catalog allows you to manage multiple versions of the products in your catalog.
 - A version can have one of three statuses:
 - Active - An active version appears in the version list and allows users to launch it.
 - Inactive - An inactive version is hidden from the version list. Existing provisioned products launched from this version will not be affected.
 - Deleted - If a version is deleted, it is removed from the version list. Deleting a version can't be undone.
- Access control
 - You apply AWS IAM permissions to control who can view and modify your products and portfolios.
 - By assigning an IAM role to each product, you can avoid giving users permission to perform unapproved operations and enable them to provision resources using the catalog.
- Constraints
 - You use constraints to apply limits to products for governance or cost control.
 - Types of constraints:
 - Template constraints restrict the configuration parameters that are available for the user when launching the product. Template constraints allow you to reuse generic AWS CloudFormation templates for products and apply restrictions to the templates on a per-product or per-portfolio basis.
 - Launch constraints allow you to specify a role for a product in a portfolio. This role is used to provision the resources at launch, so you can restrict user permissions without impacting users' ability to provision products from the catalog.
 - Notification constraints specify an Amazon SNS topic to receive notifications about stack events.
 - Tag update constraints allow administrators to allow or disallow end users to update tags on resources associated with an AWS Service Catalog provisioned product.
- Stack
 - Every AWS Service Catalog product is launched as an AWS CloudFormation stack.
 - You can use CloudFormation StackSets to launch Service Catalog products across multiple regions and accounts. You can specify the order in which products deploy sequentially within regions. Across accounts, products are deployed in parallel.



Security

- Service Catalog uses Amazon S3 buckets and Amazon DynamoDB databases that are encrypted at rest using Amazon-managed keys.
- Service Catalog uses TLS and client-side encryption of information in transit between the caller and AWS.
- Service Catalog integrates with AWS CloudTrail and Amazon SNS.

Pricing

- The AWS Service Catalog free tier includes 1,000 API calls per month.
- You are charged based on the number of API calls made to the Service Catalog beyond the free tier.

References:

<https://aws.amazon.com/servicecatalog/>

<https://docs.aws.amazon.com/servicecatalog/latest/adminguide/introduction.html>

<https://docs.aws.amazon.com/servicecatalog/latest/userguide/end-user-console.html>

<https://aws.amazon.com/servicecatalog/pricing/>

<https://aws.amazon.com/servicecatalog/faqs/>



AWS Analytics Services

Amazon OpenSearch Service

- Amazon OpenSearch lets you search, analyze, and visualize your data **in real-time**. This service manages the capacity, scaling, patching, and administration of your OpenSearch clusters for you while still giving you direct access to the OpenSearch APIs.
- The service offers open-source Elasticsearch APIs, managed Kibana, and integrations with Logstash and other AWS Services. This combination is often coined as the **ELK Stack**.
- **Concepts**
 - An Amazon OpenSearch **domain** is synonymous with an OpenSearch cluster. Domains are clusters with the settings, instance types, instance counts, and storage resources that you specify.
 - You can create multiple OpenSearch indices within the same domain. OpenSearch and Elasticsearch automatically distributes the indices and any associated replicas between the instances allocated to the domain.
 - Amazon OpenSearch uses a **blue/green deployment process** when updating domains. Blue/green typically refers to the practice of running two production environments, one live and one idle, and switching the two as you make software changes.
- **Data Ingestion**
 - Easily ingest structured and unstructured data into your Amazon OpenSearch service domain with **Logstash**, an open-source data pipeline that helps you process logs and other event data.
 - You can also ingest data into your Amazon OpenSearch domain using Amazon Kinesis Firehose, AWS IoT, or Amazon CloudWatch Logs.
 - You can get faster and better insights into your data using **Kibana**, an open-source analytics and visualization platform. Kibana is automatically deployed with your Amazon OpenSearch Service domain.
 - You can load streaming data from the following sources using AWS Lambda event handlers:
 - Amazon S3
 - Amazon Kinesis Data Streams and Data Firehose
 - Amazon DynamoDB
 - Amazon CloudWatch
 - AWS IoT
 - Amazon OpenSearch exposes three Elasticsearch logs through CloudWatch Logs:
 - error logs
 - search slow logs - These logs help fine-tune the performance of any kind of search operation on Elasticsearch.
 - index slow logs - These logs provide insights into the indexing process and can be used to fine-tune the index setup.
 - Kibana and Logstash



- Kibana is a popular open-source visualization tool designed to work with OpenSearch.
- For domains that use OpenSearch, the URL is *domain-endpoint/_dashboards/*.
- For domains using legacy Elasticsearch, the URL is *domain-endpoint/_plugin/kibana*.
- You can configure your own Kibana instance aside from using the default provided Kibana.
- Amazon OpenSearch uses **Amazon Cognito** to offer username and password protection for Kibana. (Optional feature)
- Logstash provides a convenient way to use the bulk API to upload data into your Amazon OpenSearch domain with the S3 plugin. The service also supports all other standard Logstash input plugins that are provided by OpenSearch.
- Amazon OpenSearch also supports two Logstash output plugins:
 - standard Logstash output plugins
 - *logstash-output-opensearch* plugin , which signs and exports Logstash events to Amazon OpenSearch.



Amazon Kinesis

- Makes it easy to collect, process, and analyze real-time streaming data.
- Kinesis can ingest real-time data such as video, audio, application logs, website clickstreams, and IoT telemetry data for machine learning, analytics, and other applications.

Kinesis Data Stream

- A massively scalable, highly durable data ingestion and processing service optimized for streaming data. You can configure hundreds of thousands of data producers to continuously put data into a Kinesis data stream.
- **Concepts**
 - **Data Producer** - An application that typically emits data records as they are generated to a Kinesis data stream. Data producers assign partition keys to records. Partition keys ultimately determine which shard ingests the data record for a data stream.
 - **Data Consumer** - A distributed Kinesis application or AWS service retrieving data from all shards in a stream as it is generated. Most data consumers are retrieving the most recent data in a shard, enabling real-time analytics or handling of data.
 - **Data Stream** - A logical grouping of shards. There are no bounds on the number of shards within a data stream. A data stream will retain data for **24 hours or up to 7 days** when extended retention is enabled.
 - **Shard** - The base throughput unit of a Kinesis data stream.
 - A shard is an append-only log and a unit of streaming capability. A shard contains an ordered sequence of records ordered by arrival time.
 - Add or remove shards from your stream dynamically as your data throughput changes.
 - One shard can ingest up to 1000 data records per second, or 1MB/sec. Add more shards to increase your ingestion capability.
 - When consumers use **enhanced fan-out**, one shard provides 1MB/sec data input and 2MB/sec data output for each data consumer registered to use enhanced fan-out.
 - When consumers do **not** use **enhanced fan-out**, a shard provides 1MB/sec of input and 2MB/sec of data output, and this output is shared with any consumer not using enhanced fan-out.
 - **Data Record**
 - A record is the unit of data stored in a Kinesis stream. A record is composed of a sequence number, partition key, and data blob.
 - A data blob is the data of interest your data producer adds to a stream. The maximum size of a data blob is 1 MB.
 - **Partition Key**
 - A partition key is typically a meaningful identifier, such as a user ID or timestamp. It is specified by your data producer while putting data into a Kinesis data stream, and useful



for consumers as they can use the partition key to replay or build a history associated with the partition key.

- The partition key is also used to segregate and route data records to different shards of a stream.
- **Sequence Number**
 - A sequence number is a unique identifier for each data record. Sequence number is assigned by Kinesis Data Streams when a data producer calls *PutRecord* or *PutRecords* API to add data to a Kinesis data stream.

Data Firehose

- The easiest way to load streaming data into data stores and analytics tools.
- It is a fully managed service that automatically scales to match the throughput of your data.
- It can also batch, compress, and encrypt the data before loading it.
- **Features**
 - It can capture, transform, and load streaming data into S3, Redshift, OpenSearch Service, and Splunk, enabling near real-time analytics with existing business intelligence tools and dashboards being used today.
 - Once launched, your delivery streams automatically scale up and down to handle gigabytes per second or more of input data rate and maintain data latency at levels you specify for the stream.
 - Data Firehose can convert the format of incoming data from JSON to Parquet or ORC formats before storing the data in S3.
 - You can configure Data Firehose to prepare your streaming data before it is loaded to data stores. Data Firehose provides pre-built Lambda blueprints for converting common data sources, such as Apache logs and system logs, to JSON and CSV formats. You can use these pre-built blueprints without any change, customize them further, or write your own custom functions.
- **Concepts**
 - **Data Firehose Delivery Stream** - The underlying entity of Data Firehose. You use Data Firehose by creating a Data Firehose delivery stream and then sending data to it.
 - **Record** - The data of interest that your data producer sends to a Data Firehose delivery stream. A record can be as large as 1,000 KB.
 - **Data Producer** - Producers send records to Data Firehose delivery streams.
 - **Buffer Size and Buffer Interval** - Data Firehose buffers incoming streaming data to a certain size or for a certain period of time before delivering it to destinations. Buffer Size is in MBs, and Buffer Interval is in seconds.
- **Stream Sources**
 - You can send data to your Data Firehose Delivery stream using different types of sources:
 - a Kinesis data stream,
 - the Kinesis Agent,
 - or the Data Firehose API using the AWS SDK.



- You can also use CloudWatch Logs, Amazon EventBridge, or AWS IoT as your data source.
- Some AWS services can only send messages and events to a Data Firehose delivery stream that is in the same Region.
- **Data Delivery and Transformation**
 - Data Firehose can invoke your Lambda function to transform incoming source data and deliver the transformed data to destinations.
 - Data Firehose buffers incoming data up to 3 MB by default.
 - If your Lambda function invocation fails because of a network timeout or because you've reached the Lambda invocation limit, Data Firehose retries the invocation three times by default.
 - Data Firehose can convert the format of your input data from JSON to Apache Parquet or Apache ORC before storing the data in S3. Parquet and ORC are columnar data formats that save space and enable faster queries compared to row-oriented formats like JSON.
 - Data delivery format:
 - **For data delivery to S3**, Data Firehose concatenates multiple incoming records based on the buffering configuration of your delivery stream. It then delivers the records to S3 as an S3 object.
 - **For data delivery to Redshift**, Data Firehose first delivers incoming data to your S3 bucket in the format described earlier. Data Firehose then issues a Redshift *COPY* command to load the data from your S3 bucket to your Redshift cluster.
 - **For data delivery to OpenSearch service and OpenSearch Serverless**, Data Firehose buffers incoming records based on the buffering configuration of your delivery stream. an OpenSearch Service or OpenSearch Serverless bulk request to index multiple records to your OpenSearch Service cluster.
 - **For data delivery to Splunk**, Data Firehose concatenates the bytes that you send.

Managed Service for Apache Flink

- Analyze streaming data, gain actionable insights, and respond to your business and customer needs in real-time. You can quickly build SQL queries and Java applications using built-in templates and operators for common processing functions to organize, transform, aggregate, and analyze data at any scale.
- **General Features**
 - Managed Service for Apache Flink is **serverless** and takes care of everything required to continuously run your application.
 - Managed Service for Apache Flink elastically scales applications to keep up with any volume of data in the incoming data stream.
 - Managed Service for Apache Flink delivers sub-second processing latencies so you can generate real-time alerts, dashboards, and actionable insights.
- An **application** is the primary resource in Managed Service for Apache Flink. Managed Service for Apache Flink applications continuously read and process streaming data in real-time.



- You write application code using SQL to process the incoming streaming data and produce output. Then, Managed Service for Apache Flink writes the output to a configured destination.
- You can also process and analyze streaming data using Java.
- **Components**
 - Input is the streaming source for your application. In the input configuration, you map the streaming source to an in-application data stream(s).
 - **Application code** is a series of SQL statements that process input and produce output.
 - You can create one or more in-application streams to store the **output**. You can then optionally configure an application output to persist data from specific in-application streams to an external destination.
- An **in-application data stream** is an entity that continuously stores data in your application for you to perform processing.



Amazon Athena

- A massively scalable, highly durable data ingestion and processing service optimized for streaming data. You can configure hundreds of thousands of data producers to continuously put data into a Kinesis data stream.
- An interactive query service that makes it easy to analyze data directly in Amazon S3 and other data sources using SQL.

Features

- Athena is serverless
- Has a built-in query editor.
- Uses Presto, an open source, distributed SQL query engine optimized for low latency, ad hoc analysis of data.
- Athena supports a wide variety of data formats such as CSV, JSON, ORC, Avro, or Parquet.
- Athena automatically executes queries in parallel so that you get query results in seconds, even on large datasets.
- Athena uses Amazon S3 as its underlying data store, making your data highly available and durable.
- Athena integrates with Amazon QuickSight for easy data visualization.
- Athena integrates out-of-the-box with AWS Glue.
- Athena uses a managed Data Catalog to store information and schemas about the databases and tables that you create for your data stored in S3.

Queries

- You can query geospatial data.
- You can query different kinds of logs as your datasets.
- Athena stores query results in S3.
- Athena retains query history for 45 days.
- Athena does not support user-defined functions, INSERT INTO statements, and stored procedures.
- Athena supports both simple data types, such as INTEGER, DOUBLE, VARCHAR and complex data types, such as MAPS, ARRAY, and STRUCT.
- Athena supports querying data in Amazon S3 Requester Pays buckets.

Athena Federated Queries

- Allows you to query data sources other than S3 buckets using a data connector.
- A data connector is implemented in a Lambda function that uses Athena Query Federation SDK.
- There are pre-built connectors available for some popular data sources, such as:
- MySQL, PostgreSQL, Oracle, SQL Server databases



- Amazon DynamoDB
- Amazon Managed Streaming for Apache Kafka (MSK)
- Amazon RedShift
- Amazon OpenSearch
- Amazon CloudWatch Logs and CloudWatch metrics
- Amazon DocumentDB
- Apache Kafka
- You can write your own data connector using the Athena Query Federation SDK if your data source is not natively supported by Athena.
- You may also customize the pre-built connectors to fit your use case.

Optimizing query performance

- Data partitioning. For instance, partitioning data based on column values such as date, country, and region makes it possible to limit the amount of data that needs to be scanned by a query.
- Converting data format into columnar formats such as Parquet and ORC
- Compressing files
- Making files splittable. Athena can read a splittable file in parallel; thus, the time it takes for a query to complete is faster.
- AVRO, Parquet, and Orc are splittable files regardless of the compression codec used
- Only text files (TSV, CSV, JSON, and custom SerDes for text) compressed with BZIP2 and LZ0 are splittable.

Cost controls

- You can create to isolate queries for teams, applications, or different workloads and enforce cost controls.
- There are two types of cost controls available in a workgroup:
- Per-query limit – specifies a threshold for the total amount of data scanned per query. Any query running in a workgroup is canceled once it exceeds the specified limit. Only one per-query limit can be created in a workgroup.
- Per-workgroup limit – this limits the total amount of data scanned by all queries running within a specific time frame. You can establish multiple limits based on hourly or daily data scan totals for queries within the workgroup.



AWS Developer Tools

AWS CodeBuild

- A fully managed **continuous integration service** that compiles source code, run tests, and produces software packages that are ready to deploy.
- **Concepts**
 - A **build project** defines how CodeBuild will run a build. It includes information such as where to get the source code, which builds environment to use, the build commands to run, and where to store the build output.
 - A **build environment** is the combination of operating system, programming language runtime, and tools used by CodeBuild to run a build.
 - The **build specification** is a YAML file that lets you choose the commands to run at each phase of the build and other settings. Without a build spec, CodeBuild cannot successfully convert your build input into build output or locate the build output artifact in the build environment to upload to your output bucket.
 - If you include a build spec as part of the source code, by default, the build spec file must be named `buildspec.yml` and placed in the root of your source directory.
 - A collection of input files is called **build input artifacts or build input**, and a deployable version of a source code is called **build output artifact or build output**.
- **Features**
 - AWS CodeBuild runs your builds in preconfigured build environments that contain the operating system, programming language runtime, and build tools (such as Apache Maven, Gradle, and npm) required to complete the task. You just specify your source code's location and select settings for your build, such as the build environment to use and the build commands to run during a build.
 - AWS CodeBuild builds your code and stores the artifacts in an Amazon S3 bucket, or you can use a build command to upload them to an artifact repository.
 - AWS CodeBuild provides build environments for
 - Java
 - Python
 - Node.js
 - Ruby
 - Go
 - Android
 - .NET Core for Linux
 - Docker
 - You can define the specific commands that you want AWS CodeBuild to perform, such as installing build tool packages, running unit tests, and packaging your code.



- You can integrate CodeBuild into existing CI/CD workflows using its source integrations, build commands, or Jenkins integration.
- CodeBuild can connect to S3, GitHub, and GitHub Enterprise and Bitbucket to pull source code for builds.
- CodeBuild allows you to use Docker images stored in another AWS account as your build environment by granting resource-level permissions.
- It now allows you to access Docker images from any private registry as the build environment. Previously, you could only use Docker images from public DockerHub or Amazon ECR in CodeBuild.
- You can access your past build results through the console, CloudWatch, or the API. The results include outcome (success or failure), build duration, output artifact location, and log location.
- You can automate your release process by using **AWS CodePipeline** to test your code and run your builds with CodeBuild.
- **Steps in a Build Process**
 - CodeBuild will create a temporary compute container of the class defined in the build project
 - CodeBuild loads it with the specified runtime environment
 - CodeBuild downloads the source code
 - CodeBuild executes the commands configured in the project
 - CodeBuild uploads the generated artifact to an S3 bucket
 - Then it destroys the compute container
- Build Duration is calculated in minutes, from the time you submit your build until your build is terminated, rounded up to the nearest minute.
- You can save time when your project builds by using a cache. A build project can use one of two types of caching:
 - Amazon S3 - stores the cache in an Amazon S3 bucket that is available across multiple build hosts. This is a good option for small intermediate-build artifacts that are more expensive to build than to download. Not the best option for large build artifacts because they can take a long time to transfer over your network, which can affect build performance.
 - Local - stores a cache locally on a build host that is available to that build host only. This is a good option for large intermediate build artifacts because the cache is immediately available on the build host. Build performance is not impacted by network transfer time.
 - If you use a local cache, you must choose one or more of three cache modes:
 - source cache
 - Docker layer cache
 - custom cache



AWS CodeDeploy

- A fully managed deployment service that automates software deployments to a variety of compute services such as Amazon EC2, AWS Fargate, AWS Lambda, and your on-premises servers.
- **Concepts**
 - An Application is a name that uniquely identifies the application you want to deploy. CodeDeploy uses this name, which functions as a container, to ensure the correct combination of revision, deployment configuration, and deployment group are referenced during a deployment.
 - Compute platform is the platform on which CodeDeploy deploys an application (EC2, ECS, Lambda, On-premises servers).
 - Deployment configuration is a set of deployment rules and deployment success and failure conditions used by CodeDeploy during a deployment.
 - Deployment group contains individually tagged instances, Amazon EC2 instances in Amazon EC2 Auto Scaling groups, or both.
 1. In an Amazon ECS deployment, a deployment group specifies the Amazon ECS service, load balancer, optional test listener, and two target groups. It also specifies when to reroute traffic to the replacement task set and when to terminate the original task set and ECS application after a successful deployment.
 2. In an AWS Lambda deployment, a deployment group defines a set of CodeDeploy configurations for future deployments of an AWS Lambda function.
 3. In an EC2/On-Premises deployment, a deployment group is a set of individual instances targeted for deployment.
 - In an in-place deployment, the instances in the deployment group are updated with the latest application revision.
 - In a blue/green deployment, traffic is rerouted from one set of instances to another by deregistering the original instances from a load balancer and registering a replacement set of instances that typically has the latest application revision already installed.
 - A deployment goes through a set of predefined phases called deployment lifecycle events. A deployment lifecycle event gives you an opportunity to run code as part of the deployment.
 1. ApplicationStop
 2. DownloadBundle
 3. BeforeInstall
 4. Install
 5. AfterInstall
 6. ApplicationStart
 7. ValidateService
 - Features
 - CodeDeploy protects your application from downtime during deployments through rolling updates and deployment health tracking.
 - AWS CodeDeploy tracks and stores the recent history of your deployments.



- CodeDeploy is platform and language agnostic.
 - CodeDeploy uses a file and command-based install model, which enables it to deploy any application and reuse existing setup code. The same setup code can be used to consistently deploy and test updates across your environment release stages for your servers or containers.
 - CodeDeploy integrates with Amazon Auto Scaling, which allows you to scale EC2 capacity according to conditions you define such as traffic spikes. Notifications are then sent to AWS CodeDeploy to initiate an application deployment onto new instances before they are placed behind an Elastic Load Balancing load balancer.
 - When using AWS CodeDeploy with on-premises servers, make sure that they can connect to AWS public endpoints.
 - AWS CodeDeploy offers two types of deployments:
 - With in-place deployments, the application on each instance in the deployment group is stopped, the latest application revision is installed, and the new version of the application is started and validated. Only deployments that use the EC2/On-Premises compute platform can use in-place deployments.
 - With blue/green deployments, once the new version of your application is tested and declared ready, CodeDeploy can shift the traffic from your old version (blue) to your new version (green) according to your specifications.
 - Deployment groups are used to match configurations to specific environments, such as a staging or production environments. An application can be deployed to multiple deployment groups.
 - You can integrate AWS CodeDeploy with your continuous integration and deployment systems by calling the public APIs using the AWS CLI or AWS SDKs.
 - Application Specification Files
 - The AppSpec file is a YAML-formatted or JSON-formatted file that is used to manage each deployment as a series of lifecycle event hooks.
 - For ECS Compute platform, the file specifies
 - The name of the ECS service and the container name and port used to direct traffic to the new task set.
 - The functions to be used as validation tests.
 - For Lambda compute platform, the file specifies
 - The AWS Lambda function version to deploy.
 - The functions to be used as validation tests.
 - For EC2/On-Premises compute platform, the file is always written in YAML and is used to
 - Map the source files in your application revision to their destinations on the instance.
 - Specify custom permissions for deployed files.
 - Specify scripts to be run on each instance at various stages of the deployment process.
 - Deployments
-



- You can use the CodeDeploy console or the create-deployment command to deploy the function revision specified in the AppSpec file to the deployment group.
- You can use the CodeDeploy console or the stop-deployment command to stop a deployment. When you attempt to stop the deployment, one of three things happens:
 - The deployment stops, and the operation returns a status of SUCCEEDED.
 - The deployment does not immediately stop, and the operation returns a status of pending. After the pending operation is complete, subsequent calls to stop the deployment return a status of SUCCEEDED.
 - The deployment cannot stop, and the operation returns an error.
- With Lambda functions and EC2 instances, CodeDeploy implements rollbacks by redeploying, as a new deployment, a previously deployed revision.
- With ECS services, CodeDeploy implements rollbacks by rerouting traffic from the replacement task set to the original task set.
- The CodeDeploy agent is a software package that, when installed and configured on an EC2/on-premises instance, makes it possible for that instance to be used in CodeDeploy deployments. The agent is not required for deployments that use the Amazon ECS or AWS Lambda.
- CodeDeploy monitors the health status of the instances in a deployment group. For the overall deployment to succeed, CodeDeploy must be able to deploy to each instance in the deployment and deployment to at least one instance must succeed.
- You can specify a minimum number of healthy instances as a number of instances or as a percentage of the total number of instances required for the deployment to be successful.
- CodeDeploy assigns two health status values to each instance:
 - Revision health - based on the application revision currently installed on the instance. Values include Current, Old and Unknown.
 - Instance health - based on whether deployments to an instance have been successful. Values include Healthy and Unhealthy.
- Blue/Green Deployments
 - EC2/On-Premises compute platform
 - You must have one or more Amazon EC2 instances with identifying Amazon EC2 tags or an Amazon EC2 Auto Scaling group.
 - Each Amazon EC2 instance must have the correct IAM instance profile attached.
 - The CodeDeploy agent must be installed and running on each instance.
 - During replacement, you can either
 - use the Amazon EC2 Auto Scaling group you specify as a template for the replacement environment; or
 - specify the instances to be counted as your replacement using EC2 instance tags, EC2 Auto Scaling group names, or both.
 - AWS Lambda platform

- You must choose one of the following deployment configuration types to specify how traffic is shifted from the original Lambda function version to the new version:
 - Canary: Traffic is shifted in two increments. You can choose from predefined canary options that specify the percentage of traffic shifted to your updated Lambda function version in the first increment and the interval, in minutes, before the remaining traffic is shifted in the second increment.
 - Linear: Traffic is shifted in equal increments with an equal number of minutes between each increment. You can choose from predefined linear options that specify the percentage of traffic shifted in each increment and the number of minutes between each increment.
 - All-at-once: All traffic is shifted from the original Lambda function to the updated Lambda function version all at once.
- With Amazon ECS, production traffic shifts from your ECS service's original task set to a replacement task set all at once.
- Advantages of using Blue/Green Deployments vs In-Place Deployments
 - An application can be installed and tested in the new replacement environment and deployed to production simply by rerouting traffic.
 - If you're using the EC2/On-Premises compute platform, switching back to the most recent version of an application is faster and more reliable. Traffic can just be routed back to the original instances as long as they have not been terminated. With an in-place deployment, versions must be rolled back by redeploying the previous version of the application.
 - If you're using the EC2/On-Premises compute platform, new instances are provisioned and contain the most up-to-date server configurations.
 - If you're using the AWS Lambda compute platform, you control how traffic is shifted from your original AWS Lambda function version to your new AWS Lambda function version.
- With AWS CodeDeploy, you can also deploy your applications to your on-premises data centers. Your on-premises instances will have a prefix of "mi-xxxxxxx" as shown in the image below:



Developer Tools

CodeDeploy

► Source • CodeCommit

► Build • CodeBuild

▼ Deploy • CodeDeploy

Getting started

Deployments

Applications

Deployment configurations

On-premises instances

► Pipeline • CodePipeline

Developer Tools > CodeDeploy > On-premises instances

On-premises instances

Q TutorialsDojo-Manila-On-Premises

X

< 1 >

Instance name	IAM ARN	Status
Not found		
No results found for the following search: TutorialsDojo-Manila-On-Premises		



AWS CodePipeline

- A fully managed **continuous delivery service** that helps you automate your release pipelines for application and infrastructure updates.
- You can easily integrate AWS CodePipeline with third-party services such as GitHub or with your own custom plugin.
- **Concepts**
 - A **pipeline** defines your release process workflow, and describes how a new code change progresses through your release process.
 - A pipeline comprises a series of **stages** (e.g., build, test, and deploy), which act as logical divisions in your workflow. Each stage is made up of a sequence of actions, which are tasks such as building code or deploying to test environments.
 - Pipelines must have **at least two stages**. The first stage of a pipeline is required to be a source stage, and the pipeline is required to additionally have at least one other stage that is a build or deployment stage.
 - Define your pipeline structure through a **declarative JSON** document that specifies your release workflow and its stages and actions. These documents enable you to update existing pipelines as well as provide starting templates for creating new pipelines.
 - A **revision** is a change made to the source location defined for your pipeline. It can include source code, build output, configuration, or data. A pipeline can have multiple revisions flowing through it at the same time.
 - A **stage** is a group of one or more actions. A pipeline can have two or more stages.
 - An **action** is a task performed on a revision. Pipeline actions occur in a specified order, in serial or in parallel, as determined in the configuration of the stage.
 - You can add actions to your pipeline that are in an AWS Region different from your pipeline.
 - There are six types of actions
 - Source
 - Build
 - Test
 - Deploy
 - Approval
 - Invoke
 - When an action runs, it acts upon a file or set of files called **artifacts**. These artifacts can be worked upon by later actions in the pipeline. You have an artifact store which is an S3 bucket in the same AWS Region as the pipeline to store items for all pipelines in that Region associated with your account.
 - The stages in a pipeline are connected by **transitions**. Transitions can be disabled or enabled between stages. If all transitions are enabled, the pipeline runs continuously.



- An **approval action** prevent a pipeline from transitioning to the next action until permission is granted. This is useful when you are performing code reviews before the code is deployed to the next stage.
- **Features**
 - AWS CodePipeline provides you with a graphical user interface to create, configure, and manage your pipeline and its various stages and actions.
 - A pipeline starts automatically (default) when a change is made in the source location or when you manually start the pipeline. You can also set up a rule in CloudWatch to automatically start a pipeline when events you specify occur.
 - You can model your build, test, and deployment actions to run **in parallel** in order to increase your workflow speeds.
 - AWS CodePipeline can pull source code for your pipeline directly from Git, GitHub, Amazon ECR, or Amazon S3.
 - It can run builds and unit tests in AWS CodeBuild.
 - It can deploy your changes using AWS CodeDeploy, AWS Elastic Beanstalk, Amazon ECS, AWS Fargate, Amazon S3, AWS Service Catalog, and/or AWS CloudFormation.
 - You can use the CodePipeline Jenkins plugin to easily register your existing build servers as a custom action.
 - When you use the console to create or edit a pipeline that has a GitHub source, CodePipeline creates a **webhook**. A webhook is an HTTP notification that detects events in another tool, such as a GitHub repository and connects those external events to a pipeline. CodePipeline deletes your webhook when you delete your pipeline.
- As a best practice, when you use a Jenkins build provider for your pipeline's build or test action, install Jenkins on an Amazon EC2 instance and configure a separate EC2 instance profile. Make sure the instance profile grants Jenkins only the AWS permissions required to perform tasks for your project, such as retrieving files from Amazon S3.



AWS X-Ray

- AWS X-Ray analyzes and debugs production, distributed applications, such as those built using a microservices architecture. With X-Ray, you can identify performance bottlenecks, edge case errors, and other hard-to-detect issues.
- **Features**
 - AWS X-Ray can be used with applications running on Amazon EC2, Amazon ECS, AWS Lambda, and AWS Elastic Beanstalk. You just integrate the X-Ray SDK with your application and install the X-Ray agent.
 - AWS X-Ray provides an end-to-end, cross-service, application-centric view of requests flowing through your application by aggregating the data gathered from individual services in your application into a single unit called a *trace*.
 - You can set the **trace sampling rate** that is best suited for your production applications or applications in development. X-Ray continually traces requests made to your application and stores a sampling of the requests for your analysis.
 - AWS X-Ray creates a map of services used by your application with trace data. This provides a view of connections between services in your application and aggregated data for each service, including average latency and failure rates. You can create dependency trees, perform cross-availability zone or region call detections, and more.
 - AWS X-Ray lets you add annotations to data emitted from specific components or services in your application.

AWS SDKs and Tools

- AWS SDKs and Tools provide programming interfaces and development tools that allow developers to easily build applications on AWS.
- AWS SDKs are available for popular programming languages like Java, Python, Ruby, .NET, JavaScript etc. The SDKs allow developers to access AWS services programmatically from their code. It handles tasks like authentication, request signing and error handling.
- The Amazon Web Services CLI (Command Line Interface) is a comprehensive tool for managing multiple AWS services using commands. For example `aws ec2 run-instances` to launch EC2 instances.
- AWS provides plugins and extensions for popular IDEs like Eclipse, IntelliJ, VS Code, etc. These plugins help in developing, debugging and deploying AWS applications from within the IDEs.
- AWS X-Ray helps developers analyze and debug distributed applications on AWS. It provides service maps and traces to help identify issues.
- AWS Amplify is a framework for frontend web and mobile applications that can help with user authentication, analytics, storage etc.



References:

<https://docs.aws.amazon.com/sdk-for-net/v3/developer-guide/sdks-and-tools-ref.html>

<https://docs.aws.amazon.com/transcribe/latest/dg/getting-started-sdk.html>

<https://aws.amazon.com/developer/tools/>



AWS Application Services

Amazon SNS

- A web service that makes it easy to set up, operate, and send notifications from the cloud. SNS follows the “**publish-subscribe**” (**pub-sub**) **messaging** paradigm, with notifications being delivered to clients using a “**push**” mechanism rather than to periodically check or “poll” for new information and updates.

Features

- SNS is an **event-driven** computing hub that has native integration with a wide variety of AWS event sources (including EC2, S3, and RDS) and AWS event destinations (including SQS, and Lambda).
 - **Event-driven computing** is a model in which subscriber services automatically perform work in response to events triggered by publisher services. It can automate workflows while decoupling the services that collectively and independently work to fulfill these workflows.
- **Message filtering** allows a subscriber to create a filter policy, so that it only gets the notifications it is interested in.
- **Message fanout** occurs when a message is sent to a topic and then replicated and pushed to multiple endpoints. Fanout provides asynchronous event notifications, which in turn allows for parallel processing.
- **SNS mobile notifications** allow you to fanout mobile push notifications to iOS, Android, Fire OS, Windows, and Baidu-based devices. You can also use SNS to fanout text messages (SMS) to 200+ countries and fanout email messages (SMTP).
- **Application and system alerts** are notifications triggered by predefined thresholds, sent to specified users by SMS and/or email.
- **Push email** and **text messaging** are two ways to transmit messages to individuals or groups via email and/or SMS.

SNS provides simple APIs and easy integration with applications.

Publishers and Subscribers

- Publishers communicate asynchronously with subscribers by producing and sending a message to a topic, which is a logical access point and communication channel.
- Subscribers consume or receive the message or notification over one of the supported protocols when they are subscribed to the topic.
- Publishers create topics to send messages, while subscribers subscribe to topics to receive messages.
- SNS FIFO topics support the forwarding of messages to SQS FIFO queues. You can also use SNS to forward messages to standard queues.

SNS Topics



- Instead of including a specific destination address in each message, a publisher sends a message to a **topic**. SNS matches the topic to a list of subscribers who have subscribed to that topic and delivers the message to each of those subscribers.
- Each topic has a unique name that identifies the SNS endpoint for publishers to post messages and subscribers to register for notifications.
- A topic can support subscriptions and notification deliveries over multiple transports.

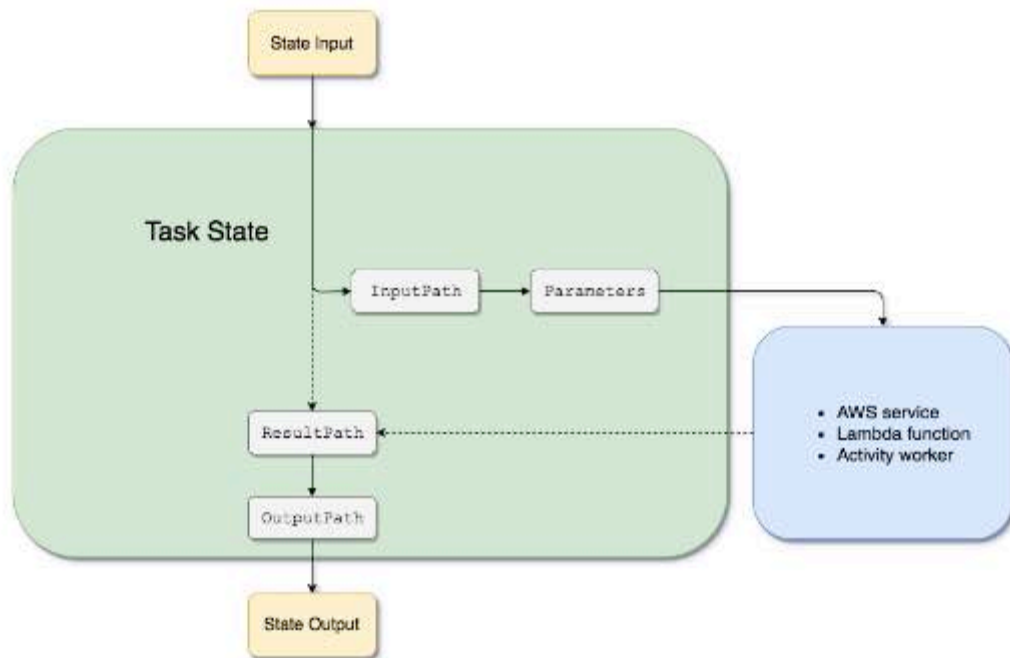
System to System Messaging

- When a message is published to an SNS topic that has a **Lambda function** subscribed to it, the Lambda function is invoked with the payload of the published message. The Lambda function receives the message payload as an input parameter and can manipulate the information in the message, publish the message to other SNS topics, or send the message to other AWS services.
- When you subscribe a **SQS queue** to a SNS topic, you can publish a message to the topic, and SNS sends a SQS message to the subscribed queue. The SQS message contains the subject and message that were published to the topic, along with metadata about the message in a JSON document.
- When you subscribe to an **HTTP/s endpoint** to a topic, you can publish a notification to the topic, and SNS sends an HTTP POST request delivering the contents of the notification to the subscribed endpoint. When you subscribe to the endpoint, you select whether SNS uses HTTP or HTTPS to send the POST request to the endpoint.



AWS Step Functions

- AWS Step Functions is a web service that provides **serverless orchestration** for modern applications. It enables you to coordinate the components of distributed applications and microservices using visual workflows.
- Concepts
 - Step Functions is based on the concepts of **tasks** and **state machines**.
 - A task performs work by using an activity or an AWS Lambda function or by passing parameters to the API actions of other services.
 - A finite state machine can express an algorithm as a number of states, their relationships, and their input and output.
 - You define state machines using the **JSON-based Amazon States Language**.
 - A state is referred to by its *name*, which can be any string, but must be unique within the scope of the entire state machine. An instance of a state exists until the end of its execution.
 - There are different types of states in AWS Step Functions
 - Task state – Do some work in your state machine. AWS Step Functions can invoke Lambda functions directly from a task state.
 - Choice state – Make a choice between branches of execution
 - Fail state – Stops execution and marks it as failure
 - Succeed state – Stops execution and marks it as a success
 - Pass state – Simply pass its input to its output or inject some fixed data
 - Wait state – Provide a delay for a certain amount of time or until a specified time/date
 - Parallel state – Begin parallel branches of execution
 - Map state – Adds a for-each loop condition
 - Common features between states
 - Each state must have a *Type* field indicating what type of state it is.
 - Each state can have an optional *Comment* field to hold a human-readable comment about, or description of, the state.
 - Each state (except a Succeed or Fail state) requires a *Next* field or, alternatively, can become a terminal state by specifying an **End** field.
 - **Activities** enable you to place a task in your state machine where the work is performed by an **activity worker** that can be hosted on Amazon EC2, Amazon ECS, or mobile devices.
 - Activity tasks let you assign a specific step in your workflow to code running in an activity worker. Service tasks let you connect a step in your workflow to a supported AWS service.
 - With **Transitions**, after executing a state, AWS Step Functions uses the value of the *Next* field to determine the next state to advance to. States can have multiple incoming transitions from other states.
 - Individual states receive JSON as input and usually pass JSON as output to the next state.



- A **state machine execution** occurs when a state machine runs and performs its tasks. Each Step Functions state machine can have multiple simultaneous executions.
- **State machine updates** in AWS Step Functions are **eventually consistent**.
- By default, when a state reports an error, AWS Step Functions causes the execution to **fail entirely**.
 - Task and Parallel states can have a field named *Retry* and *Catch* to retry an execution or to have a fallback state.
- The Step Functions console displays a graphical view of your state machine's structure, which provides a way to visually check a state machine's logic and monitor executions.



Comparison of AWS Services

AWS CloudTrail vs Amazon CloudWatch

- CloudWatch is a monitoring service for AWS resources and applications. CloudTrail is a web service that records API activity in your AWS account. They are both useful monitoring tools in AWS.
- By default, CloudWatch offers free basic monitoring for your resources, such as EC2 instances, EBS volumes, and RDS DB instances. CloudTrail is also enabled by default when you create your AWS account.
- With CloudWatch, you can collect and track metrics, collect and monitor log files, and set alarms. CloudTrail, on the other hand, logs information on who made a request, the services used, the actions performed, parameters for the actions, and the response elements returned by the AWS service. CloudTrail Logs are then stored in an S3 bucket or a CloudWatch Logs log group that you specify.
- You can enable detailed monitoring from your AWS resources to send metric data to CloudWatch more frequently, with an additional cost.
- CloudTrail delivers one free copy of management event logs for each AWS region. Management events include management operations performed on resources in your AWS account, such as when a user logs in to your account. Logging data events are charged. Data events include resource operations performed on or within the resource itself, such as S3 object-level API activity or Lambda function execution activity.
- CloudTrail helps you ensure compliance and regulatory standards.
- CloudWatch Logs reports on application logs, while CloudTrail Logs provide you with specific information on what occurred in your AWS account.
- Amazon EventBridge is a near real time stream of system events describing changes to your AWS resources. CloudTrail focuses more on AWS API calls made in your AWS account.
- Typically, CloudTrail delivers an event within 15 minutes of the API call. CloudWatch delivers metric data in 5 minutes periods for basic monitoring and 1-minute periods for detailed monitoring. The CloudWatch Logs Agent will send log data every five seconds by default.

CloudWatch Agent vs SSM Agent vs Custom Daemon Scripts

CloudWatch Agent	SSM Agent (AWS Systems Manager)	Custom Daemon Scripts
CloudWatch agent allows you to collect more system-level metrics from your EC2 and on-premises servers than just the standard CloudWatch metrics. It also enables you to retrieve custom metrics from your applications or services using the <i>StatsD</i> and <i>collectd</i> protocols. <i>StatsD</i> is supported on both Linux servers and servers running Windows Server. <i>collectd</i> is supported only on Linux servers. You can use CloudWatch agent to collect logs from your servers and send them to CloudWatch Logs. Metrics collected by the CloudWatch agent are billed as custom metrics. You can install CloudWatch Agent using three ways: • via Command Line • via SSM Agent • via AWS CloudFormation	SSM Agent is Amazon software that runs on your EC2 instances and your hybrid instances that are configured for Systems Manager. SSM Agent processes requests from the Systems Manager service in the cloud and configures your machine as specified in the request. You can manage servers without having to log in to them using automation. SSM Agent sends status and execution information back to the Systems Manager service by using the <i>EC2 Messaging</i> service. SSM Agent runs on Amazon EC2 instances using root permissions (Linux) or SYSTEM permissions (Windows). CloudWatch agent replaces SSM agent in sending metric logs to CloudWatch Logs.	You use custom scripts (such as cron or bash scripts) if the two previously mentioned agents do not fit your needs. CloudWatch agent is useful for collecting system-level metrics and logs. You can create custom scripts that perform some modifications before the metrics are sent out. SSM Agent is also useful for automation purposes, though Systems Manager does not have a document for every case scenario. You may also have some compliance requirements that would require SSM Agent to be disabled (recall that SSM agent runs at root level permissions).



EC2 Container Services ECS vs Lambda

Amazon EC2 Container Service (ECS)	AWS Lambda
• Amazon ECS is a highly scalable, high performance container management service that supports Docker container and allows you to easily run applications on a managed cluster of Amazon EC2 instances. ECS eliminates the need for you to install, operate, and scale your own cluster management infrastructure. • With ECS, deploying containerized applications is easily accomplished. This service fits well in running batch jobs or in a microservice architecture. You have a central repository where you can upload your Docker Images from ECS container for safekeeping called Amazon ECR. • Application in ECS can be written in a stateful or stateless matter. • The Amazon ECS CLI supports Docker Compose, which allows you to simplify your local development experience as well as easily set up and run your container on Amazon ECS. • Since your application still run on EC2 instances, server management is your responsibility. This gives you more granular control over your system.	• AWS Lambda is a function-as-a-service offering that runs your code in response to events and automatically manages the compute resources for you, since Lambda is a serverless compute service. With Lambda, you do not have to worry about managing servers, and directly focus on your application code. • Lambda automatically scales your function to meet demands. It is noteworthy, however, that Lambda has a maximum execution duration of 900 seconds or 15 minutes. • To allow your Lambda function to access other services such as CloudWatch Logs, you would need to create an execution role that has the necessary permission to do so. • You can easily integrate your function with different services such as API Gateway, Dynamo DB, CloudFront, etc. using the Lambda console. • You can test your function code locally in the Lambda console before launching it into production. Currently, Lambda supports only a number of programming languages such as Java, Go, PowerShell, Node.js, C#, Python, and Ruby. In contrast, ECS is not limited by programming languages, since it focuses primarily on containerization with Docker.



Amazon EC2 Container Service (ECS)	AWS Lambda
• It is up to you to manage scaling and load balancing of your EC2 instances as well, unlike in AWS Lambda where functions scale automatically. • You are charged for the costs incurred by your EC2 instances in your clusters. Most of the time, Amazon ECS costs more than using AWS Lambda since your active EC2 instances will be charged by the hour. • One version of Amazon ECS, known as AWS Fargate, will fully manage your so you can just focus on deploying containers. AWS Fargate has a different pricing model from the standard EC2 cluster. • ECS will automatically recover unhealthy containers to ensure that you have the desired number of containers supporting your application.	• Lambda functions are stateless since you do not have volumes for data storage. • You are charged based on the number of requests for your functions and the duration, the time it takes for your code to execute. To minimize costs, you can throttle the number of concurrent executions running at a time, and the execution time limit of the function. • With Lambda@Edge, AWS Lambda can run your code across AWS locations globally in response to Amazon CloudFront events, such as requests for content to or from origin servers and viewers. This makes it easier to deliver content to end users with low latency



EC2 Instance Health Check vs ELB Health Check vs Auto Scaling and Custom Health Check

EC2 Instance Health Check	Elastic Load Balancer (ELB) Health Check	Auto Scaling and Custom Health Checks
Amazon EC2 performs automated checks on every running EC2 instance to identify hardware and software issues. Status checks are performed every minute and each returns a pass or a fail status. • If all check pass, the overall status of the instance is OK. • If one or more check fails, the overall status is impaired. Status checks are built into Amazon EC2, so they cannot be disabled or deleted You can create or delete alarms that are triggered based on the result of the status checks. There are two types of status checks	To discover the availability of your registered EC2 instances, a load balancer periodically send pings, attempts connections, or sends requests to test the EC2 instances. The status of the instances that are healthy at the time of the health check is <code>InService</code>. The status of any instances that are unhealthy at the time of the health check is <code>OutOfService</code>. When configuring a health check, you would need to provide the following: • A specific port • Protocol to use ○ HTTP/HTTPS health check succeeds if the instance returns a 200 response code within the health check interval. ○ A TCP health check succeeds if the TCP connection succeeds.	All instances in your Auto Scaling group start in the healthy state. Instances are assumed to be healthy unless EC2 Auto Scaling receives notification that they are unhealthy. This notifications can come from one or more of the following sources: • Amazon EC2 (default) • Elastic Load Balancing • A custom health check After Amazon EC2 Auto Scaling marks an instance as unhealthy, it is scheduled for replacement. If you do not want instances to be replaced, you can suspend the health check process for any individual Auto Scaling Group. If an instance is in any state other than running or if the system status is impaired, Amazon EC2 Auto Scaling considers the instance to be unhealthy and launches a replacement instance.



System Status Checks - These checks detect underlying problems with your instance that require AWS involvement to repair. When a system status check fails, you can choose to wait for AWS to fix the issue, or you can resolve it yourself. Instance Status Checks - Monitor the software and network configuration of your individual instance. Amazon EC2 checks the health of an instance by sending an address resolution protocol (ARP) request to the ENI. These checks detect problems that require your involvement to repair.	- An SSL health check succeeds if the SSL handshake succeeds. - ping path ELB health checks do not support Web Sockets. The load balancer routes requests only to the healthy instances. When an instance becomes impaired, the load balancer resumes routing requests to the instance only when it has been restored to a healthy state. The load balancer checks the health of the registered instances using either: - the default health check configuration provided by Elastic Load Balancing or - a health check configuration that you can configure (auto scaling or custom health checks for example). Network Load Balancers use active and passive health checks to determine whether a target is available to handle requests.	If you attached a load balancer or target group to your Auto Scaling Group, Amazon EC2 Auto Scaling determines the health status of the instances by checking both the EC2 status checks and the Elastic Load Balancing health checks. Amazon EC2 Auto Scaling waits until the health check grace period ends before checking the health status of the instance. Ensure that the health check grace period covers the expected startup time for your application. Health check grace period does not start until lifecycle hook actions are completed and the instance enters the InService state. With custom health checks, you can send an instance's health information directly from your system to Amazon EC2 Auto Scaling.
---	--	--



	• With active health checks, the load balancer periodically sends a request to each registered target to check its status. After each health check is completed, the load balancer node closes the connection that was established. • With passive health checks, the load balancer observes how targets respond to connections, which enables it to detect an unhealthy target before it is reported as unhealthy by active health checks. You cannot disable, configure, or monitor passive health checks.	
--	--	--

Elastic Beanstalk vs CloudFormation vs CodeDeploy

AWS Elastic Beanstalk	AWS CloudFormation	AWS CodeDeploy
AWS Elastic Beanstalk makes it even easier for developers to quickly deploy and manage applications in the AWS Cloud. Developers simply upload their application, and Elastic Beanstalk automatically handles the deployment details of capacity provisioning, load balancing, auto-scaling, and application health monitoring.	AWS CloudFormation is a service that gives developers and businesses an easy way to create a collection of related AWS resources and provision them in an orderly and predictable fashion. This is typically known as "infrastructure as code" .	AWS CodeDeploy is a service that coordinates application deployments across EC2 instances and instances running on-premises. It makes it easier for you to rapidly release new features, helps you avoid downtime during deployment and handles the complexity of updating your applications.
This platform-as-a-service solution is typically for those who want to deploy and manage their applications within minutes in the AWS Cloud without worrying about the underlying infrastructure.	The main difference between CloudFormation and Elastic Beanstalk is that CloudFormation deals more with the AWS infrastructure rather than applications. AWS CloudFormation introduces two concepts: • The template, a JSON or YAML-format, text-based file that describes all the AWS resources and configurations you need to deploy to run your application. • The stack, which is the set of AWS resources that created and managed as a single unit when AWS CloudFormation instantiates a template.	Unlike Elastic Beanstalk, CodeDeploy does not automatically handle capacity provisioning, scaling and monitoring.
AWS Elastic Beanstalk supports the following languages and development stacks: • Apache Tomcat for Java applications	CloudFormation also supports a rollback feature through template version controls. When you try to update your stack but the deployment failed midway.	Unlike CloudFormation, CodeDeploy does not deal with infrastructure configuration and orchestration.



• Apache HTTP Server for PHP applications • Apache HTTP Server for Python applications • Nginx or Apache HTTP Server for Node.js applications • Passenger or Puma for Ruby applications • Microsoft IIS for .NET applications • Java SE • Docker • Go		
Elastic Beanstalk also supports deployment versioning. It maintains a copy of older deployments so that it is easy for the developer to rollback any changes made on the application.	CloudFormation will automatically revert the changes back to their previous working states.	AWS CodeDeploy is a building block service focused in helping developers deploy and update software on any instance, including EC2 instances and instances running on-premises. AWS Elastic Beanstalk is end-to-end application management solutions.
	CloudFormation supports Elastic Beanstalk application environments. This allows you, for example, to create and manage an AWS Elastic Beanstalk-hosted application along with an RDS database to store the application data.	You create deployment configuration file to specify how deployments proceed.
	AWS CloudFormation can be used to bootstrap both Chef (Server and Client) and Puppet (Master and Client) softwares on your EC2 instances.	CodeDeploy complements CloudFormation well when deploying code to infrastructure that is provisioned and managed with CloudFormation.
	AWS CodeDeploy is a recommended adjunct to CloudFormation for managing the application deployments and	



	updates.	
--	----------	--

Additional Notes:

- Elastic Beanstalk, or CloudFormation are particularly useful for the **blue-green deployment method** as they provide a simple way to clone your running application stack.
- CloudFormation is best suited for the **prebaking AMIs**.
- CodeDeploy is best suited for performing **in-place application upgrades**. For **disposable upgrades**, you can set up a cloned environment with Elastic Beanstalk, and CloudFormation.

Service Control Policies vs IAM Policies

Service Control Policies (SCP)	IAM Policies
SCPs are mainly used along with AWS Organizations organizational units (OUs).	IAM Policies operate at the Principal level. There are two types of IAM policies: • Identity-based policies - attached to an IAM user, group or role. • Resource-based policies - attached to an AWS resource such as an S3 bucket.
SCPs do not replace IAM Policies such they do not provide actual permissions. To perform an action, you would still need to grant appropriate IAM Policy permissions.	IAM Policies can grant/deny a Principal permissions to perform certain actions to certain resources. This can be used together with SCP to ensure stricter controls in AWS Organizations. An IAM policy can be applied only to IAM users, groups, or roles, and it can never restrict the root identity of the AWS account.
Even if a Principal is allowed to perform a certain action (granted through IAM Policies), an attached SCP will override that capability if it enforces a Deny	IAM Policies cannot be attached to OUs.



on that action.	
SCP takes precedence over IAM Policies.	An IAM Policy can allow or deny actions. An explicit allow overrides an implicit deny. An explicit deny overrides an explicit allow.
SCPs can be applied to the root of an organization or to individual accounts in an OU.	
When you apply an SCP to an OU or an individual AWS account, you choose to either enable (whitelist), or disable (blacklist) the specified AWS service. Access to any service that isn't explicitly allowed by the SCPs associated with an account, its parent OUs, or the management account is denied to the AWS accounts or OUs associated with the SCP.	
Any account has only those permissions permitted by every parent above it. If a permission is blocked at any level above the account, either implicitly (by not being included in an Allow policy statement) or explicitly (by being included in a Deny policy statement), a user or role in the affected account can't use that permission, even if there is an attached IAM policy granting Administrator permissions to the user.	
SCPs affect only principals that are managed by accounts that are part of the organization.	



FINAL REMARKS AND TIPS

That's a wrap! Thank you once again for choosing our Study Guide and Cheat Sheets for the AWS Certified DevOps Engineer Professional (DOP-C02) exam. The [Tutorials Dojo](#) team spent considerable time and effort to produce this content to help you pass the AWS exam.

We also recommend that before taking the actual DOP-C02 exam, allocate some time to check your readiness by taking our **[AWS practice test course](#)** in the Tutorials Dojo Portal. You can also try the free sampler version of our full practice test course [here](#). This will help you identify the topics that you need to improve on and help reinforce the concepts that you need to fully understand in order to pass this certification exam. It also has different training modes that you can choose from such as Timed mode, Review mode, Section-Based tests, and Final test plus bonus flashcards. In addition, you can read the technical discussions in our forums or post your queries if you have one. If you have any issues, concerns or constructive feedback on our eBook, feel free to contact us at support@tutorialsdojo.com.

On behalf of the Tutorials Dojo team, we wish you all the best on your upcoming AWS Certified DevOps Engineer Professional exam. May it help advance your career, as well as increase your earning potential.

With the right strategy, hard work, and unrelenting persistence, you can definitely make your dreams a reality! You can make it!

Sincerely,
Jon Bonso, Kenneth Samonte, and the Tutorials Dojo Team

ABOUT THE AUTHORS



Jon Bonso (10x AWS Certified)

Born and raised in the Philippines, Jon is the Co-Founder of **Tutorials Dojo**. Now based in Sydney, Australia, he has over a decade of diversified experience in Banking, Financial Services, and Telecommunications. He's 8x AWS Certified and has worked with various cloud services such as Google Cloud and Microsoft Azure. Jon is passionate about what he does and dedicates a lot of time creating educational courses. He has given IT seminars to different universities in the Philippines for free and has launched educational websites using his own money and without any external funding.



Kenneth Samonte (3x AWS Certified)

Kenneth is a registered Electronics Engineer and a Cloud Architect by profession. He's been tinkering with Linux servers and VMs since 2010 for his University degree. He's certified with AWS and Google Cloud platforms as well as Red Hat and VMware systems. When he's not busy typing away bash commands, you'll find him online playing League of Legends.